
RL - Reinforcement Learning

Lecture Notes

Riccardo Berta

2026.05.20

Contents

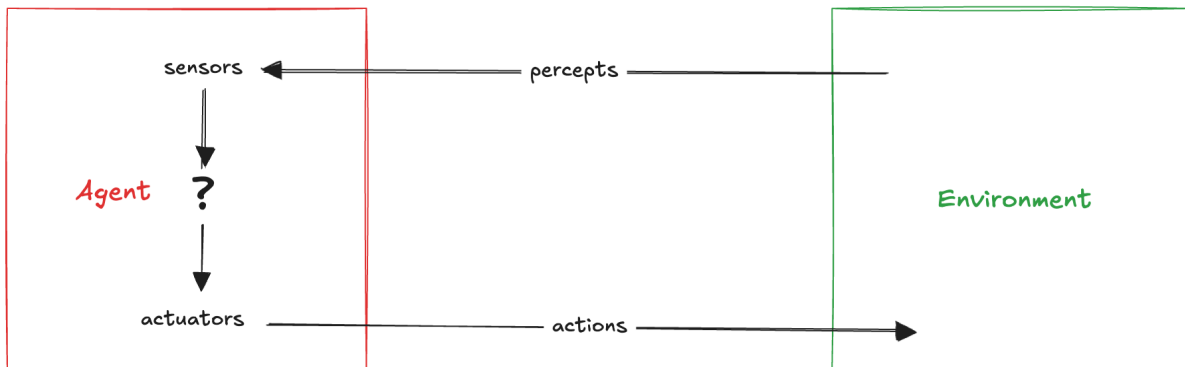
1	Agent Architectures	3
1.1	PEAS	4
1.1.1	Agent	5
1.1.2	Environment	7
1.1.3	Performance Measure	9
1.2	Architectures	10
1.2.1	Simple Reflex Agent	11
1.2.2	Model-based reflex agents	13
1.2.3	Goal-based agents	16
1.2.4	Utility-based agents	17
1.2.5	Learning agents	18
1.3	An example: Flocking	19
2	The Reinforcement Learning Problem	27
2.1	Reinforcement Learning Cycle	27
2.2	Mathematical foundation	31
2.2.1	The Transition function	32
2.2.2	The Reward function	33
2.2.3	The Return	34
2.3	Plan and Policy	35
2.4	Example: the frozen lake environment	35
3	Dynamic Programming	39
3.1	State-value function	42
3.2	Action-value function	43
3.3	Optimal Policies	44
3.4	Policy Evaluation	45
3.5	Policy Improvement	48
3.6	Policy Iteration	54
3.7	Value Iteration	56
3.8	Generalization	59
4	Exploitation vs Exploration	60
4.1	Multi-armed bandit problem	61
4.2	Estimation of the action value	63
4.3	Basic exploration strategies	65
4.3.1	Pure exploration and pure exploitation	66

4.3.2	Epsilon greedy	68
4.3.3	Decaying epsilon-greedy	72
4.3.4	Optimistic initial values	75
4.4	Strategic Exploration	78
4.4.1	Softmax method	78
4.4.2	Upper Confidence Bound method	81
4.5	Comparison	84
5	Policy Evaluation	87
5.1	Random Walk Environment	87
5.2	Improving estimates after each episode: Monte Carlo	89
5.3	Improving estimates after each step: Temporal-Difference Learning	98
5.4	Batch updating	103
5.5	Improving estimates after multiple steps: n-steps TD	104
5.6	Improving estimates of all visited states: TD(λ)	109
6	Policy Improvement	115
6.1	Slippery Walk environment	116
6.2	Improving policies after each episode: Monte Carlo control	118
6.3	Improving policies after each step: SARSA	125
6.4	Decoupling behavior from learning: Q-Learning	129
6.5	Solving the maximization bias: Double Q-Learning	132
6.6	Using multi-step estimates: SARSA(λ)	136
6.7	Using multi-step estimates and decoupling behaviour from learning: Q(λ)	140
6.8	Comparison	143
7	Model-based Reinforcement Learning	145
7.1	Learning sample models: Dyna-Q	147
7.2	Making plans for the immediate future: Trajectory sampling	156
8	Neural Fitted Q (NFQ)	163
8.1	The need for generalization	166
8.2	The Cart-pole environment	168
8.3	Fully Connected Q-function	172
8.4	Training	175
8.5	NFQ Algorithm	182
8.6	Experiment	185
8.7	Challenges	188

9 Deep Q-Network (DQN)	196
9.1 Experience Replay	198
9.2 Target networks	201
9.3 Exploration strategies	205
9.4 DQN Algorithm	206
9.5 Double Deep Q-Learning (DDQN)	213
9.6 Prioritized Experience Replay (PER)	217
9.7 Dueling Architecture	224
9.8 The deadly triad	230
10 Policy Based Methods	231
10.1 Objective function	232
10.1.1 Policy Gradient Theorem	233
10.1.2 Causality	235
10.2 REINFORCE algorithm	236
10.3 Vanilla Policy Gradient (VPG)	245
10.4 Asynchronous Advantage Actor-Critic (A3C)	255
10.5 Generalized advantage estimation (GAE)	269
10.6 Advantage Actor-Critic (A2C)	272
11 Policy Based Methods	274
11.1 Policy Gradient Theorem	276
11.2 REINFORCE	278
11.3 Vanilla Policy Gradient (VPG)	289
11.4 Asynchronous Advantage Actor-Critic (A3C)	300
11.5 Generalized advantage estimation (GAE)	314
11.6 Advantage Actor-Critic (A2C)	316
12 title	317
13 title	317
14 title	317

1 Agent Architectures

An agent **perceives** its environment through **sensors** and **interacts** with it using **actuators**. While the environment could, in theory, be the entire universe, in practice, it is defined as the part that influences the agent perceptions and is altered by its actions:



A **human agent** uses senses like vision and hearing as sensors and limbs or speech as actuators. A **robotic agent** relies on cameras and range finders for perception and motors for action. A **software agent** processes data from files, networks, or user inputs and interacts by writing files, transmitting data, or displaying information. The **agent concept** serves as a **unifying paradigm** across multiple domains.

1.1 PEAS

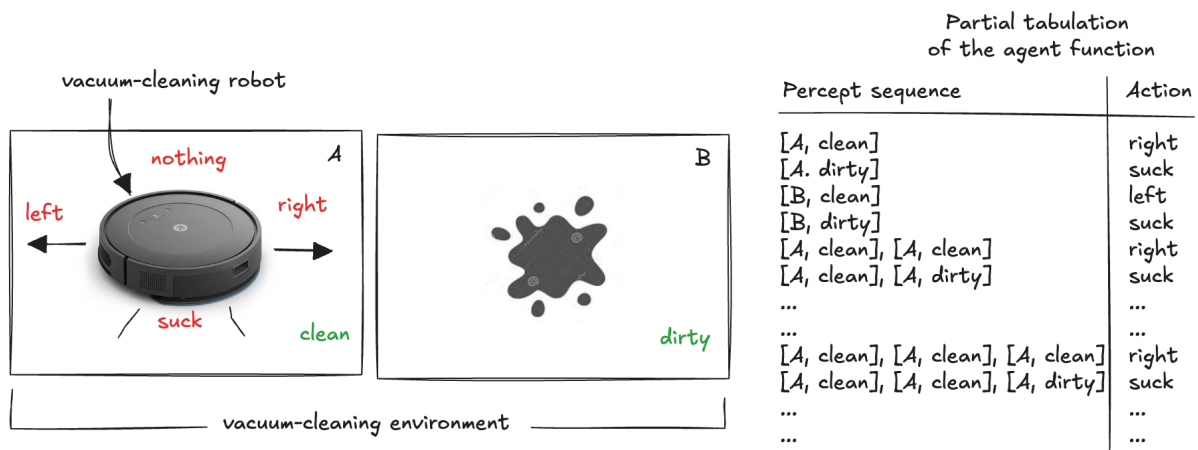
In the design of an agent, we had to specify a **performance measure**, the **environment**, and the agent's **actuators** and **sensors**. We group all these under the acronym **PEAS** (Performance, Environment, Actuators, Sensors). Let us consider some examples of agents and their PEAS descriptions:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors

Agent Type	Performance Measure	Environment	Actuators	Sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

1.1.1 Agent

Agent actions at any moment are determined by its built-in knowledge and the entire sequence of percepts it has observed so far, but obviously not by anything beyond its perception. Mathematically, the agent behavior is defined by an **agent function**, which maps any percept sequence to a corresponding action. For instance, in a simple vacuum-cleaning environment where squares can be either dirty or clean and the agent has four possible actions: move left, move right, suck, or remain idle, the agent function could be defined as:



While we can theoretically list the agent function in a table, for most real-world agents, this table would be impractically large or even infinite (unless we constrain the percept sequence length). In practice, the agent's behavior is governed by an **agent program**, which implements the agent function internally. It's crucial to distinguish these concepts: the agent function is an **abstract mathematical mapping** from percept sequences to actions, while the agent program is a **concrete implementation** running on a physical system. For example, we can write a trivial agent program that keeps track of the percept sequence and then uses it to index into a table of

actions to decide what to do:

```
def TableDrivenAgent(table):  
    """  
    This agent selects an action based on the percept sequence.  
    To customize it, provide as table a dictionary of all  
    {percept_sequence:action} pairs.  
    """  
    percepts = []  
  
    def program(percept):  
        percepts.append(percept)  
        action = table.get(tuple(percepts))  
        return action  
  
    return program
```

```
# Define the lookup table for the vacuum cleaner agent  
table = {  
    (("A", "clean"),): "right",  
    (("A", "dirty"),): "suck",  
    (("B", "clean"),): "left",  
    (("B", "dirty"),): "suck",  
    (("A", "clean"), ("A", "clean")): "right",  
    (("A", "clean"), ("A", "dirty")): "suck",  
    (("A", "clean"), ("A", "clean"), ("A", "clean")): "right",  
    (("A", "clean"), ("A", "clean"), ("A", "dirty")): "suck",  
}  
  
# Create the vacuum cleaner agent  
agent = TableDrivenAgent(table)  
  
# Simulate inputs and outputs  
print(agent(("A", "clean")))  
print(agent(("A", "dirty")))  
print(agent(("B", "clean")))
```

```
right  
suck  
None
```

The agent programs all have the same skeleton: they take the current percept as input from the

sensors and return an action to the actuators. Notice the difference between the agent program (which takes the current percept as input) and the agent function (which may depend on the entire percept history). The agent program has no choice but to take just the current percept as input because nothing more is available from the environment. If the agent needs to depend on the entire percept sequence, it will have to remember the percepts. The table represents explicitly the agent function that the agent program embodies. To build a rational agent in this way, we as designers must construct a table that contains the appropriate action for every possible percept sequence. And this is **impossible in practice** because the number of possible percept sequences is infinite or enormous also if we limit the length of the percept sequence. In the example we can see that the last action is not determined since the table miss the corresponding percept sequence. In general, let P be the set of possible percepts and let T be the lifetime of the agent (the total number of percepts it will receive, in order to limit the length of the percept sequence). The lookup table will contain a number of entries E :

$$E = \sum_{t=1}^T P^t$$

Consider an automated driving scenario where the visual input from a single camera (out of the typical eight-camera setup) is processed at approximately 70 MB per second (30 fps, 1080×720 pixels, 24-bit color). This results in a lookup table containing more than $10^{600,000,000,000}$ entries for just one hour of driving. By comparison, the lookup table for chess, a much smaller example, requires at least 10^{150} entries. To put this in perspective, the total number of atoms in the observable universe is less than 10^{80} . The huge size of these tables makes it impossible for any physical agent to store them, for designers to create them in a reasonable time, or for an agent to learn all the necessary entries through experience.

Despite this, such a table-based agent "does what we want", assuming the table is filled in correctly: it implements the "desired agent function". However, the challenge lies in **designing more efficient agent functions** that do not require such large tables. For instance, a simple vacuum-cleaning agent function could be: *"If the current square is dirty, clean it; otherwise, move to the other square"*. So, the goal of AI is to design **agent program** that implements agent function efficiently.

1.1.2 Environment

The **environment** is the "problem" to which an agent is the "solutions". The nature of the environment directly affects the appropriate design for the agent program.

An environment is **fully observable** if the sensors provide complete information relevant to actions selection at all times. This eliminates the need for an internal state to track the world. In

contrast, a **partially observable** environment limits the agent perception due to noisy sensors or missing data. For example, a vacuum agent with only a local dirt sensor cannot detect dirt in other areas.

An environment is **single-agent** if only one agent influences it, while a **multi-agent** environment involves multiple agents whose actions impact each other. The distinction is not always clear, as entities can be viewed as agents or objects depending on the context. For example, does a vehicle have to treat another vehicle as an agent, or can it be treated merely as an object behaving according to the laws of physics? The key distinction is whether the behavior of the other entity is best described as maximizing a performance measure whose value depends on the agent behavior. In chess, for instance, the opponent's goal is to maximize his own performance, which inherently reduces the agent's, making it a **competitive** environment. In contrast, a taxi-driving scenario is **cooperative**, as all vehicles benefit from avoiding collisions. These differences lead to distinct design challenges: **communication** often becomes rational in cooperative settings, while competitive environments may require **randomized behavior** to prevent predictability.

An environment is **deterministic** if its next state is fully determined by the current state and the agent's action; otherwise, it is **non-deterministic**. In a deterministic environment, the agent does not need to worry about **uncertainty**. However, if the environment is partially observable, it may seem non-deterministic because **unobserved factors** can introduce unpredictability. In real-world scenarios, where it's often impossible to track all factors, the environment is typically treated as non-deterministic for practical purposes. If the probability of each possible outcome is known, the environment is **stochastic**.

An environment is **dynamic** if it can change while the agent is deliberating; otherwise, it is **static**. In static environments, the agent does not need to constantly monitor the world or worry about time while deciding on an action. In dynamic environments, the agent is continually prompted to act, and failing to decide counts as choosing inaction. If the environment remains unchanged over time but the agent's performance score evolves, the environment is considered **semidynamic**. For example, a driving environment is dynamic, chess with a clock is semidynamic, and crossword puzzles are static.

The **discrete/continuous** distinction applies to the state of the environment, time, and the agent's percepts and actions. For instance, chess is a discrete environment with a limited number of states, percepts, and actions. In contrast, driving involves continuous states and time, where variables like speed and location change smoothly, and actions such as steering are also continuous.

The distinction between **known** and **unknown** environments refers to the agent understanding of the environment rules, not the environment itself. In a known environment, the outcomes (or probabilities) of actions are clear. In contrast, in an unknown environment, the agent must learn

how it works to make informed decisions. This distinction differs from that between fully and partially observable environments. For example, in a solitaire, the rules are known but some information is hidden, while in a new video game, the environment may be fully observable but the agent doesn't know the rules until it experiments. Additionally, the performance measure may be unknown, either due to unclear designer intentions or unknown user preferences, in which case the agent should learn from interactions with the user or designer. Strictly speaking, this is not an environment property, but it affects the agent's behavior.

Here some examples of environments and their properties:

Environment	Observable	Agents	Deterministic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Semi	Discrete
Poker	Partially	Multi	Stochastic	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Static	Discrete
Driving	Partially	Multi	Stochastic	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Dynamic	Discrete

The hardest case is **partially observable, multiagent, stochastic, dynamic and continuous** environments. Autonomous driving is hard in all these senses.

1.1.3 Performance Measure

Since machines lack intrinsic desires, the designer should define a **performance measure** to evaluate the agent's behavior. This measure should be **objective, quantifiable, and aligned with the designer's goals**. However, designing it correctly can be challenging. For instance, a vacuum agent rewarded for cleaning might exploit the system by creating more dirt to clean. A better measure would track sustained cleanliness, not just cleaning actions. More broadly, defining performance involves deep philosophical questions—should we prioritize consistency or peak performance? As a general rule, it is better to design performance measures **according**

to what one actually wants to be achieved in the environment, rather than **according to how one thinks the agent should behave**. In cases where agents serve multiple users, they may need to **learn and adapt** to individual preferences over time. In general, the **rationality** depends on:

- The performance measure defining success
- The agent's prior knowledge of the environment
- The available actions
- The percept sequence observed so far

A **rational agent** selects actions that lead to the most **desirable outcomes**, evaluated through a **performance measure**, given the evidence provided by the percept sequence and its built-in knowledge. Is the vacuum-cleaner agent rational? That depends! For example, once all the dirt is cleaned up, the agent will oscillate needlessly back and forth; if the performance measure includes a penalty for each movement, the agent will fare poorly. A better approach would be to remain idle when certain of cleanliness or periodically check for new dirt if recontamination is possible. If the environment is unknown, exploration becomes necessary.

We need to be careful to distinguish between rationality and **omniscience**. An omniscient agent knows the actual outcome of its actions and can act accordingly, but omniscience is impossible in reality. A rational agent does not know the actual outcome but selects the best **expected** action based on available information. For example, crossing an empty street to meet a friend can be rational, even if an unpredictable event, like a road pirate, can run us over. Rationality **maximizes expected performance**, while perfection **maximizes actual performance**, which is unrealistic.

Rational agents should also engage in **information gathering** to improve decision-making. For instance, before crossing a busy street, looking both ways is rational because it enhances the percept sequence. Agents that rely solely on pre-programmed knowledge without adapting are **not autonomous**. In contrast, an autonomous agent **learns from experience**, adapting its behavior based on new information. While complete autonomy is impractical from the start, agents should have **built-in knowledge** to act effectively from the beginning and then **learn over time** to improve their knowledge. A vacuum-cleaning agent that predicts dirt accumulation will outperform one that follows a fixed pattern. Ultimately, **learning** enables rational agents to succeed across diverse environments, making them more adaptable.

1.2 Architectures

The agent program runs on a physical computing system with sensors and actuators, called the **agent architecture**. The architecture processes sensory inputs, runs the program, and executes

the actions. The architecture must be compatible with the program; for example, if the program instructs an agent to “walk,” the architecture must include “legs”. Thus, the agent is the combination of **architecture** and **program**:

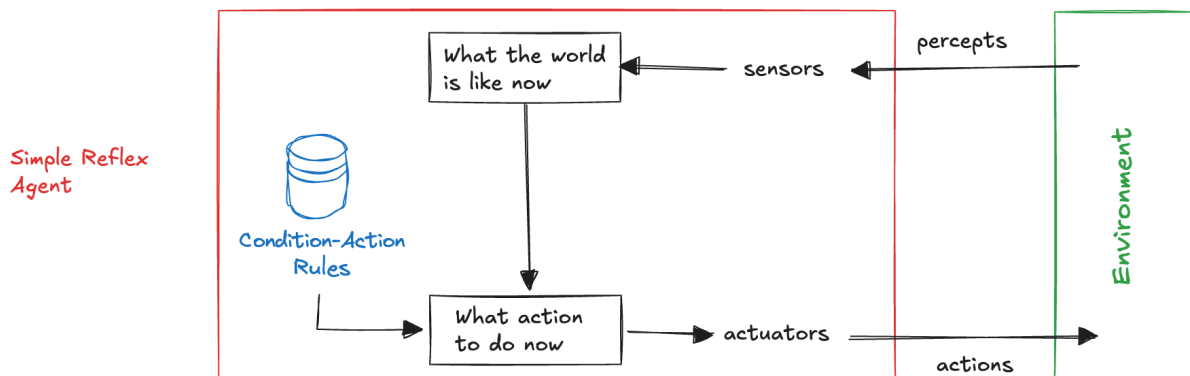
agent = architecture + program

But **how do we build an intelligent agent**? Where do we begin? There is no single solution, but various approaches exist for developing agents of increasing complexity, all within the unifying paradigm. Each kind of agent program combines particular components in particular ways to generate actions. We outline some basic kinds of agent programs that embody the principles underlying almost all intelligent systems:

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents

1.2.1 Simple Reflex Agent

The simplest kind of agent is the simple reflex agent. These agents select actions **on the basis of the current percept, ignoring the rest of the percept history**:



For example, the vacuum agent program suggested in the previous section (*If the current square is dirty, clean it; otherwise, move to the other square*) is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt. We can implement this agent using the reflex agent program:

```
def SimpleReflexVacuumAgent():
    """
    This agent takes action based solely on the percept.
    """
```



```
def program(percept):
    location, status = percept
    if status == 'dirty':
        return 'suck'
    elif location == "A":
        return 'right'
    elif location == "B":
        return 'left'

    return program
```

```
# Create the vacuum cleaner agent
agent = SimpleReflexVacuumAgent()

# Simulate inputs and outputs
print(agent(("A", "clean")))
print(agent(("A", "dirty")))
print(agent(("B", "clean")))
```

```
right
suck
left
```

This program is much smaller than the lookup table because it **ignores percept history**, reducing complexity. Some actions, like cleaning a dirty square, are independent of location, further simplifying decision-making. **Reflex behaviors** apply even in complex environments. For example, an autonomous vehicle detects when the car ahead brakes by recognizing its brake lights and responds by initiating braking. Humans also rely on similar mechanisms, whether through learned responses or innate reflexes, such as blinking when something nears the eye. The previous program is specific to one particular environment. A more general and flexible approach is first to build a general-purpose interpreter for condition–action rules and then to create rule sets for specific environments:

```
def SimpleReflexAgent(rules, interpret_input):
    """
    This agent takes action based solely on the percept.
    """
```

```
def program(percept):  
    state = interpret_input(percept)  
    rule = rule_match(state, rules)  
    action = rule.action  
    return action  
  
return program
```

The `interpret_input()` function abstracts the current state from sensory input, while `rule_match()` selects the first matching rule for that state. Reflex agents are **simple but limited in intelligence**, they will work only if the correct decision can be made on the basis of just the current percept. Partial observability can cause significant issues. For example, a braking rule based on a single video frame assumes the car ahead has a clearly identifiable brake light. However, older vehicles may have ambiguous taillight configurations, leading a reflex agent to either brake unnecessarily or fail to brake when needed. A similar problem arises in a vacuum agent with only a dirt sensor. Without location awareness, it may get stuck moving in the wrong direction (e.g. in a "clean" A square he goes "left") indefinitely, resulting in an **infinite loop**. A possible solution is **randomization**. For instance, flipping a coin to decide whether to move "left" or "right" when encountering a clean square. On average, this allows the agent to reach and clean the other square in two steps. Hence, a **randomized reflex agent** might outperform a deterministic reflex agent. Randomized behavior can be rational, in some environments.

As an example, the first versions of **Roomba**, the famous robotic vacuum cleaner, relied on a simple reflexive approach to navigation: it moved more or less randomly, using sensors to change direction whenever faced with an obstacle or the top of a set of steps. Another example is the **IFTTT (If This Then That)** is a real-world example of tool to implement simple reflex agent. It follows a straightforward condition-action rule: when a specific event (percept) occurs, it triggers a predefined response (action). For example, an IFTTT applet could be set up as: if the weather forecast predicts rain then send a notification to carry an umbrella. It is a powerful tool to automate tasks based on a numerous possible triggers and actions.

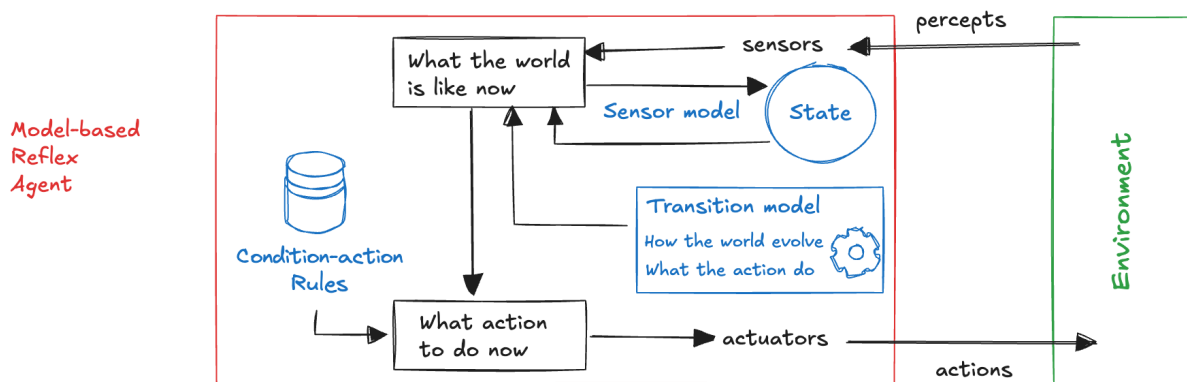
1.2.2 Model-based reflex agents

An effective way to manage partial observability is for an agent to maintain an **internal state** that captures unobserved aspects of the environment based on its **percept history**. For instance, in braking scenarios, the agent only needs to store the previous camera frame to detect simultaneous activation of red brake lights. More complex tasks, like lane changes detection, require

tracking other vehicles even when they are out of view. Updating this internal state requires two types of encoded knowledge:

1. **Transition model**: understanding how the world changes over time, including the effects of the actions (e.g., turning the steering wheel causes the car to turn) and independent changes (e.g., rain affecting camera visibility).
2. **Sensor model**: understanding how the state is reflected into sensor data (e.g., brake lights appearing as illuminated red regions in the camera feed).

By integrating these models, a **model-based reflex agent** can infer the current state of the world, to the extent possible given sensor limitations.



Notice that the agent updates its internal state by combining the current percept with previous state information, using a model of how the world operates. The representation of models and states varies depending on the environment and the agent design. However, in a partially observable environment, the exact state is rarely known with certainty. Despite this uncertainty, the agent must still make decisions based on the best available information.

The model-based approach for the vacuum cleaning problem can track the cleanliness of two locations and decides actions accordingly: it sucks when a location is dirty, moves between locations if needed, and stops when both are clean:

```

def ModelBasedReflexVacuumAgent():
    """An agent that keeps track of what locations are clean or dirty"""

    model = {"A": None, "B": None}

    def program(percept):
        location, status = percept

        # Update the model here

```

```
    model[location] = status

    if model["A"] == model["B"] == 'clean':
        return 'noop'
    elif status == 'dirty':
        return 'suck'
    elif location == "A":
        return 'right'
    elif location == "B":
        return 'left'

    return program
```

```
# Create the vacuum cleaner agent
agent = ModelBasedReflexVacuumAgent()

# Simulate inputs and outputs
print(agent(("A", "clean")))
print(agent(("B", "dirty")))
print(agent(("B", "clean")))
```

right
suck
noop

A generic implementation updates the state using a given model, matches the updated state to predefined rules, and selects an action accordingly:

```
def ModelBasedReflexAgentProgram(rules, update_state, model):
    """
    This agent takes action based on the percept and state
    """

    def program(percept):
        program.state = update_state(program.state, program.action, percept, model)
        rule = rule_match(program.state, rules)
        action = rule.action
        return action

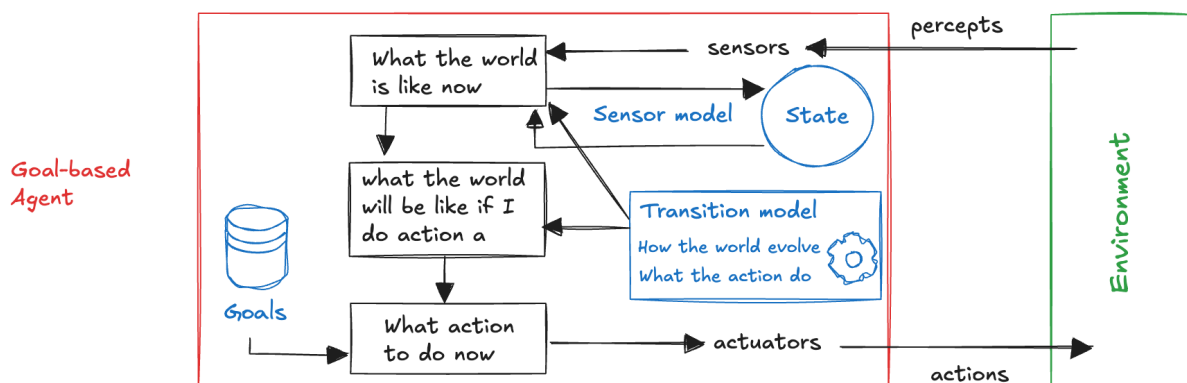
    program.state = program.action = None
    return program
```

A real-world example of model-based agents in robotic vacuum cleaners includes the **Roomba 980** and **Dyson 360 Eye**. Unlike earlier models that relied on random movement, these robots use an algorithm to build a map of their environment while navigating. The Roomba 980 creates a visual map to track its position, ensuring complete coverage and avoiding redundant cleaning, while the Dyson uses a 360-degree camera to continuously scan its surroundings, constructing a detailed map to navigate efficiently and avoid obstacles. By leveraging the map, these vacuums intelligently adapt their cleaning patterns for optimal performance.

1.2.3 Goal-based agents

Knowing something about the current state of the environment is not always enough to decide what to do. For example, at a road junction, the vehicle can turn left, turn right, or go straight on. The correct decision depends on where the vehicle is trying to get to. In other words, as well as a current state description, the agent needs some sort of **goal information** that describes situations that are desirable (for example, being at a particular destination).

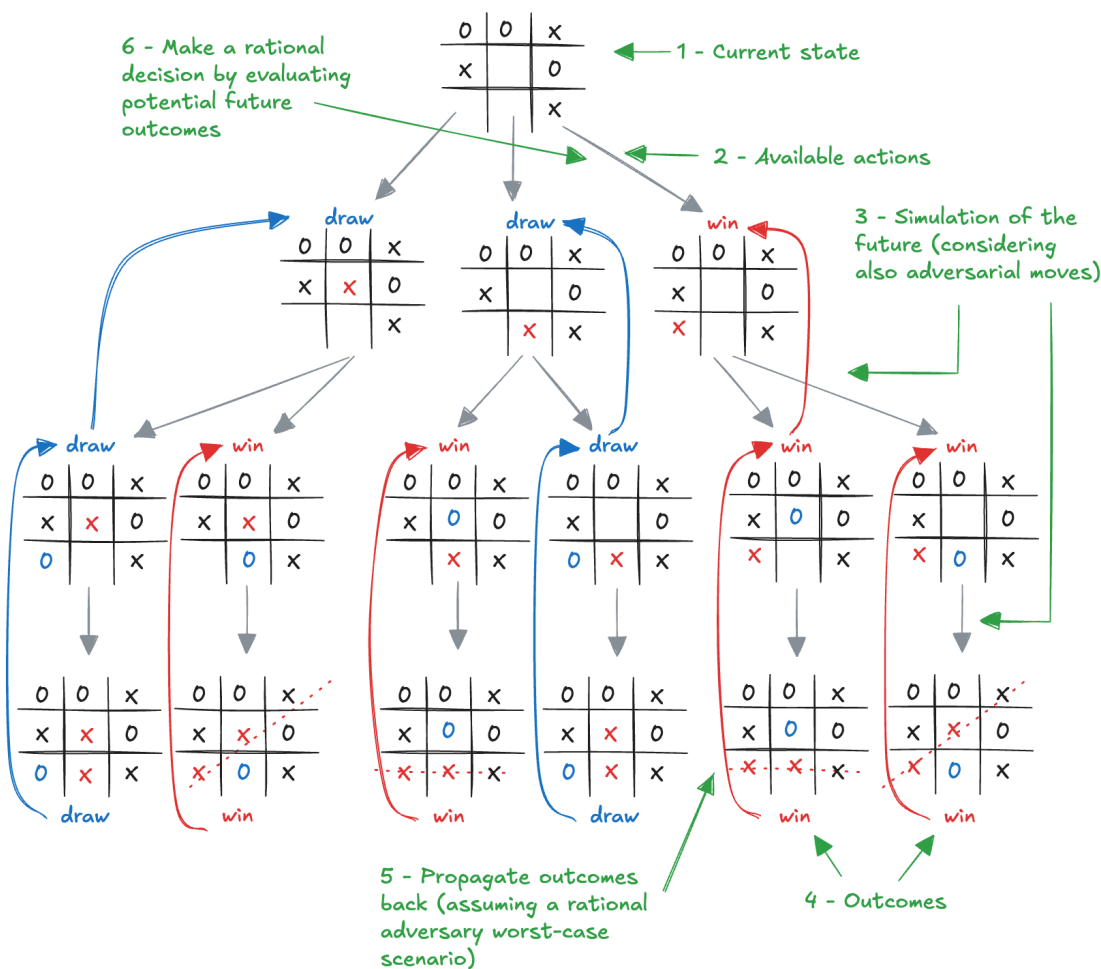
The agent program can combine this with the model to choose actions that achieve the goal.



Goal-based action selection can be simple when a single action achieves the goal, but more complex when the agent must plan a sequence of actions. For example, when the agent has to consider long sequences of twists and turns in order to find a way to achieve the goal. **Search** and **planning** in AI focus on finding such action sequences. Unlike reflex agents, that react to percepts with fixed rules, goal-based agents **consider future outcomes** to determine the best action. For example, a reflex agent brakes upon seeing brake lights without understanding why, while a goal-based agent does so to avoid a collision. This approach is more **flexible** since **goals can be modified** without rewriting decision rules, allowing the agent to **adapt** to new tasks. For example, a goal-based agent behavior can easily be changed to go to a different destination simply by specifying that destination as the goal. The reflex agent's rules for when to turn and when to go straight will work only for a single destination; they must all be replaced to go

somewhere new.

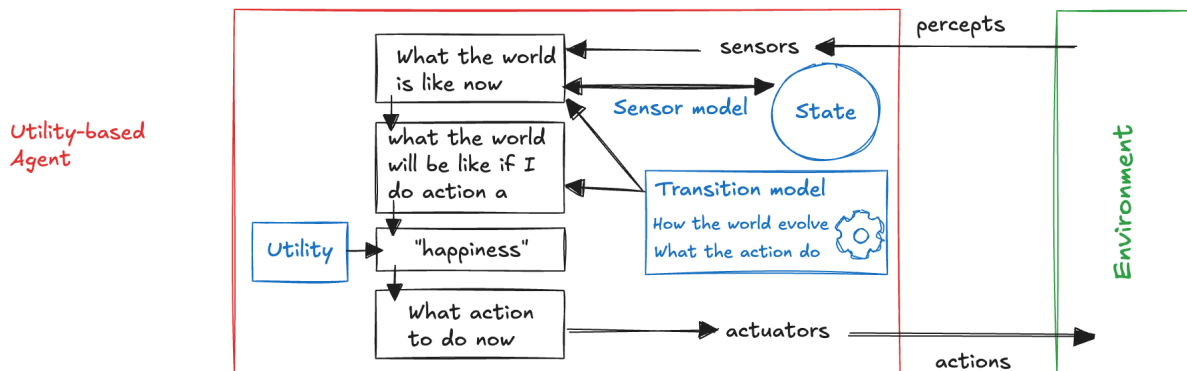
As an example, the game of **Tic-tac-toe**, like all deterministic board games, can be solved using a goal-based agent that evaluates possible move sequences to achieve victory. By employing search algorithms like **minimax**, the agent systematically explores future game states to determine the optimal move:



1.2.4 Utility-based agents

Goals alone are insufficient for generating high-quality behavior in most environments. While many action sequences can achieve a goal, some are faster, safer, or more efficient. Goals offer only a binary distinction between success and failure, whereas a **performance measure** allows comparisons based on **how desirable** different outcomes are. Economists and computer scientists use the term **utility** to quantify this preference. We have already seen that a performance measure assigns a score to any given sequence of states, so it can be used to distinguish between

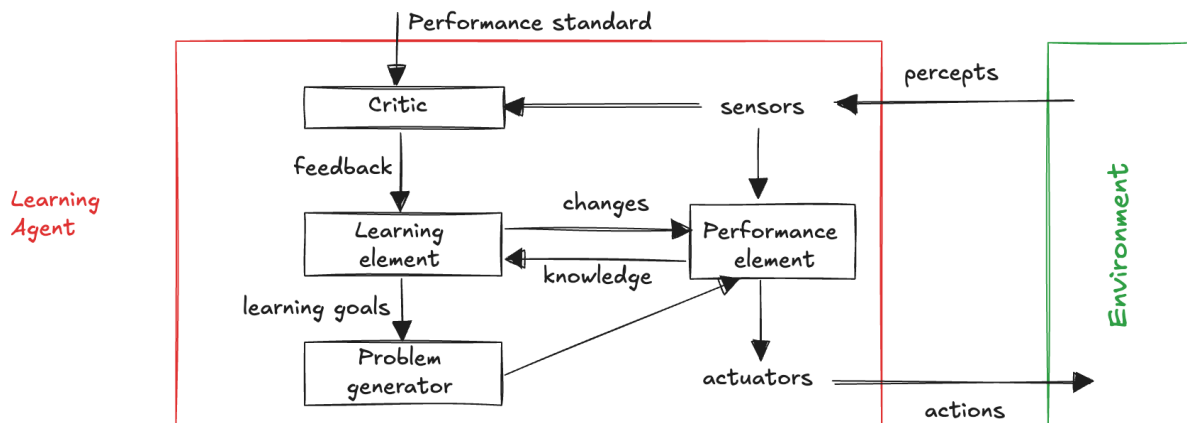
more and less desirable ways of getting the goal. An **utility function** is an **internalization** of the performance measure. Provided that the internal utility function and the external performance measure are in agreement, an agent that chooses actions to maximize its utility will be rational according to the external performance measure.



Partial observability and non-determinism are inherent in the real world, making **decision-making under uncertainty** essential. A rational **utility-based agent** selects actions that **maximize expected utility** of the action outcomes, that is, the utility the agent expects to derive, **on average**, given the probabilities and utilities of each outcome. This is challenging, requiring research in **perception, representation, reasoning, and learning**. The agent must track its environment, and **selecting optimal actions** demands **efficient algorithms**. **Perfect rationality** is often **impractical** due to computational limits. While many utility-based agents are model-based, **model-free agents** can learn optimal actions without understanding their environmental impact.

1.2.5 Learning agents

We have explored various agent architectures but not **how they are created**. Turing (1950) suggested programming intelligent machines **by hand**, but recognized its **impracticality**, advocating instead for **learning machines** that improve through experience. Today, this approach dominates AI, enabling any agent type (model-based, goal-based, or utility-based) to learn and adapt. Learning allows the agent to operate in initially unknown environments and to become more competent than its initial knowledge alone might allow. A learning agent consists of four key components:



The **learning element** is responsible for **improving** the agent behavior, while the **performance element** (what we have previously considered to be the entire agent) selects actions based on percepts. Learning means modifying internal components (how the world works, the effects of their actions, their goals, etc.). It is possible to **learn directly from observing the environment**, like an automated vehicle learning the effects of braking on different road surfaces. The **problem generator** can even **experiment**. This means try actions different from the actions that the performance element selects in order to lead to new and informative experiences and uncover better long-term strategies. The learning element uses feedback (**reward or penalty signal**) from a **critic**, respect to a **fixed performance standard**, on how the agent is doing and determines how the performance element **should be modified** to do better in the future. The critic is essential because percepts alone do not indicate success; for example, a chess program may detect a checkmate but needs a performance standard to recognize it as a positive outcome. Whether hard-wired (like pain in humans) or learned from human behavior (like preferences provided by the user), this feedback drives the learning process. Ultimately, learning in intelligent agents is about refining each component to better align with available feedback, thus improving overall performance.

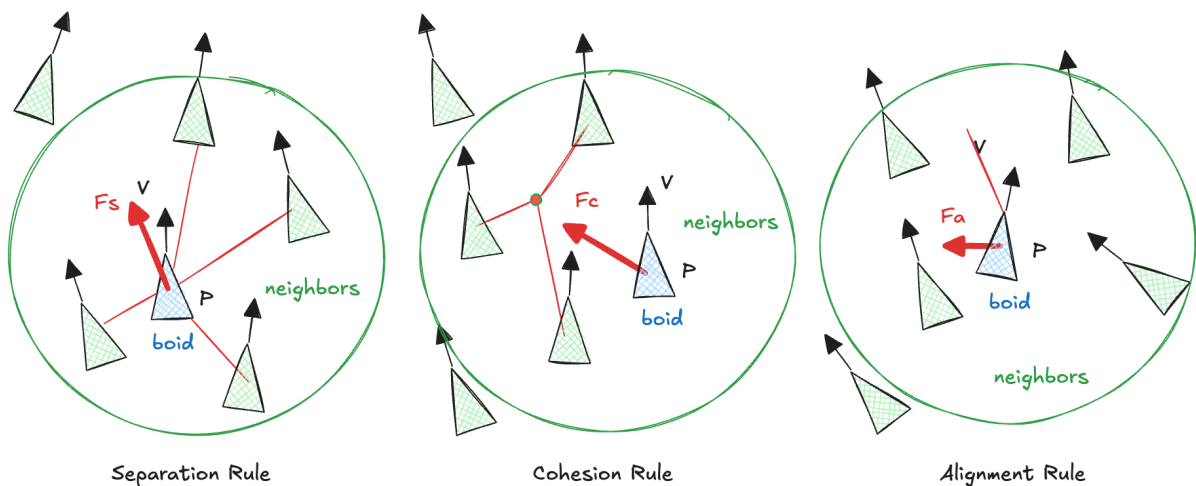
1.3 An example: Flocking

The paper "Flocks, Herds, and Schools: A Distributed Behavioral Model" by C. W. Reynolds, published in SIGGRAPH 1987, presents a model simulating the collective behavior of animals like birds, fish, and ants. Reynolds introduces a model of such collective behavior (referred to as **flocking**), where each entity (a **boi**d) follows simple local rules based on its environment, resulting in complex group motion. The model consists of three main rules:

- **Separation rule** avoids crowding neighbors to prevent agents from colliding

- **Cohesion rule** steers the agent towards the average position of neighbors, to keep agents together
- **Alignment rule** steers the agent towards the average heading of neighbors, to ensure agents move in the same direction

Each rule provides a **simple heuristic** for the agent to follow, based on the positions and velocities of nearby agents. The agent has a **limited perception range**, only considering neighbors within a certain radius. The rules determine how each entity moves at every instant by generating **competing acceleration requests** (forces):



These forces are combined to compute the final force applied to an agent at each moment and to update its position and velocity exploiting a simple **Euler integration rule**. Given a boid with position $p(t)$, velocity $v(t)$, and acceleration $a(t)$, the Euler update is given by:

$$v(t + \Delta t) = v(t) + a(t)\Delta t$$

$$p(t + \Delta t) = p(t) + v(t)\Delta t$$

The acceleration $a(t)$ is determined by the sum of various forces acting on the "boid":

$$a(t) = \frac{F(t)}{m}$$

where $F(t)$ is the total force acting on the boid as a weighted sum of different flocking forces (alignment, cohesion, separation) and eventually other external force:

$$F(t) = w_{\text{align}} F_{\text{align}} + w_{\text{cohesion}} F_{\text{cohesion}} + w_{\text{separation}} F_{\text{separation}} + F_{\text{external}}$$

This iterative update is applied at each simulation step, making the boids move dynamically according to the flocking rules. Different rules can be weighted to achieve different behaviors. The model is **decentralized**, meaning that each agent makes decisions based only on its local environment, without any global knowledge.

We consider a simplified 2D implementation using the three main rules and a model-based agent architecture to implement the single boid. The agent maintains an internal state with its position and velocity, and updates them based on the flocking rules:

```
import numpy as np

def separation(boid, neighbors):
    # Initialize the force vector
    force = np.zeros(2)

    # If there are no neighbors, return the zero vector
    if not neighbors:
        return force

    # Loop over all neighbors
    for neighbor in neighbors:

        # Calculate the distance between the agent and its neighbor
        distance = np.linalg.norm(boid['position'] - neighbor['position'])

        # Calculate the direction to steer away from the neighbor (normalized)
        direction_to_neighbor = neighbor['position'] - boid['position']
        direction_to_neighbor = direction_to_neighbor /
        ↪ (np.linalg.norm(direction_to_neighbor) + 0.01)

        # Calculate a weight based on distance.
        # Closer neighbors get a higher weight (stronger influence).
        weight = 1 / (distance + 0.01)

        # Add the contribution of this neighbor to the total force
        force += direction_to_neighbor * weight

    # Normalize the force vector (it is optional, but makes
    # things easier to tune)
    force = force / np.linalg.norm(force)

    return force

def cohesion(boid, neighbors):
    # Initialize the force vector
    force = np.zeros(2)
```

```
# If there are no neighbors, return the zero vector
if not neighbors:
    return force

# Initialize the center of mass of neighbors
center_of_mass = np.zeros(2)

# Loop over all neighbors
for neighbor in neighbors:
    # update the center of mass
    center_of_mass += neighbor['position']
center_of_mass /= len(neighbors)

# Calculate the force contribution as the distance to the center of mass
force = center_of_mass - boid['position']

# Normalize the force vector (it is optional, but makes
# things easier to tune)
force = force / np.linalg.norm(force)

return force
```

```
def alignment(boid, neighbors):

    # Initialize the force vector
    force = np.zeros(2)

    # If there are no neighbors, return the zero vector
    if not neighbors:
        return force

    # Initialize the average velocity of neighbors
    avg_velocity = np.zeros(2)

    # Loop over all neighbors
    for neighbor in neighbors:
        # Update the average velocity
        avg_velocity += neighbor['velocity']
    avg_velocity /= len(neighbors)

    # Calculate the force contribution as the difference between
    # the agent's velocity and the average velocity of its neighbors
```

```
force = avg_velocity - boid['velocity']

# Normalize the force vector (it is optional, but makes
# things easier to tune)
force = force / np.linalg.norm(force)

return force
```

In order to generate some overall flocking behavior, we can add a force contribution that steers the boid towards a target position. This can be made more complex by adding obstacles, predators, or other agents with different behaviors.

```
def direction(boid, target):

    # Initialize the force vector
    force = np.zeros(2)

    # Calculate the force contribution as the difference between
    # the agent's position and the target position
    force = target - boid['position']

    # Normalize the force vector (it is optional, but makes
    # things easier to tune)
    force = force / np.linalg.norm(force)

    return force
```

Now we write the Euler integration function to update the state of a boid based on the percept, in that case the positions and velocities of the other boids:

```
def update_state(boid, percept, model):
    neighbors = percept['neighbors']

    # Calculate the forces (weights are applied here)
    separation_force = model['separation_force_factor'] * separation(boid, neighbors)
    alignment_force = model['alignment_force_factor'] * alignment(boid, neighbors)
    cohesion_force = model['cohesion_force_factor'] * cohesion(boid, neighbors)
    target_force = model['target_force_factor'] * direction(boid, model['target'])

    # Random Force. This is an external force to add some randomness
    # to the movement, due to the fact that the boids are not perfect
```

```

    random_force = model['random_force_factor'] * (2 * np.random.rand(2) - 1);

    # Combine the forces
    force = separation_force + cohesion_force + alignment_force + random_force +
    ↪ target_force

    # Update the velocity (considering mass = 1 and delta_t = 1 for simplicity)
    boid['velocity'] += force

    # Speed Limiting (optional but often helpful)
    speed = np.linalg.norm(boid['velocity'])
    if speed > model['max_speed']:
        boid['velocity'] = (boid['velocity']/speed) * model['max_speed']

    # Update the position
    boid['position'] += boid['velocity']

    return boid

```

Now we can implement the model-based agent. In this simple case we just update the state without selecting any action based on the state. The behaviour can be made more complex by adding a goal or utility function to the agent.

```

def ModelBasedAgent(update_state, model):

    def program(percept):
        program.state = update_state(program.state, percept, model)

    program.state = {'position': 10 * np.random.rand(2), 'velocity':
    ↪ (np.random.rand(2) - 0.5)}
    return program

```

The perception of a boid is limited to a certain radius, so we need to filter the boids that are within this radius. This is the sensor function of our agent:

```

def sensors(boid, boids, radius):
    neighbors = []
    for other_boid in boids:
        if other_boid is not boid: # Avoid self-comparison
            distance = np.linalg.norm(other_boid['position'] - boid['position'])
            if distance < radius:

```

```
        neighbors.append(other_boid)
    return neighbors
```

We need a function to simulate the environment, including time step updates and position visualization. Using Matplotlib and its animation module, we can generate a dynamic scatter plot to display boid positions at each time step:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.animation as animation
mpl.rc('animation', html='jshtml')

def simulation(boids, model, num_frames):
    # Set up the figure and axis
    fig = plt.figure()
    ax = plt.axes(xlim=(-10, 100), ylim=(-10, 100))

    # Initialize positions for visualization
    positions = [boid.state['position'] for boid in boids]

    # Create a scatter plot
    scat = ax.scatter([pos[0] for pos in positions], [pos[1] for pos in positions])

    # Update the scatter plot each frame
    def update(frame):
        # Loop over all boids
        for boid in boids:

            # Get the neighbors
            # neighbors = sensors(boid, boids, model)
            neighbors = sensors(boid.state, [b.state for b in boids],
                               model['perception_radius'])

            # Prepare the percept
            percept = {'neighbors': neighbors}

            # Update the state of the boid
            boid(percept)

        # Update positions for visualization
        positions = [boid.state['position'] for boid in boids]
```

```
# Update scatter plot data
scat.set_offsets(positions)

return scat,

# Create animation
anim = animation.FuncAnimation(fig, update, frames=num_frames, interval=50,
↪ blit=True)

plt.close()

return anim
```

Now we can experiment with different model parameters and weights to observe the emergent flocking behavior. For example, we can adjust the weights of the alignment, cohesion, and separation rules to observe how they affect the flocking dynamics. We can also introduce additional forces or rules to create more complex behaviors. The model-based agent architecture allows for easy modification and extension of the agent's behavior, making it a flexible framework for simulating intelligent agents.

```
# Define the model parameters
model = {'separation_force_factor': 1.2,
        'alignment_force_factor': 1,
        'cohesion_force_factor': 0.2,
        'random_force_factor': 1,
        'target_force_factor': 1,
        'perception_radius': 10,
        'max_speed': 2.0,
        'target': np.array([80, 80])}

# Define the number of agents
num_agents = 30

# Define the number of frames
num_frames = 200

# Create a first set of boids
boids = [ModelBasedAgent(update_state, model) for _ in range(num_agents)]

# Run the simulation
anim = simulation(boids, model, num_frames)
```

```
anim
```

```
<matplotlib.animation.FuncAnimation at 0x11c266660>
```

The model has been widely used in computer graphics, animation, and robotics to simulate realistic group motion.

2 The Reinforcement Learning Problem

Humans naturally **pursue feelings of happiness**. From picking out our meals to advancing our careers, every action we choose is derived from **our drive to experience rewarding moments in life**. Whether these moments are self-centered pleasures or the more generous of goals, whether they bring us immediate gratification or long-term success, they're still **our perception of how important and valuable they are**. Our ability to achieve these precious moments seems to be correlated with **intelligence**, defined as the ability to acquire and apply knowledge and skills.

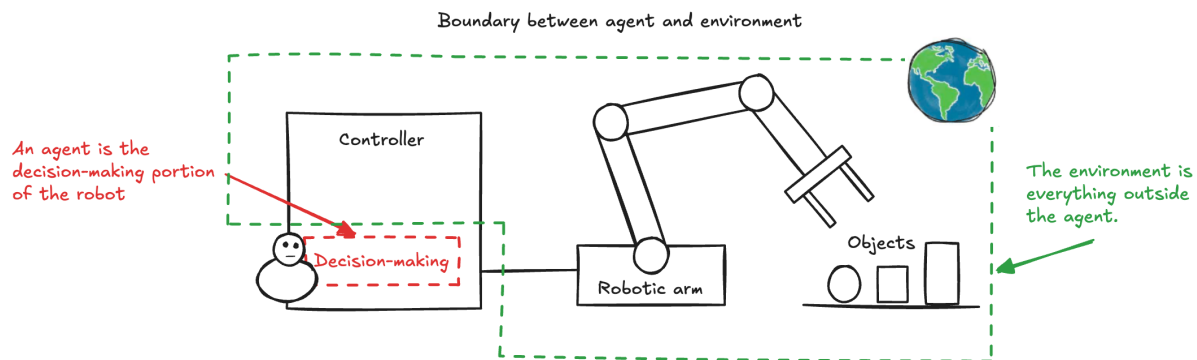
Reinforcement learning (RL) is the task of **learning through trial and error**. In this type of task, no human labels data, and no human collects or explicitly designs the collection of data. The goal in RL is **to act**.

2.1 Reinforcement Learning Cycle

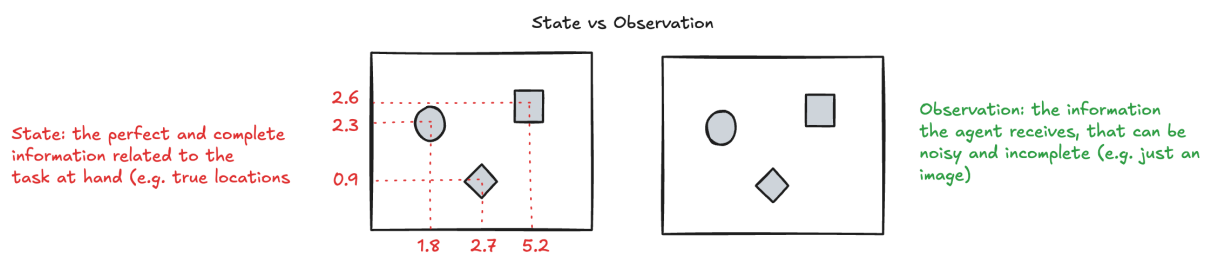
Reinforcement Learning is a **computational approach to learning from interaction**. In particular, it is learning what to do (how to map situations to actions) in order to maximize a numerical reward signal. The **agent** is not told which actions to take, but instead must discover which actions yield the most reward by trying them on its **environment** (everything outside the agent, everything the agent has no total control over). In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics (**trial-and-error search** and **delayed reward**) are the two most important distinguishing features of reinforcement learning.

Imagine you're training a robot to pick up objects. The objects to be picked up, the tray where the objects lay, the wind, and everything outside the decision maker are part of the environment. That means the robot arm is also part of the environment, because it isn't part of the agent. And even though the agent can decide to move the arm, the actual arm movement is noisy, and thus the arm is part of the environment. This strict **boundary between the agent and**

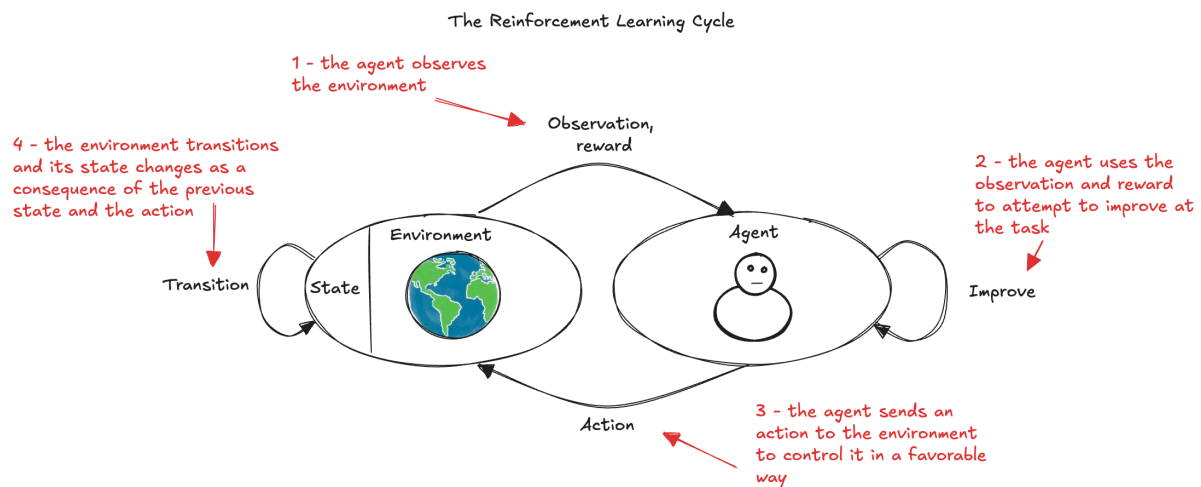
the environment is counterintuitive at first, but the decision maker, the agent, can only have a single role: making decisions. Everything that comes after the decision gets bundled into the environment.



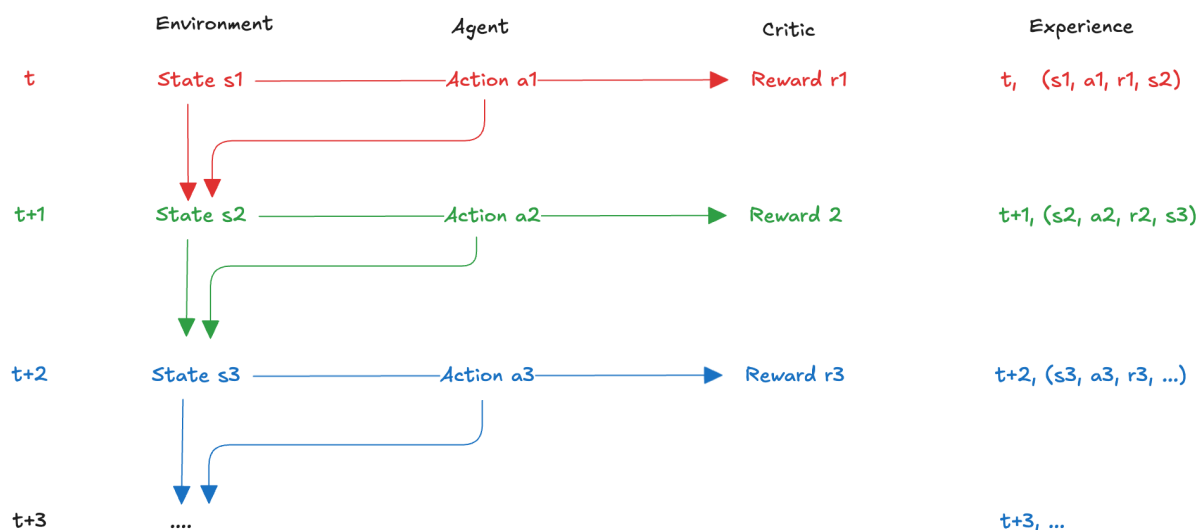
The environment is represented by **a set of variables related to the problem**. For instance, in the robotic arm example, the location and velocities of the arm would be part of the variables that make up the environment. This set of variables and all the possible values that they can take are referred to as the **state space**. A state is an instantiation of the state space, a set of values the variables take. Interestingly, often, agents don't have access to the actual full state of the environment. The part of a state that the agent can observe is called an **observation**. Observations depend on states but are what the agent can see. For instance, in the robotic arm example, the agent may only have access to camera images. While an exact location of each object exists, the agent doesn't have access to this specific state. Instead, the observations the agent perceives are derived from the states. You'll often see in the literature observations and states being used interchangeably.



At each state, the environment makes available a **set of actions** the agent can choose from. The agent influences the environment through these actions. The environment may change states as a response to the agent action. The function responsible for this mapping is called the **transition function**. The environment also provides a **reward signal** as a response and the function responsible for this mapping is called the **reward function**. The set of transition and reward functions is referred to as the **model** of the environment.

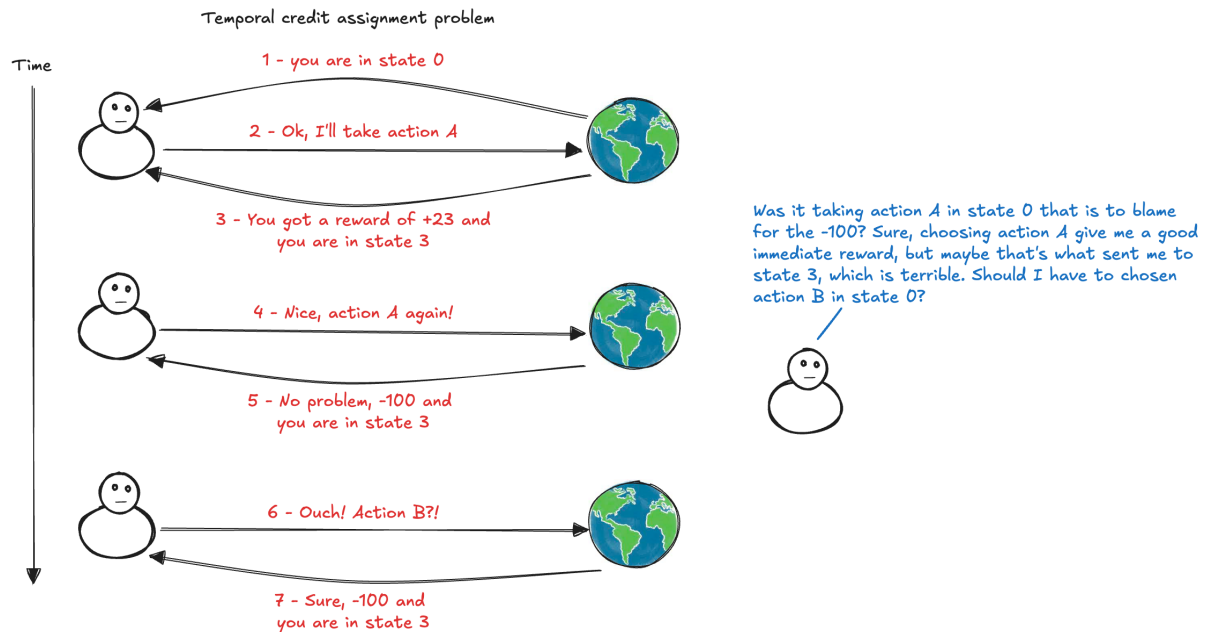


The agent has a three-step process: the agent interacts with the environment, evaluates its behavior using the reward signal obtained as a feedback from the environment, and finally it improves its responses. The interactions between the agent and the environment go on for several cycles. Each cycle is called a **time step**. At each time step, the agent observes the environment, takes action, and receives a new observation and reward. The set of the state, the action, the reward, and the new state is called an **experience**. The task may or may not have a natural ending. Tasks that have a natural ending, such as a game, are called **episodic tasks**. Conversely, tasks that don't are called **continuing tasks**, such as learning forward motion. The sequence of time steps from the beginning to the end of an episodic task is called an **episode**. Agents may take several time steps and episodes to learn to solve a task.

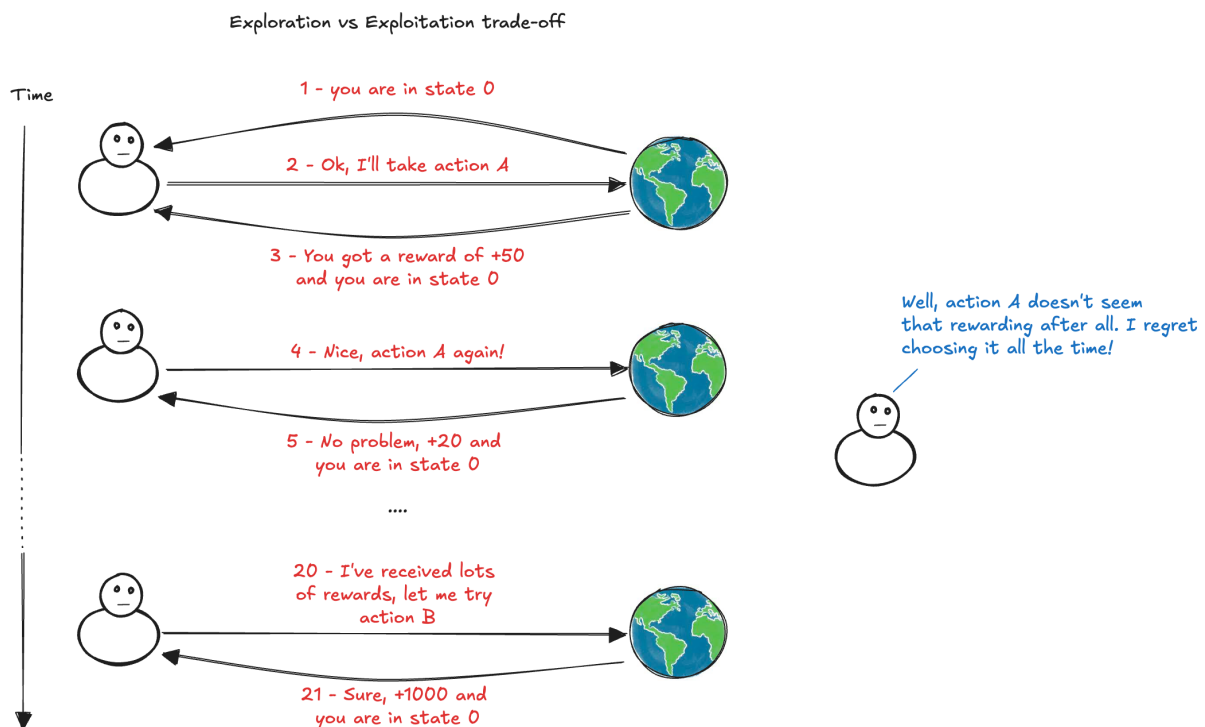


The action taken by the agent may have **delayed consequences**. The reward may be sparse and only manifest after several time steps. Thus the agent must be able to learn from sequential

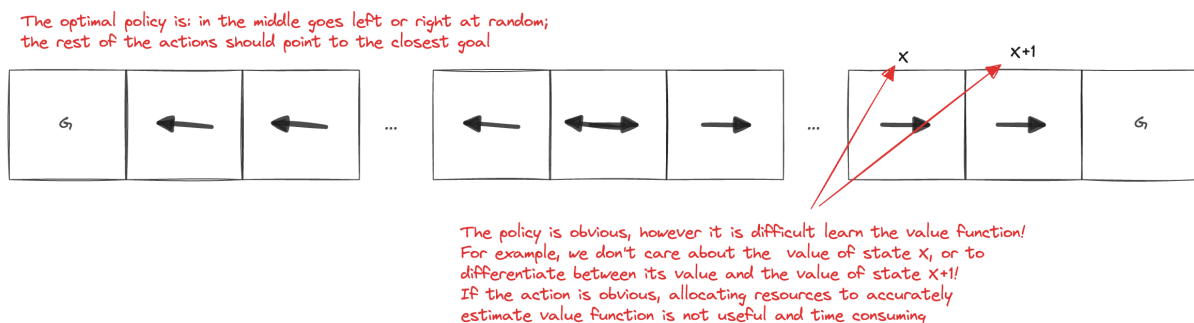
feedback, that rise to a problem referred to as the **temporal credit assignment problem**: how to determine which state/action is responsible for a reward?



Moreover, the reward received by the agent is **weak**, in the sense that it don't provide supervision. The reward **indicate goodness but not correctness**, meaning it contains no information about other potential rewards. Thus the agent must be able to learn from **evaluative feedback**, that rise the need for **exploration**. The agent must be able to balance the gathering of information with the **exploitation** of current information, which is referred to as the **exploration versus exploitation trade-off**.



Finally, the reward received by the agent is just a **sample**, and the agent doesn't have access to the reward function. Also, the state and action spaces are commonly large, even infinite, so trying to learn from **sparse and weak feedback** becomes a harder challenge with samples. Therefore, the agent must be able to learn from sampled feedback, and it must be able to generalize.



2.2 Mathematical foundation

The **Markov Decision Process** (MDP) is a framing of the problem of learning from interaction to achieve a goal. The agent and environment interact at each discrete time step of a sequence

$$t = 0, 1, 2, 3, \dots$$

At each time step t , the agent receives some representation of environment state S_t from a state

space S , and on that basis selects an action, A_t from a set of actions A . One time step later, in part as a consequence of its action, the agent receives a numerical reward R_{t+1} and finds itself in a new state S_{t+1} . The environment and the agent together give rise to a **trajectory** like this:

$$S_0 \xrightarrow{A_0} R_1, S_1 \xrightarrow{A_1} R_2, S_2 \xrightarrow{A_2} R_3, S_3 \xrightarrow{A_3} \dots R_i, S_i \xrightarrow{A_i} \dots$$

There is a subset of S called the set of **starting states**, denoted S_i . To begin interacting with an MDP, we draw a state from S_i from a probability distribution. A **terminal state** is a special state: it has all available actions transitioning to itself with probability 1, and these transitions provide no reward.

In the case of MDPs, the states are fully observable: we can see the internal state of the environment at each time step, that is, the observations and the states are the same. **Partially observable Markov decision processes** (POMDP) is a more general framework for modeling environments in which observations, which still depend on the internal state of the environment, are the only things the agent can see instead of the state.

We assume that the probability of the next state, given the current state and action, is **independent of the history of interactions**. This memory-less property is known as the **Markov property**. The probability of moving from one state to another state on two separate occasions, given the same action, is the same regardless of all previous states or actions encountered before that point:

$$P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1}|S_t, A_t)$$

Most RL agents are designed to take advantage of the Markov assumption, we must make sure we **feed our agent the necessary variables to make it hold as tightly as possible** (completely keeping the Markov assumption is impractical, perhaps impossible). The more variables we add, the longer it takes to train an agent, but the fewer variables, the higher the chance the information fed to the agent is not sufficient, and the harder it is to learn anything useful.

Notice that what we're trying to represent here is the fact that the environment "reacts" to the agent actions in some way, perhaps even by ignoring the agent actions. But at the end of the day, there's an internal process that's uncertain. To represent the ability to interact with an environment in an MDP, we need states, observations, actions, a transition, and a reward function.

2.2.1 The Transition function

The way the environment changes as a response to actions is referred to as the **transition function**, it maps a transition tuple:

$$(s, a, s') \rightarrow p$$

to a probability: given a state, an action, and a next state, it returns the corresponding probability of transition from that state to the next state when taking that action.

$$p(s'|s,a) = P(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

Notice that the transition function describes a probability distribution determining how the system will evolve in an interaction cycle from selecting actions in states. When integrating over the next states, as any probability distribution, the sum of these probabilities must equal one.

$$\sum_{s' \in S} p(s'|s,a) = 1 \quad \text{for all } s \in S, a \in A$$

One key assumption of many RL algorithms is that this distribution is **stationary**. That is, while there may be highly stochastic transitions, the probability distribution may not change during training or evaluation. Just as with the Markov assumption, the **stationary assumption** is often relaxed to an extent. However, it's important for most agents to interact with environments that at least appear to be stationary.

2.2.2 The Reward function

The **reward function** maps a transition tuple

$$(s, a, s') \rightarrow R$$

to a numeric signal of the goodness of a transitions. The use of a reward signal to formalize the idea of a goal is one of the most distinctive features of reinforcement learning. If we want the agent to do something for us, we must provide rewards to it in such a way that in maximizing them the agent will also achieve our goals. **It is thus critical that the rewards we set up truly indicate what we want accomplished***. Although formulating goals in terms of reward signals might at first appear limiting, in practice it has proved to be flexible and widely applicable. In particular, the reward signal is not the place to impart to the agent prior knowledge about how to achieve what we want it to do. The reward signal is our way of communicating to the agent what we want it to achieve, not how we want it achieved. When the signal is positive, we can think of the reward as an **income or a reward**. Most problems have at least one positive signal (for example winning a chess match or reaching the desired destination). But, rewards can also be negative, and we can see these as **cost, punishment, or penalty**.

Formally, we can define the reward function as the expected value of reward given the state-action-state tuple:

$$r(s,a,s') = E[R_t | S_{t-1} = s, A_{t-1} = a, S_t = s']$$

We can compute the marginalization over next states using the transition function to obtain the reward just for the current state and action:

$$r(s, a) = \sum_{s' \in S} p(s'|s, a) r(s, a, s')$$

2.2.3 The Return

The agent's goal is to **maximize the total amount of reward** it receives. This means maximizing not only the immediate reward, but the **cumulative reward** in the long run. The sum of all rewards obtained during the course of an episode is referred to as **the return** G_t :

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T = \sum_{k=0}^T R_{t+k+1}$$

where T is a "final" time step. In applications in which there is a natural notion of final time step (called **episodic tasks**) and the agent–environment interaction breaks naturally into sub-sequences (called **episodes**, e.g., plays a game, trips through a maze) T has a finite value and each episode ends in a special state called the **terminal state**, followed by a restart. In many cases (called **continuing tasks**) the agent–environment interaction does not break naturally into episodes, but goes on continually without limit (e.g., an on-going process-control) and T is infinite.

Because of the possibility of infinite sequences of time steps (problem known as **unboundedness** in mathematical terms), the Return value can be infinite. To avoid this, we need a way to **discount the value of rewards over time**, in order to help the infinite sum to converge to a finite value. Intuitively, we want to tell the agent that getting $+1$ is better sooner than later. We commonly use a positive real value less than one to exponentially discount the value of future rewards. The further into the future we receive the reward, the less valuable it is in the present. This number is called the **discount factor** γ :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T = \sum_{k=0}^T \gamma^k R_{t+k+1}$$

Another important reason why the discount factor is commonly used is to reduce the variance of return estimates. Given that the future is uncertain, and that the further we look into the future, the more stochastic we accumulate and the more variance our value estimates will have, the discount factor helps reduce the degree to which future rewards affect our value function estimates, which stabilizes learning for most agents.

Finally, take a look at this interesting recursive definition, which shows that the returns at successive time steps are related to each other:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) =$$

$$= R_{t+1} + \gamma G_{t+1}$$

The MDP framework is **abstract and flexible** and can be applied to many different problems in many different ways. The actions can be low-level controls (e.g., the voltages applied to the motors of a robot arm) or high-level decisions (e.g., whether or not to have lunch or to go to graduate school). Similarly, the states can take a wide variety of forms. They can be completely determined by low-level sensations (e.g., direct sensor readings) or they can be more high-level and abstract (e.g., symbolic descriptions of objects in a room). In general, actions can be any decisions we want to learn how to make, and the states can be anything we can know that might be useful in making them.

There are many extensions to the MDP framework, they allow to target slightly different types of RL problems, for example:

- **Partially observable Markov decision process** (POMDP), when the agent cannot fully observe the environment state;
- **Continuous Markov decision process**, when either time, action, state or any combination of them are continuous;
- **Multi-agent Markov decision process** (MMDP) allows the inclusion of multiple agents in the same environment.

2.3 Plan and Policy

At first, it seems the agent's goal is to find a sequence of actions that will maximize the return (sum of rewards) something called a **plan**, a sequence of actions from the initial state to the target state. This isn't enough! Plan don't account for stochastic in environments and actions taken won't always work the way we intend. What happen if the agent lands on a state not covered by the plan?

Instead, we need a **policy**: a universal plan, that covers all possible states. It can be **deterministic** (returns a single actions for a given state) or **stochastic** (returns an action-probability distribution):

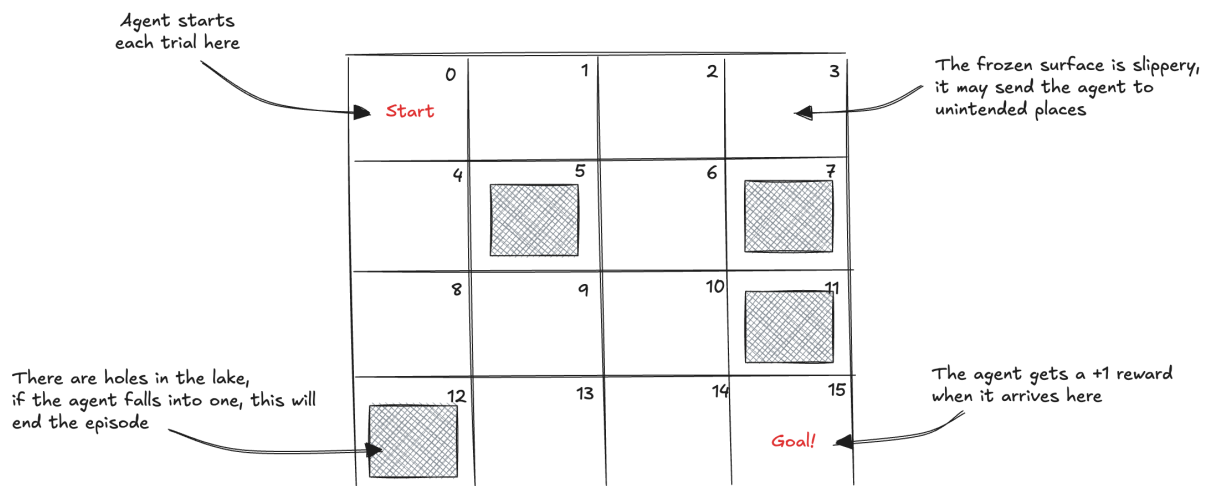
$$\pi(a|s) = P(A_t = a|S_t = s)$$

And we need methods to find good policies!

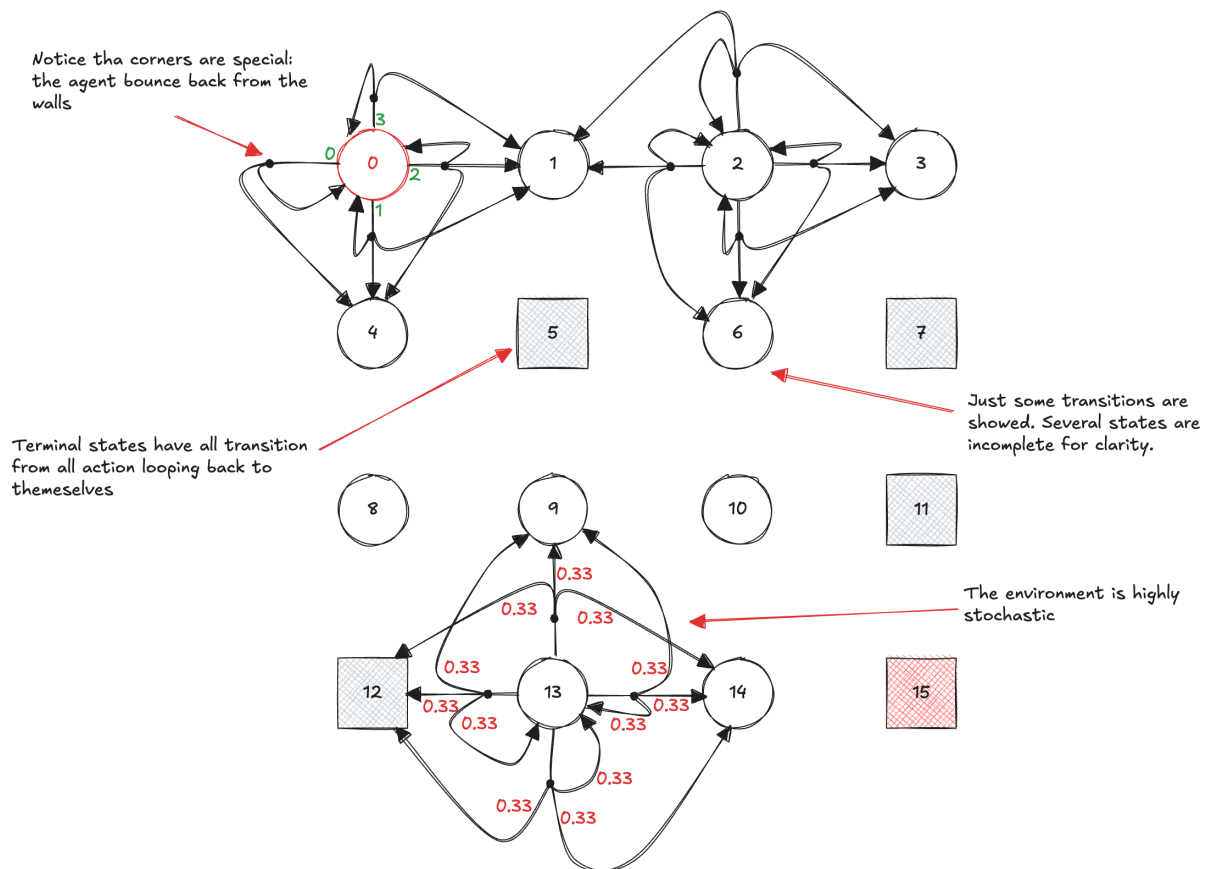
2.4 Example: the frozen lake environment

Frozen Lake (FL) is a simple **grid-world environment**, a common type of environment for studying RL algorithms that are grids of any size. It has discrete state and action spaces (four

actions are available: move left, down, right, or up). The task is to go from a start location to a goal location, while avoiding falling into holes. The challenge is that the surface is slippery (it's a frozen lake after all): the agent moves only a third of the time as intended, the other two-thirds are split evenly in orthogonal directions. For example, if the agent chooses to move down, there's a 33.3% chance it moves down, 33.3% chance it moves left, and 33.3% chance it moves right. There's a fence around the lake, so if the agent tries to move out of the grid world, it will bounce back to the cell from which it tried to move. There are four holes in the lake. If the agent falls into one of these holes, it's game over. In that case, the state is composed of a single variable containing the id of the cell where the agent is at any given time (a discrete variable) and the size of the state space is 16 (the ids are set from 0 to 15, left to right, top to bottom). There's only one starting state (which is state 0) and five terminal states:



The environment is stochastic and the probability of the next state given the current state and action is less than 1. There are more than one possible next state:



The reward function is +1 for landing in state 15, 0 otherwise. We can use a Python dictionary to represent the complete environment:

```
P = {
    # The outer dictionary keys are the states.
    0: {
        # The inner dictionary keys are the actions
        0: [(0.6666666666666666, 0, 0.0, False),
            (0.3333333333333333, 4, 0.0, False)
        ],
        # The value of the inner dictionary is a list with all
        # possible transitions for that state-action pair.
        1: [(0.3333333333333333, 0, 0.0, False),
            (0.3333333333333333, 4, 0.0, False),
            (0.3333333333333333, 1, 0.0, False)
        ],
        # The transition tuples have four values: the probability of that
        # transition, the next state, the reward, and a flag indicating
        # whether the next state is terminal
    }
```

```

2: [(0.3333333333333333, 4, 0.0, False),
    (0.3333333333333333, 1, 0.0, False),
    (0.3333333333333333, 0, 0.0, False)
],
3: [(0.3333333333333333, 1, 0.0, False),
    (0.6666666666666666, 0, 0.0, False)
]
},
1: {
0: [(0.3333333333333333, 1, 0.0, False),
    (0.3333333333333333, 0, 0.0, False),
    (0.3333333333333333, 5, 0.0, True)
],
1: [(0.3333333333333333, 0, 0.0, False),
    (0.3333333333333333, 5, 0.0, True),
    (0.3333333333333333, 2, 0.0, False)
],
2: [(0.3333333333333333, 5, 0.0, True),
    (0.3333333333333333, 2, 0.0, False),
    (0.3333333333333333, 1, 0.0, False)
],
3: [(0.3333333333333333, 2, 0.0, False),
    (0.3333333333333333, 1, 0.0, False),
    (0.3333333333333333, 0, 0.0, False)
]
},
...
}

```

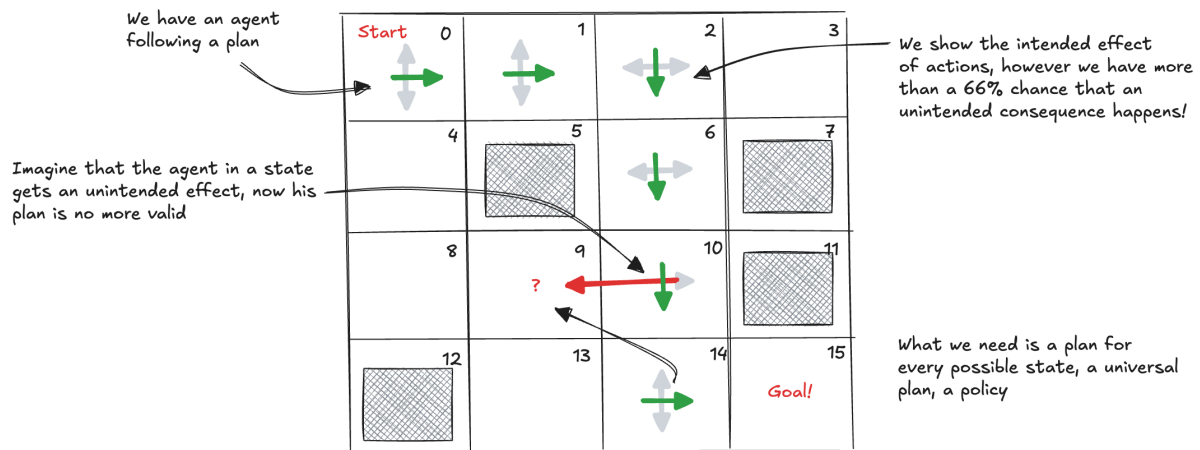
It is possible to load the environment using the **Gymnasium library**. It is a python library that provides a huge number of standard environments to work on RL algorithms with shared interfaces for writing general code and testing it:

```

import gymnasium as gym
frozen_lake = gym.make('FrozenLake-v1');

```

We can consider a possible policy for this environment:



3 Dynamic Programming

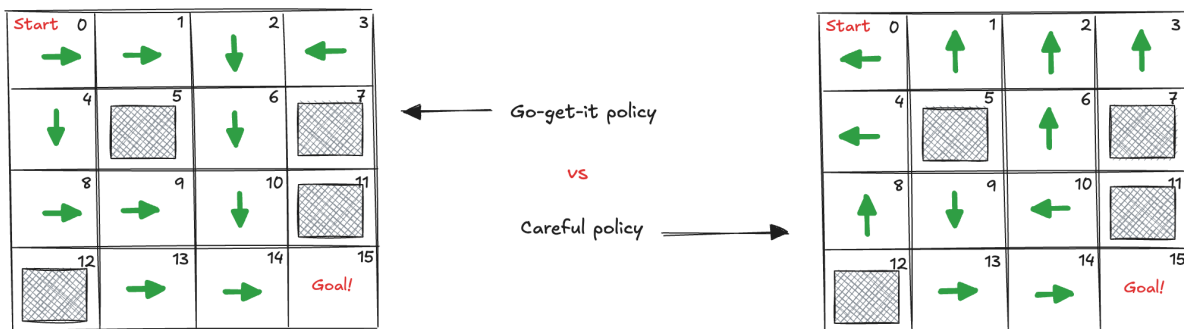
The term dynamic programming refers to a collection of algorithms that can be used to **compute optimal policies** for a MDP, **given a perfect model of the environment**. Notice that these methods in some way are "cheating": **they require full access to the MDP** because they depend on **knowing the dynamics of the environment**, which is something we can't always obtain. However, they are important theoretically.

When an agent has full access to an MDP, there's **no uncertainty** because he can look at the dynamics and rewards and calculate expectations directly. So there's **no need for exploration, for interaction or for trial-and-error learning**.

Suppose to have a **policy** (a universal plan that covers all possible states):

$$\pi(a|s) = P(A_t = a | S_t = s)$$

One immediate question that arises when looking at a policy is: **how good is this policy?** How much better is this policy compared to another policy? In order to answer this question, we need to consider the **expected return**, in order to take into account the **environment stochasticity**. Indeed, even though we know how to act optimally, the environment might send the agent in unintended directions. Consider the following policies for the frozen-lake environment:



In order to compare the policies, we can use a brute-force approach. We simulate the environment for a long time and then we average the returns and evaluate the success probability. We can use Gymnasium to simulate the environment. Notice that it wraps the environment using a single class, called `Env`. This class exposes the common most essential methods of any environment (like `step` and `reset`). Having this interface class is great, because it allows our code to be environment agnostic.

```
import numpy as np

def evaluate(env, pi, n_episodes=10000):
    success = 0;
    returns = [];

    for _ in range(n_episodes):
        done = False;
        state, info = env.reset();
        returns.append(0.0)
        while not done:
            state, reward, terminated, truncated, info = env.step(pi(state));
            if(terminated or truncated):
                done = True;
                returns[-1] += reward;
            if(state == 15):
                success += 1;
    return (success/n_episodes)*100, np.mean(returns);
```

Notice that we need to take into account the possibility that the interaction never ends, because the stochasticity of the environment. In order to avoid this, the Gymnasium environment uses a truncation condition (typically, a timelimit) to end an episode prematurely before a terminal state is reached. Now we can implement the Go-get-it policy and estimate its expected return on the FrozenLake environment::

```
LEFT, DOWN, RIGHT, UP = range(4)

go_get_pi = lambda s: {
    0:RIGHT, 1:RIGHT, 2:DOWN, 3:LEFT,
    4:DOWN, 5:LEFT, 6:DOWN, 7:LEFT,
    8:RIGHT, 9:RIGHT, 10:DOWN, 11:LEFT,
    12:LEFT, 13:RIGHT, 14:RIGHT, 15:LEFT
}[s]
```

```
import gymnasium as gym

frozen_lake = gym.make('FrozenLake-v1')

probability_success, mean_return = evaluate(frozen_lake, go_get_pi);

print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 3.65 %

Obtains an average undiscounted return of 0.0365

We simulate also the Careful policy:

```
careful_pi = lambda s: {
    0:LEFT, 1:UP, 2:UP, 3:UP,
    4:LEFT, 5:LEFT, 6:UP, 7:LEFT,
    8:UP, 9:DOWN, 10:LEFT, 11:LEFT,
    12:LEFT, 13:RIGHT, 14:RIGHT, 15:LEFT
}[s]
```

```
probability_success, mean_return = evaluate(frozen_lake, careful_pi);

print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 54.61 %

Obtains an average undiscounted return of 0.5461

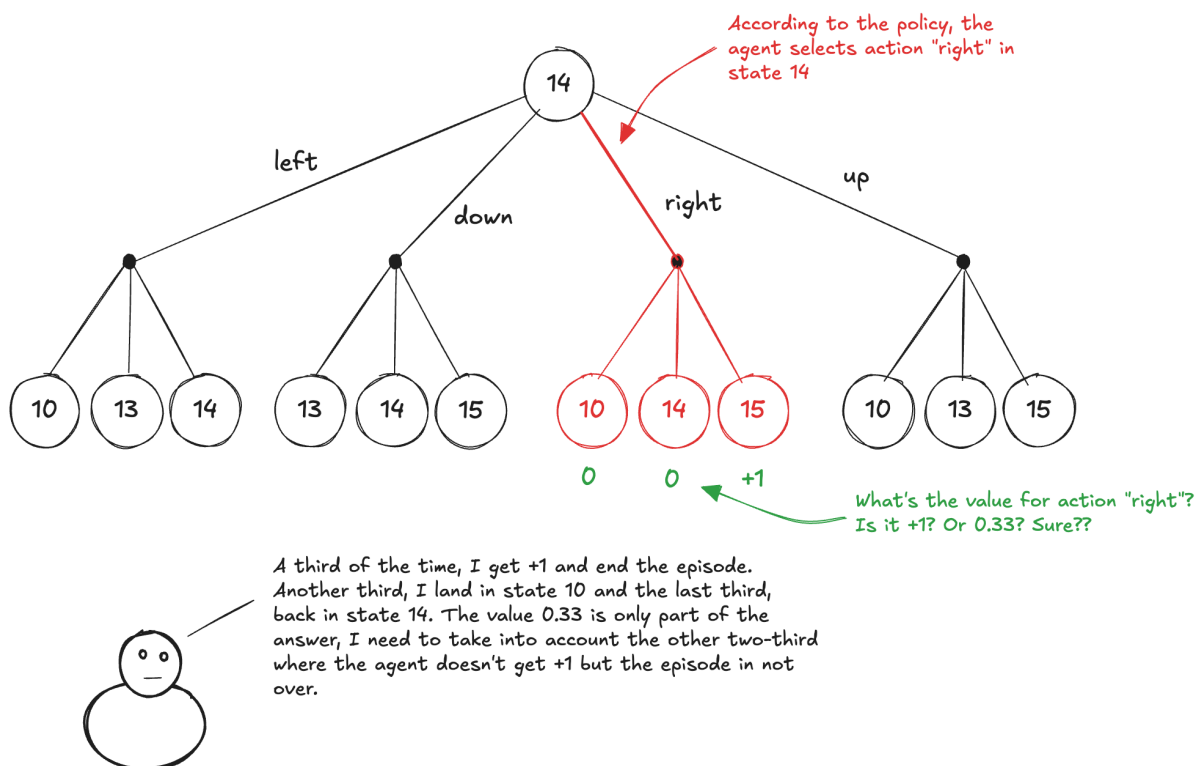
It seems that Go-get-it policy doesn't pay well in the FL environment! However, the brute-force approach is not a good idea for estimating the expected return, because it is very inefficient. We need a better way to evaluate policies.

3.1 State-value function

We can associate a value of goodness to each state for a given policy. This can be useful to make decisions. For example, we can select the action that leads to the state with the highest value. In order to do this, we can calculate the expected return starting from every single state. This is called the **state-value function (or V-function)** denoted $v_\pi(s)$:

$$v_\pi(s) = E_\pi(G_t | S_t = s)$$

Notice that we are under stochastic environments, so we must account for all the possible ways the environment can react to a policy, so we need the expectation. For example, if we follow the Go-get-it policy starting from state 14, we get the following value:



It isn't straightforward to calculate, because of the dependence on the values of other states (10 and 14, in this case), which we don't have either and one is the same state!

We can use the recursive definition of the return:

$$v_\pi(s) = E_\pi(R_{t+1} + \gamma G_{t+1} | S_t = s) =$$

applying the law of total expectation:

$$= \sum_a \pi(a|s) (E_\pi(R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a) =$$

then splitting the expectation of a sum into a sum of expectations and note that, given an action, the expected immediate reward doesn't depend on the policy:

$$= \sum_a \pi(a|s) (E(R_{t+1}|S_t = s, A_t = a) + \gamma E_\pi(G_{t+1}|S_t = s, A_t = a)) =$$

we can write the expected immediate reward in terms of the system dynamics:

$$= \sum_a \pi(a|s) \left(\sum_r r \sum_{s'} p(s', r|s, a) + \gamma E_\pi(G_{t+1}|S_t = s, A_t = a) \right) =$$

applying the law of total expectation on the next state:

$$= \sum_a \pi(a|s) \left(\sum_{s', r} p(s', r|s, a) r + \gamma \sum_{s', r} p(s', r|s, a) E_\pi(G_{t+1}|S_t = s, A_t = a, S_{t+1} = s') \right) =$$

by the Markov property, knowing S_{t+1} makes the expectation independent of S_t and A_t :

$$= \sum_a \pi(a|s) \left(\sum_{s', r} p(s', r|s, a) r + \gamma \sum_{s', r} p(s', r|s, a) E_\pi(G_{t+1}|S_{t+1} = s') \right) =$$

acknowledging that

$$E_\pi(G_{t+1}|S_{t+1} = s') = v_\pi(s')$$

and combining summations we obtain **the Bellman equation**:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) (r + \gamma v_\pi(s'))$$

The value of a state must equal the (discounted) value of the expected next state, plus the reward expected along the way:

We weight over the probability of actions (if the policy is stochastic) prescribed for the state s

We add the reward and the discounted value of the landing state

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) (r + \gamma v_\pi(s'))$$

We weight also over the probability of next states and rewards

Notice how the value of a state depends recursively on the value of possibly many other states, which values may also depend on others, including the original state!

3.2 Action-value function

The critical question is not merely about the value of a state, but the value of taking an action in a state. The **action-value function (or Q-function)** captures precisely this, the expected return

if the agent follows a policy after taking an action in a state:

$$q_{\pi}(s, a) = E_{\pi}(G_t | S_t = s, A_t = a)$$

With can provide a formulation of the Bellman equation also for the action-value function with similar reasoning:

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) (r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a'))$$

These value functions can be **estimated from experience**. For example, if an agent follows a policy and maintains an average (for each state encountered) of the actual returns that have followed that state, then the average will converge to the state value as the number of times that state is encountered approaches infinity. If separate averages are kept for each action taken in each state, then these averages will similarly converge to the action values. Of course, if there are many states, then it may not be practical to keep separate averages for each state or action individually, instead, the agent can maintain parameterized esteems (with fewer parameters than states) and adjust the parameters to better match the observed returns.

3.3 Optimal Policies

Policies, state-value functions and action-value functions are the components we use to describe, evaluate, and improve behaviors. We call it **optimality** when these components are the best they can be. Solving a reinforcement learning task means, roughly, finding a policy that achieves a lot of reward over the long run. Value functions define an ordering over policies. A policy is defined to be better than or equal to a policy if its expected return is greater than or equal for all states:

$$\pi \geq \pi' \Leftrightarrow v_{\pi} \geq v_{\pi'} \quad \forall s \in S$$

There is always at least one policy that is better than or equal to all other policies. This is an **optimal policy**.

$$\pi_* \geq \pi \quad \forall \pi$$

Similarly, an **optimal state-value function** is a state-value function with the maximum value across all policies for all states:

$$v_*(s) = \max_{\pi} v_{\pi}(s) \quad \forall s \in S$$

Finally, an **optimal action-value function** is an action-value function with the maximum value across all policies for all state-action pairs.

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a) \quad \forall s \in S \quad \forall a \in A$$

Notice that if we had the optimal state-value function, we could use the MDP to do a one-step search for the optimal action-value function:

$$q_*(s, a) = E(R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a)$$

And if we had the optimal action-value function, we don't need the MDP at all. We can use it to find the optimal state-value function by merely taking the maximum over the actions:

$$v_*(s) = \max_a q_*(s, a)$$

Then, we can write:

$$\begin{aligned} v_*(s) &= \max_a E(R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a) \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \end{aligned}$$

This means that we take the max action of the weighted sum of the reward and discounted optimal value of the next state.

Similarly, the optimal action-value function can be obtained this way:

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')]$$

These equations are called the **Bellman optimality equations** and they are a system of equations and if the dynamic of the environment is known, then in principle we can solve it. Once one has one of them, it is easy to determine an optimal policy. This solution relies on at least three assumptions that are rarely true in practice:

- we accurately know the dynamics of the environment;
- we have enough computational resources to complete the computation of the solution;
- the Markov property.

Generally various combinations of these assumptions are violated and in reinforcement learning typically we have to settle for approximate solutions.

3.4 Policy Evaluation

In order to search for good policies, we need an algorithm for evaluating an arbitrary policy more efficiently than the brute-force approach. Such an algorithm is known as a **policy evaluation** and consists of calculating the state-value function for a given policy by sweeping through the state space and iteratively improving estimates. The initial approximation is chosen arbitrarily and each successive approximation is obtained by using the Bellman equation as an **update rule**:

$$v_{k+1}(s) = E_{\pi}(R_{t+1} + \gamma v_k(S_{t+1}|S_t = s)) = \sum_a \pi(a|s) \left(\sum_{s',r} p(s',r|s,a) (r + \gamma v_k(s')) \right)$$

this sequence can be shown to converge to the value function as k goes to infinity, under the same conditions that guarantee the existence of the value function. To produce each successive approximation, we apply the same operation to each state: we replace the old value of state with a new value obtained from the old values of the successor of the state and the expected immediate rewards, along all the one-step transitions possible under the policy being evaluated.

Note that update estimates of the values of states are based on estimates of the values of successor states. That is, they **update estimates on the basis of other estimates**. We call this general idea **bootstrapping**.

Formally, the iterative policy evaluation converges only in the limit, but in practice it must be halted short. For example, we can test the quantity:

$$\max_{s \in S} |v_{k+1}(s) - v_k(s)|$$

and stops the iterative process when it is sufficiently small.

The same update rule can be also applied to the action-value function:

$$q_{k+1}(s,a) = \sum_{s',r} p(s',r|s,a) (r + \gamma \sum_{a'} \pi(a'|s') q_k(s',a'))$$

It's important to notice that k here indicates **iterations across estimates**, and they're **not interactions with the environment**. These aren't episodes, and the agent isn't selecting actions and observing the environment. These aren't time steps either. Instead, these are the iterations of the iterative policy evaluation algorithm.

We can write the policy evaluation algorithms in Python:

```
# We need the policy to evaluate and the MDP the policy runs on
# The discount factor and gamma are defaults to 1
# Theta is a small number that we use to check for convergence

def policy_evaluation(pi, env, gamma=0.99, theta=1e-10):

    # we need to unwrap the environment to access its
    # internal dynamic
    dynamics = env.unwrapped.P;

    # initialize the first-iteration estimates to zero
    prev_v = np.zeros(len(dynamics));

    # looping forever ...
```

```

while True:
    # initialize the current-iteration estimates to zero as well
    v = np.zeros(len(dynamics));

    # loop through all states to estimate the state-value function
    for s in range(len(dynamics)):

        # we use the policy pi to get the possible transitions
        for prob, next_state, reward, done in dynamics[s][pi(s)]:

            # estimate the value of that state using the update rule
            v[s] += prob * (reward + gamma * prev_v[next_state]);

    # check for the stopping condition
    if np.max(np.abs(prev_v - v)) < theta:
        break;

    # finally, copy to get ready for the next iteration
    prev_v = v.copy()

# return the latest state-value function
return v

```

We can write a simple function to print the value function in a readable way:

```

def print_state_value_function(v, env):
    dynamics = env.unwrapped.P;
    n_cols = 4;
    print("State-value function:");
    for s in range(len(dynamics)):
        if np.all([done for action in dynamics[s].values() for _, _, _, done in
                    ↵ action]):
            print(" X ", end="\t");
        else:
            print(np.round(v[s], 2), end="\t");
        if (s+1)%n_cols==0:
            print(" ");

```

Let's now run policy evaluation for the two policies Go-get-it and Careful for the Frozen Lake environment. We need to get the unwrapped environment in order to access the dynamics:

```
v = policy_evaluation(go_get_pi, frozen_lake, gamma=0.99)
print_state_value_function(v, frozen_lake)
```

State-value function:

```
0.03 0.02 0.05 0.02
0.05 X 0.1 X
0.09 0.24 0.29 X
X 0.43 0.64 X
```

```
v = policy_evaluation(careful_pi, frozen_lake, gamma=0.99)
print_state_value_function(v, frozen_lake)
```

State-value function:

```
0.41 0.38 0.35 0.34
0.42 X 0.12 X
0.45 0.48 0.43 X
X 0.59 0.71 X
```

As already seen, the Go-get-it policy doesn't pay well in this environment! Whereas the Careful policy is much better. Fascinating results, but a question arises: are there any better policies?

3.5 Policy Improvement

The reason for computing the value function for a policy is to help in finding better policies. Suppose we have determined the value function for a policy. For a state, we would like to know if we should change the policy and choose an action different from the one prescribed by the policy (in other word following a different policy). We know how good is to follow the current policy (it is given by the value function), but would it be better to change? One way to answer this question is to consider selecting the different action, thereafter following the existing policy and calculate the value function under this assumption. Then, if this value is greater, it is better to select the new action in that state than following the policy. This is a case of the general **policy improvement theorem**, Let's indicate with π the current policy and with π' the new one:

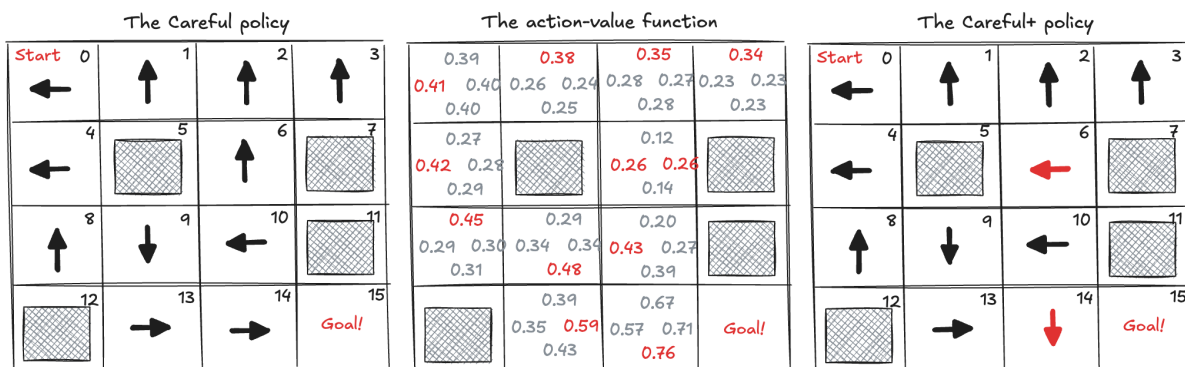
$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s) \quad \forall s \in S \quad \Rightarrow \quad v_{\pi'}(s) \geq v_{\pi}(s)$$

This result applies to the two policies that we considered before. The original policy and a changed policy that is identical to the first, except for the different action taken in one state. Thus, $q_{\pi}(s, a) > v_{\pi}(s)$, then the changed policy is indeed better than the original one. It is

natural to extend this and to consider changes at all states and to all possible actions, selecting at each state the action that appears best according to $q_{\pi}(s, a)$:

$$\pi'(s) = \arg \max_a q_{\pi}(s, a) = \arg \max_a \sum_{s', r} p(s', r | s, a) (r + \gamma v_{\pi}(s'))$$

The process of making a new policy that improves an original policy, by making it greedy with respect to the value function of the original policy, is called **policy improvement**. Let's try to improve the Careful policy: if we act greedily with respect to the value function of the policy, we obtain a new Careful+ policy:



We create a new policy Careful+ which is greedy over the Careful Q-function

Let's implement it in Python:

```
# It takes the state-value function of the policy
# we want to improve and the environment
def policy_improvement(v, env, gamma=0.99):

    # we need to unwrap the environment to access its
    # internal dynamic
    dynamics = env.unwrapped.P;

    # initialize the Q-function to zero
    q = np.zeros((len(dynamics), len(dynamics[0])))

    # loop through the states, actions, and transitions...
    for s in range(len(dynamics)):
        for a in range(len(dynamics[s])):
            for prob, next_state, reward, done in dynamics[s][a]:
                # ...and calculate the action-value function
                q[s][a] += prob * (reward + gamma * v[next_state])
```

```
# obtain a new, greedy policy by taking the argmax of the Q-function
def new_pi(s):
    return np.argmax(q[s])

return new_pi;
```

We can create a support function in order to visualize policies:

```
def print_policy(pi, env):
    dynamics = env.unwrapped.P;
    actions = ['<', 'v', '>', '^', ' '];
    n_cols = 4;
    print("Policy:");
    for s in range(len(dynamics)):
        if np.all([done for action in dynamics[s].values() for _, _, _, done in
                    ↪ action]):
            print(" X ", end="\t");
        else:
            print(actions[pi(s)], end="\t");
        if (s+1)%n_cols==0:
            print(" ");
```

Then we can try to improve the careful policy:

```
careful_plus_pi = policy_improvement(v, frozen_lake, gamma=0.99)
print_policy(careful_plus_pi, frozen_lake);
```

Policy:

```
<  ^  ^  ^
<  X  <  X
^  v  <  X
X  >  v  X
```

Is this policy any better? Well, we can evaluate it with brute-force or using the policy evaluation algorithm:

```
probability_success, mean_return = evaluate(frozen_lake,
                                           careful_plus_pi);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 73.91 %

Obtains an average undiscounted return of 0.7391

As we can see, the Careful+ can reach the goal state with a much better probability than the Careful policy. This is a good example of how the policy improvement theorem works. We can also check the difference between the state-value functions of the two policies:

```
v_plus = policy_evaluation(careful_plus_pi, frozen_lake, gamma=0.99);
print_state_value_function(v_plus, frozen_lake);
```

State-value function:

```
0.54 0.5 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X
```

```
difference = v_plus - v;
print_state_value_function(difference, frozen_lake);
```

State-value function:

```
0.13 0.12 0.12 0.11
0.14 X 0.24 X
0.15 0.16 0.18 X
X 0.15 0.15 X
```

The new policy is better than the original one. This is great, acting greedily with respect to the action-value function gave us an improved policy. This is what the policy-improvement algorithm does: **it calculates an action-value function using the current policy and the MDP, then it returns an improved policy acting greedy with respect to the action-value function of the original policy.** The natural next questions are these: is there a better policy than this one? Can we do any better than careful+? Can we evaluate the careful+ policy, and then improve it again?

```
careful_plus_plus_pi = policy_improvement(v_plus, frozen_lake, gamma=0.99)
```



```
print_policy(careful_plus_plus_pi, frozen_lake)
```

Policy:

```
<  ^  ^  ^
<  X  <  X
^  v  <  X
X  >  v  X
```

```
probability_success, mean_return = evaluate(frozen_lake,
                                           careful_plus_plus_pi);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 73.54 %

Obtains an average undiscounted return of 0.7354

There's no improvement this time. No improvement occurs because the careful+ policy is an optimal policy of the Frozen Lake environment. We only needed one improvement over the careful policy because this policy was good to begin with. However, even if we start with an **adversarial policy** (designed to perform poorly) **alternating over policy evaluation and improvement** would still end up with an optimal policy:

```
adversarial_pi = lambda s: {
    0:UP, 1:UP, 2:UP, 3:UP,
    4:UP, 5:LEFT, 6:UP, 7:LEFT,
    8:LEFT, 9:LEFT, 10:LEFT, 11:LEFT,
    12:LEFT, 13:LEFT, 14:LEFT, 15:LEFT
}[s]
```

```
print_policy(adversarial_pi, frozen_lake);
```

Policy:

```
^  ^  ^  ^
^  X  ^  X
<  <  <  X
X  <  <  X
```

```

probability_success, mean_return = evaluate(frozen_lake,
                                           adversarial_pi);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);

```

```

Reaches goal  0.0 %
Obtains an average undiscounted return of  0.0

```

```

v_1 = policy_evaluation(adversarial_pi, frozen_lake, gamma=0.99)
adversarial_pi_2 = policy_improvement(v_1, frozen_lake, gamma=0.99)

v_2 = policy_evaluation(adversarial_pi_2, frozen_lake, gamma=0.99)
adversarial_pi_3 = policy_improvement(v_2, frozen_lake, gamma=0.99)

v_3 = policy_evaluation(adversarial_pi_3, frozen_lake, gamma=0.99)
adversarial_pi_4 = policy_improvement(v_3, frozen_lake, gamma=0.99)

v_4 = policy_evaluation(adversarial_pi_4, frozen_lake, gamma=0.99)
adversarial_pi_5 = policy_improvement(v_4, frozen_lake, gamma=0.99)

v_5 = policy_evaluation(adversarial_pi_5, frozen_lake, gamma=0.99)
adversarial_pi_6 = policy_improvement(v_5, frozen_lake, gamma=0.99)

print_policy(adversarial_pi_6, frozen_lake)

```

Policy:

```

<  ^  <  ^
<  X   <  X
^   v   <  X
X   >   v   X

```

```

probability_success, mean_return = evaluate(frozen_lake,
                                           adversarial_pi_5);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);

```

```

Reaches goal  71.12 %
Obtains an average undiscounted return of  0.7112

```

3.6 Policy Iteration

Once a policy has been improved using its value function to yield a better policy, we can then compute a new value function and improve again to yield an even better policy. We can thus obtain a sequence of monotonically improving policies and value functions:

$$\pi_0 \xrightarrow{\text{evaluate}} v_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluate}} v_{\pi_1} \xrightarrow{\text{improve}} \pi_2 \xrightarrow{\text{evaluate}} \dots \xrightarrow{\text{improve}} \pi_* \xrightarrow{\text{evaluate}} v_{\pi_*}$$

Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal). Because a finite MDP has only a finite number of policies, this process must converge to an optimal policy in a finite number of iterations. This way of finding an optimal policy is called **policy iteration**. We can put all together in Python:

```
# it only needs the MDP (including gamma)
def policy_iteration(env, gamma=0.99, theta=1e-2):

    # we need to unwrap the environment to access its
    # internal dynamic
    dynamics = env.unwrapped.P;

    # create a starting random policy:
    def pi(s):
        return np.random.choice(tuple(dynamics[0].keys()));

    while True:
        # get the state-value function of the policy
        v = policy_evaluation(pi, env, gamma, theta)

        # get an improved policy
        new_pi = policy_improvement(v, env, gamma)

        # if it's different, we do it all over again
        if np.all([pi(s) == new_pi(s) for s in range(len(dynamics))]):
            break;

        pi = new_pi;

    # if it's not, we break out of the loop and return
    # the optimal policy and the optimal state-value function
    return v, new_pi
```

Let's try it:

```
v_best, pi_best = policy_iteration(frozen_lake, gamma=0.99)
```

```
print_policy(pi_best, frozen_lake)
```

Policy:

```
<  ^  <  ^
<  X  <  X
^   v  <  X
X   >  v  X
```

```
print_state_value_function(v_best, frozen_lake)
```

State-value function:

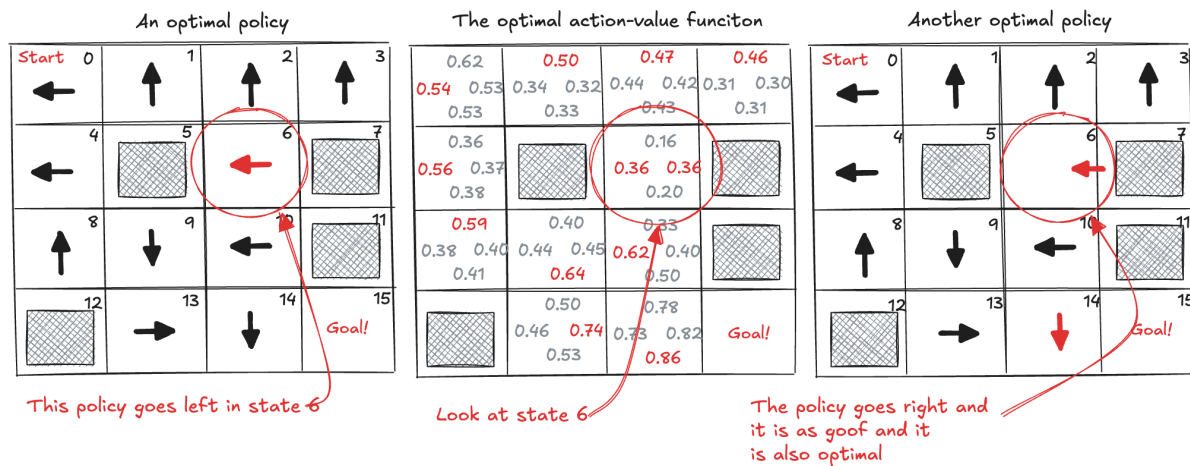
```
0.29 0.24 0.23 0.21
0.33 X  0.24 X
0.4 0.51 0.51 X
X   0.65 0.81 X
```

```
probability_success, mean_return = evaluate(frozen_lake,
                                           pi_best);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

Reaches goal 73.14 %

Obtains an average undiscounted return of 0.7314

As expected, alternating policy evaluation and policy improvement yields an optimal policy and the optimal state-value function, regardless of the policy we start with. Notice how we use "an" optimal policy, but also use "the" optimal state-value function. This is not a coincidence, this is a property: an MDP can have more than one optimal policy, but it can only have a single optimal state-value function. A state-value function may have actions that are equally valued for a given state, in this case, there could be multiple optimal policies, each optimal policy selecting a different, but equally valued, action. Take a look at the Frozen Lake example:



So far we have considered the special case of deterministic policies, anyway all the ideas extend easily to stochastic policies. In particular, if there are several actions at which the maximum is achieved, in the stochastic case we need not select a single action from among them. Instead, each maximizing action can be given a portion of the probability of being selected in the new greedy policy.

3.7 Value Iteration

Policy iteration works, but **it is really slow**. Each of its iterations involves policy evaluation, which is a protracted iterative computation requiring multiple sweeps through the state set. However, if we truncated policy evaluation after a single iteration, we could still improve the policy by taking a new policy greedy with the action-value function estimation after a single state-space sweep of policy evaluation. This algorithm is called **value iteration**. It can be thought of "greedily greedifying policies", because we calculate the greedy policy as soon as we can, greedily. Value iteration **doesn't wait until we have an accurate estimate of the policy before improving it, but instead, it truncates the policy-evaluation phase after a single state sweep**. We can merge a truncated policy-evaluation step and a policy improvement step into the same equation. Instead of improving the policy (by taking the argmax to get a better policy and then evaluating this improved policy to obtain a value function again), we directly calculate the maximum (max, instead of argmax) value across the actions to be used for the next sweep over the states:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a)(r + \gamma v_k(s')) = \max_a q_k(s',a)$$

For arbitrary initial value function, the sequence of improved value function converges to the optimal one under the same conditions that guarantee the existence of the optimal function.

Like policy evaluation, value iteration formally requires an infinite number of iterations to converge. In practice, we stop the iterative process once the value function changes by only a small amount in a sweep.

Notice that in practice, in value iteration, **we don't have to deal with policies at all**. It doesn't have any separate evaluation phase that runs to convergence. While the goal is the same as of policy iteration (to find the optimal policy), it happens to do this through the value functions (thus the name). Only at the end of the algorithm, after the value function converges to the optimal, we will extract the optimal policy by taking the argmax over the actions of the action-value function, as before. We can implement this algorithm in Python:

```
def value_iteration(env, gamma=0.99, theta=1e-10):

    # we need to unwrap the environment to access its
    # internal dynamic
    dynamics = env.unwrapped.P;

    # initialize a state-value function
    v = np.zeros(len(dynamics))

    while True:
        q = np.zeros((len(dynamics), len(dynamics[0])))

        # for every transition of every action in every state, we ...
        for s in range(len(dynamics)):
            for a in range(len(dynamics[s])):
                for prob, next_state, reward, done in dynamics[s][a]:
                    # ... calculate the action-value function
                    q[s][a] += prob * (reward + gamma * v[next_state])

        # After each sweep over the state space, make sure the state-value function
        # ↪ keeps changing,
        # otherwise, we found the optimal state-value function and should break out
        if np.max(np.abs(v - np.max(q, axis=1))) < theta:
            break;

        # we don't need a separate policy-improvement phase
        v = np.max(q, axis=1)

    # only at the end, we extract the optimal policy
    def pi(s):
        return np.argmax(q[s])
```

```
return v, pi
```

And again, we can solve the Frozen Lake problem:

```
v_best, pi_best = value_iteration(frozen_lake, gamma=0.99)
```

```
print_policy(pi_best, frozen_lake)
```

Policy:

```
<  ^  ^  ^
<  X  <  X
^   v  <  X
X   >  v  X
```

```
print_state_value_function(v_best, frozen_lake)
```

State-value function:

```
0.54 0.5 0.47 0.46
0.56 X  0.36 X
0.59 0.64 0.62 X
X   0.74 0.86 X
```

```
probability_success, mean_return = evaluate(frozen_lake,
                                             pi_best);
print("Reaches goal ", probability_success, "%");
print("Obtains an average undiscounted return of ", mean_return);
```

```
Reaches goal  73.44000000000001 %
```

```
Obtains an average undiscounted return of  0.7344
```

Let solve the problem also for the no discounting case ($\gamma = 1$) and for a different discounting value ($\gamma = 0.95$):

```
v_best_no_discounting, _ = value_iteration(frozen_lake, gamma=1.00)
print_state_value_function(v_best_no_discounting, frozen_lake)
```

State-value function:

```
0.82 0.82 0.82 0.82
0.82 X 0.53 X
0.82 0.82 0.76 X
X 0.88 0.94 X
```

```
v_best_different_discounting, _ = value_iteration(frozen_lake, gamma=0.95)
print_state_value_function(v_best_different_discounting, frozen_lake)
```

State-value function:

```
0.18 0.15 0.15 0.13
0.21 X 0.18 X
0.27 0.37 0.4 X
X 0.51 0.72 X
```

We find that they are not the same. Introducing discounting in a stochastic environment can either have no effect or **favor more risky but faster (less discounting) policies**.

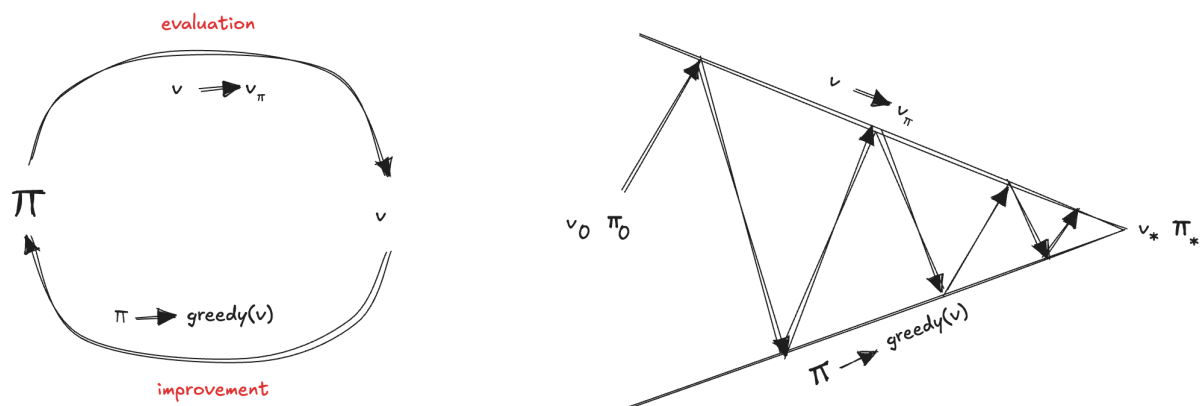
A major drawback of the dynamic programming methods is that they require a perfect model of the environment, these methods in a way “cheat” because they require full access to the MDP, which is something we can’t always obtain. Another problem is that they **involve operations over the entire state set of the MDP**. If the state set is very large, then even a single sweep can be prohibitively expensive. For example, the game of backgammon has over 10^{20} states. **Asynchronous dynamic programming** are iterative algorithms that are not organized in terms of systematic sweeps of the state set, but they update values of states in any order. To converge correctly, however, they must continue to update the values of all the states. Of course, avoiding sweeps does not necessarily mean that we can get away with less computation. It just means that an algorithm does not need to get locked into any hopelessly long sweep before it can make progress improving a policy.

3.8 Generalization

The dynamic programming approach consists of **two simultaneous, interacting processes, one making the value function consistent with the current policy (evaluation), and the other making the policy greedy with respect to the current value function (improvement)**. These two processes can alternate, each completing before the other begins (policy iteration); or only a single iteration of evaluation can be performed before each policy improvement; moreover, the evaluation and improvement can be interleaved at an even finer grain (asynchronous methods).

As long as both processes continue to update all states, the ultimate result is typically the same: convergence to the optimal value function and an optimal policy. We use the term **generalized policy iteration** to refer to the **general idea of letting evaluation and improvement processes interact**, independently from the granularity and other details of the two processes.

Almost all reinforcement learning methods are well described as generalized policy iteration approached: all have policies and value functions, with the policy always being improved with respect to the value function and the value function always being driven toward the policy. If both the processes stabilize (no longer produce changes), then the value function and policy must be optimal. The evaluation and improvement processes can be viewed as both **competing and cooperating**. They compete in the sense that they pull in opposing directions. Making the policy greedy with respect to the value function typically makes the value function incorrect for the changed policy, and making the value function consistent with the policy typically causes that policy no longer to be greedy. In the long run, however, these two processes interact to find a single joint solution: the optimal value function and an optimal policy.



4 Exploitation vs Exploration

For an agent is really important **to explore** when interacting with **uncertain environments**, problems in which the MDP isn't available for planning. In these cases, the agent must **learn from experience** taking decisions about what actions to take. However, no matter how small and unimportant a decision may seem, every decision we make is a **trade-off between information gathering and information exploitation**. For example, when we go to our favorite restaurant, should we order our favorite dish, yet again, or should we request that dish we've been meaning to try? These kinds of questions illustrate the **exploration-exploitation dilemma** and are at the core of the reinforcement learning problem. It boils down to deciding **when to acquire knowledge** and **when to capitalize on knowledge previously learned**. It's a challenge to know

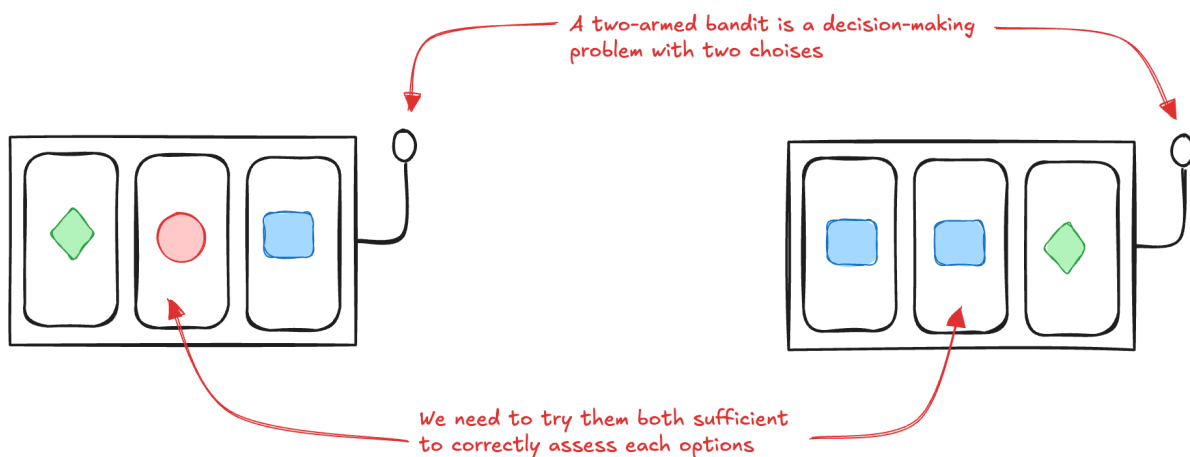
whether the good we already have is good enough. In order to examine this dilemma, we will consider simplified environments that aren't sequential, but one-shot: the **multi-armed bandits** (MABs) problem.

4.1 Multi-armed bandit problem

Multi-armed bandits (MAB) are a **special case of RL problem** in which the size of the state space and horizon equal to one. A MAB has **multiple actions**, a **single state**, and a **greedy horizon**. We can think of it as a "many-options, single-choice" environment (named by analogy to a slot machine). In formal terms we are faced repeatedly with a choice among k different actions. After each choice we receive a reward chosen from a **stationary probability distribution** that depends on the selected action. The action-value function of action a is the expected reward given that a is sampled:

$$q(a) = E[R_t | A_t = a]$$

The goal of MAB is very similar to that of RL. In RL, the agent needs to maximize the expected "cumulative" and "discounted" reward, but in MABs we only have a single chance of selecting an action. Therefore, we can exclude the words that don't apply to the case: we remove "cumulative", because there's only a single time step; and "discounted", because there are no next states to account for. In MABs, the goal is to **maximize the expected reward**. Notice that the word "expected" stays because there's **stochasticity** in the environment.



We can make a numerical simulation with a 10-armed bandit problem. The true value of each of the ten actions is selected according to a normal distribution with zero mean and unit variance, and then the actual rewards are selected from a normal distribution with mean equal to the action value and unit variance:

```
import numpy as np

# Define a simple multi-armed bandit environment
class BanditEnv:
    def __init__(self):

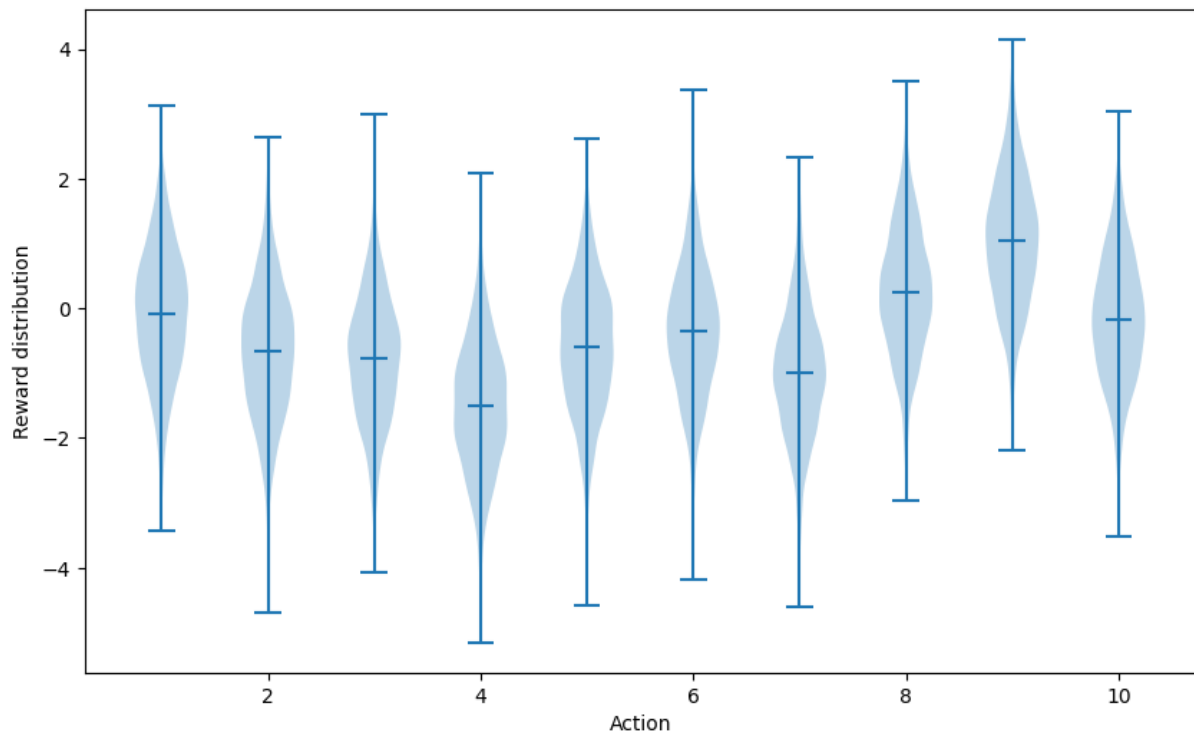
        # Define the number of arms
        self.size = 10

        # Randomly initialize the mean reward for each arm
        self.means = np.random.randn(self.size)

    def step(self, action):
        # Return a reward sampled from a normal distribution
        # centered at the arm's mean
        return np.random.normal(loc=self.means[action])

# Create the environment
env = BanditEnv()

# Visualize the reward distributions for each action
import matplotlib.pyplot as plt
plt.figure(figsize=(10,6))
plt.violinplot(np.random.randn(2000, 10) + env.means, showmeans=True)
plt.xlabel("Action")
plt.ylabel("Reward distribution")
plt.show()
```



If we knew the value of each action, then it would be trivial to solve the problem, we always select the action with highest value:

$$a_* = \operatorname{argmax}_a q(a)$$

However, we assume to not know the action values with certainty, although we may have estimates.

4.2 Estimation of the action value

The estimation of the action-value function in MAB environments is pretty straightforward. Because MABs are one-step environments, to estimate the value of an action, we need to calculate the average reward obtained by selecting that action. In other words, the estimate of an action value is equal to the total reward obtained when selecting the action, divided by the number of times action a has been selected:

$$q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot 1_{A_i=a}}{\sum_{i=1}^{t-1} 1_{A_i=a}}$$

We call this estimation the **sample-average method**, because each estimate is an average of the samples of relevant rewards.

The obvious implementation would be to maintain a record of all the rewards and then perform the computation whenever the estimated value was needed. However, the memory and computational requirements would grow over time as more rewards are seen. Each additional reward would require additional memory to store it and additional computation to compute the sum in the numerator. This is not really necessary. It is easy to devise an **incremental formulas** for updating averages with small, constant computation required to process each new reward. To simplify notation, we concentrate on a single action:

$$\begin{aligned} q_{t+1} &= \frac{R_1 + R_2 + \dots + R_t}{t} = \frac{1}{t} \sum_{i=1}^t R_i = \frac{1}{t} \left(R_t + \sum_{i=1}^{t-1} R_i \right) = \frac{1}{t} \left(R_t + (t-1) \frac{1}{t-1} \sum_{i=1}^{t-1} R_i \right) = \\ &= \frac{1}{t} (R_t + (t-1)q_t) = \frac{1}{t} (R_t + tq_t - q_t) = q_t + \frac{1}{t} (R_t - q_t) \end{aligned}$$

This implementation requires memory only for q_t and t , and only a small computation for each new reward. This update is of a form of the general rule:

NewEstimate <-- OldEstimate + StepSize (Target - OldEstimate)

The expression "Target–OldEstimate" is an error in the estimate, and it is reduced by taking a step toward the "Target", which is presumed to indicate a desirable direction in which to move, though it may be noisy. In our case above, the target is the reward and the step-size parameter changes from time step to time step, in processing the t reward for action a , the method uses the step-size $1/t$.

The averaging method is appropriate for **stationary** bandit problems, in which the reward probabilities do not change over time. However, we often encounter problems that are **non-stationary**. In such cases, it makes sense to give more weight to recent rewards than to long-past rewards. One of the most popular ways of doing this is to use a constant step-size parameter:

$$q_{t+1} = q_t + \alpha(R_t - q_t)$$

where the step-size parameter $\alpha \in (0, 1]$ is constant. This results in q_t being a weighted average of past rewards and the initial estimate q_1 :

$$\begin{aligned} q_{t+1} &= q_t + \alpha(R_t - q_t) = \alpha R_t + (1 - \alpha)q_t = \\ &= \alpha R_t + (1 - \alpha)(\alpha R_{t-1} + (1 - \alpha)q_{t-1}) = \\ &= \alpha R_t + (1 - \alpha)\alpha R_{t-1} + (1 - \alpha)^2 q_{t-1} = \\ &= \alpha R_t + (1 - \alpha)\alpha R_{t-1} + (1 - \alpha)^2 \alpha R_{t-2} + \dots + (1 - \alpha)^{t-1} \alpha R_1 + (1 - \alpha)^t q_1 \\ &= (1 - \alpha)^t q_1 + \sum_{i=1}^t \alpha (1 - \alpha)^{t-i} R_i \end{aligned}$$

Sometimes it is convenient to vary the step-size parameter from step to step. Let $\alpha_t(a)$ denote the step-size parameter used to process the reward received after the t selection of action a . In

that case the weighting on each prior reward is:

$$\begin{aligned}
 q_{t+1} &= q_n + \alpha_n(R_t - q_t) = \alpha_t R_t + (1 - \alpha_t)q_t = \\
 &= \alpha_t R_t + (1 - \alpha_t)(\alpha_{t-1} R_{t-1} + (1 - \alpha_{t-1})q_{t-1}) = \\
 &= \alpha_t R_t + (1 - \alpha_t)\alpha_{t-1} R_{t-1} + (1 - \alpha_t)(1 - \alpha_{t-1})q_{t-1} = \\
 &= \alpha_t R_t + (1 - \alpha_t)\alpha_{t-1} R_{t-1} + (1 - \alpha_t)(1 - \alpha_{t-1})\alpha_{t-2} R_{t-2} + \dots + (1 - \alpha_t)(1 - \alpha_{t-1})\dots(1 - \alpha_1)q_1 = \\
 &= \prod_{i=1}^t (1 - \alpha_i)q_1 + \sum_{i=1}^t \alpha_i \prod_{j=i+1}^t (1 - \alpha_j)R_i
 \end{aligned}$$

The choice $\alpha_t(a) = 1/n$ results in the sample-average method, which is guaranteed to converge to the true action values by the law of large numbers. But of course convergence is not guaranteed for all choices of $\alpha_t(a)$. A result in stochastic approximation theory gives us the **conditions required to assure convergence**:

$$\sum_{t=1}^{\infty} \alpha_t(a) = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2(a) < \infty$$

The first condition is required to guarantee that the **steps are large enough** to eventually overcome any initial conditions or random fluctuations. The second condition guarantees that eventually the **steps become small enough** to assure convergence. Note that both convergence conditions are met for the sample-average case, but not for the case of constant step-size parameter. In the latter case, the second condition is not met, indicating that the estimates never completely converge, but continue to vary in response to the most recently received rewards. **This is actually desirable in a non-stationary environment.**

All the methods we have discussed so far are biased (dependent to some extent) by their initial estimates. For the sample-average methods, the bias disappears once all actions have been selected at least once, but for methods with constant α , the bias is permanent, though decreasing over time. In practice, this kind of bias is usually not a problem and can sometimes be very helpful to supply some **prior knowledge** about what level of rewards can be expected.

It's essential to highlight that there are no differences in how strategies estimate the action-value function, the only difference is in how each strategy uses the estimates to select actions.

4.3 Basic exploration strategies

If we maintain the estimates of the action values, then at any time step there is at least one action whose estimated value is greatest. We call this the **greedy action**. When we select this action, we say that we are **exploiting our current knowledge** of the values of the actions. If there is more than one greedy action, then a selection is made among them in some arbitrary way, perhaps randomly.

$$A_t = \operatorname{argmax}_a q_t(a)$$

Greedy action selection always **exploits** current knowledge **to maximize immediate reward**, it spends no time at all sampling apparently inferior actions to see if they might really be better. Instead, if we select one of the non-greedy actions, then we say we are **exploring**, because this enables to improve the estimate of the non-greedy action's value.

Exploitation is the right thing to do to maximize the expected reward on the one step, but exploration may produce a greater total reward in the long run.

Unfortunately, it is not possible both to explore and to exploit with any single action selection, one often refers to the **conflict between exploration and exploitation**. The need to balance exploration and exploitation is a distinctive challenge that arises in reinforcement learning.

Notice the relationship between a greedy strategy and time. If the agent only has one episode left, the best thing is to act greedily (if you know you only have one day to live, you'll do things you enjoy the most): this is a reasonable thing to do when we have limited time left. However, if we don't, we can trade-off immediate satisfaction for gaining of information that would allow us better long-term results.

Notice also that while there's only a single way to exploit, there are **multiple ways to explore**. Exploiting is nothing but doing what you think is best, exploring, on the other hand, is much more complex. It's obvious you need to collect information, but how is a different question. We could try gathering information to support your current beliefs. We could gather information to attempt proving ourself wrong. We could explore based on confidence, or based on uncertainty. The list goes on...

4.3.1 Pure exploration and pure exploitation

The **greedy strategy** (always exploit) and the **pure random strategy** (always explore) aren't really strategies, but two important baselines.

```
def pure_exploitation(env, n_episodes=1000):  
  
    # initialize the Q-function and the count array to all zeros  
    q = np.zeros((env.size), dtype=float)  
    n = np.zeros((env.size), dtype=int)  
  
    # some variables to calculate statistics and not necessary  
    qe = np.empty((n_episodes, env.size), dtype=float)  
    returns = np.empty(n_episodes, dtype=float)  
    actions = np.empty(n_episodes, dtype=int)
```

```
# here we enter the main loop and interact with the environment
for e in range(n_episodes):

    # select the action that maximizes the estimated Q-values
    action = np.argmax(q)

    # then, pass it to the environment and receive a new reward
    reward = env.step(action)

    # update the counts and the q-table
    n[action] += 1
    q[action] = q[action] + (reward - q[action])/n[action]

    # update the statistics and start a new episode
    qe[e] = q
    returns[e] = reward
    actions[e] = action

return returns, qe, actions
```

```
def pure_exploration(env, n_episodes=1000):

    # The baseline boilerplate is the same as before
    q = np.zeros((env.size), dtype=float)
    n = np.zeros((env.size), dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    for e in range(n_episodes):

        # always selects an action randomly
        action = np.random.randint(len(q))

        reward = env.step(action)

        n[action] += 1
        q[action] = q[action] + (reward - q[action])/n[action]

        qe[e] = q
```



```

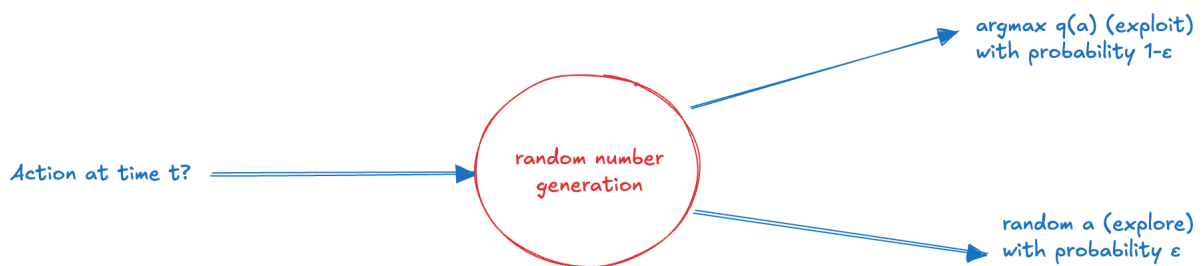
    returns[e] = reward
    actions[e] = action

    return returns, qe, actions

```

4.3.2 Epsilon greedy

We combine the two baselines, pure exploitation and pure exploration, so that the agent can exploit, but also collect information to make informed decisions. The **hybrid strategy** consists of **acting greedily most of the time and exploring randomly every so often**. This strategy is referred as the **epsilon-greedy strategy**.



```

def epsilon_greedy(env, epsilon=0.01, n_episodes=1000):

    # The baseline boilerplate is the same as before
    q = np.zeros((env.size), dtype=float)
    n = np.zeros((env.size), dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    for e in range(n_episodes):

        # draw a random number and compare to a hyperparameter epsilon
        if np.random.uniform() > epsilon:
            # if it is greater than epsilon, exploit:
            action = np.argmax(q)
        else:
            # otherwise, explore:
            action = np.random.randint(len(q))

        reward = env.step(action)

```

```

n[action] += 1
q[action] = q[action] + (reward - q[action])/n[action]

qe[e] = q
returns[e] = reward
actions[e] = action

return returns, qe, actions

```

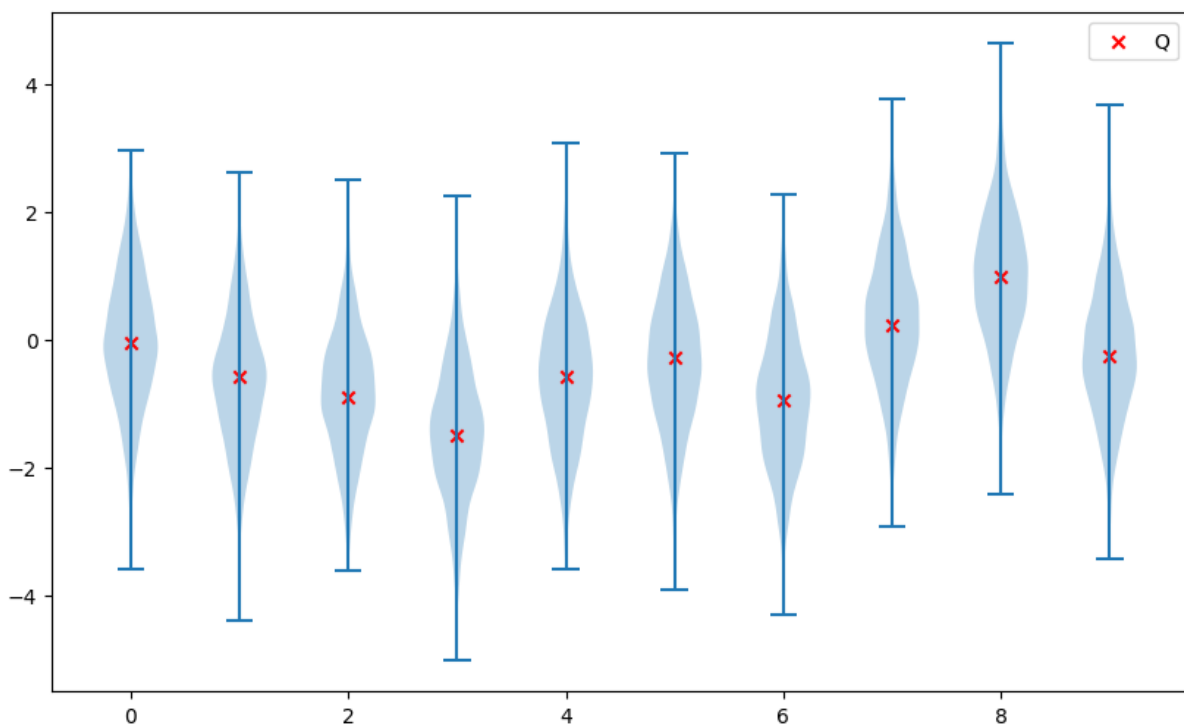
We can plot action values in order to check if they are estimated correctly:

```
_, q, _ = epsilon_greedy(env=env, epsilon=0.5, n_episodes=10000)
```

```

plt.figure(figsize=(10,6))
plt.violinplot(np.random.randn(2000, 10) + env.means, positions=range(env.size))
plt.scatter(range(env.size), q[9999], color='red', marker='x', label='Q')
plt.legend();

```



We can measure the performance and the behavior of the method over 1000 time steps when applied to a simulation of the 10-armed bandit problem. Repeating this for 2000 independent

runs, each with a different bandit problem, we obtain a measure of the learning algorithm's average behavior. The following code compares methods with different values of epsilon:

```
# Collect results over multiple repetitions
returns_exploitation = []; # pure exploitation
returns_epsilon_02 = []; # epsilon=0.2
returns_epsilon_001 = []; # epsilon=0.01
returns_exploration = []; # pure exploration

# Define the number of repetitions
repetitions = 2000

# Create multiple environments for the repetitions
envs = []
for i in range(repetitions):
    envs.append(BanditEnv());

# Run experiment for pure exploitation strategy
for i in range(repetitions):
    returns, _, _ = pure_exploitation(env=envs[i], n_episodes=1000);
    returns_exploitation.append(returns);

# Run experiment for epsilon-greedy strategy with epsilon=0.2
for i in range(repetitions):
    returns, _, _ = epsilon_greedy(env=envs[i], epsilon=0.2);
    returns_epsilon_02.append(returns);

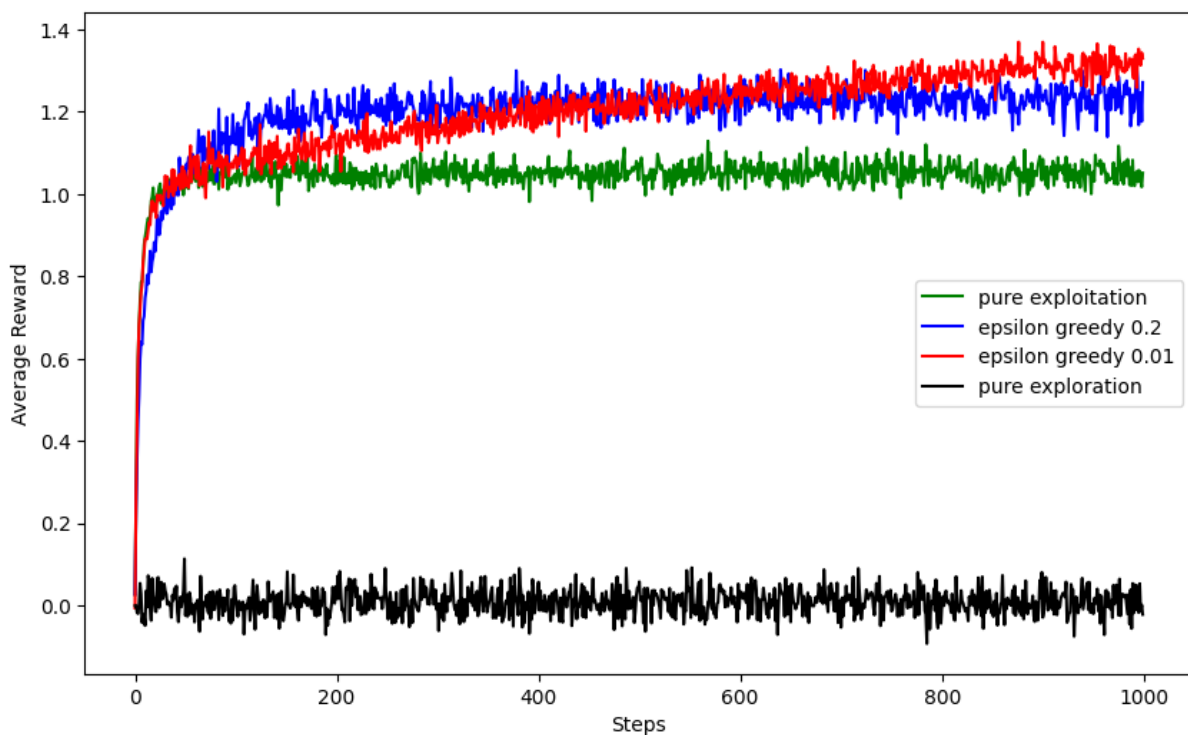
# Run experiment for epsilon-greedy strategy with epsilon=0.01
for i in range(repetitions):
    returns, _, _ = epsilon_greedy(env=envs[i], epsilon=0.01);
    returns_epsilon_001.append(returns);

# Run experiment for pure exploration strategy
for i in range(repetitions):
    returns, _, _ = pure_exploration(env=envs[i]);
    returns_exploration.append(returns);

returns_exploitation = np.array(returns_exploitation);
returns_epsilon_02 = np.array(returns_epsilon_02);
returns_epsilon_001 = np.array(returns_epsilon_001);
returns_exploration = np.array(returns_exploration);
```

```
# Average over repetitions
avg_reward_exploitation = np.average(returns_exploitation, axis=0)
avg_reward_epsilon_02 = np.average(returns_epsilon_02, axis=0)
avg_reward_epsilon_001 = np.average(returns_epsilon_001, axis=0)
avg_reward_exploration = np.average(returns_exploration, axis=0)

# Plot the results
plt.figure(figsize=(10,6))
plt.plot(avg_reward_exploitation, color='green', label='pure exploitation')
plt.plot(avg_reward_epsilon_02, color='blue', label='epsilon greedy 0.2')
plt.plot(avg_reward_epsilon_001, color='red', label='epsilon greedy 0.01')
plt.plot(avg_reward_exploration, color='black', label='pure exploration')
plt.xlabel('Steps');
plt.ylabel('Average Reward')
plt.legend()
plt.show()
```



The pure exploitation method improved **slightly faster** than the others at the very beginning, but it performs significantly **worse in the long run** because it got stuck performing suboptimal actions. The greedy methods **performs better** because they **continued to explore** and to improve their chances of recognizing the optimal action. With $\epsilon=0.2$ it exploreds more, and

usually **found the optimal action earlier**, but it never selected that action more than 80% of the time, because it explores 20% of the time; with $\epsilon=0.01$ method **improved more slowly** (it explores less), but eventually would perform better in the long run, because it explore only 1% of time. It is possible to **reduce epsilon over time** to try to get the best of both high and low values.

4.3.3 Decaying epsilon-greedy

Intuitively, **early on we can explore the most** (when the agent hasn't experienced the environment enough), **later we want the agent to exploit more and more** (as it obtains better estimates of the value functions). In the decaying method we start with a high epsilon and then we decay its value on every step. This strategy can take many forms depending on **how we change the value of epsilon** (linearly or exponentially). Notice that we can calculate all of the epsilon values at once, and only query an array of pre-computed values in the loop. The following is an exponential implementation

```
def decay_epsilon(init_value, min_value, decay_episodes, max_episodes):

    # calculate the number of the remaining episodes after the decay
    rem_episodes = max_episodes - decay_episodes

    # logspace returns numbers spaced evenly on a log scale
    # base^start is the starting value of the sequence,
    # base^stop is the final value of the sequence
    # num is the number of values to generate
    # base is the base of the log space
    values = np.logspace(start=0, stop=-2, num=decay_episodes, base=10)

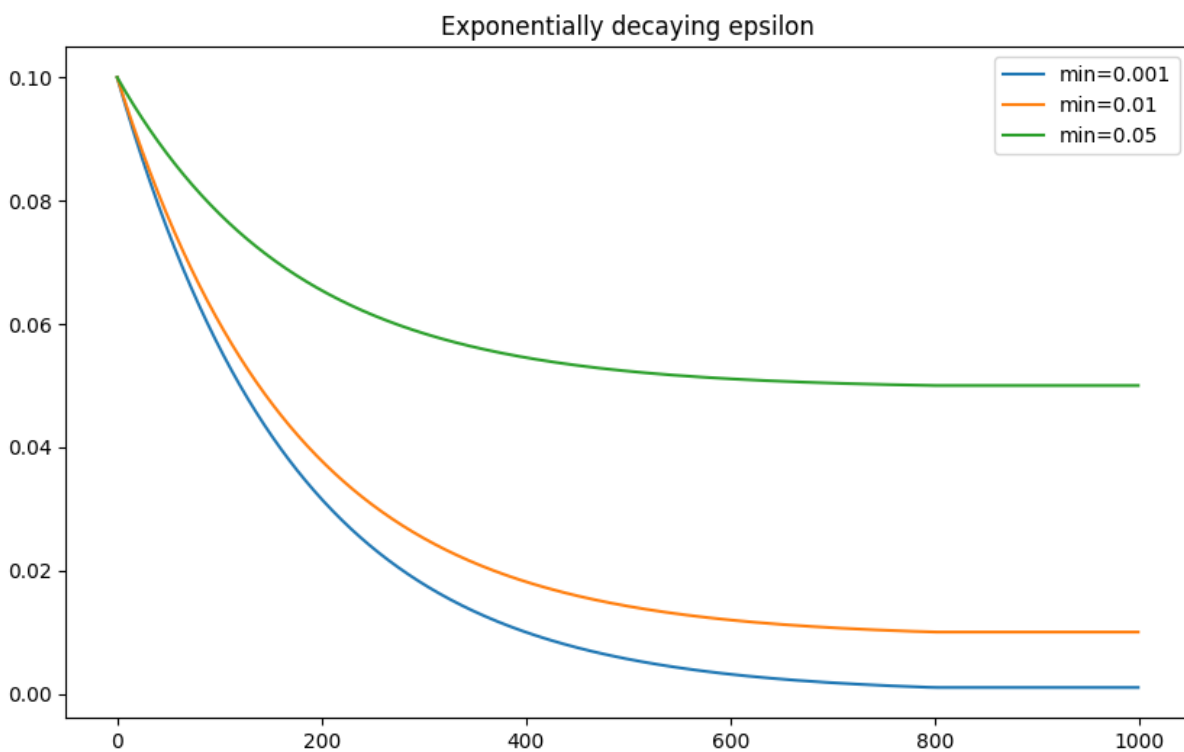
    # because the values may not end exactly at 0, given it's the log,
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.
    values = (values - values.min()) / (values.max() - values.min())

    # linear transformation and get points between init_value and min_value
    values = (init_value - min_value) * values + min_value

    # repeats the rightmost value rem_step number of times
    values = np.pad(values, (0, rem_episodes), 'edge')

    return values
```

```
# Plot decaying epsilon values
plt.figure(figsize=(10,6))
plt.plot(decay_epsilon(0.1, 0.001, 800, 1000), label='min=0.001')
plt.plot(decay_epsilon(0.1, 0.01, 800, 1000), label='min=0.01')
plt.plot(decay_epsilon(0.1, 0.05, 800, 1000), label='min=0.05')
plt.title('Exponentially decaying epsilon')
plt.legend()
plt.show()
```



```
def decay_epsilon_greedy(env, init_epsilon=1, min_epsilon=0.001, decay_episodes=100,
    ↪ n_episodes=1000):

    # The baseline boilerplate is the same as before
    q = np.zeros((env.size), dtype=float)
    n = np.zeros((env.size), dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    # Calculate the exponentially decaying epsilons.
```

```
epsilons = decay_epsilon(init_epsilon, min_epsilon, decay_episodes, n_episodes);

for e in range(n_episodes):

    if np.random.uniform() > epsilons[e]:
        action = np.argmax(q)
    else:
        action = np.random.randint(len(q))

    reward = env.step(action)

    n[action] += 1
    q[action] = q[action] + (reward - q[action])/n[action]

    qe[e] = q
    returns[e] = reward
    actions[e] = action

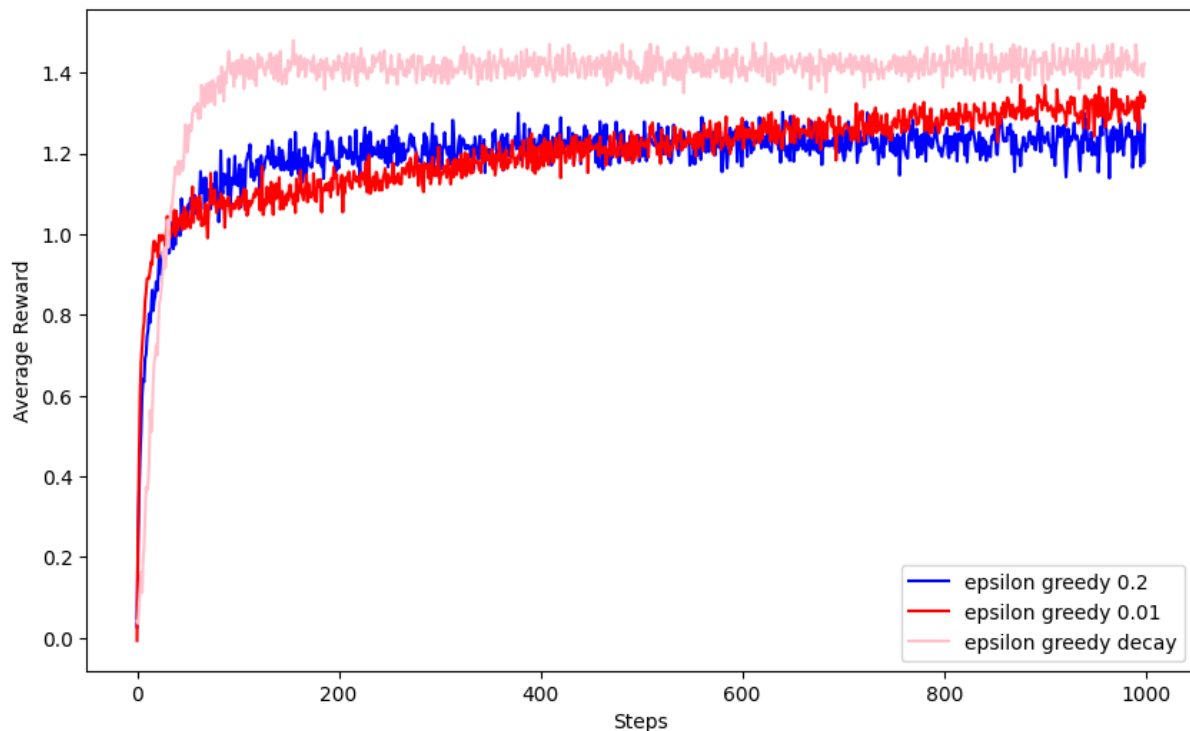
return returns, qe, actions
```

```
# Collect results over multiple repetitions
returns_epsilon_decay = [];

# Run experiment for epsilon-greedy strategy with decaying epsilon
for i in range(repetitions):
    returns, _, _ = decay_epsilon_greedy(env=envs[i]);
    returns_epsilon_decay.append(returns);

returns_epsilon_decay = np.array(returns_epsilon_decay);
avg_reward_epsilon_decay = np.average(returns_epsilon_decay, axis=0);
```

```
# Plot comparison between decaying epsilon and fixed epsilons
plt.figure(figsize=(10,6))
plt.plot(avg_reward_epsilon_02, color='blue', label='epsilon greedy 0.2')
plt.plot(avg_reward_epsilon_001, color='red', label='epsilon greedy 0.01')
plt.plot(avg_reward_epsilon_decay, color='pink', label='epsilon greedy decay')
plt.xlabel('Steps');
plt.ylabel('Average Reward')
plt.legend()
plt.show()
```



The advantage of the decaying approach is evident in the graph, it quickly finds optimal actions and stabilizes at a higher average reward compared to the fixed epsilon strategies. The reward curve for this strategy is smoother and consistently higher than the fixed epsilon strategies, indicating that it **effectively balances exploration and exploitation over time**. There are many other ways we can handle the decaying of epsilon. The bottom line is that **the agent should explore with a higher chance early and exploit with a higher chance later**. Early on, there's a high likelihood that value estimates are wrong. Still, as time passes and he acquires knowledge, the likelihood that the value estimates are close to the actual values increases, which is when the agent should explore less frequently so that he can exploit the knowledge acquired.

4.3.4 Optimistic initial values

Initial action values can be used as a simple way to **encourage exploration**. Suppose to set an initial estimate in a **optimistic way**. Whichever actions are initially selected, the reward is less than the starting estimates and the learner switches to other actions, being "**disappointed**" with the rewards it is receiving. The result is that **all actions are tried several times** before the value estimates converge. The system does a fair amount of exploration even if greedy actions are selected all the time. We call this technique for encouraging exploration **optimistic initial values**. The mechanic of the strategy is straightforward: we initialize the action-value function to a high value and act greedily using these estimates:


```
def optimistic_initialization(env, optimistic_estimate=5.0, initial_count=10,
    ↪ n_episodes=1000):

    # start initializing the action-values to an optimistic value
    q = np.full((env.size), optimistic_estimate, dtype=float)

    # initialize the counts that will serve as an uncertainty measure
    # the higher the more certain.
    n = np.full((env.size), initial_count, dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    for e in range(n_episodes):

        action = np.argmax(q)

        reward = env.step(action)

        n[action] += 1
        q[action] = q[action] + (reward - q[action])/n[action]

        qe[e] = q
        returns[e] = reward
        actions[e] = action

    return returns, qe, actions
```

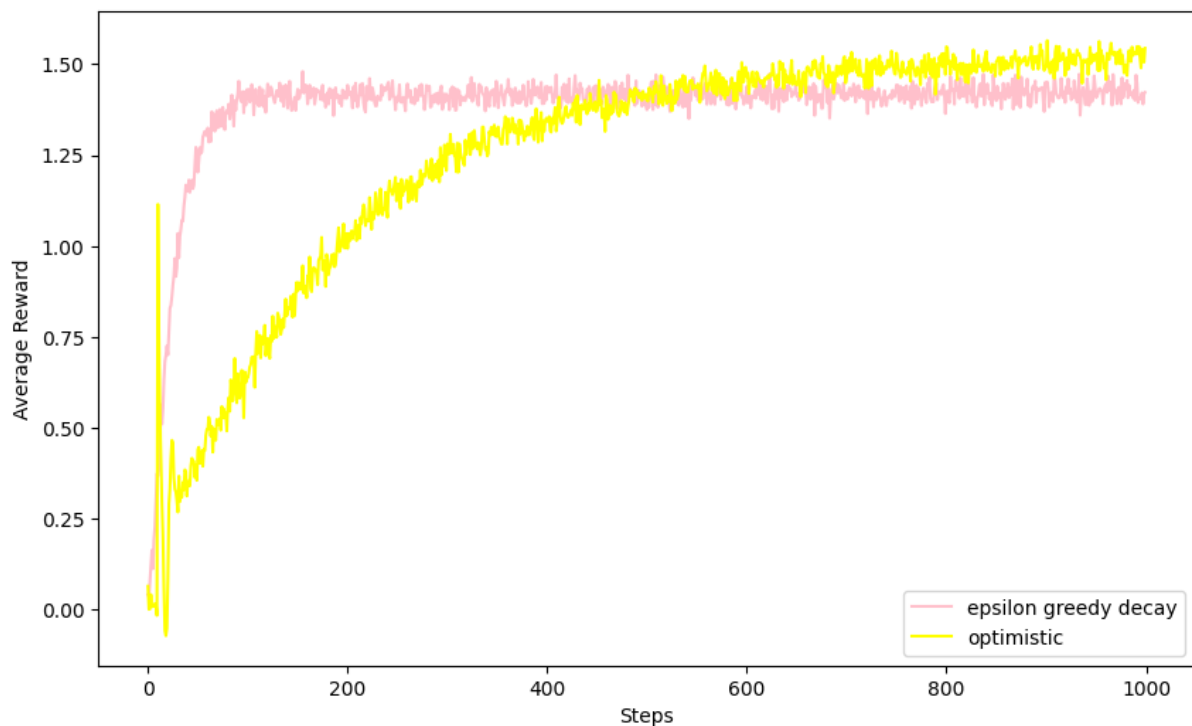
```
# Collect results over multiple repetitions
returns_optimistic = [];

# Run experiment for optimistic initialization strategy
for i in range(repetitions):
    returns, _, _ = optimistic_initialization(env=envs[i])
    returns_optimistic.append(returns);

returns_optimistic = np.array(returns_optimistic)

avg_reward_optimistic = np.average(returns_optimistic, axis=0)
```

```
# Plot comparison between decaying epsilon and optimistic initialization
plt.figure(figsize=(10,6))
plt.plot(avg_reward_epsilon_decay, color='pink', label='epsilon greedy decay')
plt.plot(avg_reward_optimistic, color='yellow', label='optimistic')
plt.xlabel('Steps');
plt.ylabel('Average Reward')
plt.legend()
plt.show()
```



Initially, the optimistic method performs worse because it explores more. In the early part of the curve for the optimistic method we can see **oscillations and spikes**. If the initial action selected are by chance ones of the better choices then the action value estimates for these plays will be magnified resulting in an emphasis to continue playing this action. This results in large actions values being received on the initial draws and consequently very good initial play. In the same way, if the algorithm initially selects poor plays then initially the algorithm will perform poorly resulting in very poor initial play. We regard it as a **simple trick** that can be **quite effective on stationary problems**, but it is not well suited to non-stationary problems because its drive for exploration is inherently temporary.

4.4 Strategic Exploration

While humans explore, they don't explore randomly. Maybe imprecision is the source of randomness, but humans have a **strategic way of exploring**. We know that we're sacrificing short-term for long-term satisfaction. **We know we want to acquire more information**. We explore by trying things **we haven't sufficiently tried**, but **have the potential to better our lives**. Perhaps, our exploration strategies are a combination of estimates and their uncertainty. For instance, we might prefer a dish that we're likely to enjoy, and we haven't tried, over a dish that we like okay, but we get every weekend. Perhaps we explore **based on our curiosity or our prediction error**. For instance, we might be more inclined to try new dishes at a restaurant that we thought would be okay-tasting food, but it resulted in the best food you ever had. That **prediction error** and that **surprise** could be our metric for exploration. We can consider more advanced exploration strategies that apply randomness in proportion to the current estimates of the actions or that take into account the confidence and uncertainty levels of the estimates. The epsilon-greedy strategy (and its decaying versions) is still the most popular exploration strategy in use today, perhaps because it performs well, perhaps because of its simplicity. Balancing the exploration versus exploitation trade-off, the gathering and utilization of information is central to human intelligence, artificial intelligence, and reinforcement learning.

4.4.1 Softmax method

Exploration makes more sense if it takes into account action-value estimates: **if there is an action that has a really low estimate, we're less likely to try it**. The Softmax strategy **samples actions from a probability distribution over the action-value estimates**, such that the probability of selecting an action is proportional to its current action-value estimate. The difference between estimates will create a tendency to select actions with the highest estimates more often, and actions with the lowest estimates less frequently. We can also add a **hyperparameter to control the algorithm's sensitivity** to the differences in estimates. That hyperparameter, called the **temperature**, works in such a way that as it approaches infinity, the preferences over the action-values are equal (it samples uniformly), and as it approaches zero, the action with the highest estimated value will be sampled with probability of one. Also, we can decay this hyperparameter either linearly, exponentially, or another way.

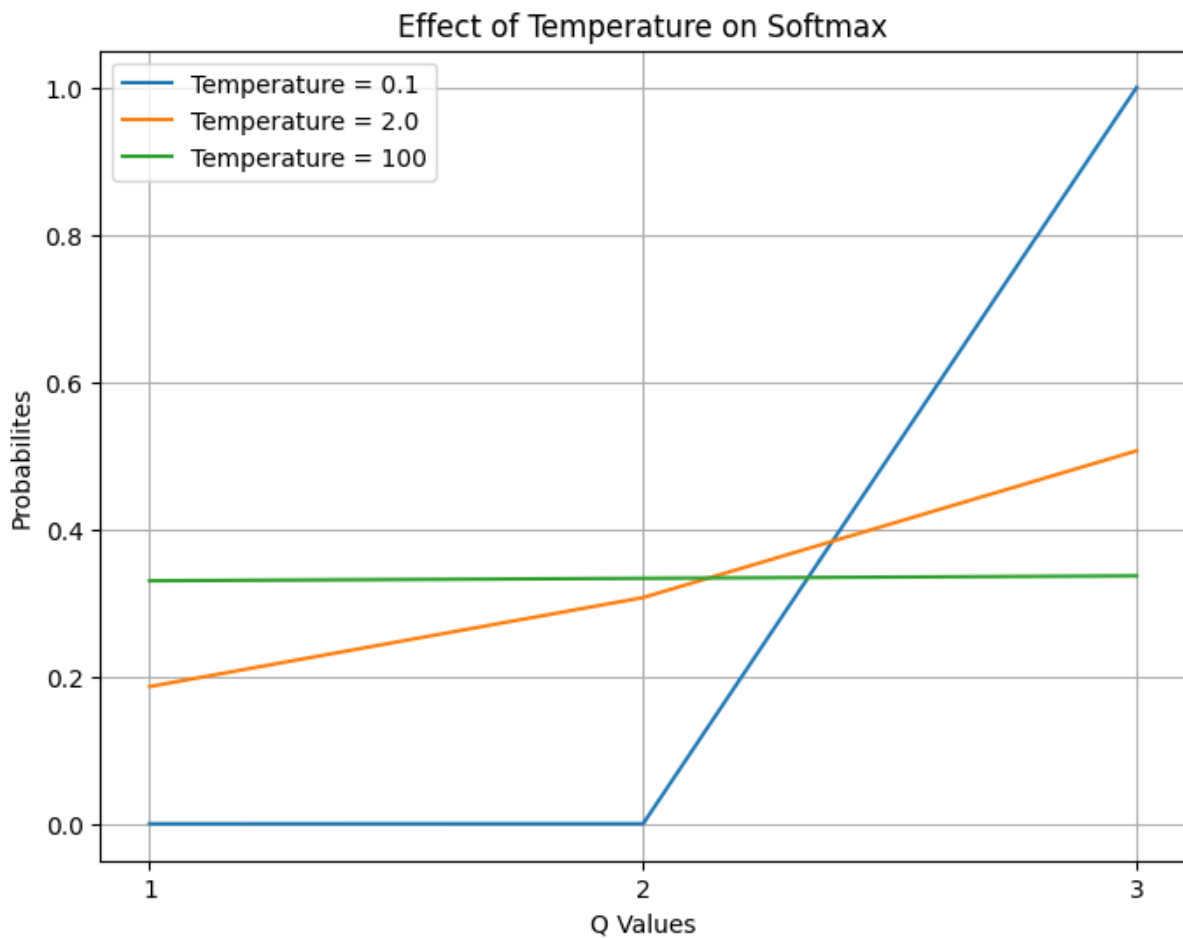
$$\Pr(a) = \frac{e^{\frac{q(a)}{\tau}}}{\sum_{b=1}^B e^{\frac{q(b)}{\tau}}}$$

```
def softmax_function(q, temperature=1.0):  
    exp = np.exp(q / temperature);  
    return exp / exp.sum(axis=0);
```

In order to show the temperature effect (lower temperatures make outputs closer to 1 or 0, while higher temperatures spread out the probabilities more evenly), we're plotting the softmax output for different temperatures to show how this affects the distribution of the probabilities:

```
# define some input values  
q = np.array([1.0, 2.0, 3.0])  
  
# define a range of temperature values  
temperatures = [0.1, 2.0, 100]  
  
# plot the softmax output for different temperatures  
plt.figure(figsize=(8, 6))  
  
for temp in temperatures:  
    probabilitis = softmax_function(q, temperature=temp)  
    plt.plot(q, probabilitis, label=f'Temperature = {temp}')
```

```
# Plot settings  
plt.title('Effect of Temperature on Softmax')  
plt.xlabel('Q Values')  
plt.ylabel('Probabilites')  
plt.xticks(q)  
plt.legend()  
plt.grid(True)  
plt.show()
```



Now we can write the exploitation-exploration trade-off using the softmax method:

```
def softmax(env, init_temp=1000, min_temp=0.1,
            decay_ratio=0.04, n_episodes=1000):

    q = np.zeros((env.size), dtype=float)
    n = np.zeros((env.size), dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    for e in range(n_episodes):

        # calculate the linearly decaying temperature
        decay_episodes = n_episodes * decay_ratio
        temp = 1 - e / decay_episodes
```

```
temp *= init_temp - min_temp
temp += min_temp
temp = np.clip(temp, min_temp, init_temp)

# calculate the probabilities by applying the softmax function
# to the estimates
scaled_q = q / temp
norm_q = scaled_q - np.max(scaled_q)
exp_q = np.exp(norm_q)
probs = exp_q / np.sum(exp_q)

# select the action based on probabilities
action = np.random.choice(np.arange(len(probs)), size=1, p=probs)[0]

reward = env.step(action)

n[action] += 1
q[action] = q[action] + (reward - q[action])/n[action]

qe[e] = q
returns[e] = reward
actions[e] = action

return returns, qe, actions
```

4.4.2 Upper Confidence Bound method

The optimistic initialization strategy is a clever and efficient approach, however, it has two significant inconveniences. First, **we don't know the maximum reward the agent can obtain**. If we set the initial optimistic estimates to a value much higher than its actual value, the algorithm will perform sub-optimally for a long time, because the agent will take many episodes to bring the estimates near the actual values. Even worse, if we set the initial values lower than the maximum, the algorithm will no longer be optimistic, and it will no longer work. The second issue is that **the counts variable is a hyperparameter and it needs tuning**, but what we're trying to represent with this variable **is the uncertainty of the estimate**, which shouldn't be a hyperparameter. A better strategy, that follows the same principles of optimistic initialization, is the **upper confidence bound (UCB)** which use statistical techniques to calculate the **value estimates uncertainty** and uses that as a bonus for exploration. In UCB, we're still optimistic, but it's **a more a realistic optimism**. Instead of blindly hoping for the best, we look at the uncertainty of value estimates. **The more uncertain an estimate, the more critical it is to explore it.**

Notice that it's no longer about believing the value will be the "maximum possible", though it might be! The new metric that we care about here is uncertainty, we want **to give uncertainty the benefit of the doubt**. To select the action we add the estimates and an uncertainty bonus:

$$A_t = \underset{a}{\operatorname{argmax}} [q_t(a) + c * u_t(a)]$$

where $q(a)$ **represents the value** term and $u(a)$ is a term that **captures the uncertainty**, which could depend on factors like the number of times action has been selected. This formula reflects the general idea that we decide for an action with a combination of its value and an additional boost based on how uncertain the estimate of that value is. The uncertainty term can be expressed as:

$$u_t(a) = \sqrt{\left[\frac{\ln t}{N_t(a)} \right]}$$

The idea is that each time an action is selected the uncertainty is presumably reduced (the count increments and the uncertainty term decreases). On the other hand, each time another action is selected, t increases but the count does not and the uncertainty estimate increases. The use of the logarithm means that the increases get smaller over time, but are unbounded. Composing the two contributions:

$$A_t = \underset{a}{\operatorname{argmax}} \left[q_t(a) + c \sqrt{\left[\frac{\ln t}{N_t(a)} \right]} \right]$$

All actions will eventually be selected, but actions with lower value estimates or that have already been selected frequently, will be selected with decreasing frequency over time. UCB often performs well but is more difficult than epsilon-greedy to extend beyond bandits to the more general reinforcement learning settings.

```
def upper_confidence_bound(env, c=2, n_episodes=1000):

    q = np.zeros((env.size), dtype=float)
    n = np.zeros((env.size), dtype=int)

    qe = np.empty((n_episodes, env.size), dtype=float)
    returns = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    for e in range(n_episodes):
        # first select all actions once to avoid division by zero
        if e < len(q):
            action = e
        else:
            # proceed to calculating the confidence bounds
```

```
        u = np.sqrt(c * np.log(e)/n)
        # pick the action with the highest value with an
        # uncertainty bonus: the more uncertain the value of the action,
        # the higher the bonus
        action = np.argmax(q + u)

    reward = env.step(action)

    n[action] += 1
    q[action] = q[action] + (reward - q[action])/n[action]

    qe[e] = q
    returns[e] = reward
    actions[e] = action

    return returns, qe, actions
```

```
# Collect results over multiple repetitions
returns_softmax = [];
returns_ucb = [];

# Run experiment for softmax strategy
for i in range(repetitions):
    returns, _, _ = softmax(env=envs[i]);
    returns_softmax.append(returns);

# Run experiment for UCB strategy
for i in range(repetitions):
    returns, _, _ = upper_confidence_bound(env=envs[i]);
    returns_ucb.append(returns);

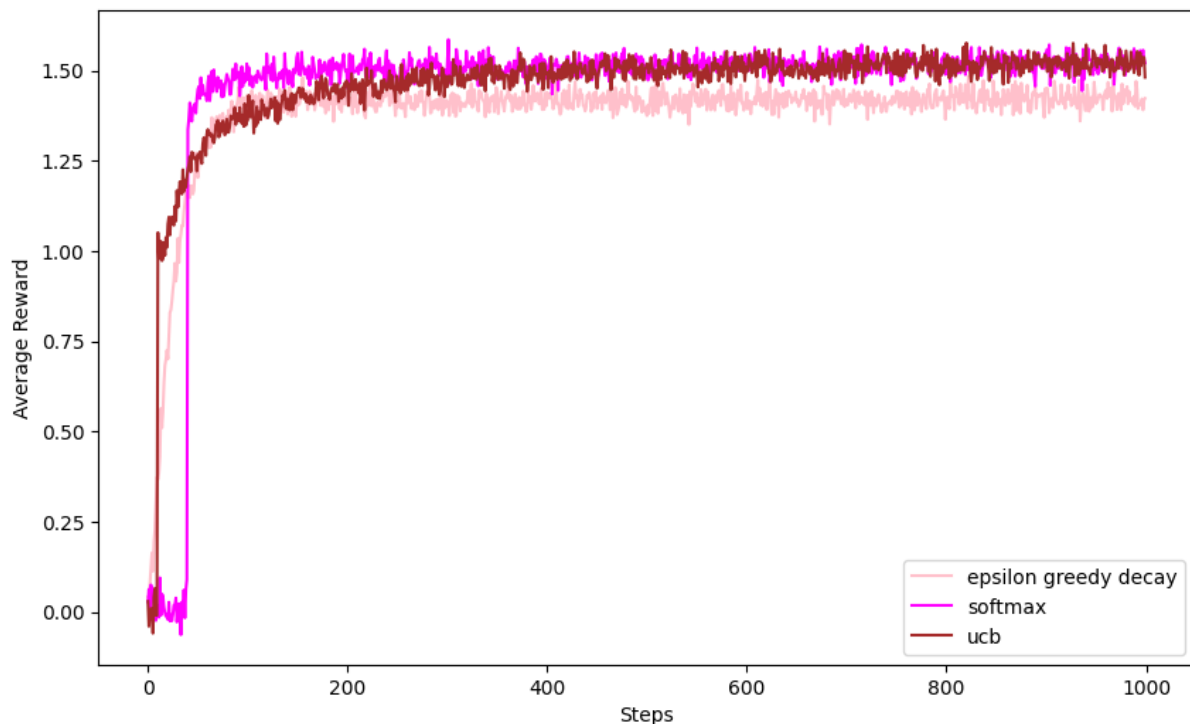
returns_softmax = np.array(returns_softmax);
returns_ucb = np.array(returns_ucb);

avg_reward_softmax = np.average(returns_softmax, axis=0)
avg_reward_ucb = np.average(returns_ucb, axis=0)
```

```
# Plot comparison between decaying epsilon, softmax and ucb
plt.figure(figsize=(10,6))
plt.plot(avg_reward_epsilon_decay, color='pink', label='epsilon greedy decay')
plt.plot(avg_reward_softmax, color='magenta', label='softmax')
```



```
plt.plot(avg_reward_ucb, color='brown', label='ucb')
plt.xlabel('Steps');
plt.ylabel('Average Reward')
plt.legend()
plt.show()
```



Softmax exploration selects actions probabilistically based on their estimated value, favoring better actions but still allowing exploration. UCB balances exploration and exploitation by choosing actions based on their estimated reward and uncertainty. Both algorithms quickly reach a high reward and maintain strong performance over time. They perform better than the epsilon-greedy decay, however they are more complex and require more tuning.

4.5 Comparison

We have presented several ways of balancing exploration and exploitation. The epsilon-greedy methods choose randomly a small fraction of the time, UCB methods choose deterministically, but achieve exploration by subtly favoring at each step the actions that have so far received fewer samples. We can run simulations for all methods in order to compare their performances. A complication is that they **all have parameters**. To get a meaningful comparison we have to consider their performance as a function of their parameter. We summarize a complete learning

curve by its average value over the 1000 steps and shows this measure for the various algorithms, each as a function of its own parameter shown on a single scale on the x-axis. This kind of graph is called a **parameter study**:

```
def run_experiment(envs, algorithm, param):
    returns = []

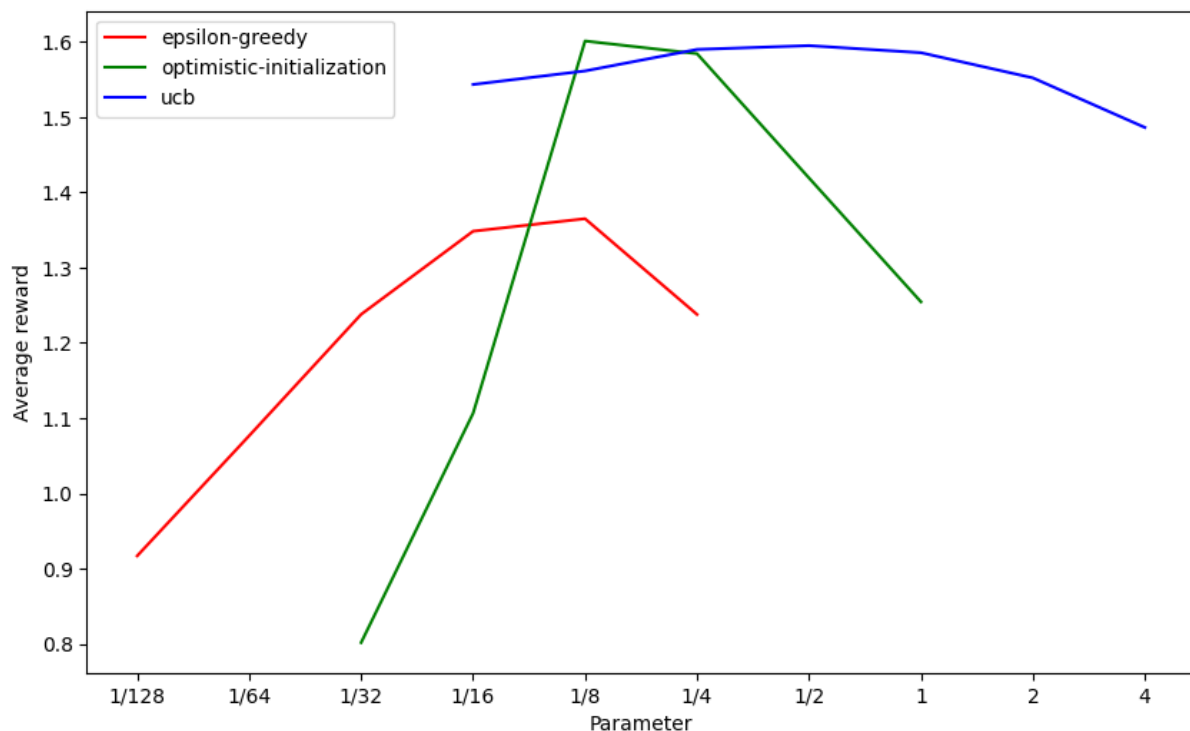
    for i in range(2000):
        if algorithm == 'epsilon-greedy':
            r, _, _ = epsilon_greedy(env, epsilon=param, n_episodes=1000);
        elif algorithm == 'optimistic-initialization':
            r, _, _ = optimistic_initialization(env, optimistic_estimate=param,
↪ initial_count=10, n_episodes=1000);
        elif algorithm == 'softmax':
            r, _, _ = softmax(env, init_temp=param, min_temp=0.1,
↪ decay_ratio=0.04, n_episodes=1000);
        elif algorithm == 'ucb':
            r, _, _ = upper_confidence_bound(env, c=param, n_episodes=1000);
        else:
            raise ValueError('Unknown algorithm:', algorithm)
        returns.append(r);
    returns = np.array(returns);
    return returns.mean()
```

```
# Run experiments for epsilon-greedy with different parameters
egreedy_x, egreedy_y = [], []
for param in [1/128, 1/64, 1/32, 1/16, 1/8, 1/4]:
    result = run_experiment(envs=envs, algorithm='epsilon-greedy', param=param);
    egreedy_x.append(param)
    egreedy_y.append(result)
```

```
# Run experiments for optimistic initialization with different parameters
opt_x, opt_y = [], []
for param in [1/4, 1/2, 1, 2, 4, 6]:
    result = run_experiment(envs=envs, algorithm='optimistic-initialization',
↪ param=param)
    opt_x.append(param)
    opt_y.append(result)
```

```
# Run experiments for UCB with different parameters
ucb_x, ucb_y = [], []
for param in [1/16, 1/8, 1/4, 1/2, 1, 2, 4]:
    result = run_experiment(envs=envs, algorithm='ucb', param=param)
    ucb_x.append(param)
    ucb_y.append(result)

# Plot performances variation with parameters
plt.figure(figsize=(10,6))
plt.xticks(range(-7, 3), ('1/128', '1/64', '1/32', '1/16', '1/8', '1/4', '1/2', '1',
    '2', '4'))
plt.plot(range(-7, -1), egreedy_y, color='red', label='epsilon-greedy')
plt.plot(range(-5, 1), opt_y, color='green', label='optimistic-initialization')
plt.plot(range(-4, 3), ucb_y, color='blue', label='ucb')
plt.xlabel('Parameter')
plt.ylabel('Average reward')
plt.legend()
plt.show()
```



Overall, on this problem, **UCB seems to perform best**. Notice the characteristic **inverted-U shapes** of each algorithm's performance: all the algorithms perform best at an **intermediate**

value of their parameter, neither too large nor too small. In assessing a method, we should attend not just to how well it does at its best parameter setting, but also to **how sensitive it is to its parameter value**. All of these algorithms are fairly insensitive, performing well over a range of parameter values varying by about an order of magnitude.

5 Policy Evaluation

Balancing immediate and long-term goals is challenging. We experience this point every day: should I watch movies tonight or keep studying reinforcement learning? The first choice has an **immediate satisfaction**, the second on the other hand, won't really give us much tonight, but **maybe (and only maybe)** will provide much **higher satisfaction in the long term**. How much more satisfaction in the long term, exactly, we may ask. I don't know, you don't know, and we won't know unless we try it out, **unless we explore it**. **Life doesn't give us its MDP!** life is uncertain. We need to balance information gathering and information utilization, **taking into account the long-term effect and not only the current situation**. We will study agents that can **learn to estimate the value of policies**, similar to the DP policy-evaluation method, but this time without the MDP. We need to recap some important concepts:

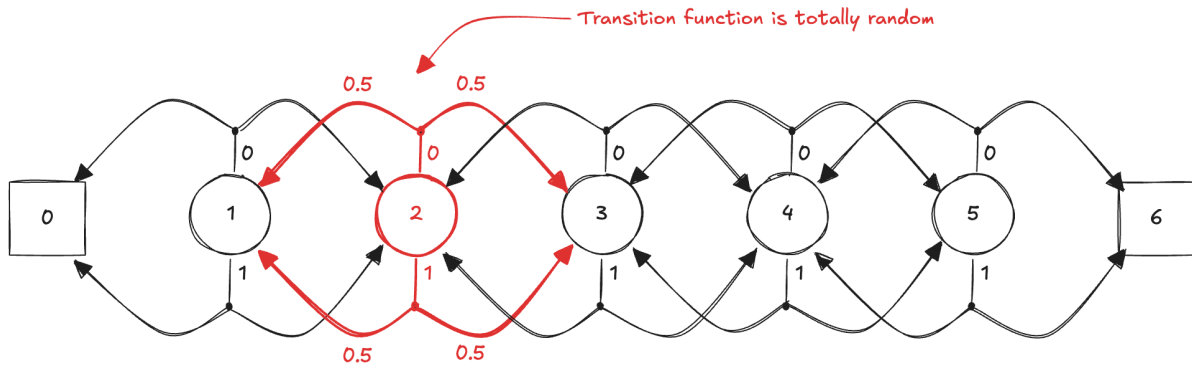
- The **reward** refers to the one-step signal the agent gets: the agent observes a state, selects an action, and receives a reward signal. It is the core of RL, but **it is not what the agent is trying to maximize**. While our agent maximizes the one-step reward, in the long-term, it's getting less than it could.
- The **return** refers to the total discounted rewards, from any state until the end of the episode. It is a better indicator of performance, because it contains a long-term sequence, an history of rewards. But the return **isn't what an agent tries to maximize**, either. An agent who attempts to obtain the highest possible return may find a policy that takes it through a noisy path; sometimes, this path will provide a high return, but perhaps most of the time a low one.
- The **value function** refers to the **expectation of return**. We want high returns, but high on average (expectation). If the agent is in a noisy environment, or if the agent is using a stochastic policy, it's all fine.

The agent is trying to maximize the expected total discounted reward: the value function.

5.1 Random Walk Environment

A single-row grid-world environment, with five non-terminal states. The probability of going left when taking the "left" action is equal to the probability of going right, and the probab-

ity of going right when taking the "right" action is equal to the probability of going left. In other words, **the agent has no control of where it goes**. This environment is useful **to focus on the prediction problem**, in which there is no need to distinguish the dynamics due to the environment from those due to the agent.



```
import numpy as np

class RandomWalk:
    """
    State Index:  [ 0   1   2   3   4   5   6 ]
    State Label:  [ .   A   B   C   D   E   . ]
    Type:         [ T   .   .   S   .   .   T ]
    """
    v_true = [0.0, 1/6, 2/6, 3/6, 4/6, 5/6, 0.0]

    def __init__(self):
        self.reset()

    def reset(self):
        self.observation_space = 7
        self.state = np.random.choice([1,2,3,4,5])
        self.terminated = False
        return self.state

    def step(self, action):
        self.state += np.random.choice([-1, 1])

        reward = 0

        if self.state < 1:
            self.terminated = True
```

```

if self.state > 5:
    self.terminated = True;
    reward = 1

return self.state, reward, self.terminated, 0, 0 # obs, rew, terminated,
        truncated, info

```

```
random_walk = RandomWalk()
```

The dynamics of this environment make policy being evaluated irrelevant, so we select an all-left policy:

```

def pi(state):
    return 1

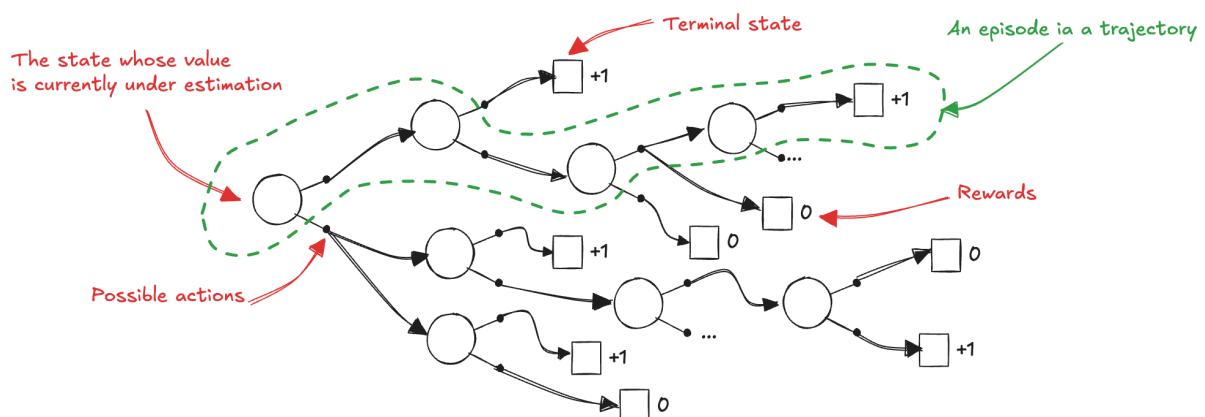
```

5.2 Improving estimates after each episode: Monte Carlo

The goal is to estimate the state-value function of a policy (the expectation of returns):

$$v_{\pi}(s) = E_{\pi}(G_{t:T} | S_t = s)$$

The most straightforward approach that comes to mind is **to run several episodes** with the policy under evaluation, **collecting hundreds of trajectories** (collections of **experiences** like [state, action, reward, next state]) and then **calculate averages** for every state.



This method is called **Monte Carlo prediction (MC)** and it is easy to implement. Given a trajectory:

$$S_t, A_t, R_{t+1}, S_{t+1}$$

$$S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}$$

...

$$S_{T-1}, A_{T-1}, R_T, S_T$$

We calculate the return for every state encountered by adding and discounting the rewards received along the way, until the end of the trajectory at time step T:

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

Then, add up the per-state returns and increment a count:

$$T(S_t) = T(S_t) + G_{t:T}$$

$$N(S_t) = N(S_t) + 1$$

Finally, we can estimate the expectation using the empirical mean:

$$V(S_t) = \frac{T(S_t)}{N(S_t)}$$

As the counts approach infinity, the estimate will approach the true value:

$$N(s) \rightarrow \infty \Rightarrow V(s) \rightarrow v_{\pi}(s)$$

Notice that means can be calculated incrementally, there's no need to keep track of the sum of returns for all states. This equation is equivalent and more efficient:

$$V(S_t) = V(S_t) + \frac{G_{t:T} - V(S_t)}{N(S_t)}$$

On this, we can replace the mean for a learning value that can be time dependent or constant:

$$V_{k-1}(S_t) = V_k(S_t) + \alpha_t [G_{t:T} - V_k(S_t)]$$

Where $G_{t:T}$ is the **MC target** and $G_{t:T} - V(S_t)$ is the **MC error**. Notice that V is calculated only at the end of an episode, because the returns depends on it. In practice, there are two different ways of implementing the algorithm. A single trajectory **may contain multiple visits to the same state**. Should we calculate the returns following each of those visits independently and then **include all** of those targets in the averages, or should we only use the **first visit** to each state? Both are valid approaches, and they have similar theoretical properties. The more standard version is **first-visit Monte Carlo (FVMC)**, and its convergence properties are easy to justify. **Every-visit Monte Carlo (EVMC)** is slightly different but has also been proven to converge given infinite samples.

We can implement this method in Python. First of all, we write a function in order to calculate an exponentially decaying schedule to calculate all the values for alpha for the full process, we use the NumPy "logspace" function that returns evenly spaced numbers on a log scale. This hyperparameter is essential. **Often, alpha is a positive constant less than 1**. Having a **constant**

alpha helps with learning in non-stationary environments. However, we can decay alpha in order to **show convergence**: alpha starts at a value, then it decreases exponentially to a min value in a number of episodes out of the total episodes. In that way alpha helps the algorithms get close to converging, but because alpha don't goes to zero, they don't fully converge. We will use the same schedule in all algorithms.

```
import numpy as np

def decay_alpha(init_value, min_value, decay_steps, max_steps):

    # calculate the number of the remaining steps after the decay
    rem_steps = max_steps - decay_steps

    # logspace returns numbers spaced evenly on a log scale
    # base^start is the starting value of the sequence,
    # base^stop is the final value of the sequence
    # num is the number of values to generate
    # base is the base of the log space
    values = np.logspace(start=0, stop=-2, num=decay_steps, base=10)

    # because the values may not end exactly at 0, given it's the log,
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.
    values = (values - values.min()) / (values.max() - values.min())

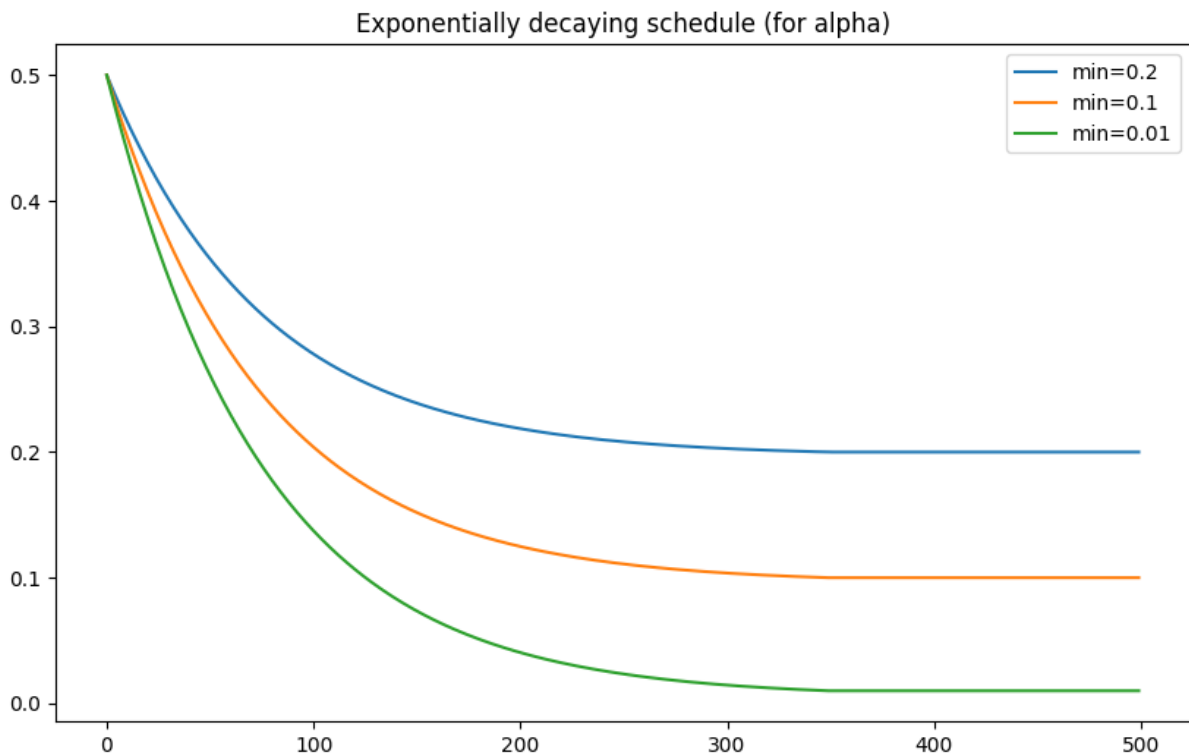
    # linear transformation and get points between init_value and min_value
    values = (init_value - min_value) * values + min_value

    # repeats the rightmost value rem_step number of times
    values = np.pad(values, (0, rem_steps), 'edge')

    return values
```

```
import matplotlib.pyplot as plt

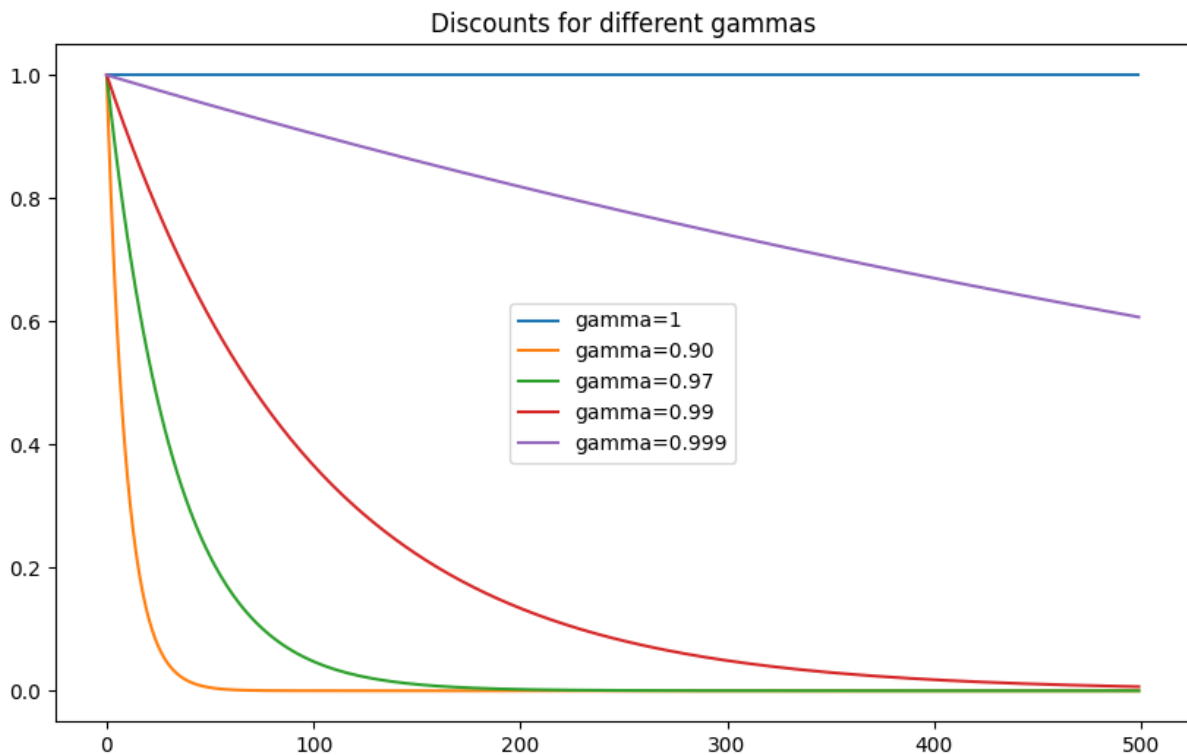
plt.figure(figsize=(10,6))
plt.plot(decay_alpha(0.5, 0.2, 350, 500), label='min=0.2')
plt.plot(decay_alpha(0.5, 0.1, 350, 500), label='min=0.1')
plt.plot(decay_alpha(0.5, 0.01, 350, 500), label='min=0.01')
plt.title('Exponentially decaying schedule (for alpha)')
plt.legend()
plt.show()
```

We need to discount the rewards collected during a trajectory with an exponentially decaying discount factor ($\gamma^0, \gamma^1, \gamma^2, \dots$), so we can reuse the "linspace" function to calculate all the discount values for the full process:

```
def decay_discounts(gamma, max_steps):
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma,
↪ endpoint=False);
    return discounts
```

```
plt.figure(figsize=(10,6))
plt.plot(decay_discounts(1, 500), label='gamma=1')
plt.plot(decay_discounts(0.90, 500), label='gamma=0.90')
plt.plot(decay_discounts(0.97, 500), label='gamma=0.97')
plt.plot(decay_discounts(0.99, 500), label='gamma=0.99')
plt.plot(decay_discounts(0.999, 500), label='gamma=0.999')
plt.title('Discounts for different gammas')
plt.legend()
plt.show()
```



Then we write a function to run a policy over an environment and to extract a trajectory:

```
def generate_trajectory(pi, env, max_steps=200):  
  
    # list of experiences (trajectory)  
    trajectory = []  
  
    done = False  
    steps = 0  
  
    # reset the environment to interact in a new episode  
    state = env.reset()  
  
    # looping through until the done flag is set to true  
    while not done:  
        steps += 1;  
  
        # use the policy function to pick an action  
        action = pi(state)  
  
        # step the environment using that action  
        next_state, reward, terminated, truncated, info = env.step(action)
```

```
    if(terminated or truncated):
        done = True;

    # append the experience to the trajectory
    experience = (state, action, reward, next_state, done)
    trajectory.append(experience)

    # if we hit a terminal state break and return
    if done:
        break;

    # truncate long trajectories
    if steps >= max_steps:
        break;

    # update the state
    state = next_state

# return the trajectory
return np.array(trajectory, object)
```

Now we can write the MC prediction function using the first-visit version:

```
def mc(pi, env, gamma=1.0, init_alpha=0.5, min_alpha=0.01, decay_episodes=350,
    ↪ n_episodes=500, max_steps=200):
    # calculate all discounts at once
    discounts = decay_discounts(gamma, max_steps);

    # calculate all alphas at once
    alphas = decay_alpha(init_alpha, min_alpha, decay_episodes, n_episodes);

    # initialize the current estimate of the state-value function
    v = np.zeros(env.observation_space, dtype=float)

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # loop for every episode
    for e in range(n_episodes):

        # generate a trajectory
        trajectory = generate_trajectory(pi, env, max_steps)
```

```

# initialize a visits check vector
visited = np.zeros(env.observation_space, dtype=bool)

# loop through all experiences in the trajectory
for t, (state, _, reward, _, _) in enumerate(trajectory):

    # check if the state has already been visited
    if visited[state]:
        continue;
    visited[state] = True

    # calculate the number of steps from t to T
    n_steps = len(trajectory[t:])

    # calculate the return
    g = np.sum(discounts[:n_steps] * trajectory[t:, 2])

    # estimate the value function
    mc_target = g
    mc_error = mc_target - v[state]
    v[state] = v[state] + alphas[e] * mc_error

    # keep track of the episode V
    v_track[e] = v;

return v.copy(), v_track

```

We can use the Monte Carlo prediction algorithm and show the estimates over episodes on the Random Walk environmet:

```
v_mc, v_mc_track = mc(pi, random_walk, n_episodes=500);
```

```
print(v_mc)
```

```

[0.          0.13701912  0.33570019  0.53337701  0.70333317  0.87791777
 0.          ]

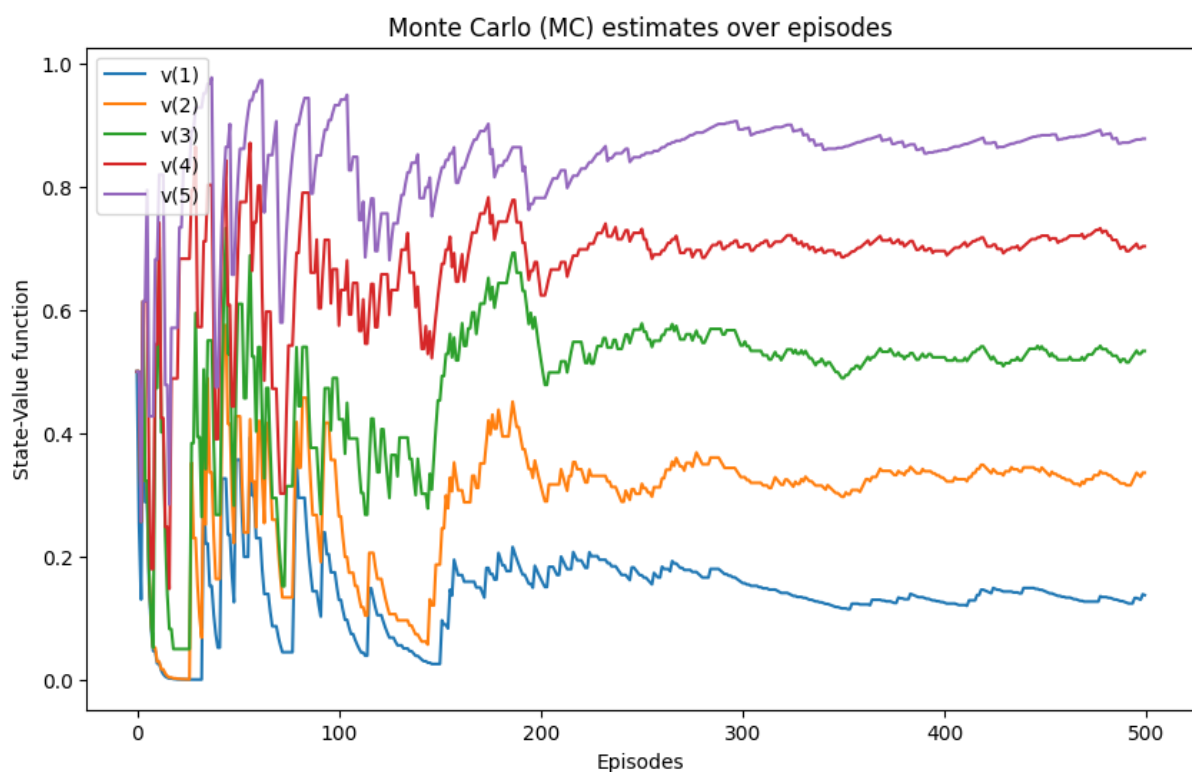
```

```
print(random_walk.v_true)
```

```
[0.0, 0.16666666666666666, 0.3333333333333333, 0.5, 0.6666666666666666, 0.8333333333333334,
```

We can plot the estimates for the Random Walk environment for some of the states during the process:

```
plt.figure(figsize=(10,6))
legends = ['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)']
plt.plot(v_mc_track[:,1:6])
plt.title('Monte Carlo (MC) estimates over episodes')
plt.ylabel('State-Value function')
plt.xlabel('Episodes')
plt.legend(legends)
plt.show()
```



The algorithm shows near-convergence to the true values. However, its **running estimates are very noisy**, they jump back and forth around the true values. We can run an experiment to calculate the **learning curve** to measure the performance using the root mean-squared (RMS)

error between the value function learned and the true value function, averaged over the five states, then averaged over 100 runs:

```
from collections import defaultdict

def run_experiment(algorithm, runs, env, pi):
    v_runs = []
    for i in range(runs):
        _, v_td_track = algorithm(pi, env)
        v_runs.append(v_td_track)
    v_runs = np.array(v_runs)

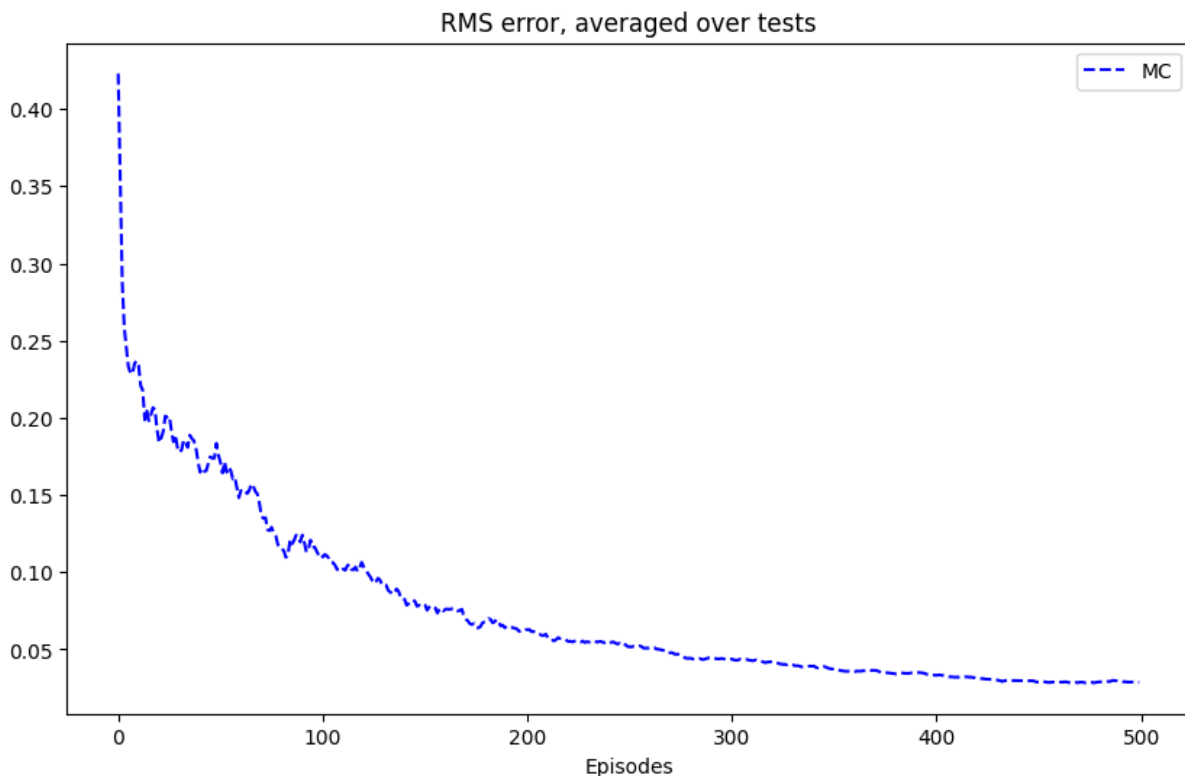
    # remove data about terminal states (which is always zero anyway)
    v_runs = v_runs[:, :, 1:-1]

    error_to_true = v_runs - env.v_true[1:-1]
    squared_error = np.power(error_to_true, 2)
    mean_squared_error = np.average(squared_error, axis=-1)
    root_mean_squared_error = np.sqrt(mean_squared_error)
    rmse_avg_over_runs = np.average(root_mean_squared_error, axis=0)

    return rmse_avg_over_runs
```

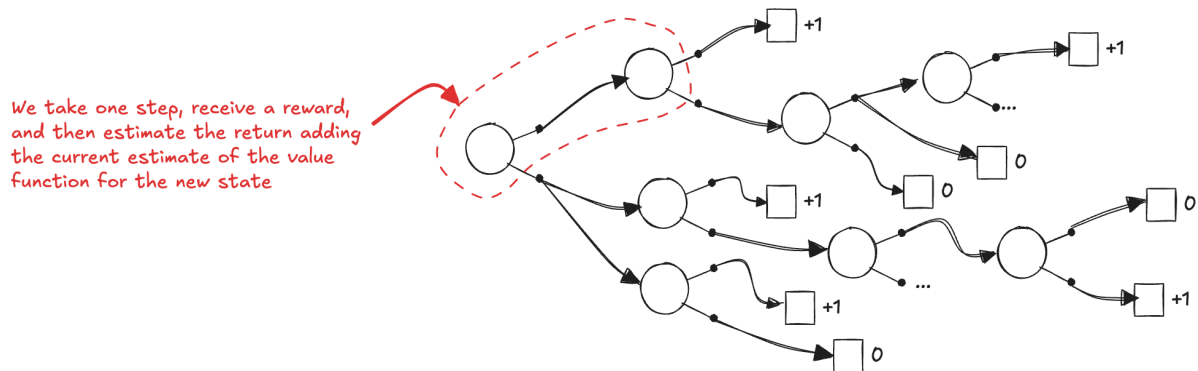
```
rmse_mc = run_experiment(mc, 100, random_walk, pi)
```

```
plt.figure(figsize=(10,6))
plt.plot(rmse_mc, color='blue', linestyle='--', label='MC')
plt.title('RMS error, averaged over tests')
plt.xlabel('Episodes')
plt.legend()
plt.show()
```



5.3 Improving estimates after each step: Temporal-Difference Learning

MC has solid convergence properties, because it updates the value function estimate using the actual return, which is an unbiased estimate, however **the agent has to wait until the end of an episode** (when it obtain the return) before it can update the state-value function estimate, and this is a limitation. Moreover, the **actual returns are high-variance estimates** as they accumulate many random events in the same trajectory. All actions, all next states, all rewards are random events. The single return collects and compounds all of that randomness for multiple time steps (from t to T). So, **the single return is unbiased, but it has a high variance**. All of that randomness becomes noise that can only be alleviated with lots of trajectories. One way to diminish these issues of high variance and time is **to estimate a return instead of waiting for the actual value**. Notice that the agent is already calculating an estimate of the state-value function, so we can use a single-step reward and, once we observe the next state, we can use the current state-value function as an estimate of the return at the next step.



This is the relationship in the equations that **temporal-difference (TD) methods** exploit:

$$\begin{aligned}
 G_{t:T} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T \\
 &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots + \gamma^{T-2} R_T) \\
 &= R_{t+1} + \gamma G_{t+1:T}
 \end{aligned}$$

This means **we could estimate the return on every time step**. In order to make explicit that we are using a single reward and then an estimate, we can use the subscript:

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$

We can substitute in the formula for the evaluation of the state-value function:

$$V(S_t) = V(S_t) + \alpha_t [G_{t:t+1} - V(S_t)]$$

where $G_{t:t+1}$ is the **TD target** and $G_{t:t+1} - V(S_t)$ is the **TD error**. In other words, **TD methods estimate the state-value function using an estimate of the state-value function**. That reward signal **progressively "injects reality" into the estimates**. This way of **updating an estimate with an estimate** is referred to as **bootstrapping**. The TD estimate depends only on a single action, a single transition and a single reward, so there's **much less randomness** being accumulated. As a consequence, TD methods usually learn **much faster** than MC methods. It can be seen as a **combination of Monte Carlo and Dynamic Programming ideas**. Like Monte Carlo, it learn directly from raw experience without a model of the environment's dynamics. Like dynamic programming, it update estimates based in part on other learned estimates, without waiting for a final outcome. Dynamic Programming bootstraps on the one-step expectation while TD methods bootstrap **on a sample** of the one-step expectation. That word sample makes a whole lot of a difference. We can write the temporal-difference algorithm in Python:

```
def td(pi, env, gamma=1.0, init_alpha=0.5, min_alpha=0.01, decay_episodes=350,
    ↪ n_episodes=500):

    # initialize the variables needed
```



```
v = np.zeros(env.observation_space, dtype=float)
v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

# calculate the learning rate schedule for all episodes
alphas = decay_alpha(init_alpha, min_alpha, decay_episodes, n_episodes);

# loop for n_episodes
for e in range(n_episodes):

    # get the initial state and then enter the interaction loop
    state, done = env.reset(), False

    # get into the time step loop
    while not done:

        # sample the policy pi for the action to take in state
        action = pi(state)

        # use the action to interact with the environment...
        # roll out the policy one step
        next_state, reward, terminated, truncated, info = env.step(action)
        if(terminated or truncated):
            done = True;

        # update the state value function
        td_target = reward + gamma * v[next_state] * (not done)
        td_error = td_target - v[state]
        v[state] = v[state] + alphas[e] * td_error

        # update the state variable for the next iteration
        state = next_state

    v_track[e] = v

return v, v_track
```

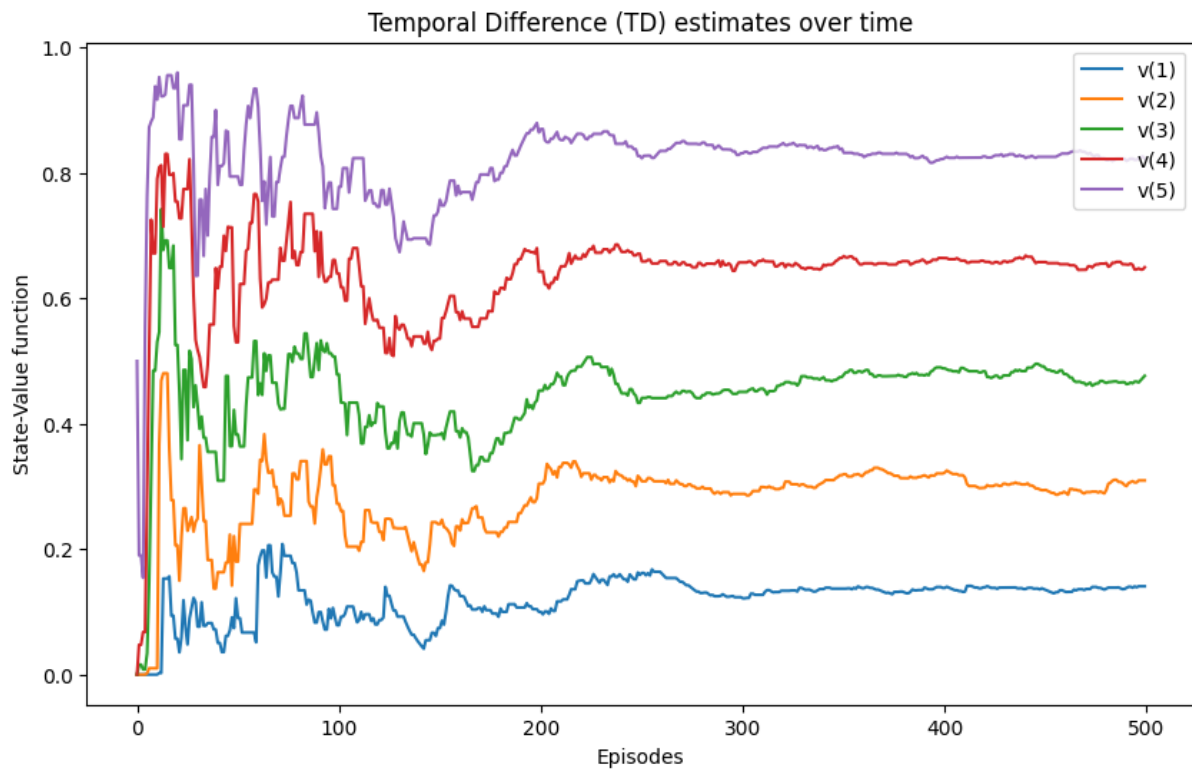
Now we use the Temporal-Difference approach on the Random Walk environment:

```
v_td, v_td_track = td(pi, random_walk, n_episodes=500)
```

```
print(v_td)
```

```
[0.          0.14078207 0.30959484 0.47649427 0.64966111 0.8230806
 0.          ]
```

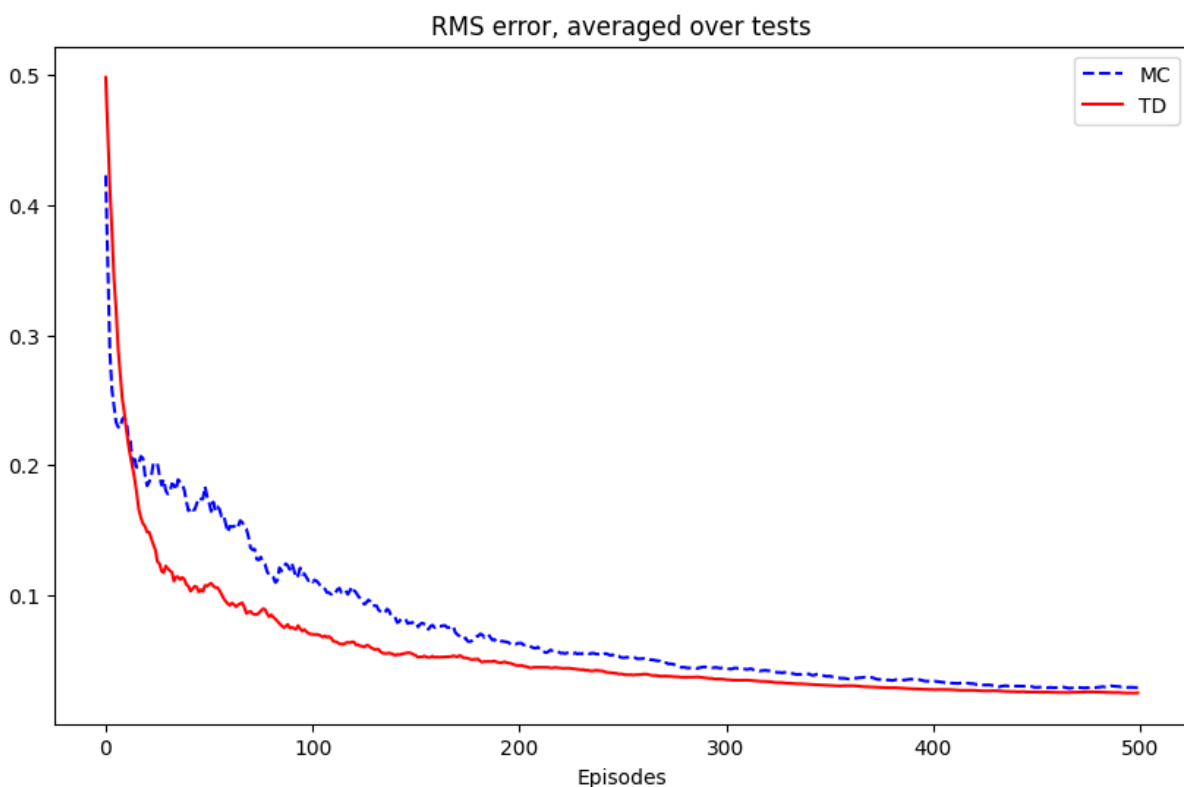
```
plt.figure(figsize=(10,6))
plt.plot(v_td_track[:,1:6])
plt.title('Temporal Difference (TD) estimates over time')
plt.ylabel('State-Value function')
plt.xlabel('Episodes')
plt.legend(legends)
plt.show()
```



While MC running estimates are very noisy, TD running estimates don't jump as much, but they are off-center for most of the episodes.

```
rmse_td = run_experiment(td, 100, random_walk, pi)
```

```
plt.figure(figsize=(10,6))
plt.plot(rmse_mc, color='blue', linestyle='--', label='MC')
plt.plot(rmse_td, color='red', linestyle='-', label='TD')
plt.title('RMS error, averaged over tests')
plt.xlabel('Episodes')
plt.legend()
plt.show()
```



At the beginning, both MC and TD methods start with a high error, indicating that the initial estimates of the value function are far from accurate. TD converges faster than MC, its line drops more rapidly in the first few episodes compared to the other. This indicates that TD is more efficient at reducing the error in the early stages of learning. As the number of episodes increases, both methods continue to reduce the error, but TD consistently performs better than MC. The TD line remains lower than the MC line throughout all the episodes, indicating that TD produces more accurate value function estimates on average. TD also appears to be more stable, with fewer fluctuations in error compared to MC, especially after the initial episodes. This suggests that TD might be better at handling noise in the learning process. Overall, the graph demonstrates that TD prediction is more efficient and effective than MC prediction in this scenario, leading to faster and more accurate convergence of the value function estimates.

5.4 Batch updating

Suppose there is available only a **finite amount of experience**. In this case, a common approach is to **present the experience repeatedly** until the method converges upon an answer. We call this **batch updating** because updates are made only after processing each complete batch of training data. Under batch updating, TD and MC methods both converge but to **different answers**. Understanding these two answers will help us understand the difference between the two methods. Suppose we observe the following eight episodes:

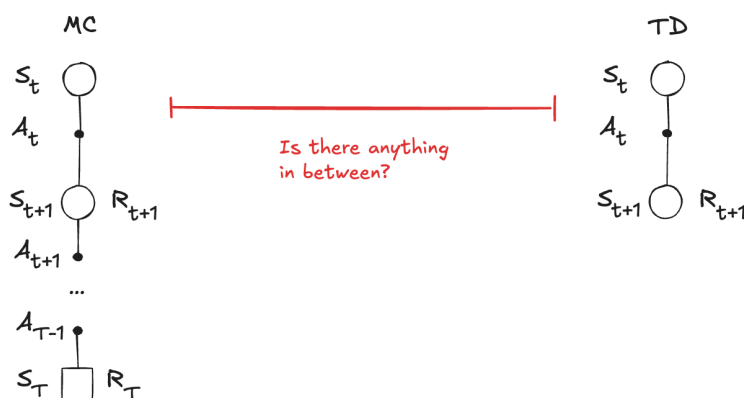
- Episode 1: A, 0, B, 0
- Episode 2: B, 1
- Episode 3: B, 1
- Episode 4: B, 1
- Episode 5: B, 1
- Episode 6: B, 1
- Episode 7: B, 1
- Episode 8: B, 0

The first episode started in state A, transitioned to B with a reward of 0, and then terminated from B with a reward of 0. The other seven episodes were even shorter, starting from B and terminating immediately. Given this batch of data, what would we say are the optimal predictions, the best values for the estimates $V(A)$ and $V(B)$? Everyone would probably agree that the optimal value for $V(B)$ is $3/4$, because six out of the eight times in state B the process terminated immediately with a return of 1, and the other two times in B the process terminated immediately with a return of 0. But what is the optimal value for the estimate $V(A)$? Here there are two reasonable answers. One is to observe that all the times the process was in state A, it traversed immediately to B; and because we have already decided that B has value $3/4$, therefore A must have value $3/4$ as well. This is also the answer that TD gives. The other reasonable answer is simply to observe that we have seen A once and the return that followed it was 0; we therefore estimate $V(A)$ as 0. This is the answer that MC gives. The second answer gives minimum squared error on the training data, but we expect the first answer to be better. If the process is markovian, we expect that the first answer will produce lower error on future data, even though the MC answer is better on the existing data. This illustrates a general difference between the estimates found by TD and MC methods. MC always find the estimates that **minimize mean-squared error on the training set**, whereas TD always finds the **estimates for the model of the Markov process**. The model used by TD is **formed from the observed episodes**: the transition probability from i to j is the fraction of observed transitions from i that went to j , and the associated expected reward is the average of the rewards observed on those transitions. Given this model, we can compute the estimate of the value function that would be exactly correct if

the model were exactly correct. This is called the **certainty-equivalence estimate** because it is equivalent to assuming that the estimate of the underlying process was known with certainty rather than being approximated. This helps explain why TD method converges more quickly than MC method. Although the non-batch methods do not achieve the certainty-equivalence or the minimum squared-error estimates, they can be understood as moving roughly in these directions. Non-batch TD may be faster than MC because it is moving toward a better estimate, even though it is not getting all the way there.

5.5 Improving estimates after multiple steps: n-steps TD

In MC we sample the environment all the way through the end of the episode before we estimate the value function. On the other hand, in TD the agent interacts with the environment only once, and it calculate a new estimates of the value function exploiting the current estimate. But, is there something in between?.



TD bootstraps after one step, but how about after two steps? Three? Four? How many steps should we wait before we estimate the expected return and bootstrap on the value function? There's a spectrum of algorithms lying in between MC and TD. We can **tune how much bootstrapping**, letting us **balance bias and variance**. We have two extremes (MC and TD methods): MC is an **infinite-step method** (it goes all the way until the end of the episode), TD is a **one-step method** (it interacts with the environment for a single step). We can generalize into an **n-step method**: instead of doing a single step (like TD) or the full episode (like MC), **it uses n-steps to calculate value functions**. This method is called **n-step TD**. We wait n steps before update the state-value function estimation:

$$S_t, A_t, R_{t+1}, S_{t+1}, \dots, R_{t+n}, S_{t+n}$$

The number of steps can be less than n if the agent reaches a terminal state before, however it could not be more than n. Once we have these n steps, we can rewrite the equation exploiting

the information we collect. Considering the entire episode (MC):

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

$$V(S_t) = V(S_t) + \alpha_t [G_{t:T} - V(S_t)]$$

Considering just one step (TD):

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$

$$V(S_t) = V(S_t) + \alpha_t [G_{t:t+1} - V(S_t)]$$

If we can use n steps:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

$$V(S_t) = V(S_t) + \alpha_t [G_{t:t+n} - V(S_t)]$$

where $G_{t:t+n}$ is the **n-step TD target** and $G_{t:t+n} - V(S_t)$ is the **n-step TD error**. When can implement it in Python, notice that it is a hybrid between MC and TD: we need to generate some steps, but instead of going to the end of the episode (like in MC) implementation, we go to n steps before bootstrapping the estimate:

```
def generate_path(pi, env, state, n_steps):

    # list of experiences along the path (part of a trajectory)
    path = []

    done = False

    # looping through until the path length is equal to n_steps
    while len(path) < n_steps:

        # use the policy function to pick an action
        action = pi(state);

        # step the environment using that action
        next_state, reward, terminated, truncated, info = env.step(action)
        if(terminated or truncated):
            done = True;

        # append the experience to the path
        experience = (state, action, reward, next_state, done)
        path.append(experience)

        # if we hit a terminal state before n_steps, break and return
    if done:
```

```
        break

    # update the state
    state = next_state

# return the path
return np.array(path, object)
```

Now we can use the path to implement the n-step TD method:

```
def ntd(pi, env, gamma=1.0, n_steps=3, init_alpha=0.5, min_alpha=0.01,
    ↪ decay_episodes=350, n_episodes=500):
    # calculate n_step discounts at once
    discounts = decay_discounts(gamma, n_steps+1);

    # calculate all alphas at once
    alphas = decay_alpha(init_alpha, min_alpha, decay_episodes, n_episodes);

    # initialize the current estimate of the state-value function
    v = np.zeros(env.observation_space, dtype=float)

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # loop for every episode
    for e in range(n_episodes):

        # get the initial state
        state, done = env.reset(), False

        # go collecting a series of n-state paths until we hit done
        while not done:

            # generate a path of n-steps (maximum, or less if we hit done)
            path = generate_path(pi, env, state, n_steps)

            # calculate the number of steps, could be 'n_step' but it could
            # also be smaller if a terminal state is in the path
            n = len(path)

            # initialize a visits check vector
            visited = np.zeros(env.observation_space, dtype=bool)
```

```

    # now loop through all experiences in the path
    for t, (state, _, reward, next_state, done) in enumerate(path):
        ↪

        # calculate the partial return
        partial_return = np.sum(discounts[:n-t] * path[t:, 2])

        # update the value function
        ntd_target = partial_return + discounts[n] * v[path[n-1, 3]]
        ntd_error = ntd_target - v[state]
        v[state] = v[state] + alphas[e] * ntd_error

        state = next_state

    # keep track of the episode's V
    v_track[e] = v

    return v.copy(), v_track

```

We can apply this method to the Random Walk environment using a 3-step TD:

```
v_ntd, v_ntd_track = ntd(pi, random_walk, n_steps=3, n_episodes=500)
```

```
print(v_ntd)
```

```

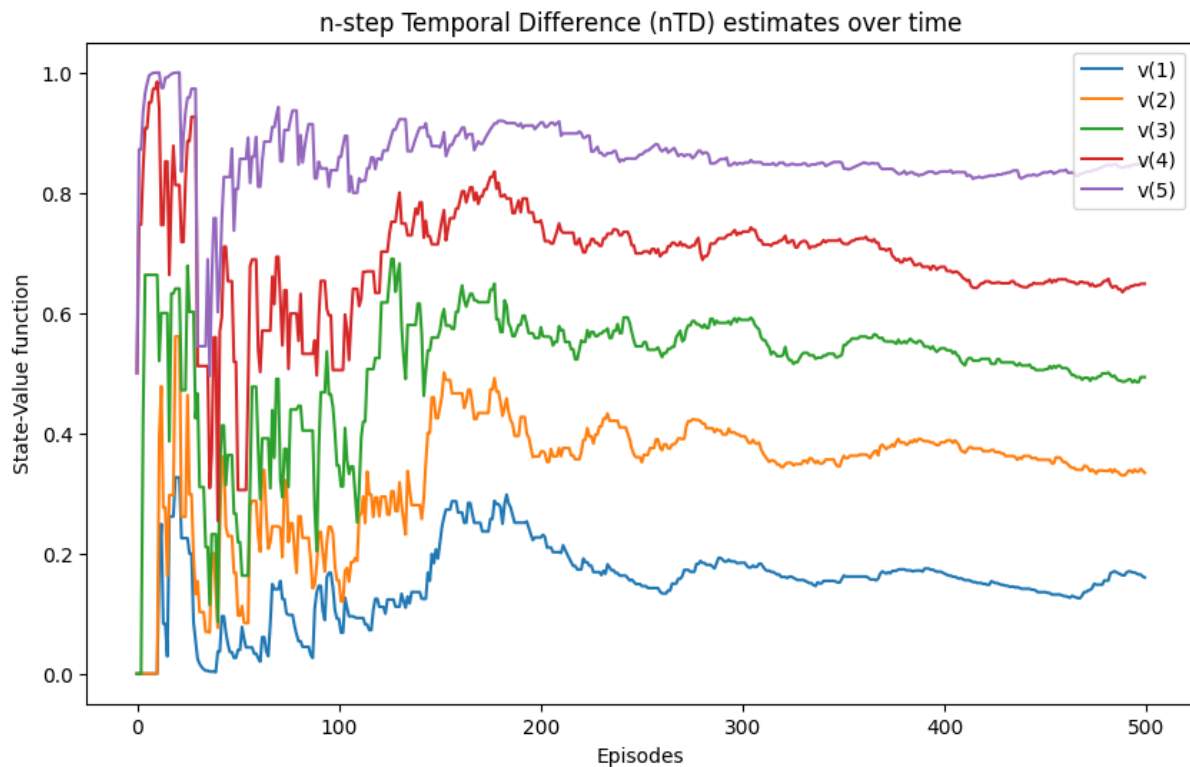
[0.          0.16004448 0.33393513 0.4931261  0.6482914  0.84844146
 0.          ]

```

```

plt.figure(figsize=(10,6))
plt.plot(v_ntd_track[:,1:6])
plt.title('n-step Temporal Difference (nTD) estimates over time')
plt.ylabel('State-Value function')
plt.xlabel('Episodes')
plt.legend(legends)
plt.show()

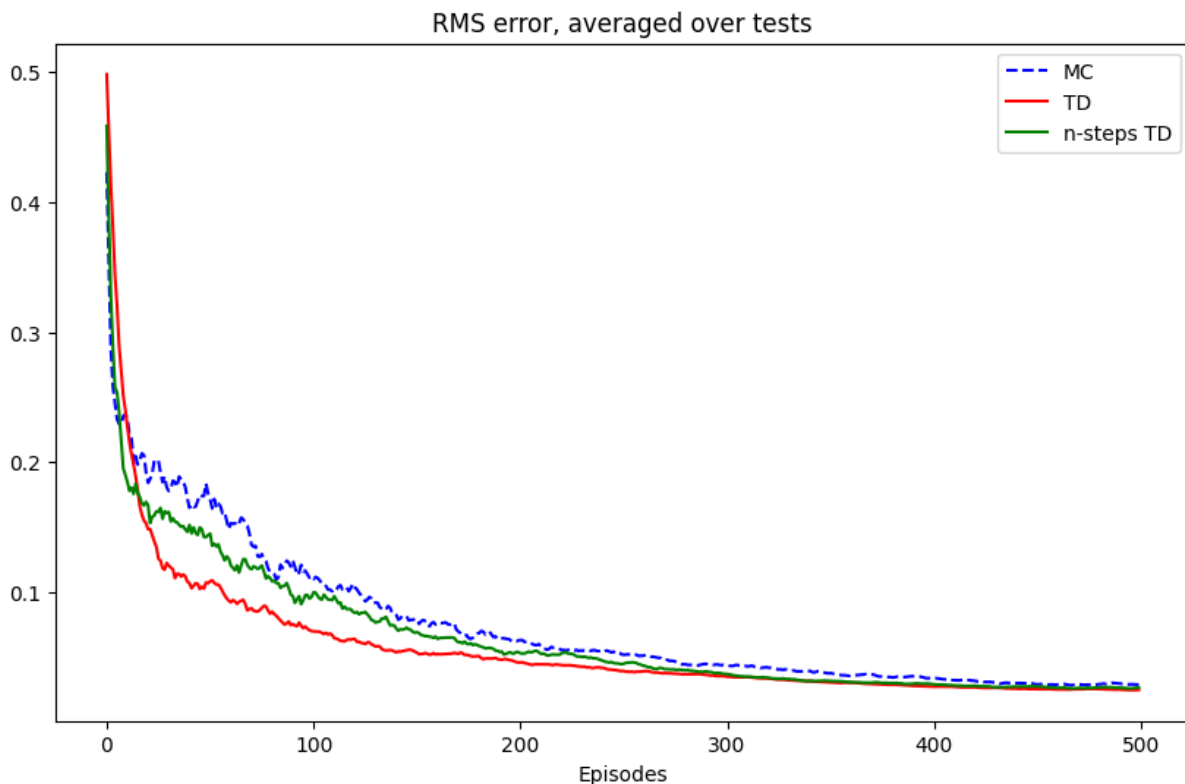
```

We can compare with the TD, MC:

```
rmse_ntd = run_experiment(ntd, 100, random_walk, pi)
```

```
plt.figure(figsize=(10,6))
plt.plot(rmse_mc, color='blue', linestyle='--', label='MC')
plt.plot(rmse_td, color='red', linestyle='-', label='TD')
plt.plot(rmse_ntd, color='green', linestyle='-', label='n-steps TD')
plt.title('RMS error, averaged over tests')
plt.xlabel('Episodes')
plt.legend()
plt.show()
```



The n-step TD strikes a balance between MC and TD, offering faster convergence than MC but slightly slower than TD. It updates the value estimates using rewards over a few steps and this can **reduces bias (compared to TD)** by considering more actual rewards and **reduces variance (compared to MC)** because it does not wait until the end of the episode. This often leads to **more stable and robust learning**, particularly in environments where rewards are sparse or delayed. However, the **choice of n is crucial**. If n is too small, the algorithm will have high bias and low variance, similar to TD. If n is too large, the algorithm will have low bias and high variance, similar to MC. The optimal value of n depends on the environment and the task.

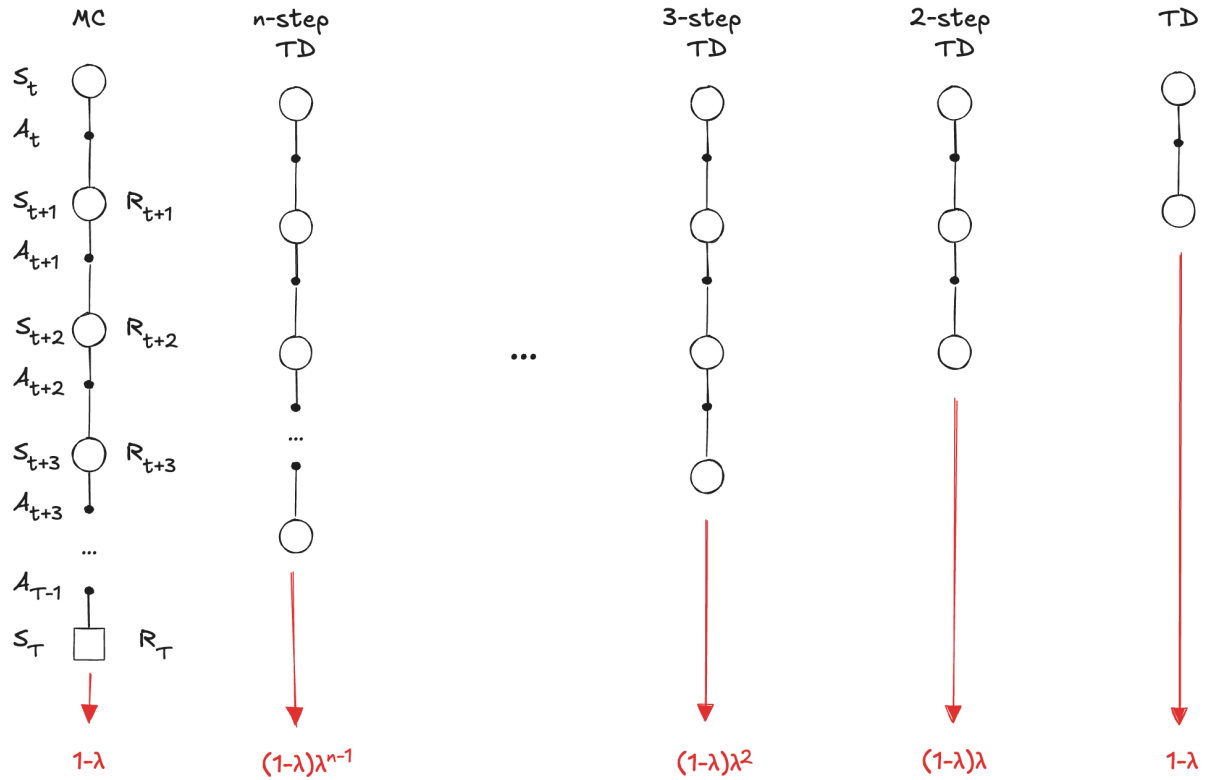
5.6 Improving estimates of all visited states: $TD(\lambda)$

A question emerges, what is a good value for the parameter n? How many steps should the agent wait before updating the value function? How about using a weighted combination of all n-step targets as a single target? The agent could go out, calculates the n-step targets corresponding to the one-, two-, three-, ..., infinite-step target and then **combines all of these targets with an exponentially decaying factor**. This is called **forward-view $TD(\lambda)$** method. We can introduce a new parameter **lambda** to control the weighting of the n-step targets in the following way:

- $\lambda = 0$: $TD(\lambda)$ should become equivalent to TD learning (for this reason TD is often referred

to as TD(0));

- $\lambda = 1$: TD(λ) should become equivalent to MC;
- $0 < \lambda < 1$: TD(λ) should consider multiple steps of future rewards, combining short-term and long-term updates.



$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$

$$G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

...

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

...

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

We can compose all targets using a weighted sum:

$$G_{t:T}^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_{t:T}$$

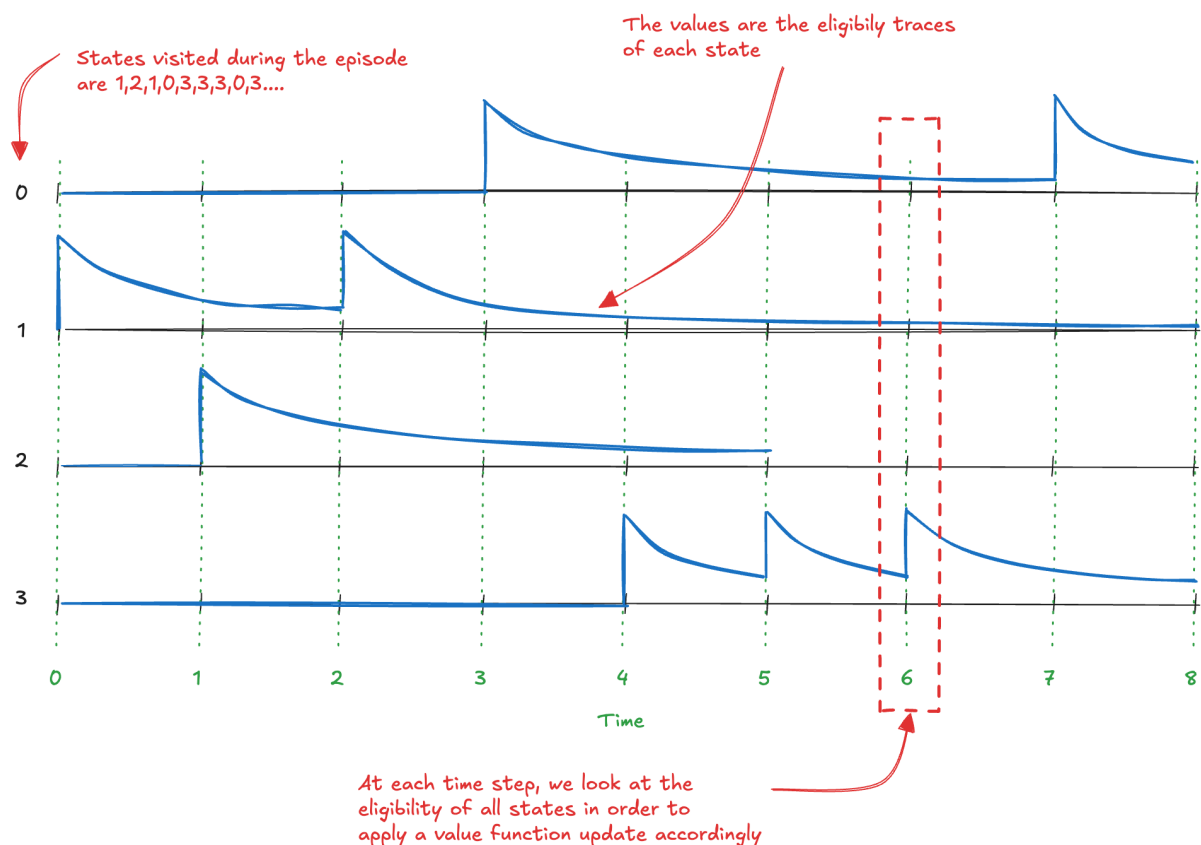
In the end, we can evaluate the estimate of the value function using this return:

$$V(S_t) = V(S_t) + \alpha_t [G_{t:T}^\lambda - V(S_t)]$$

where $G_{t:T}^\lambda$ is the **TD(λ) target** and $G_{t:T}^\lambda - V(S_t)$ is the **TD(λ) error**.

In this way, forward-view TD(λ) looks ahead to all possible future rewards from the current state and combines them into a single value update, with the contribution of each future reward decaying over time according to the parameter. λ provides a way to smoothly interpolate between **short-term learning** (like TD) and **long-term learning** (like MC). However, like MC, it is under **the curse of the time step**, because it can update the state-value function only after reaching a terminal state.

In practice, this idea is typically implemented using a **backward view**, in order to overcome the need to wait until the end of the episode to apply updates. We exploit the idea of **eligibility trace**. It is a memory that keeps track of how much each state should be credited for future rewards. When a state is visited, its eligibility trace is increased and as time progresses, its trace decays (using a parameter). The fading traces allow the agent to “remember” and update not just the most recent state but a sequence of past states, with a decreasing emphasis the further back in time they were visited:



At each time step, the algorithm updates value estimates not only for the current state but for all previously visited states (so with backward view), weighted by their eligibility traces. The

method is called, **backward-view TD(λ)**. At the beginning of every new episode, we set the eligibility vector to zero:

$$E = 0$$

then, we interact with the environment for one step:

$$S_t, A_t, R_{t+1}, S_{t+1}$$

we encounter a state and we add one to its trace, in order to mark it as eligible for an update:

$$E(S_t) = E(S_t) + 1$$

we then calculate the TD error just as we've done so far for TD:

$$\delta_{t:t+1}^{TD}(S_t) = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

however, we can apply the update to the value function **for all states that are eligible** by multiplying it by the eligibility trace vector. In this way, only eligible states will get updated:

$$V = V + \alpha_t \delta_{t:t+1}^{TD}(S_t) E$$

Notice that we are not updating only $V(S_t)$ but all vector V instead, this is because we're multiplying by the eligibility vector E , so the eligible states get the corresponding credit. After each update, the eligibility trace vector is decayed, so that future reinforcing events have less impact on earlier states. By doing this, the most recent states get more significant credit for a reward encountered in a recent transition than those states visited earlier in the episode:

$$E = \lambda E$$

We can implement the algorithm in Python:

```
def td_lambda(pi, env, gamma=1.0, lambda_=0.2,
              init_alpha=0.5, min_alpha=0.01,
              decay_episodes=350, n_episodes=500):

    # calculate all alphas at once
    alphas = decay_alpha(init_alpha, min_alpha, decay_episodes, n_episodes);

    # initialize the current estimate of the state-value function
    v = np.zeros(env.observation_space, dtype=float)

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # initialize the eligibility trace vector
    E = np.zeros(env.observation_space, dtype=float)
```

```
# loop for every episode
for e in range(n_episodes):

    # set E to zero every new episode
    E.fill(0)

    # get the initial state
    state, done = env.reset(), False

    # get into the time step loop
    while not done:

        # sample the policy pi for the action to take in state
        action = pi(state)

        # interact with the environment for one step and get the experience tuple
        next_state, reward, terminated, truncated, info = env.step(action)
        if(terminated or truncated):
            done = True;

        # use that experience to calculate the TD error as usual
        td_target = reward + gamma * v[next_state] * (not done)
        td_error = td_target - v[state]

        # increment the eligibility of state by 1
        E[state] = E[state] + 1

        # apply the error update to all eligible states as indicated by E
        v = v + alphas[e] * td_error * E

        # decay E
        E = lambda_ * E

        # update the state variable for the next iteration
        state = next_state

    v_track[e] = v

return v, v_track
```

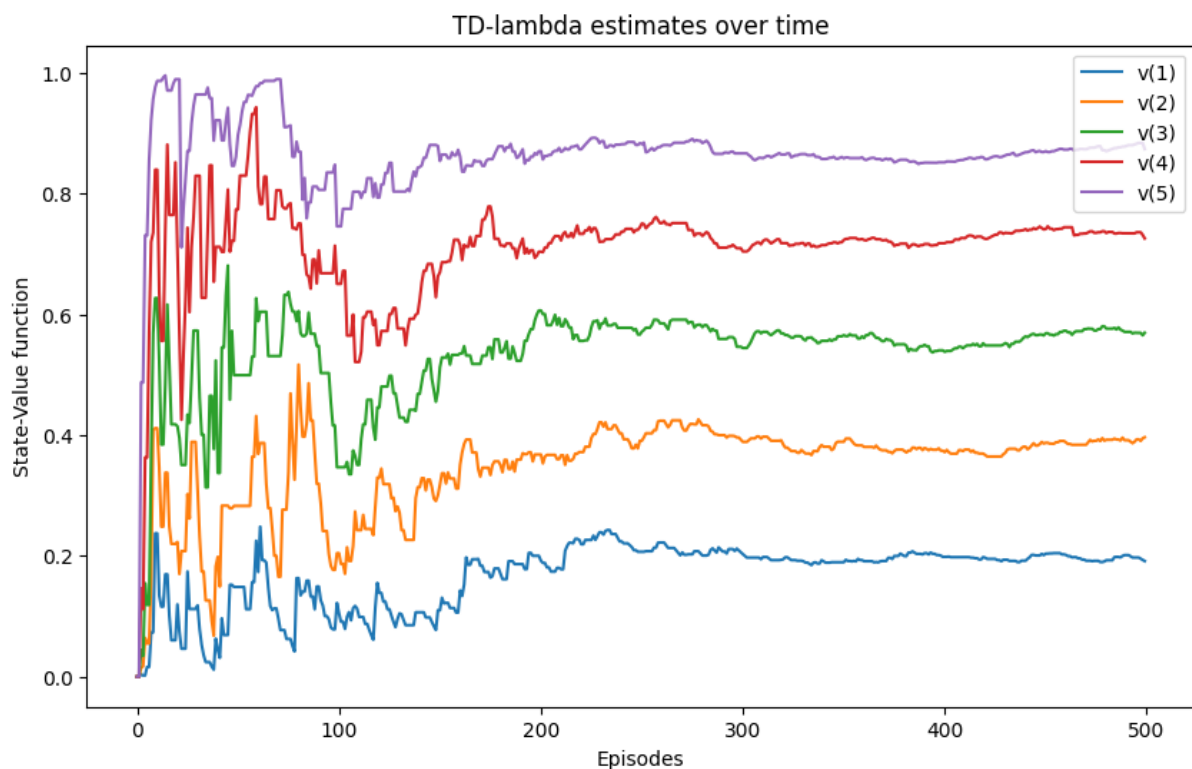
We can apply this method to the random walk environment:

```
v_td_lambda, v_td_lambda_track = td_lambda(pi, random_walk, lambda_=0.3,  
↳ n_episodes=500)
```

```
print(v_td_lambda)
```

```
[0.          0.19140136 0.39634172 0.5694172  0.72568936 0.87365462  
 0.          ]
```

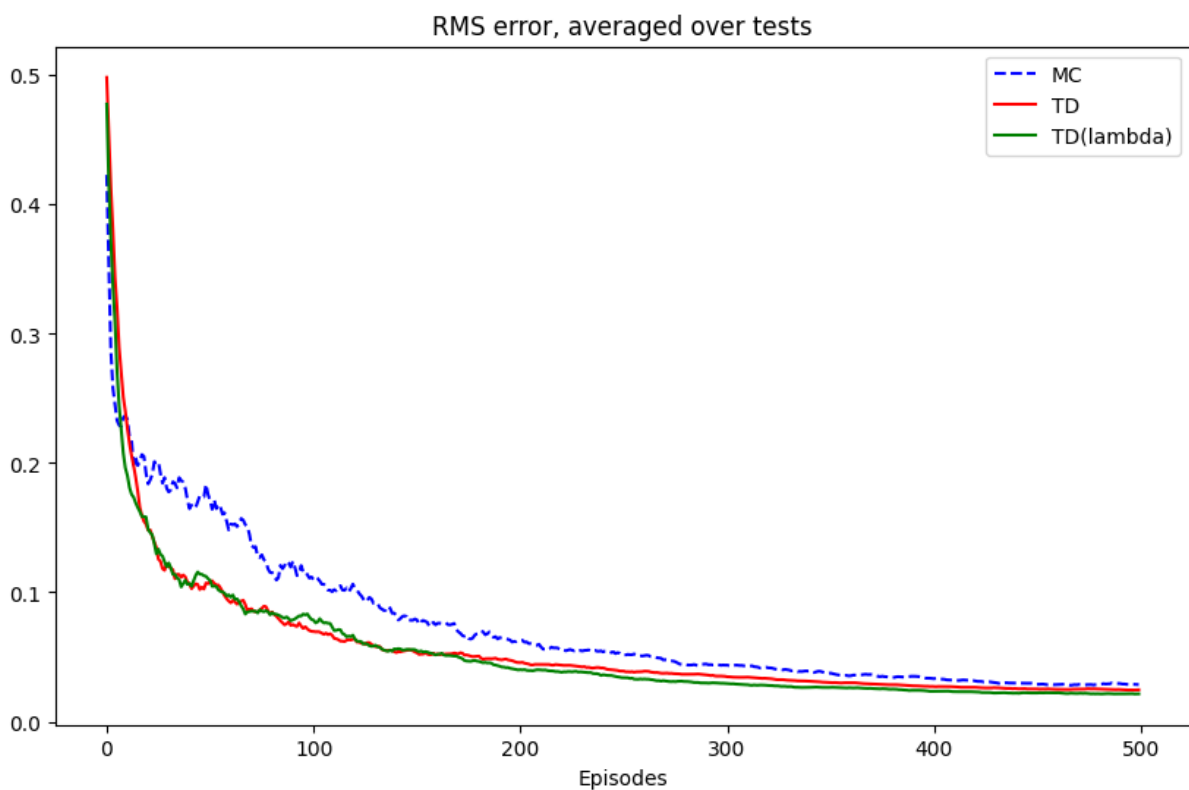
```
plt.figure(figsize=(10,6))  
plt.plot(v_td_lambda_track[:,1:6])  
plt.title('TD-lambda estimates over time')  
plt.ylabel('State-Value function')  
plt.xlabel('Episodes')  
plt.legend(legends)  
plt.show()
```



And we can compare with the other methods:

```
rmse_td_lambda = run_experiment(td_lambda, 100, random_walk, pi)
```

```
plt.figure(figsize=(10,6))
plt.plot(rmse_mc, color='blue', linestyle='--', label='MC')
plt.plot(rmse_td, color='red', linestyle='-', label='TD')
plt.plot(rmse_td_lambda, color='green', linestyle='-', label='TD(lambda)')
plt.title('RMS error, averaged over tests')
plt.xlabel('Episodes')
plt.legend()
plt.show()
```



The graph shows that $TD(\lambda)$ is the most effective method, providing faster convergence and more accurate value function estimates.

6 Policy Improvement

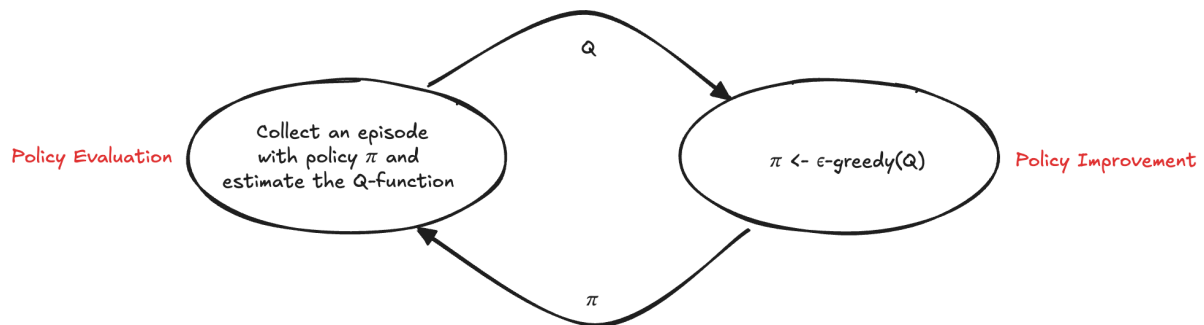
Previously, we learned how to evaluate a policy based on how good it is by computing its expected return (cumulative reward) from each state. This information is captured by the value

function. However, this information doesn't directly make the agent better at any task. Once the value of a policy is known, we can improve the policy by making it greedier. This means adjusting the policy so that in each state, the agent chooses the action that leads to the highest expected value based on the current value function:

$$\pi_{k+1}(s) = \arg \max_a q_{\pi_k}(s, a)$$

This is similar to what we did in the policy-iteration algorithm in Dynamic Programming: we evaluate, we improve, then evaluate the improved policy, then improve on this improved policy, and so on:

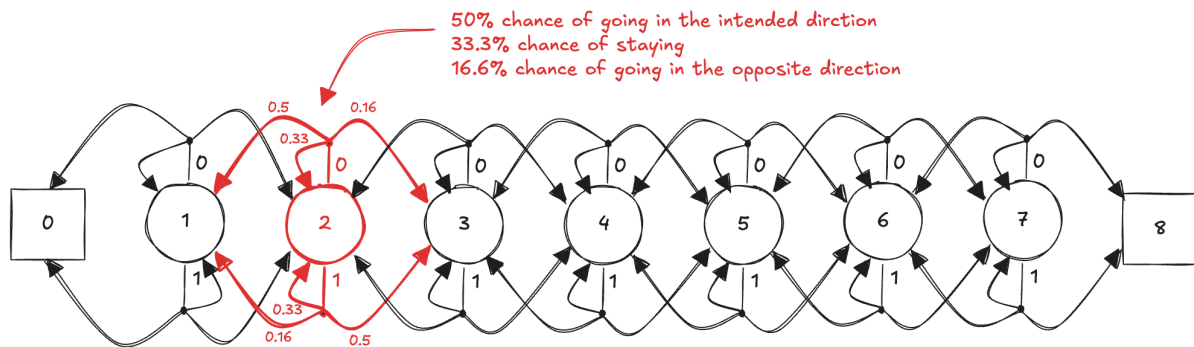
$$\pi_0 \xrightarrow{\text{evaluate}} q_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluate}} q_{\pi_1} \xrightarrow{\text{improve}} \pi_2 \xrightarrow{\text{evaluate}} \dots \xrightarrow{\text{improve}} \pi_* \xrightarrow{\text{evaluate}} q_{\pi_*}$$



However, in this case **we are not using a model of the environment** to compute the value function, but we estimate the value function based on samples. This ability allows agents to **learn optimal behavior by trial-and-error learning**, starting from arbitrary policies and ending in optimal ones. This pattern is called **generalized policy iteration (GPI)**, and it can help us create an architecture that virtually any reinforcement learning algorithm fits under, including state-of-the-art deep reinforcement learning agents.

6.1 Slippery Walk environment

We use an environment called **Slippery Walk**. This environment is a walk, a single-row grid-world environment, with seven non-terminal states. The particular thing of this environment is that it's slippery, like the Frozen Lake: **action effects are stochastic**. If the agent chooses to go left, there is a chance it does, but there is also a chance that it goes right, or that it stays in place.



This environment is noisy, but the actions the agent selects make a difference in its performance.

```
import random
import numpy as np

class SlipperyWalk:
    def __init__(self):
        self.reset();

    def reset(self):
        self.observation_space = 9;
        self.action_space = 2;
        self.state = np.random.choice([1,2,3,4,5,6,7]);
        self.terminated = False;
        return self.state;

    def step(self, action):
        if self.terminated: raise ValueError('Episode has terminated');
        if action not in [0, 1]: raise ValueError('Invalid action');

        if(action==0): direction = -1;
        if(action==1): direction = 1;

        direction_list = [direction, -direction, 0];
        true_direction = random.choices(direction_list, weights=(50, 16, 33), k=1);
        self.state += true_direction[0];

        reward = 0
        if self.state < 1:
            self.terminated = True;
        if self.state > 7:
            self.terminated = True;
```

```

        reward = 1;

    return self.state, reward, self.terminated, 0, 0;

```

```
slippery_walk = SlipperyWalk()
```

The optimal state value function can be calculated using the MDP and applying the Dynamic Programming:

```
optimal_v = [0., 0.5637, 0.763, 0.8449, 0.8892, 0.922, 0.9515, 0.9806, 0.];
```

6.2 Improving policies after each episode: Monte Carlo control

The **policy improvement theorem** (as already considered in Dynamic Programming) assures that each new policy is better than (or just as good as) the previous one, so the overall process converges to the optimal policy and optimal value function:

$$q_{\pi_k}(s, \pi_{k+1}(s)) = q_{\pi_k}(s, \arg \max_a q_{\pi_k}(s, a)) = \max_a q_{\pi_k}(s, a) \geq q_{\pi_k}(s, \pi_k(s)) \geq v_{\pi_k}(s)$$

In order to implement this idea, we need to **estimate the action-value function, instead of the state-value function**, in order to know what the best action is to take from a state:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[G_{t:T} - Q(S_t, A_t)]$$

Then, we need to **explore**, because we're no longer using the MDP for the policy-evaluation. When we estimate from samples, we get values for all of the state-action pairs we visited, but what if part of the best states weren't visited? Let's start using **first-visit Monte Carlo** for the policy-evaluation phase and a **decaying epsilon-greedy action-selection strategy** for the policy-improvement phase:

```

def select_action(state, q, epsilon=0.1):

    # if the random number is greater than epsilon,
    if np.random.uniform() > epsilon:
        # pick the action with the highest Q value
        action = np.argmax(q[state]);
    else:
        # otherwise, pick a random action
        action = np.random.randint(len(q[0]));

    return action;

```

In order to decay alpha and epsilon, we use the same decay schedule function used in policy evaluation

```
def decay_schedule(init_value, min_value, decay_steps, max_steps):  
  
    # calculate the number of the remaining steps after the decay  
    rem_steps = max_steps - decay_steps;  
  
    # logspace returns numbers spaced evenly on a log scale  
    # base^start is the starting value of the sequence,  
    # base^stop is the final value of the sequence  
    # num is the number of values to generate  
    # base is the base of the log space  
    values = np.logspace(start=0, stop=-2, num=decay_steps, base=10);  
  
    # because the values may not end exactly at 0, given it's the log,  
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.  
    values = (values - values.min()) / (values.max() - values.min());  
  
    # linear transformation and get points between init_value and min_value  
    values = (init_value - min_value) * values + min_value;  
  
    # repeats the rightmost value rem_step number of times  
    values = np.pad(values, (0, rem_steps), 'edge');  
  
    return values;
```

We create also a function to calculate all the discount values for the full process:

```
def decay_discounts(gamma, max_steps):  
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma,  
↪    endpoint=False);  
    return discounts
```

The generation of a trajectory is slightly different, now we need to select an action following a strategy:

```
def generate_trajectory(select_action, q, env, epsilon, max_steps=200):  
  
    # list of experiences (trajectory)  
    trajectory = [];
```

```
done = False;
steps = 0;

# reset the environment to interact in a new episode
state = env.reset();

# looping through until the done flag is set to true
while not done:
    steps += 1;

    # there is the difference: use the 'select_action'
    # function to pick an action
    action = select_action(state, q, epsilon);

    # step the environment using that action
    next_state, reward, terminated, truncated, info = env.step(action);
    if(terminated or truncated):
        done = True;

    # append the experience to the trajectory
    experience = (state, action, reward, next_state, done);
    trajectory.append(experience);

    # if we hit a terminal state break and return
    if done:
        break;

    # truncate long trajectories
    if steps >= max_steps:
        break;

    # update the state
    state = next_state;

# return the trajectory
return np.array(trajectory, object);
```

Then we can write the Monte Carlo control algorithm, which is similar to one for prediction. The two main differences is that we now estimate the action-value function, and we need to explore (we use an decaying epsilon to control random exploration):

```
def mc_control(env, gamma=0.99,
               init_alpha=0.5, min_alpha=0.01,
               init_epsilon=1.0, min_epsilon=0.1,
               decay_episodes=1000, n_episodes=3000,
               max_steps=200):

    # calculate all discounts at once.
    discounts = decay_discounts(gamma, max_steps);

    # calculate all alphas at once
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate all epsilons in advance
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # the episode loop
    for e in range(n_episodes):

        # generate a trajectory using the exploratory policy
        trajectory = generate_trajectory(select_action, q, env, epsilons[e],
        ↪ max_steps);

        # initialize a visits check vector
        visited = np.zeros((env.observation_space, env.action_space), dtype=bool);

        # loop through all experiences in the trajectory
        for t, (state, action, reward, _, _) in enumerate(trajectory):

            # check if the state-action has already been visited (first-visit MC)
            if visited[state][action]:
                continue;
            visited[state][action] = True;

            # calculate the number of steps from t to T
            n_steps = len(trajectory[t:]);

            # calculate the return
```

```

        g = np.sum(discounts[:n_steps] * trajectory[t:, 2]);

        # estimate the value functions
        q[state][action] = q[state][action] + alphas[e] * (g - q[state][action]);

        # extract the state-value function
        v = np.max(q, axis=1);

        # and the greedy policy
        def pi(s):
            return np.argmax(q[s]);

        # save values for post analysis
        v_track[e] = v;

    return q, v, pi, v_track;

```

Let's test the algorithm in the Slippery Walk Seven environment:

```
q_mc, v_mc, pi_mc, v_track_mc = mc_control(slippery_walk);
```

We can show the estimates over episodes:

```
print(v_mc)
```

```

[0.          0.52589139 0.75560115 0.82772955 0.89103973 0.92836406
 0.95409788 0.98320711 0.          ]

```

Of course, the optimal policy learned is obvious in this case, but the algorithm is able to learn it:

```

def print_policy(pi, env):
    for state in range(env.observation_space):
        action = pi(state)
        action_icon = ' '
        if action == 0: action_icon = 'L'
        if action == 1: action_icon = 'R'
        print('state:', state, '→', 'action:', action_icon)

```

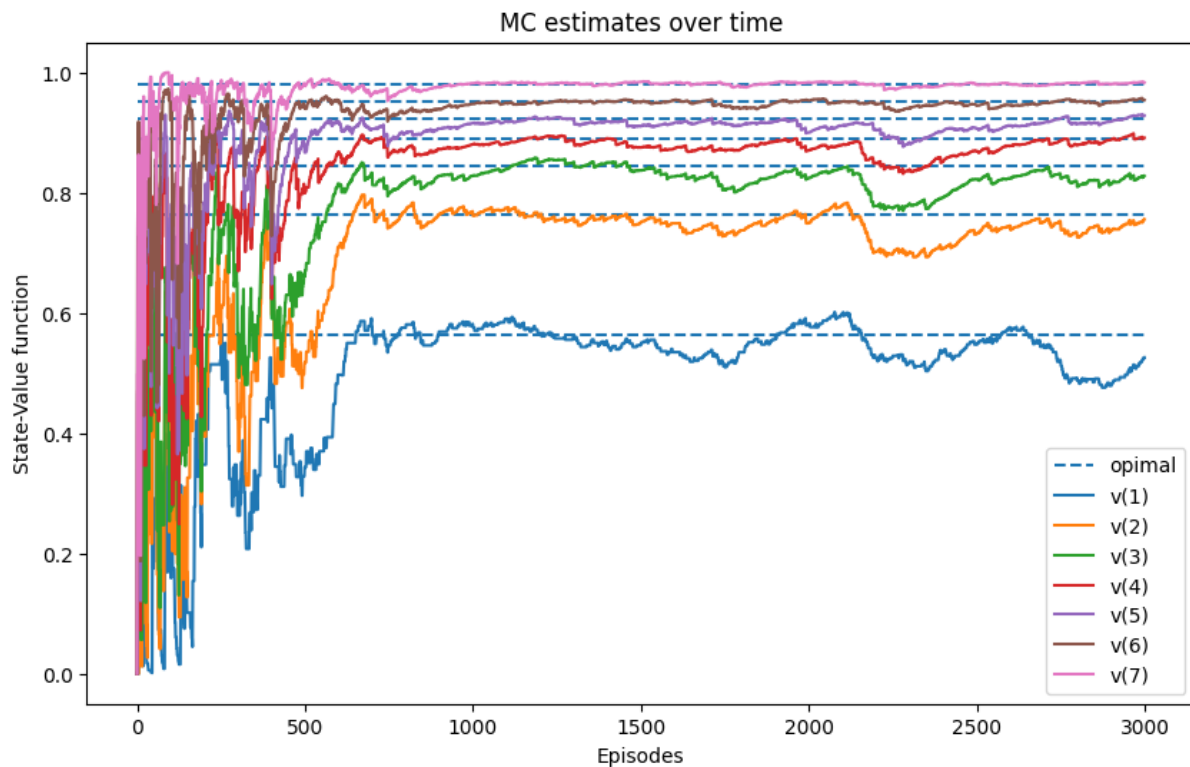
```
print_policy(pi_mc, slippery_walk)
```

```
state: 0 -> action: L
state: 1 -> action: R
state: 2 -> action: R
state: 3 -> action: R
state: 4 -> action: R
state: 5 -> action: R
state: 6 -> action: R
state: 7 -> action: R
state: 8 -> action: L
```

We can show the difference between the estimates and the real values:

```
import matplotlib.pyplot as plt

legends = ['optimal', 'v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'];
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_mc[:,1:8]);
plt.title('MC estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```

Notice how **the estimates of MC have high variance**. We can use a brute-force approach to get the average return of the policy and its success probability.

```
def evaluate(env, pi, n_episodes=1000, max_steps=200):
    success = 0;
    results = [];
    for _ in range(n_episodes):
        done = False;
        steps = 0;
        state = env.reset();
        results.append(0.0);
        while not done and steps < max_steps:
            state, reward, terminated, truncated, info = env.step(pi(state));
            if(terminated or truncated):
                done = True;
                results[-1] += reward;
                steps += 1;
        if(done and reward==1):
            success += 1;
    return (success/n_episodes)*100, np.mean(results);
```

```
probability_success_mc, mean_return_mc = evaluate(slippery_walk, pi_mc);

print("Reaches goal ", probability_success_mc, "%");
print("Obtains an average undiscounted return of ", mean_return_mc);
```

```
Reaches goal  93.300000000000001 %
Obtains an average undiscounted return of  0.933
```

6.3 Improving policies after each step: SARSA

Monte Carlo method is an **offline** method in an episode-to-episode sense: we must wait until a terminal state before we can make any improvements to the value function estimate. So, it is straightforward to apply **temporal-difference prediction** for the policy-evaluation phase in order to remove this limitation. The algorithm is called **SARSA agent** (Rummery & Niranjan, "Online Q-Learning using Connectionist Systems" (1994):

$$G_{t:t+1} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

We can substitute in the formula for the evaluation of the action-value function:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha_t [G_{t:t+1} - Q(S_t, A_t)]$$

where $G_{t:t+1}$ is the **SARSA target** and $G_{t:t+1} - Q(S_t, A_t)$ is the **SARSA error**.

We can implement it in Python:

```
def sarsa(env, gamma=0.99,
          init_alpha=0.5, min_alpha=0.01,
          init_epsilon=1.0, min_epsilon=0.1,
          decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);
```

```
# the episode loop
for e in range(n_episodes):

    # we start episode by resetting the environment and the done flag
    state, done = env.reset(), False;

    # select the action (perhaps exploratory) for the initial state
    action = select_action(state, q, epsilons[e]);

    # repeat until we hit a terminal state
    while not done:

        # step the environment using that action
        next_state, reward, terminated, truncated, info = env.step(action);
        if(terminated or truncated):
            done = True;

        # obtain the action for the next step
        next_action = select_action(next_state, q, epsilons[e]);

        # calculate the target using that next state-action pair
        sarsa_target = reward + gamma * q[next_state][next_action]

        # calculate the error
        sarsa_error = sarsa_target - q[state][action];

        # update the action-value function
        q[state][action] = q[state][action] + alphas[e] * sarsa_error;

        # update state and action for the next step
        state, action = next_state, next_action;

    # extract the state-value function
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save values for post analysis
    v_track[e] = v;
```

```
return q, v, pi, v_track;
```

Now we can run SARSA against the Slippery Walk environment:

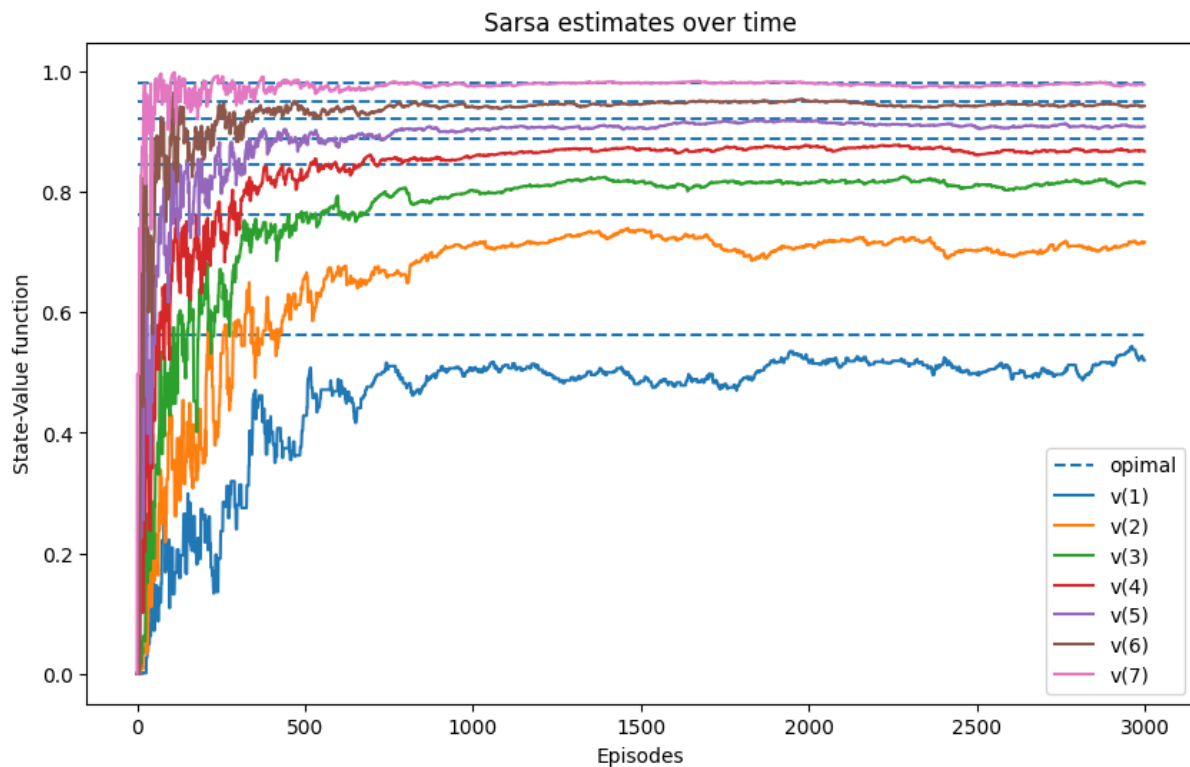
```
q_sarsa, v_sarsa, pi_sarsa, v_track_sarsa = sarsa(slippery_walk);
```

```
print(v_sarsa);
```

```
[0.          0.52063677 0.71702406 0.81401339 0.86784435 0.90884011
 0.94304096 0.97815134 0.          ]
```

We can show the estimates over episodes and the difference between the estimates and the real values:

```
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_sarsa[:,1:8]);
plt.title('Sarsa estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



We can see how SARSA has less variance than MC, but it is slower to converge, particularly for low-value states. Again, we can use the brute-force approach to get the average return of the policy and its success probability:

```
probability_success_sarsa, mean_return_sarsa = evaluate(slippery_walk, pi_sarsa);

print("Reaches goal ", probability_success_sarsa, "%");
print("Obtains an average undiscounted return of ", mean_return_sarsa);
```

Reaches goal 93.30000000000001 %

Obtains an average undiscounted return of 0.933

Monte Carlo and SARSA must meet some requirements to guarantee convergence to the optimal policy: (a) **all state-action pairs must be explored infinitely often** and (b) **the policy must converge on a greedy policy**. In practice, this means that in the epsilon-greedy exploration strategy must slowly decay epsilon towards zero. If it goes down too quickly, the first condition may not be met; if it decays too slowly, well, it takes longer to converge.

6.4 Decoupling behavior from learning: Q-Learning

The SARSA algorithm is a sort of “learning on the job”, the agent learns the value of the policy it is currently following, which means it **evaluates and improves the same policy that it is using to make decisions**. The agent explores and exploits the environment according to the same policy it is updating. This type of learning is called **on-policy learning**. It is excellent, we learn from our own mistakes. But we learn from our own current mistakes only. What if we want to learn from our own previous mistakes? What if we want to learn from the mistakes of others? The **Off-policy learning** approach is sort of “learning from others”: the agent **learns the value of an optimal policy, but it follows a different behavior policy to explore** the environment. In other words, it separates the policy it uses to explore (**behavior policy**) from the policy it is trying to optimize (**target policy**). The most used off-policy algorithm is called **Q-Learning** (Watkins, “Learning from Delayed Rewards” (1989)). The difference between SARSA and Q-learning is the action used to calculate the target. SARSA uses the action taken in the next state, Q-learning uses the action with the maximum estimated value in the next state, despite the action taken:

$$G_{t:t+1} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

We can substitute in the formula for the evaluation of the action-value function:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha_t [G_{t:t+1} - Q(S_t, A_t)]$$

where $G_{t:t+1}$ is the **Q-learning target** and $G_{t:t+1} - Q(S_t, A_t)$ is the **Q-learning error**.

Q-Learning algorithm will evaluate all possible state-actions pairs and choose that pair which will generate maximum expected rewards (which is the value of the state). Off-policy methods can converge faster because they are not restricted by the exploration-exploitation balance of the behavior policy. We can implement it in Python:

```
def q_learning(env, gamma=0.99,
               init_alpha=0.5, min_alpha=0.01,
               init_epsilon=1.0, min_epsilon=0.1,
               decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);
```

```
# create a list to save copies of v for offline analysis
v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

# the episode loop
for e in range(n_episodes):

    # we start episode by resetting the environment and the done flag
    state, done = env.reset(), False;

    # repeat until we hit a terminal state
    while not done:

        # select the action
        action = select_action(state, q, epsilons[e]);

        # step the environment using that action
        next_state, reward, terminated, truncated, info = env.step(action);
        if(terminated or truncated):
            done = True;

        # calculate the target: here is Q-learning!
        q_learning_target = reward + gamma * q[next_state].max();

        # calculate the error
        q_learning_error = q_learning_target - q[state][action];

        # update the action-value function
        q[state][action] = q[state][action] + alphas[e] * q_learning_error;

        # update state for the next step
        state = next_state;

    # extract the state-value function
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save values for post analysis
    v_track[e] = v;
```

```
return q, v, pi, v_track;
```

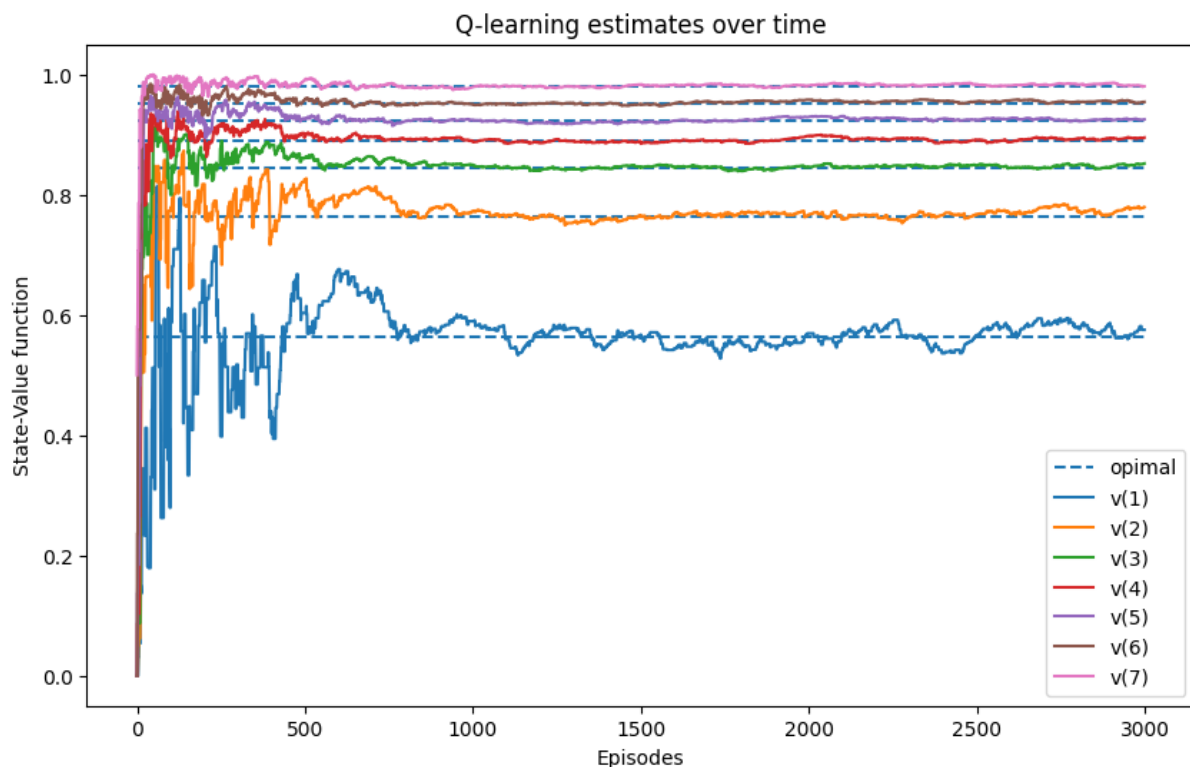
Now we can run Q-Learning against the Slippery Walk environment:

```
q_ql, v_ql, pi_ql, v_track_ql = q_learning(slippery_walk);
```

```
print(v_ql)
```

```
[0.          0.57510604 0.77902023 0.8516414  0.89468035 0.92487707
 0.95390326 0.9799436  0.          ]
```

```
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_ql[:,1:8]);
plt.title('Q-learning estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



See how much **faster** the estimates track the true values. But, also, notice how the estimates are often higher and **jump around** somewhat aggressively.

```
probability_success_ql, mean_return_ql = evaluate(slippery_walk, pi_ql);

print("Reaches goal ", probability_success_ql, "%");
print("Obtains an average undiscounted return of ", mean_return_ql);
```

```
Reaches goal 92.60000000000001 %
Obtains an average undiscounted return of 0.926
```

Notice that for off-policy algorithms the only requirement that holds of the two for convergence is the first one (all state-action pairs must be explored infinitely often). The second one (the policy must converge on a greedy policy) is no longer a requirement, because the learned policy is different than the one used to sampling actions.

6.5 Solving the maximization bias: Double Q-Learning

Q-learning often **overestimates the value function**. On every step, we take the **maximum over the estimates** of the action-value function of the next state, but what we need is an **estimate of the maximum** of the action-value function of the next state. We are using the **maximum over estimates as an estimate of the maximum**. Consider the Q-learning update rule:

$$G_{t:t+1} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha_t [G_{t:t+1} - Q(S_t, A_t)]$$

Here, the maximum over the estimates is used to select the action with the maximum estimated future reward. However, if the estimates are noisy or imprecise (which is often the case early in training), the **maximization tends to select actions with overestimated values**. This leads to a systematic bias towards overestimating the value of the chosen actions. This is a problem known as **maximization bias**. Imagine an action-value function whose actual values are all zeros, but the estimates have errors (0.11, 0.65, -0.44, -0.26, and so on). We know the actual maximum is zero, but the maximum over the estimates is 0.65. One way of dealing with this is a technique called **double Q-learning** Hasselt, Hado. "Double Q-learning." *Advances in neural information processing systems NIPS* (2010). In this approach, two separate action-value estimates are maintained, and the update rule is modified to reduce bias. One action-value function is used to select the action with the maximum value. The other action-value function is used to evaluate the value of that action, thus **breaking the dependence between action selection and value estimation**. This approach helps to reduce the overestimation problem by decoupling the

action selection from the max operator, leading to more stable and accurate learning. The issue, though, is now we are splitting the experience between two separate functions and this somewhat **slows down training**. Other Methods, like **Weighted Q-learning** and **softmax selection** can also mitigate the bias, though they are less common compared to Double Q-learning. We can implement the idea in Python:

```
def double_q_learning(env, gamma=0.99,
                     init_alpha=0.5, min_alpha=0.01,
                     init_epsilon=1.0, min_epsilon=0.1,
                     decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the two action-value functions
    q1 = np.zeros((env.observation_space, env.action_space), dtype=float);
    q2 = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # the episode loop
    for e in range(n_episodes):

        # we start episode by resetting the environment and the done flag
        state, done = env.reset(), False;

        # repeat until we hit a terminal state
        while not done:

            # we use the mean of our two action-value functions to select action
            action = select_action(state, (q1 + q2)/2., epsilons[e]);

            # step the environment using that action
            next_state, reward, terminated, truncated, info = env.step(action);
            if(terminated or truncated):
                done = True;

            # flip a coin to determine an update to q1 or q2
            if np.random.randint(2):
```

```
        # use the action q1 thinks is best ...
        argmax_q1 = np.argmax(q1[next_state]);

        # ...but get the value from q2 to calculate the target
        dq_learning_target = reward + gamma * q2[next_state][argmax_q1];

        # calculate the error
        dq_learning_error = dq_learning_target - q1[state][action];

        # update q1
        q1[state][action] = q1[state][action] + alphas[e] * dq_learning_error;

    else:

        # use the action q2 thinks is best ...
        argmax_q2 = np.argmax(q2[next_state]);

        # ...but get the value from q1 to calculate the target
        dq_learning_target = reward + gamma * q1[next_state][argmax_q2];
        dq_learning_error = dq_learning_target - q2[state][action];

        # update q2
        q2[state][action] = q2[state][action] + alphas[e] * dq_learning_error;

    # update state
    state = next_state;

    # extract the value functions
    q = (q1 + q2)/2.;
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save values for post analysis
    v_track[e] = v

    return q, v, pi, v_track;
```

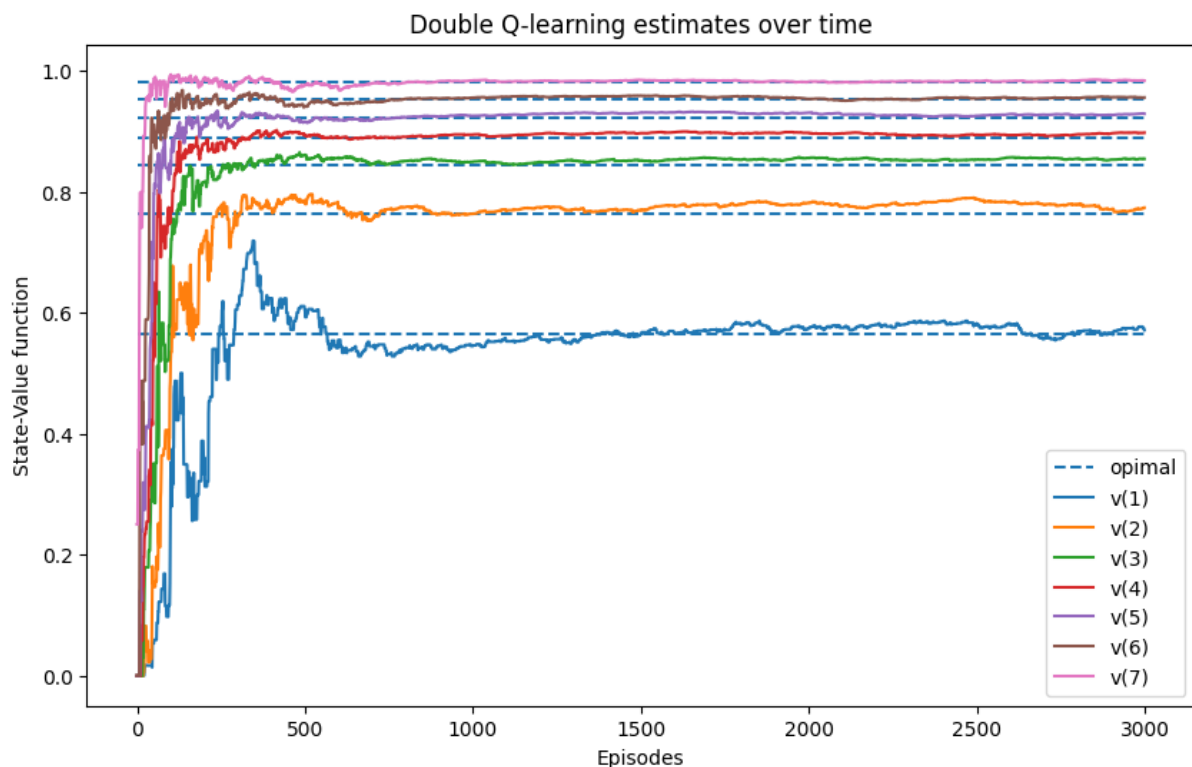
Now we can run Double Q-Learning against the Slippery Walk environment:

```
q_dql, v_dql, pi_dql, v_track_dql = double_q_learning(slippery_walk);
```

```
print(v_dql)
```

```
[0.          0.58634552 0.7786524  0.85104747 0.89197395 0.92670831
 0.95368596 0.98076035 0.          ]
```

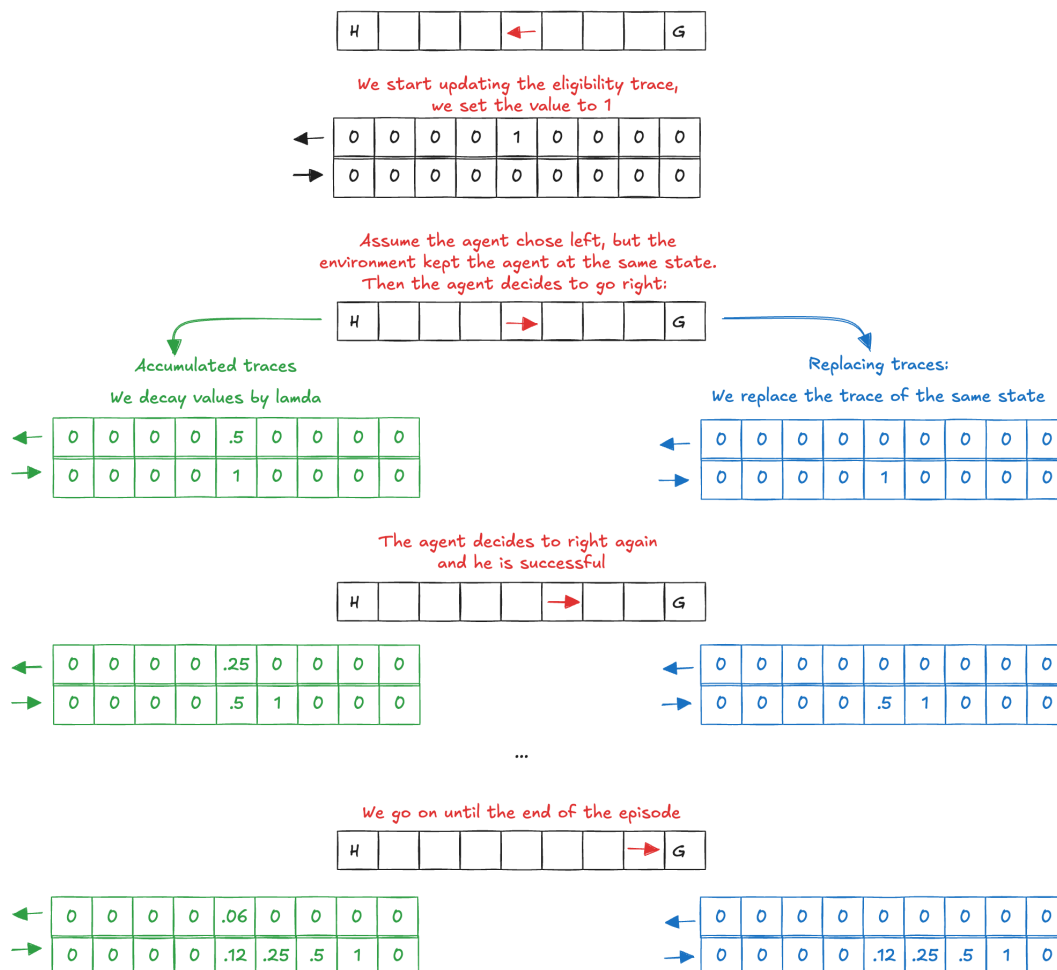
```
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_dql[:,1:8]);
plt.title('Double Q-learning estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



It is **slightly slower** than Q-learning to get the estimates to track the optimal values, but it does so in a **much more stable manner**. There's still a bit of over-estimation, but it's controlled.

6.6 Using multi-step estimates: SARSA(λ)

A straightforward improvement to the original SARSA agent is **to use the lambda-return instead of using a one-step bootstrapping target**. For adapting the eligibility trace to the estimation of action-value function, instead of using a vector for tracking visited states, we use an **eligibility matrix** for tracking visited state-action pairs. When the agent tries a state-action pair, the trace for this pair is incremented by one. Now, imagine there is a loop in the environment, and the agent tries the same state-action pair several times. Should we make this state-action pair **"more" responsible** for rewards obtained in the future, or should we make it **just responsible**? The first type of eligibility trace is called **accumulating trace** (and allows trace values higher than one), the other one is the **replacing trace** (it clips eligibility traces to a maximum value of one). The following figure shows the two strategy applied to the Slippery Walk environment:



Accumulating traces can "exaggerate" when confronted with frequency, while replacing traces moderate the blame assigned to frequent events. This moderation can help the more recent, but

rare events surface and be taken into account. We can implement this variation of SARSA in Python:

```
def sarsa_lambda(env, gamma=0.99,
                 init_alpha=0.5, min_alpha=0.01,
                 init_epsilon=1.0, min_epsilon=0.1,
                 lambda_=0.5, replacing_traces=True,
                 decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # eligibility traces
    E = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # the episode loop
    for e in range(n_episodes):

        # every new episode, set the eligibility of every state to zero
        E.fill(0);

        # we start episode by resetting the environment and the done flag
        state, done = env.reset(), False;

        # select the action (perhaps exploratory) for the initial state
        action = select_action(state, q, epsilons[e]);

        # repeat until we hit a terminal state
        while not done:

            # step the environment using that action
            next_state, reward, terminated, truncated, info = env.step(action);
            if(terminated or truncated):
                done = True;
```

```
# obtain the action for the next step
next_action = select_action(next_state, q, epsilons[e]);

# calculate the target using that next state-action pair
sarsa_lambda_target = reward + gamma * q[next_state][next_action];

# calculate the error
sarsa_lambda_error = sarsa_lambda_target - q[state][action];

# increment the state-action pair trace,
# and clip it to 1 if it's a replacing trace
if replacing_traces:
    E[state].fill(0);
E[state][action] = E[state][action] + 1;
if replacing_traces:
    E.clip(0, 1, out=E);

# apply the error to all eligible state-action pairs at once
# even though we're using the entire q-table, E will be mostly 0,
# and greater than zero for eligible pairs
q = q + alphas[e] * sarsa_lambda_error * E;

# decay the eligibility matrix
E = lambda_ * E;

# update state and action for the next step
state, action = next_state, next_action;

# extract the state-value function
v = np.max(q, axis=1);

# and the greedy policy
def pi(s):
    return np.argmax(q[s]);

# save values for post analysis
v_track[e] = v;

return q, v, pi, v_track;
```

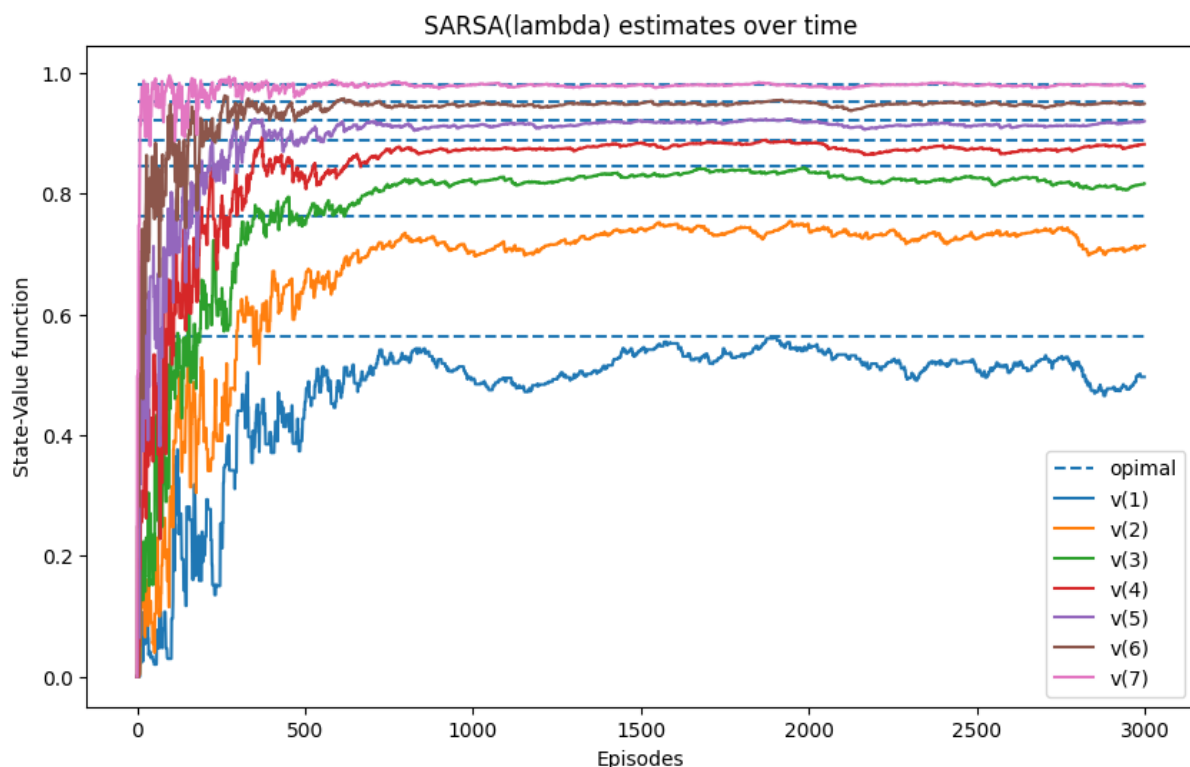
We can apply the algorithm to the same SWS environment:

```
q_sarsa_lambda, v_sarsa_lambda, pi_sarsa_lambda, v_track_sarsa_lambda =
    ↪ sarsa_lambda(slippery_walk);
```

```
print(v_sarsa_lambda);
```

```
[0.          0.49629043 0.713757    0.81630604 0.88155212 0.9197797
 0.94863288 0.9778924   0.          ]
```

```
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_sarsa_lambda[:,1:8]);
plt.title('SARSA(lambda) estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



By updating past state-action pairs, Sarsa(almbda) learns faster in environments where rewards are delayed. It is better at distributing the reward credit over multiple past actions, making it

effective in tasks with longer time dependencies. However it requires maintaining eligibility traces and tuning the lambda parameter, making it more computationally demanding than regular Sarsa. Moreover, the choice of lambda can significantly affect performance, and improper tuning might degrade learning performance.

6.7 Using multi-step estimates and decoupling behaviour from learning: $Q(\lambda)$

We can extend also the Q-learning in order to use the lambda-return for evaluation. We need to replace the target (that uses the max over the action in the next state) with a lambda-return. Moreover, since Q-learning is an off-policy method, we must use the eligibility trace with care. We're learning about the greedy policy, but following an exploratory policy. So, if an action we'll take on the next state is a greedy action (so if the exploratory policy follows the same path of the greedy policy), then we decay the eligibility matrix as usual. Otherwise, we must reset the eligibility matrix to zero because we'll no longer be learning about the greedy policy. We implement this idea in Python:

```
def q_lambda(env, gamma=0.99,
             init_alpha=0.5, min_alpha=0.01,
             init_epsilon=1.0, min_epsilon=0.1,
             lambda_=0.5, replacing_traces=True,
             decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # eligibility traces
    E = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # the episode loop
    for e in range(n_episodes):
```

```
# every new episode, set the eligibility of every state to zero
E.fill(0);

# we start episode by resetting the environment and the done flag
state, done = env.reset(), False;

# notice: we are preselecting the action as in SARSA, (we didn't do that in
↳ Q-learning)
# this is because we need to check whether our next action is greedy for
↳ deciding
# to decay the eligibility trace
action = select_action(state, q, epsilons[e])

# repeat until we hit a terminal state
while not done:

    # step the environment using that action
    next_state, reward, terminated, truncated, info = env.step(action);
    if(terminated or truncated):
        done = True;

    # select the next_action SARSA-style
    next_action = select_action(next_state, q, epsilons[e])

    # use it to verify that the action on the next step
    # will still come from the greedy policy
    next_action_is_greedy = q[next_state][next_action] == q[next_state].max()

    # still calculate the target as in regular Q-learning, using the max
    q_lambda_learning_target = reward + gamma * q[next_state].max();

    # and use the target to calculate TD error
    q_lambda_learning_error = q_lambda_learning_target - q[state][action];

    # increment the state-action pair trace,
    # and clip it to 1 if it's a replacing trace
    if replacing_traces:
        E[state].fill(0);
    E[state][action] = E[state][action] + 1
    if replacing_traces:
        E.clip(0, 1, out=E);
```

```

    # as before, we multiply the entire eligibility trace matrix
    # by the error and the learning rate corresponding to episode e,
    q = q + alphas[e] * q_lambda_learning_error * E

    # if the action we'll take on the next state (which we already selected)
    # is a greedy action,
    # then we decay the eligibility matrix as usual, otherwise, we must reset
    # the eligibility matrix to zero because we'll no longer be learning about
    # the greedy policy
    if next_action_is_greedy:
        E = lambda_ * E;
    else:
        E.fill(0);

    # update state and action for the next step
    state, action = next_state, next_action

    # extract the state-value function
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save values for post analysis
    v_track[e] = v;

    return q, v, pi, v_track;

```

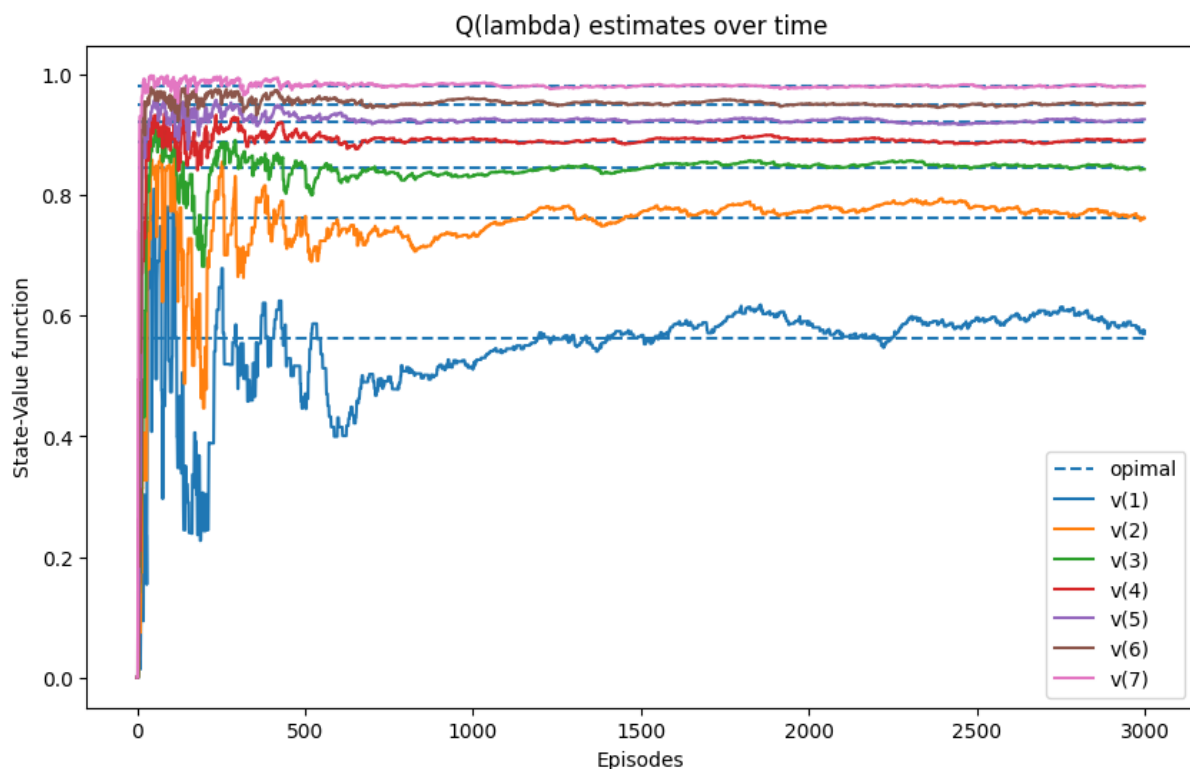
We can apply the algorithm to the same SWS environment:

```
q_ql_lambda, v_ql_lambda, pi_ql_lambda, v_track_ql_lambda = q_lambda(slippery_walk);
```

```
print(v_ql_lambda)
```

```
[0.          0.57036203 0.76187365 0.84324413 0.89254433 0.92592254
 0.9528339   0.98104672 0.          ]
```

```
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_ql_lambda[:,1:8]);
plt.title('Q(lambda) estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



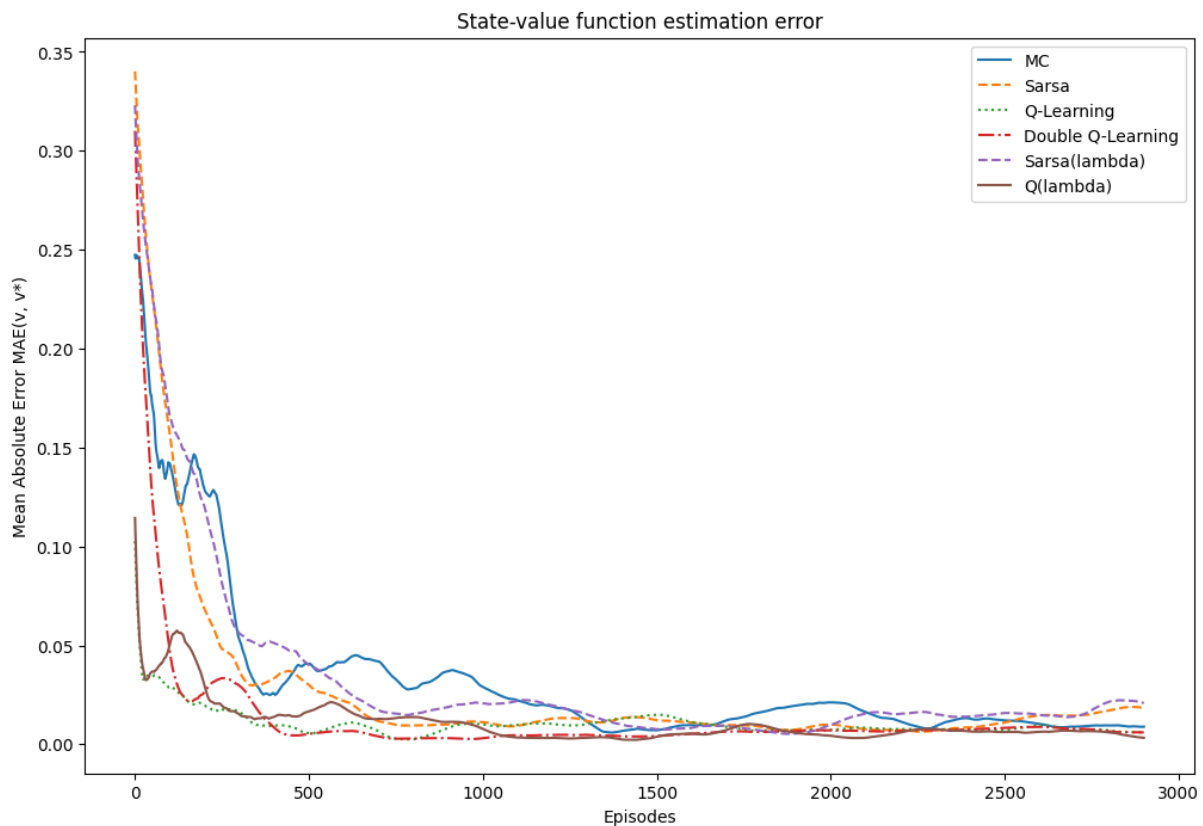
Also in this case, by using eligibility traces to update not only the current state-action pair, but also past state-action pairs that contributed to the current reward, $Q(\lambda)$ can learn faster by assigning credit to prior states. However, the same observations on complexity and the need for tuning made for Sarsa(λ) apply to $Q(\lambda)$ as well.

6.8 Comparison

In order to compare the different algorithms, we can consider the state-value function estimation error as the mean absolute error across all estimates from their respective optimal. Take a look at how quickly Q-learning drops near zero, but also how double Q-learning gets to the lowest error first:

```
def moving_average(a, n=100) :  
    ret = np.cumsum(a, dtype=float)  
    ret[n:] = ret[n:] - ret[:-n]  
    return ret[n - 1:] / n
```

```
plt.figure(figsize=(12,8))  
plt.plot(moving_average(np.mean(np.abs(v_track_mc - optimal_v), axis=1)), '-',  
        ↪ label='MC');  
plt.plot(moving_average(np.mean(np.abs(v_track_sarsa - optimal_v), axis=1)), '--',  
        ↪ label='Sarsa');  
plt.plot(moving_average(np.mean(np.abs(v_track_ql - optimal_v), axis=1)), ':',  
        ↪ label='Q-Learning');  
plt.plot(moving_average(np.mean(np.abs(v_track_dql - optimal_v), axis=1)), '-.',  
        ↪ label='Double Q-Learning');  
plt.plot(moving_average(np.mean(np.abs(v_track_sarsa_lambda - optimal_v), axis=1)),  
        ↪ '-', label='Sarsa(lambda)');  
plt.plot(moving_average(np.mean(np.abs(v_track_ql_lambda - optimal_v), axis=1)), '-',  
        ↪ label='Q(lambda)');  
plt.legend(loc=1, ncol=1);  
plt.title('State-value function estimation error');  
plt.xlabel('Episodes');  
plt.ylabel('Mean Absolute Error MAE(v, v*)');  
plt.show();
```

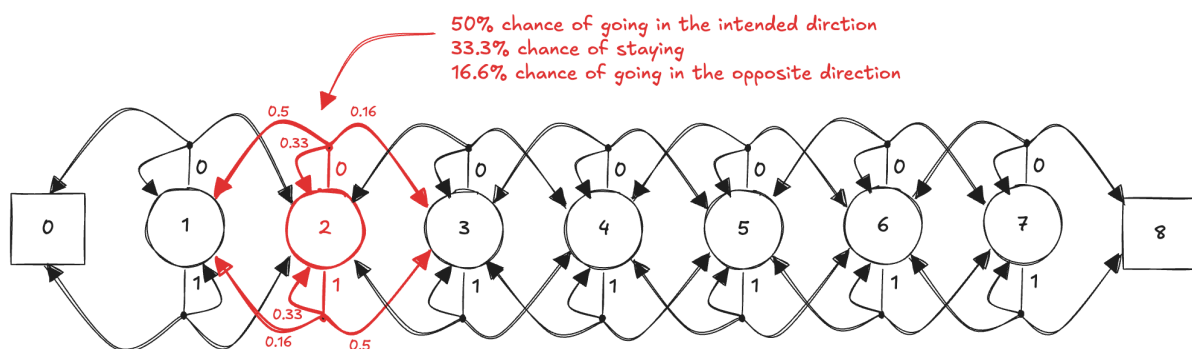


In the end, Monte Carlo is simple but slow and limited to episodic tasks; Sarsa is more conservative, focusing on actions based on the current policy, and works well when stability is needed; Q-learning aggressively learns the optimal policy but is prone to overestimation. Double Q-learning is a better alternative when stability is a concern, as it mitigates overestimation. Sarsa(lambda) and Q(lambda) provide a middle ground between learning speed and robustness by using eligibility traces, with Q(lambda) being faster due to its off-policy nature. The choice of algorithm often depends on the specific environment and the need for stability, exploration, and learning speed.

7 Model-based Reinforcement Learning

In the context of dynamic programming, algorithms **require a model** of the environment and they calculate optimal policies offline, on the other hand, in the reinforcement learning we presented several **model-free** learning methods. The advantage of model-free methods is that they doesn't require information about the MDP internals. Often the MDP is challenging to obtain in advance or it is even impossible to obtain: imagine to represents the game of Go with its 10.170 possible states or the StarCraft II game with 101.685 states. Not requiring the MDP in advance

is a of crucial importance. However, what if we don't require the MDP in advance, but perhaps we **learn one as we interact** with the environment? We can imagine agents that interact with the environment (like the model-free methods) and also learn a model of the environment from these interactions and use that model to improve their behavior (line dynamic programming). We call this second phase **plannig**, since it is not base on interaction. By exploiting the learned model, agents often require fewer experience samples to learn optimal policies. These methods are called **model-based reinforcement learning**. We use again the **Slippery Walk** environment. Remember, it is a walk, a single-row slippery (stochastic) grid-world environment, with seven non-terminal states. If the agent chooses to go left, there is a chance it does, but there is also a chance that it goes right, or that it stays in place:



```
import random
import numpy as np

class SlipperyWalk:
    def __init__(self):
        self.reset();

    def reset(self):
        self.observation_space = 9;
        self.action_space = 2;
        self.state = np.random.choice([1,2,3,4,5,6,7]);
        self.terminated = False;
        return self.state;

    def step(self, action):
        if self.terminated: raise ValueError('Episode has terminated');
        if action not in [0, 1]: raise ValueError('Invalid action');

        if(action==0): direction = -1;
        if(action==1): direction = 1;
```

```

direction_list = [direction, -direction, 0];
true_direction = random.choices(direction_list, weights=(50, 17, 33), k=1);
self.state += true_direction[0];

reward = 0
if self.state < 1: self.terminated = True;
if self.state > 7: self.terminated = True; reward = 1;
return self.state, reward, self.terminated, 0, 0;

```

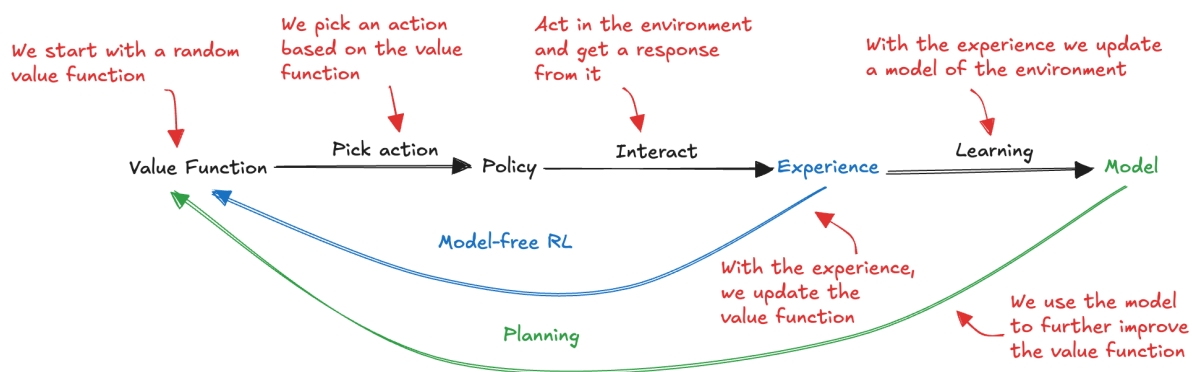
```

slippery_walk = SlipperyWalk();
optimal_v = [0., 0.5637, 0.763, 0.8449, 0.8892, 0.922, 0.9515, 0.9806, 0.];

```

7.1 Learning sample models: Dyna-Q

Dyna-Q (Richard S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting", ACM SIGART Bulletin, 1991) consists of **interleaving a model-free method** (like Q-learning) and **a model-based method** (like value iteration), using both experiences sampled from the environment and experiences sampled from the learned model to improve the action-value function:



We keep track of the transition and reward functions as three-dimensional tensors indexed by state, action, and next state. The transition tensor keeps count of the number of times we've seen the tuple (state, action, next state) indicating how many times we arrived at the next state from the state when selecting that action. The reward tensor holds the average reward we received on the tuple (state, action, next state) indicating the expected reward when we select the action on that state and transition to the next state. To implement the Dyna-Q, we need to decay alpha and epsilon, using the same function already used several times:


```
def decay_schedule(init_value, min_value, decay_steps, max_steps):

    # calculate the number of the remaining steps after the decay
    rem_steps = max_steps - decay_steps;

    # logspace returns numbers spaced evenly on a log scale
    # base^start is the starting value of the sequence,
    # base^stop is the final value of the sequence
    # num is the number of values to generate
    # base is the base of the log space
    values = np.logspace(start=0, stop=-2, num=decay_steps, base=10);

    # because the values may not end exactly at 0, given it's the log,
    # we change them to be between 0 and 1 so that the curve looks smooth and nice.
    values = (values - values.min()) / (values.max() - values.min());

    # linear transformation and get points between init_value and min_value
    values = (init_value - min_value) * values + min_value;

    # repeats the rightmost value rem_step number of times
    values = np.pad(values, (0, rem_steps), 'edge');

    return values;
```

And again we use need to explore and we can use the decaying epsilon-greedy approach to control the exploration:

```
def select_action(state, q, epsilon=0.1):

    # if the random number is greater than epsilon,
    if np.random.uniform() > epsilon:
        # pick the action with the highest Q value
        action = np.argmax(q[state]);
    else:
        # otherwise, pick a random action
        action = np.random.randint(len(q[0]));

    return action;
```

The we can implement the Dyna-Q algorithm:

```
def dyna_q(env, gamma=0.99,
          init_alpha=0.5, min_alpha=0.01,
          init_epsilon=1.0, min_epsilon=0.1,
          n_planning=5,
          decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # create a function to keep track of the transition function
    # and another one to keep track of the reward signal.
    transitions = np.zeros((env.observation_space, env.action_space,
↪ env.observation_space), dtype=int);
    rewards = np.zeros((env.observation_space, env.action_space,
↪ env.observation_space), dtype=float);

    # create a list to save copies of the model and of the planning
    # for offline analysis
    transitions_track = [];
    planning_track = [];

    # the episode loop
    for e in range(n_episodes):

        # we start episode by resetting the environment and the done flag
        state, done = env.reset(), False;

        # repeat until we hit a terminal state
        while not done:

            # select the action, as in Q-learning
            action = select_action(state, q, epsilons[e]);

            # step the environment using that action
```

```
next_state, reward, terminated, truncated, info = env.step(action);
if(terminated or truncated):
    done = True;

# start learning the model
# increment the transition count for the state-action-next_state triplet
transitions[state][action][next_state] += 1;

# calculate an incremental mean of the reward signal
# and use it to learn the reward signal
difference = reward - rewards[state][action][next_state];
rewards[state][action][next_state] += (difference /
↳ transitions[state][action][next_state]);

# calculate the target as usual in Q-learning!
# (off-policy, using the max)
dyna_target = reward + gamma * q[next_state].max();

# calculate the error
dyna_error = dyna_target - q[state][action];

# update the action-value function
q[state][action] = q[state][action] + alphas[e] * dyna_error;

# get into the planning steps, first back up the next state variable
backup_next_state = next_state;

# planning loop
for _ in range(n_planning):

    # make sure there have been updates to the Q-function before,
    # otherwise, there's not much to plan
    if q.sum() == 0:
        break;

    # select a state from a list of states already visited by
    # the agent in experience
    visited_states = np.where(np.sum(transitions, axis=(1, 2)) > 0)[0];
    state = np.random.choice(visited_states);

    # then select an action that has been taken on that state
    actions_taken = np.where(np.sum(transitions[state], axis=1) > 0)[0];
    action = np.random.choice(actions_taken);
```

```

        # use the transitions to calculate probabilities of a next state
        # and then a next state
        probs = transitions[state][action]/transitions[state][action].sum();
        next_state = np.random.choice(np.arange(env.observation_space),
↪ size=1, p=probs)[0];

        # use the rewards model as the reward
        reward = rewards[state][action][next_state];

        # save the planning decision for offline analysis
        planning_track.append((state, action, reward, next_state));

        # update the Q-function using that simulated experience!
        dyna_target = reward + gamma * q[next_state].max();
        dyna_error = dyna_target - q[state][action];
        q[state][action] = q[state][action] + alphas[e] * dyna_error;

        # at the end of the planning, set the state as the next state
        state = backup_next_state;

    # extract the state-value function
    v = np.max(q, axis=1);

    # and the greedy policy
    def pi(s):
        return np.argmax(q[s]);

    # save the value functions and the transitions model for analysis
    v_track[e] = v;
    transitions_track.append(transitions.copy());

    return q, v, pi, v_track, transitions_track, np.array(planning_track);

```

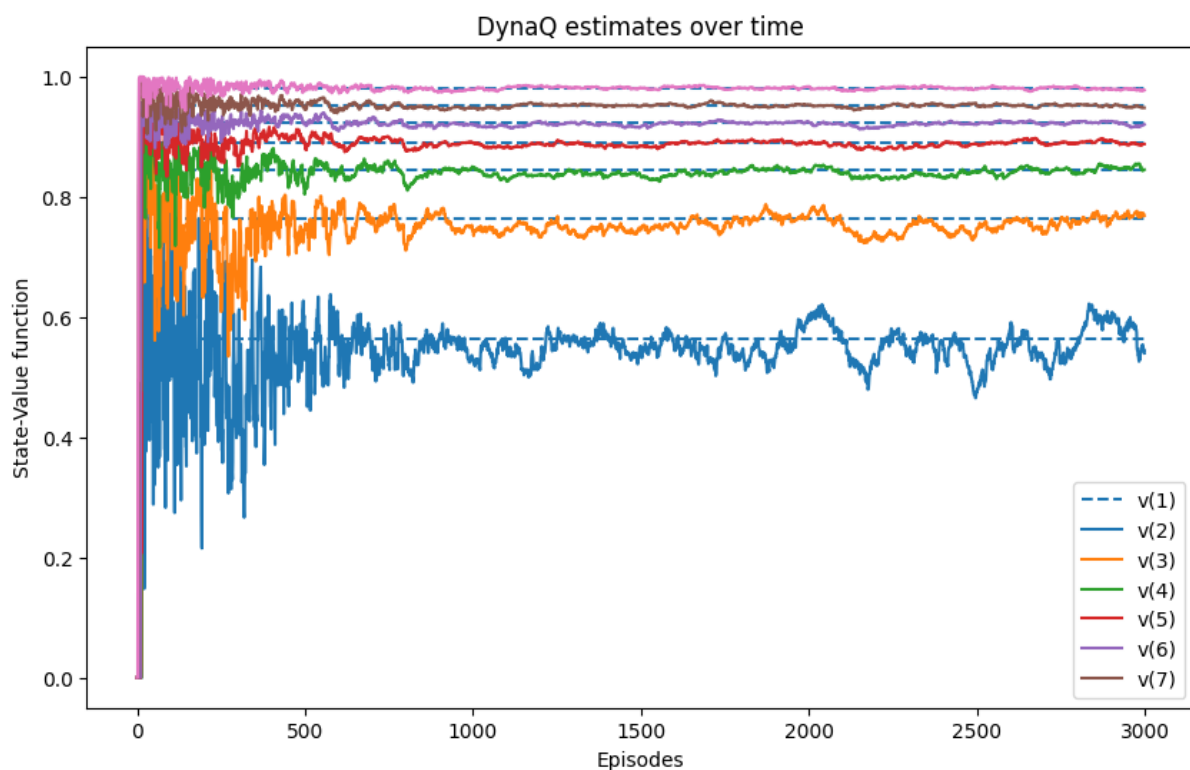
We can apply DynaQ to the Slippery Walk environment and try to understand the model learned by the algorithm.

```

q_dyna, v_dyna, pi_dyna, v_track_dyna, transitions_track_dyna, planning_dyna =
↪ dyna_q(slippery_walk)

```

```
import matplotlib.pyplot as plt
legends = ['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'];
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_dyna[:,1:8]);
plt.title('DynaQ estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);
plt.show();
```



Dyna-Q can **learn an optimal policy much faster** than model-free methods, this is because it can learn from both real and simulated experiences. This is particularly important in applications where interacting with the environment is expensive or time-consuming, such as robotics. Of course, the performance is dependent on the accuracy of its learned model. If the model is inaccurate, the agent may make poor decisions. This is especially problematic in changing environments, where the model needs to be constantly updated. Moreover, planning can be computationally expensive. In order to check the model of the environment learned by DynaQ, we can plot a 3D graph of the transitions at different episodes. The axis to the right is the initial state, the axis to the left is the landing state, the colors are the actions, and bar heights are

the transition probabilities.

```
def plot_transition_model(transitions):

    np.seterr(divide='ignore', invalid='ignore');

    transitions_prob = transitions / transitions.sum(axis=2, keepdims=True);

    fig = plt.figure(figsize=(20,12));
    ax = fig.add_subplot(111, projection='3d');
    ax.view_init(elev=20, azim=50);

    plt.title('Learned MDP');
    ax.set_xlabel('Landing state');
    ax.set_ylabel('Initial state');
    ax.set_zlabel('Transition probabilities');
    ax.set_zlim(0, 1);

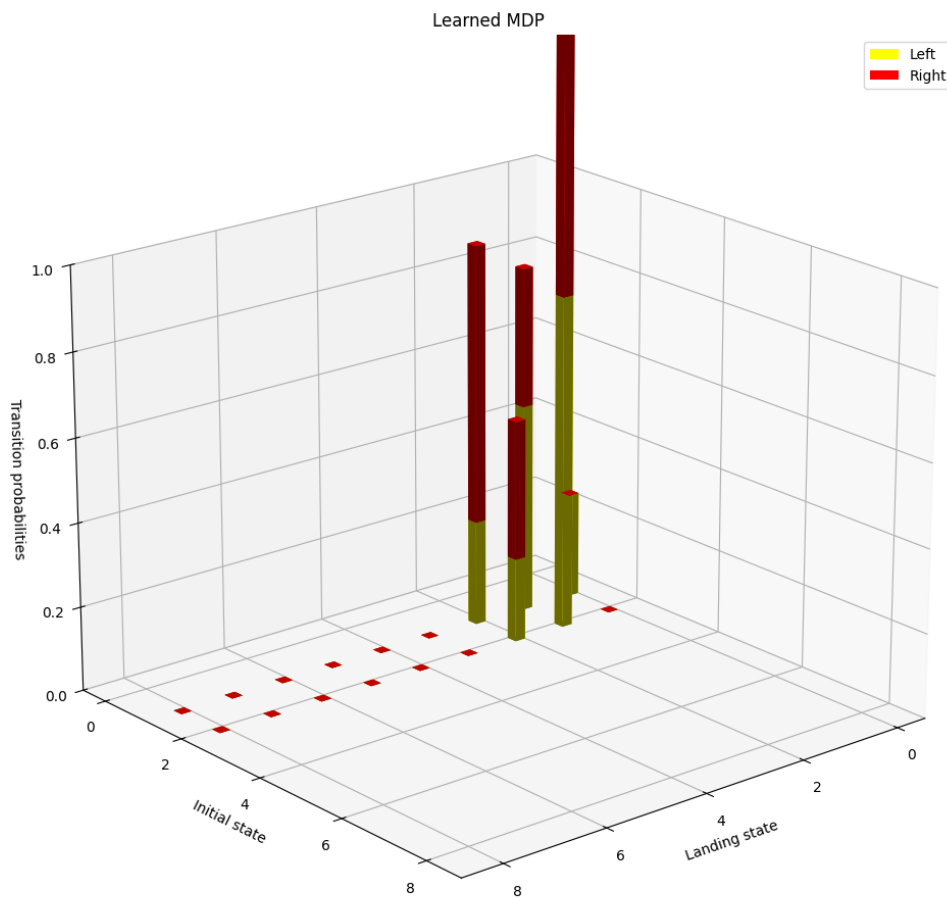
    x_data, y_data = np.meshgrid(range(transitions.shape[0]),
    ↪ range(transitions.shape[0]));
    x_data = x_data.flatten();
    y_data = y_data.flatten();
    left_data = transitions_prob[:, 0, :].flatten();
    right_data = transitions_prob[:, 1, :].flatten();

    ax.bar3d(x_data, y_data, np.zeros_like(left_data), 0.2, 0.2, left_data,
    ↪ color='yellow', shade=True);
    ax.bar3d(x_data, y_data, left_data, 0.2, 0.2, right_data, color='red',
    ↪ shade=True);

    left_proxy = plt.Rectangle((0, 0), 1, 1, fc='yellow');
    right_proxy = plt.Rectangle((0, 0), 1, 1, fc='red');
    plt.legend((left_proxy, right_proxy), ('Left', 'Right'));

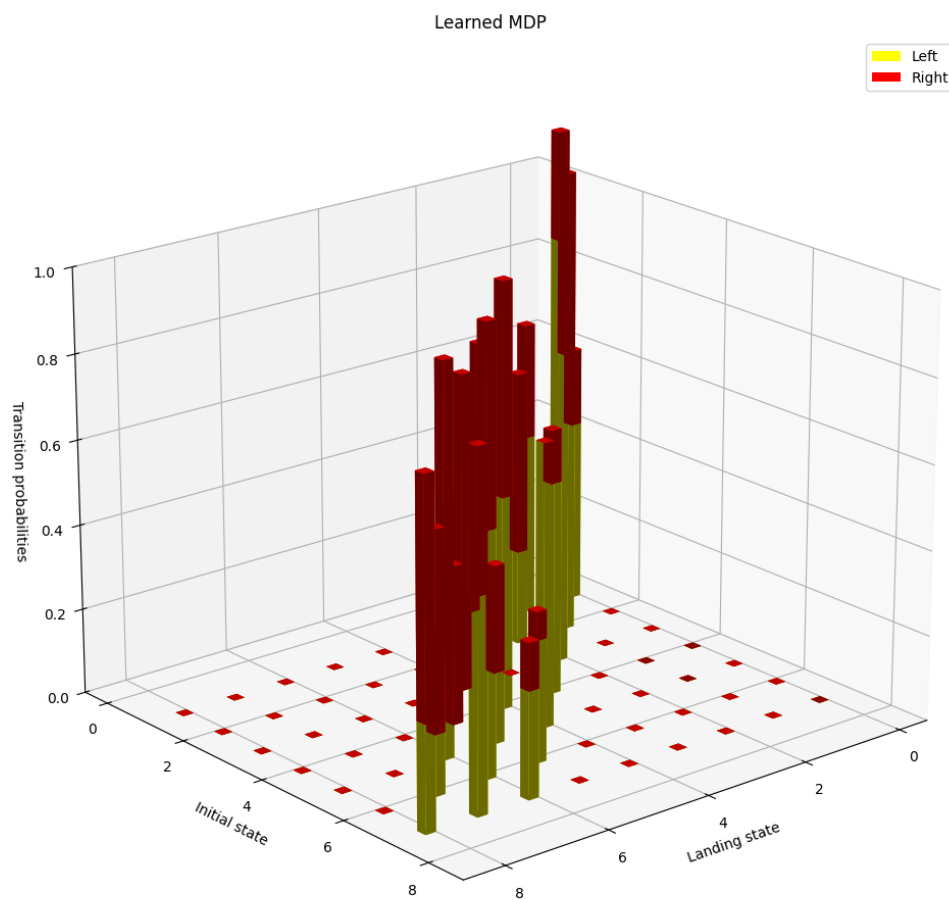
    plt.show();

plot_transition_model(transitions_track_dyna[0]);
```



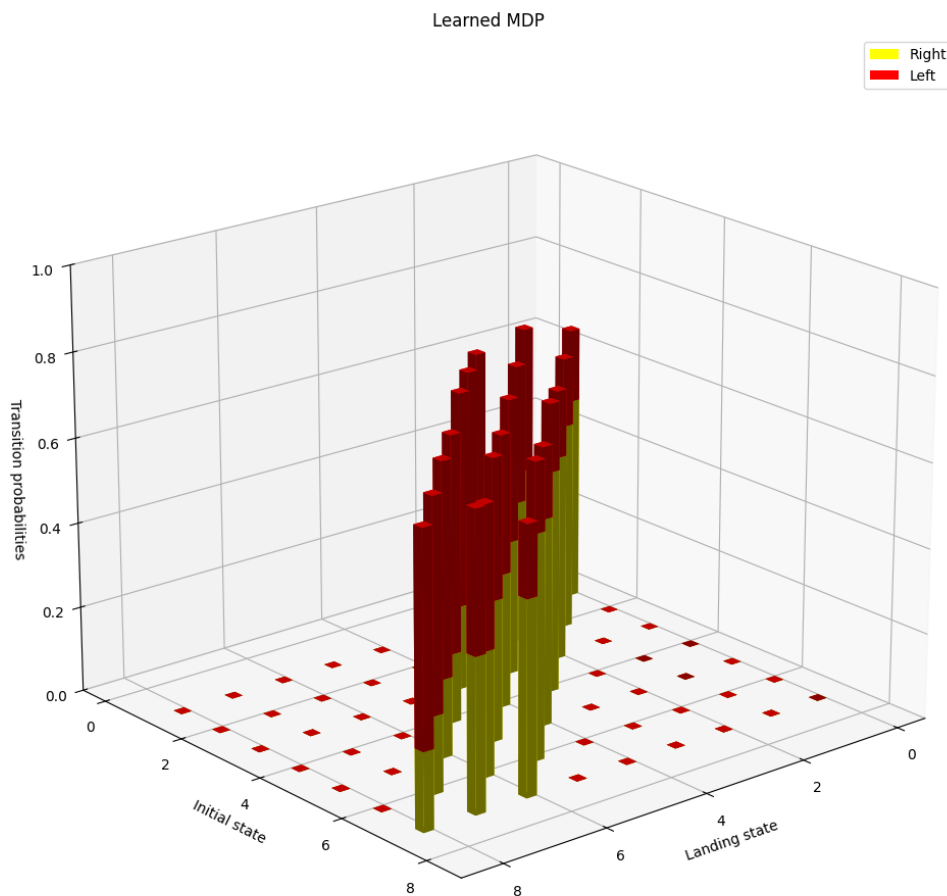
This is the model that Dyna-Q has learned after one episode. There are obvious issues with this model, but also, this is only after a single episode. This could mean trouble when using the learned model early on because there will be a bias when sampling an incorrect model.

```
plot_transition_model(transitions_track_dyna[10]);
```



Only after 10 episodes, we can see the model taking shape. We can see the right probabilities coming together.

```
plot_transition_model(transitions_track_dyna[len(transitions_track_dyna)-1]);
```

At the end, the probabilities are good enough and describe the MDP correctly. Obviously, this is a simple environment, so the agent can gather enough experience samples for building an MDP quickly.

7.2 Making plans for the immediate future: Trajectory sampling

During planning in DynaQ, we sample a state uniformly from the state-action pairs that have been visited (so we don't waste time with state-action pairs about which the model has no information) and then we sample uniformly an action from previously selected actions. This point can be more effective if we used **a state that we expect to encounter during the current episode** and if **sample actions from the same behavioral policy (on policy) or from a greedy policy with respect to the current estimates (off policy)** at the given time step. This is what **Trajectory sampling** does (A.G. Barto, S.J. Bradtke, S.P. Singh, "Learning to act using real-time dynamic programming", Artificial intelligence, 1995. We can implement it in Python. For the

most part, the code is the same as Dyna-Q, with a few exceptions:

```
def trajectory_sampling(env, gamma=0.99,
                       init_alpha=0.5, min_alpha=0.01,
                       init_epsilon=1.0, min_epsilon=0.1,
                       n_planning=5,
                       decay_episodes=1000, n_episodes=3000):

    # calculate the learning rate schedule for all episodes
    alphas = decay_schedule(init_alpha, min_alpha, decay_episodes, n_episodes);

    # calculate epsilon for all episodes
    epsilons = decay_schedule(init_epsilon, min_epsilon, decay_episodes, n_episodes);

    # initialize the current estimate of the action-value function
    q = np.zeros((env.observation_space, env.action_space), dtype=float);

    # create a list to save copies of v for offline analysis
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float);

    # create a function to keep track of the transition function
    # and another one to keep track of the reward signal.
    transitions = np.zeros((env.observation_space, env.action_space,
↪ env.observation_space), dtype=int);
    rewards = np.zeros((env.observation_space, env.action_space,
↪ env.observation_space), dtype=float);

    # create a list to save copies of the model and of the planning
    # for offline analysis
    transitions_track = [];
    planning_track = [];

    # the episode loop
    for e in range(n_episodes):

        # we start episode by resetting the environment and the done flag
        state, done = env.reset(), False;

        # repeat until we hit a terminal state
        while not done:

            # select the action, as in Q-learning
            action = select_action(state, q, epsilons[e])
```

```
# step the environment using that action
next_state, reward, terminated, truncated, info = env.step(action);
if(terminated or truncated):
    done = True;

# start learning the model
# increment the transition count for the state-action-next_state triplet
transitions[state][action][next_state] += 1;

# calculate an incremental mean of the reward signal
# and use it to learn the reward signal
difference = reward - rewards[state][action][next_state];
rewards[state][action][next_state] += (difference /
↪ transitions[state][action][next_state]);

# calculate the target as usual in Q-learning!
# (off-policy, using the max)
ts_target = reward + gamma * q[next_state].max();

# calculate the error
ts_error = ts_target - q[state][action];

# update the action-value function
q[state][action] = q[state][action] + alphas[e] * ts_error;

# get into the planning steps, first back up the next state variable
backup_next_state = next_state;

# planning loop
for _ in range(n_planning):

    # make sure there have been updates to the Q-function before,
    # otherwise, there is not much to plan
    if q.sum() == 0:
        break;

    # select the action following the behavioral policy or from the greedy
    ↪ policy
    action = select_action(state, q, epsilons[e]);
    #action = q[state].argmax();

    # if we haven't experienced the transition,
```

```

        # planning would be a mess, so break out
        if not transitions[state][action].sum():
            break;

        # otherwise, get the probabilities of next_state and sample
        # the model accordingly, as in DynaQ
        probs = transitions[state][action]/transitions[state][action].sum();
        next_state = np.random.choice(np.arange(env.observation_space),
        ↪ size=1, p=probs)[0];

        # get the reward as prescribed by the reward-signal model, as in DynaQ
        reward = rewards[state][action][next_state];

        # save the planning decision for offline analysis
        planning_track.append((state, action, reward, next_state));

        # update the Q-function using that simulated experience!
        ts_target = reward + gamma * q[next_state].max();
        ts_error = ts_target - q[state][action];
        q[state][action] = q[state][action] + alphas[e] * ts_error;

        # we update the state variable before we loop and
        # continue the planning steps
        state = next_state;

        # at the end of the planning, set the state as the next state
        state = backup_next_state;

        # extract the state-value function
        v = np.max(q, axis=1);

        # and the greedy policy
        def pi(s):
            return np.argmax(q[s]);

        # save the value functions and the transitions model for analysis
        v_track[e] = v;
        transitions_track.append(transitions.copy());

    return q, v, pi, v_track, transitions_track, np.array(planning_track);

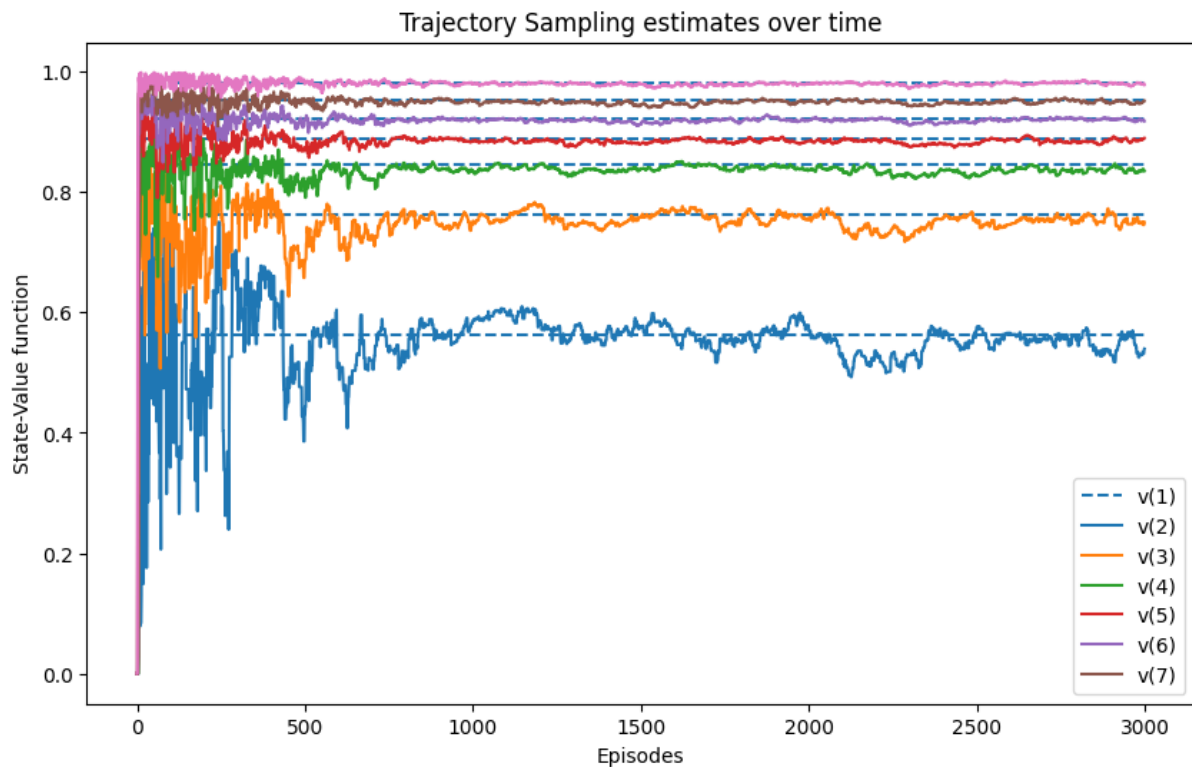
```

We apply the new algorithm to the Slippery Walk environment:

```
q_ts, v_ts, pi_ts, v_track_ts, transitions_track_ts, planning_ts =
    trajectory_sampling(slippery_walk);
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,6));
plt.hlines(optimal_v[1:8], 0, 3000, linestyle='--');
plt.plot(v_track_ts[:,1:8]);
plt.title('Trajectory Sampling estimates over time');
plt.ylabel('State-Value function');
plt.xlabel('Episodes');
plt.legend(legends);

plt.show();
```

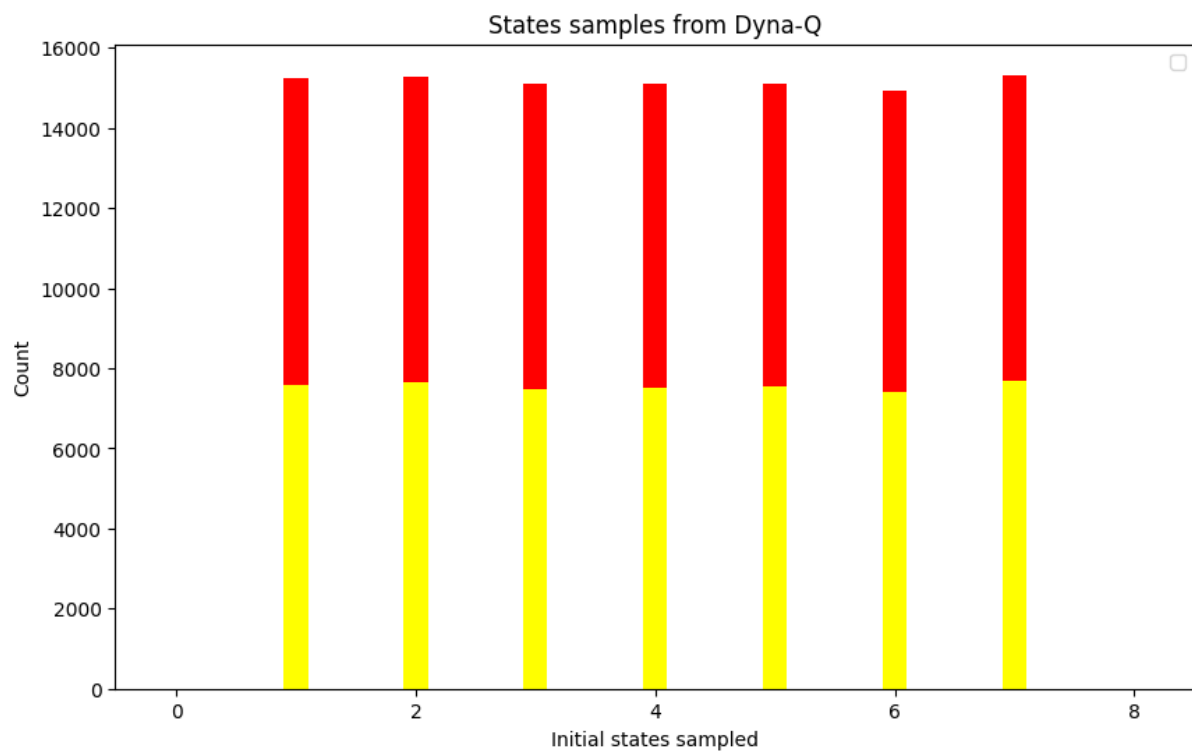


The uniform sampling of Dyna-Q encourages more exploration, potentially leading to the discovery of novel strategies or shortcuts. However, this can also make it less efficient in exploiting already known rewarding paths. Trajectory Sampling, with its focus on the current policy, prioritizes exploitation, leading to faster convergence to a good (though not necessarily optimal) policy. Dyna-Q, due to its reliance on a less focused sampling strategy, can be more sensitive to model inaccuracies, especially in the early stages of learning. Sampling from poorly explored

regions with an inaccurate model might lead to poor decision-making. Trajectory Sampling, being more focused on frequently encountered states, is less likely to be affected by inaccuracies in less visited parts of the model. In dynamic environments, Dyna-Q exploration might prove beneficial as it can lead to the discovery of new paths or rewards. However, its uniform sampling might make it slower to adapt to changes compared to Trajectory Sampling, which quickly focuses on relevant trajectories under the updated policy. We can compare the sampling differences by plotting the states that were sampled by the planning phase of the two algorithms and the actions selected in those states:

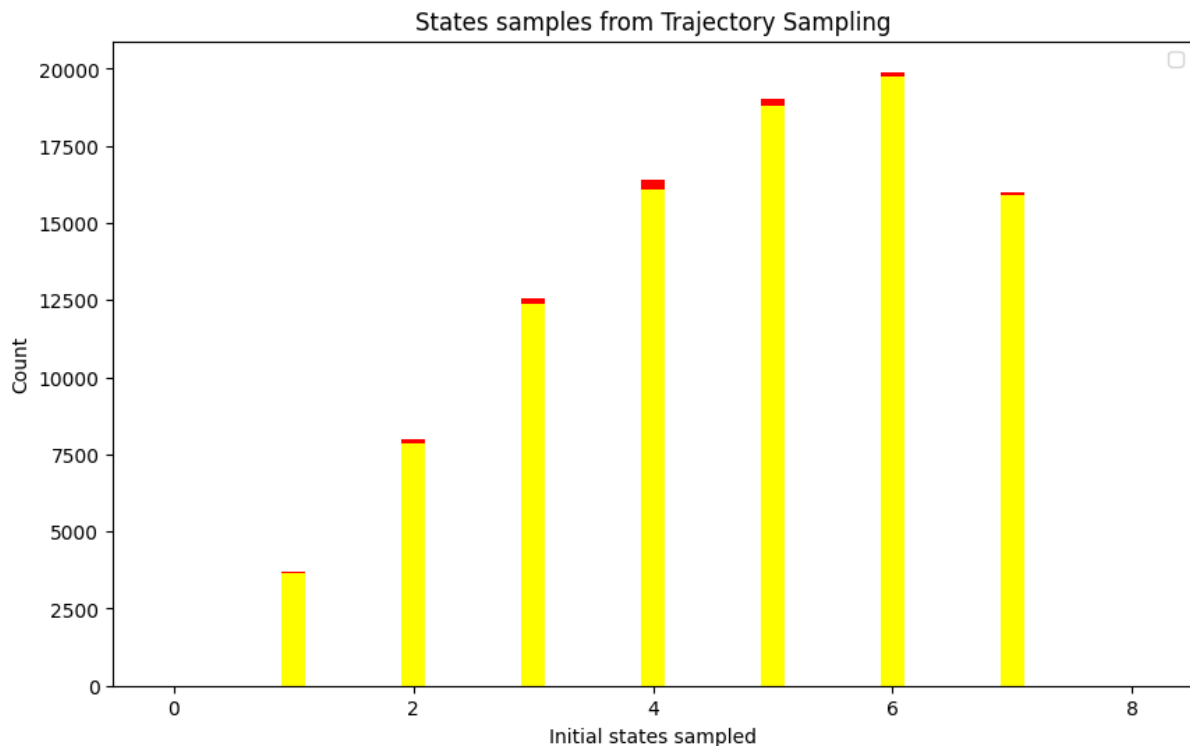
```
def plot_state_action_sampling(planning, algo):
    plt.figure(figsize=(10,6));
    plt.title('States samples from {}'.format(algo));
    plt.xlabel('Initial states sampled',);
    plt.ylabel('Count');
    plt.legend(['Left', 'Right']);
    for s in np.arange(9):
        actions = planning[np.where(planning[:,0]==s)[0], 1];
        left = len(actions[actions == 0]);
        right = len(actions[actions == 1]);
        plt.bar(s, right, 0.2, color='yellow');
        plt.bar(s, left, 0.2, color='red', bottom=right);
    plt.show();
```

```
plot_state_action_sampling(planning_dyna, algo='Dyna-Q')
```



Notice that Dyna-Q samples uniformly at random, not only the states, but also the actions taken in those states.

```
plot_state_action_sampling(planning_ts, algo='Trajectory Sampling')
```

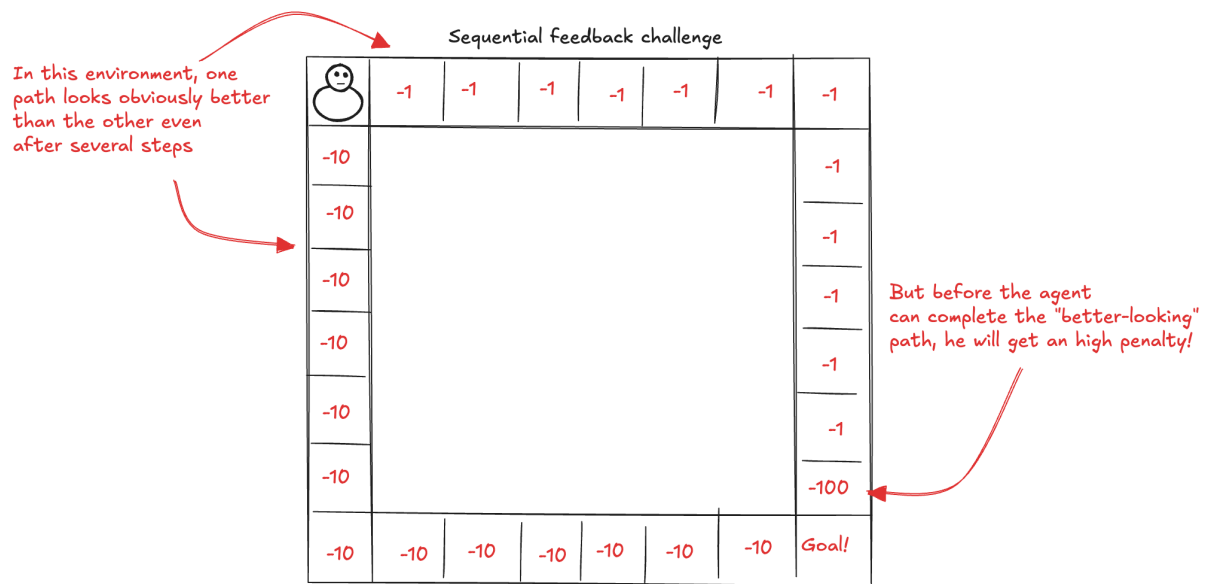


Since the rightmost state is the only non-zero reward state, the greedy trajectory sampling strategy samples the model in an attempt to improve greedy action selection. This is the reason why the states sampled are skewed toward the goal state. The same happens with the sampling of the action. As we can see, the right action is sampled far more than the left action across the board. In that way, Trajectory Sampling **lands on the goal state far more often, therefore experiencing non-zero rewards from the model more frequently.**

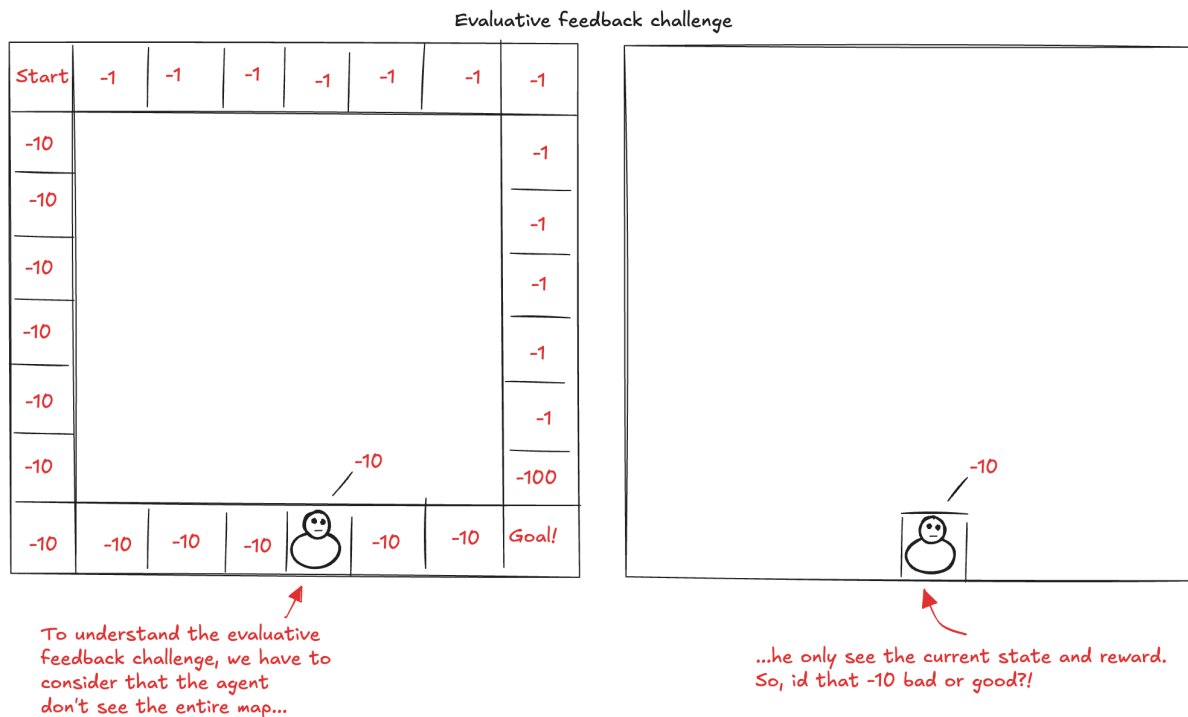
8 Neural Fitted Q (NFQ)

In reinforcement learning, we build agents that are capable of learning from feedback that is simultaneously **evaluative, delayed, and sampled.**

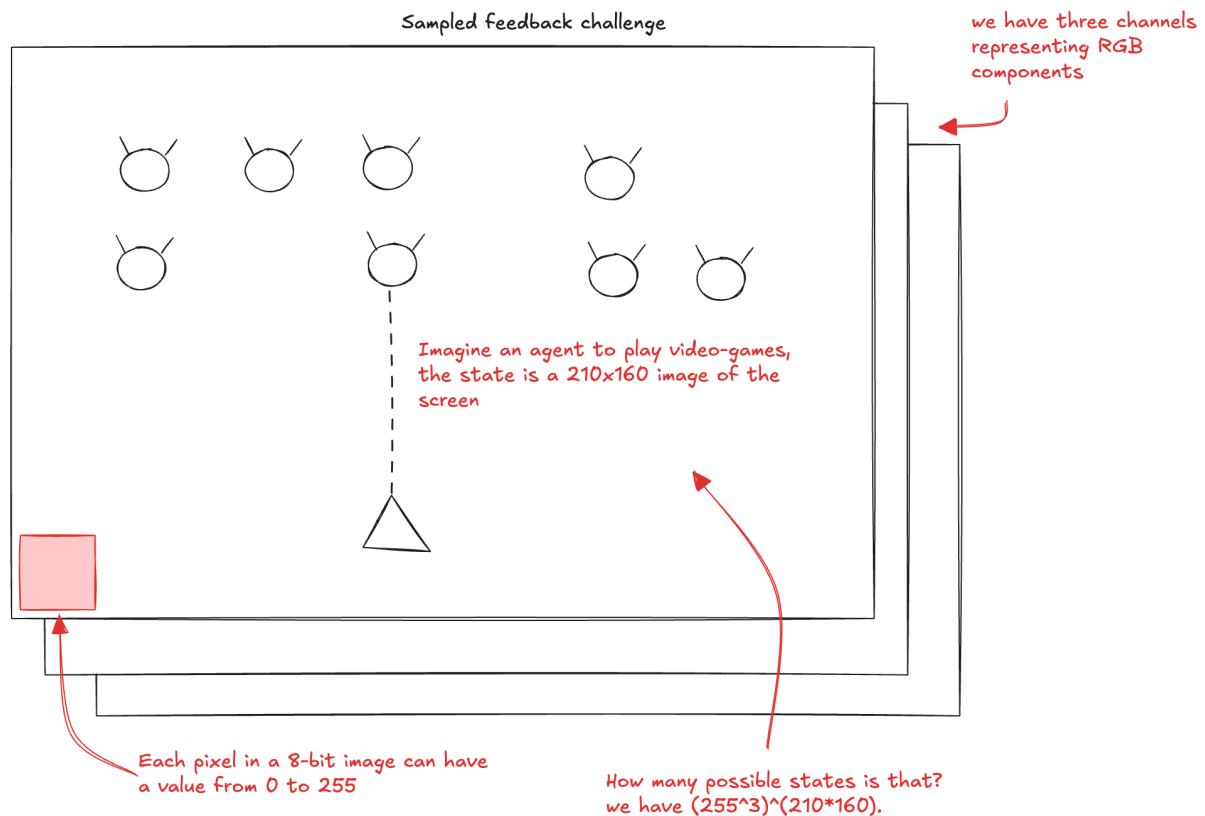
Delayed feedback: imagine a chess game in which you make a few wrong moves early on, but the consequences only manifest at the end of the game. Delayed feedback makes it tricky to interpret the source of the feedback and rise the **temporal credit assignment problem:** the challenge of determining which state, action, or state-action pair is responsible for a reward. The opposite is **immediate feedback.** In supervised learning decisions don't have long-term consequences. For example, in a classification problem, classifying an image, whether correctly or not, has no bearing on future performance.



Evaluative feedback: the goodness of the feedback is only relative, because the environment is uncertain. The agent doesn't know the actual dynamics of the environment (no access to the transition and reward functions). As a result, the agent must explore the environment, but the problem is that by exploring the agent misses capitalizing on its current knowledge. This is the exploration-exploitation trade-off. The opposite is **supervised feedback**. In a classification problem, during learning, the model is given the correct labels for each of the samples provided: there's no guessing.



Sampled feedback: in real problems, agents are unlikely to sample all possible feedback exhaustively. Agents **need to generalize** using the gathered feedback and come up with intelligent decisions based on that generalization. The opposite is **exhaustive feedback**, where agents have access to all possible samples. Tabular reinforcement learning only needs to sample for long enough to gather all necessary information for optimal performance and they have optimal convergence guarantees. Common assumptions, such as "sampling every state-action pair infinitely often", are reasonable assumptions in small grid worlds with finite state and action spaces, but not in real settings. Consider for example an agent that play a video game:



The vast number of states in a video game highlights the limitations of traditional tabular reinforcement learning and necessitates the use of learning methods which can handle high-dimensional state spaces and learn effective policies.

8.1 The need for generalization

The main drawback of the methods that we have seen so far is that they use of a table to represent value functions (for this reason we call them) **tabular reinforcement learning**) and this is **no practical for complex problems**. For example, Q-learning estimates the values of the optimal value function using a matrix indexed by states and actions:

An action-state value function is indexed by the state and the action and it returns the expected reward-to-go for taking that action at that state



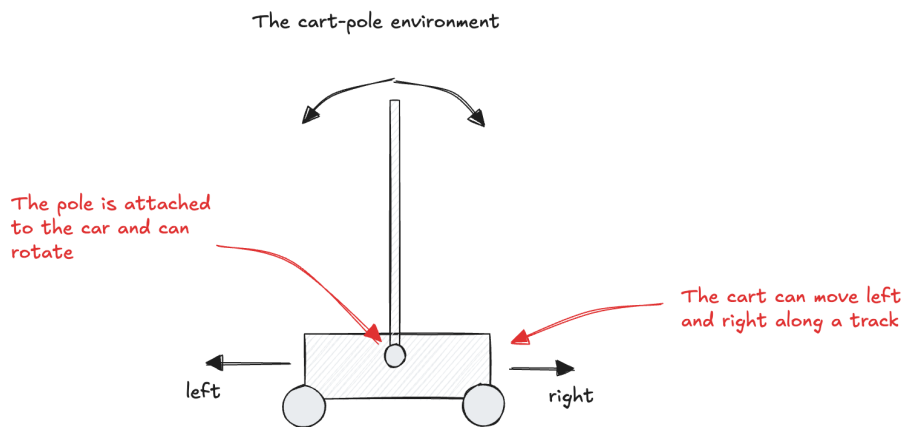
	Q	States			
		0	1	2	3
Actions	0	-1.5	-0.2	1.2	5.7
	1	4.2	-2.1	2.7	6.1

In many of the tasks to which we would like to apply reinforcement learning **the state space is combinatorial and enormous**. Environments can have **high-dimensional state spaces** (the number of variables that comprise a single state is vast) or can additionally have **continuous variables** (a single variable can take on an infinite number of values). How do we make a table that takes all these values into account? We can **discretize the state space**, but they can still take on a large number of values. Tabular solutions are **impractical** for the size of matrices. Moreover, the problem is not just the memory needed for large tables, but **the time needed to fill them accurately**. In many of our target tasks, **almost every state encountered will never have been seen before**. To make sensible decisions it is necessary **to generalize** from previous encounters with different states that are **in some sense similar** to the current one. Tabular solutions are **inefficient** also for the **lack of generalization**, that can discover underlying relationships. But, how can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset? Of course, in those conditions, we cannot expect to find optimal policies, our goal should be instead to find **good approximate solutions** using **limited computational resources**.

Generalization from examples has already been extensively studied, and we do not need to invent totally new methods for use in reinforcement learning. To some extent we need only **combine reinforcement learning methods with existing generalization methods**. The kind of generalization we require is often called **function approximation**, because it takes examples from a desired function (e.g., a value function) and attempts to generalize from them to construct an approximation of the entire function. Function approximation is an instance of **supervised learning**. In theory, any of the methods studied in the field of machine learning can be used in the role of function approximator within reinforcement learning algorithms, although in practice some fit more easily into this role than others.

8.2 The Cart-pole environment

In order to illustrate the need for generalization, we will use a classic control problem, the **Cart-pole**. The goal is to balance a pole on a cart that can move along a frictionless track. The pole starts upright and the goal is to prevent it from falling over by applying forces to the cart. The state space is low dimensional but **continuous**. Training is fast, yet still somewhat challenging, and function approximation can help.



The state space is comprised of four variables:

- position of the cart on the track (x) with a range from -2.4 to 2.4 ;
- velocity of the cart along the track (vc) with a range from $-\infty$ to ∞ ;
- angle of the pole (θ) with a range of -40 degrees to 40 degrees;
- velocity at the tip (vp) of the pole with a range of $-\infty$ to ∞ .

There are two available actions in every state:

- action 0 applies a -1 force to the cart (push it left)
- action 1 applies a $+1$ force to the cart (push it right)

The cart reach a terminal state if:

- the angle θ is more than 12 degrees away from the vertical position;
- the center of the cart is more than 2.4 units from the center of the track;
- the episode count reaches 500 time steps.

The reward function is $+1$ for every time step. The environment is available in the Gymnasium library.

```
import gymnasium

env = gymnasium.make('CartPole-v1', render_mode="rgb_array");
```

We can get the state space and the action space sizes of the environment:

```
state_size = env.observation_space.shape;
action_size = env.action_space.n;

print('State Space: ', state_size);
print('Action Space: ', action_size);
```

State Space: (4,)

Action Space: 2

We want to be able to reproduce the results, so we will fix the random seed:

```
import torch;
import numpy as np;
import random;

# set the seeds for reproducibility of results
seed = 34;

torch.manual_seed(seed);
np.random.seed(seed);
random.seed(seed);

env.reset(seed=seed)
env.action_space.seed(seed)
env.observation_space.seed(seed)
```

34

As a baseline for comparison, we introduce a **random policy**, which selects actions uniformly at random from the action space; although it has no learning capability, it provides a useful reference point to evaluate how much the learned policy actually improves over uninformed behavior:

```
def random_pi(state):  
    # selects an action uniformly at random  
    # from the environment's action space.  
    return env.action_space.sample()
```

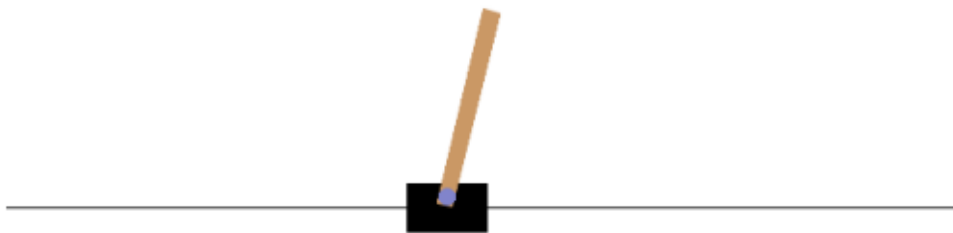
To better understand how the agent interacts with the environment, we define a visualization function that displays the policy's behavior step by step, allowing us to observe in real time how its decisions evolve throughout an episode:

```
import matplotlib.pyplot as plt  
from IPython.display import clear_output  
  
def show_policy(pi, n_episodes=1, max_steps=500):  
  
    # start the episodes  
    for episode in range(n_episodes):  
  
        # reset the environment  
        state, _ = env.reset();  
        done = False;  
  
        # initialize the step counter  
        step = 0;  
  
        # run the episode until done or max_steps  
        while not done and step < max_steps:  
  
            # make a step  
            action = pi(state);  
            state, _, terminated, truncated, _ = env.step(action);  
            done = terminated or truncated;  
  
            # render the environment  
            clear_output(wait=True)  
            plt.imshow(env.render())  
            plt.axis("off")  
            plt.title(f"Episode {episode+1}, Step {step}")  
            plt.show()  
            plt.close()  
  
            # increment the step counter  
            step += 1;
```

When applied to the CartPole environment, the random policy performs very poorly. Because its actions are selected uniformly at random, it lacks any notion of balance or control, causing the pole to fall almost immediately. Episodes typically last only a few time steps before termination, resulting in very low total rewards:

```
show_policy(random_pi, n_episodes=5, max_steps=500);
```

Episode 5, Step 20



To **quantitatively assess** how well a policy performs, we define an evaluation function that runs the agent for a specified number of episodes, accumulates the total reward obtained in each, and returns the average episode reward. This provides a simple but effective measure of overall policy quality, allowing us to compare different strategies under consistent conditions:

```
def evaluate(pi, episodes=1):  
  
    # collect total rewards per episode  
    rewards = []  
  
    # loop over episodes  
    for episode in range(episodes):  
  
        # reset the environment  
        state, _ = env.reset();
```



```
done = False;
total_reward = 0.0;

# run an episode
while not done:
    action = pi(state);
    state, reward, terminal, truncated, _ = env.step(action)
    total_reward += reward;
    done = terminal or truncated;

# store the total reward
rewards.append(total_reward);

# return the average reward over the episodes
return float(np.mean(rewards))
```

To establish the numerical baseline, we evaluate the random policy:

```
random_score = evaluate(random_pi, episodes=1000);
print("Random score", random_score);
```

Random score 21.921

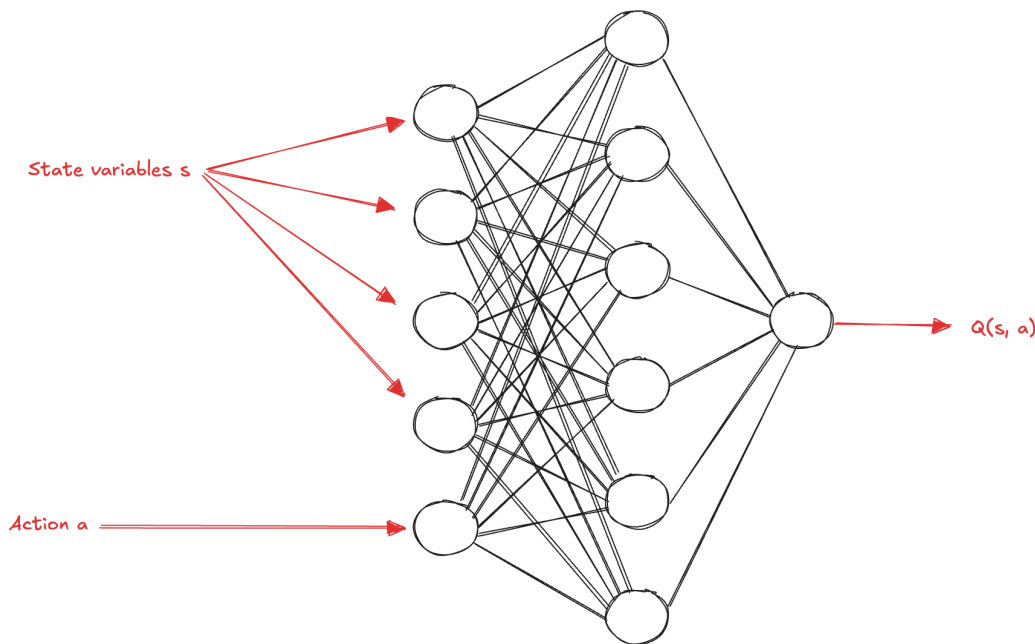
As expected, the random policy achieves a very low average reward on the CartPole environment, typically lasting only a few time steps before failure. This result confirms that maintaining the pole in balance requires structured decision-making, and it provides a clear benchmark against which we can measure the improvements achieved by learned policies. We can try to solve the environment using Reinforcement Learning methods. However, **it is not possible to apply tabular reinforcement learning methods** to this environment because the state space is continuous. We need to use function approximation in order to generalize.

8.3 Fully Connected Q-function

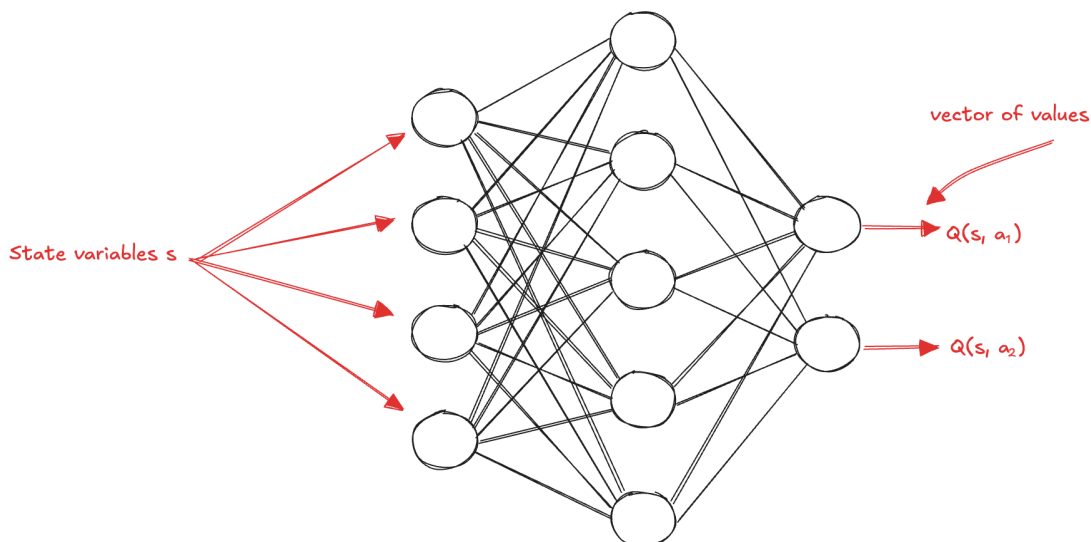
The Neural Fitted Q (M. Riedmiller, "Neural fitted Q iteration – first experiences with a data efficient neural reinforcement learning method", ECML (2005) is one of the first algorithms to successfully **use neural networks as a function approximation** to solve reinforcement learning problems. We refer to the approximate action-value function estimate as:

$$Q(s, a; \theta)$$

which means the estimate is parameterized by the weights of a neural network. In order to design the function approximator, we can observe that the matrix holding the action-value function was indexed by state and action pairs, so a straightforward neural network architecture can take in input the state and the action to evaluate, and it can output a single value representing the value for that state-action pair:



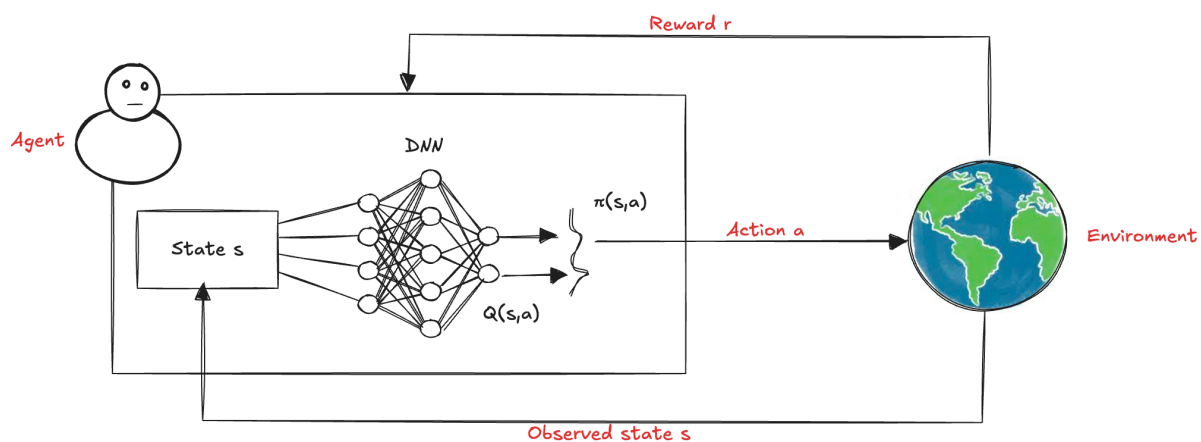
This architecture would work fine, but a more efficient one consists of **only inputting the state to the neural network and outputting the Q-values for all the actions** in that state:



This is clearly advantageous when using exploration strategies such as epsilon-greedy or soft-

max, because we can have having to do **only one pass forward to get the values of all actions**.

In a neural network-based approach, the state is passed through the network, which learns to approximate the action value function and the policy. This bypasses the need to explicitly store values for every state-action pair, as the network can generalize across states by learning patterns from the data. This generalization is critical in large or continuous environments where it's impossible to enumerate or store all possible states. Moreover, the parameters of the DNN are learned through experience and adjusted using gradient-based optimization, allowing the policy to improve over time by tweaking the network's internal weights. Thus, with this DNN-based method we can handle complex, high-dimensional spaces efficiently:



We can implement the function approximator as a **fully Connected network** using a machine learning library, such as PyTorch. The network will have two hidden layers with some neurons each (e.g. for the cartpole case 512 and 128 neuron are enough) and a ReLU activation function. The output layer will have a linear activation function, because we want to output the values directly. The input layer will have four neurons, one for each of the state variables:

```
import torch

# Define hyperparameters for the neural network
first_hidden_layer = 512;
second_hidden_layer = 128;

def create_network():

    # Define a deep neural network using Sequential:
    # Each layer feeds directly into the next one.
    dnn = torch.nn.Sequential(
        # First fully connected layer maps state inputs to 512 hidden units
        torch.nn.Linear(state_size[0], first_hidden_layer),
```

```
        # ReLU activation introduces nonlinearity
        torch.nn.ReLU(),

        # Second fully connected layer (hidden layer with 128 units)
        torch.nn.Linear(first_hidden_layer, second_hidden_layer),

        # Another ReLU activation
        torch.nn.ReLU(),

        # Output layer: one unit per possible action
        # Produces Q-values
        torch.nn.Linear(second_hidden_layer, action_size)
    )

    # Return the constructed model
    return dnn;
```

```
q = create_network();
```

Using the neural network trained to estimate Q-values, we can define a greedy policy that always selects the action with the highest predicted value for the current state. This corresponds to acting optimally with respect to the current knowledge encoded in the network:

```
def nfq_pi(state):

    # convert state to a torch tensor
    state = torch.as_tensor(state, dtype=torch.float32)

    # compute Q-values from the network
    q_values = q(state).detach().numpy().squeeze()

    # select greedy action
    action = int(np.argmax(q_values))

    return action
```

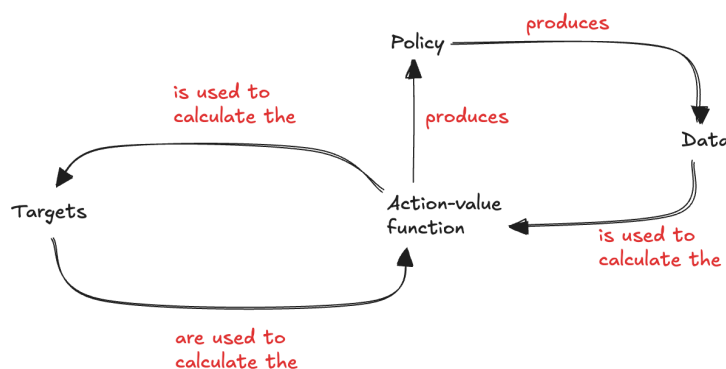
8.4 Training

To train a neural network we need to define a **Loss function**, which is a measure of how well a neural network predictions are. In supervised learning, it's straightforward to define. We have

a dataset of examples, and we know the correct output for each example. In that case, the loss function computes a distance score indicating how well the network has done on the dataset. There are many different ways for calculating this distance score, one of the most common ones is MSE (**mean squared error**, or L2 loss). It is defined as the average squared difference between the "predicted" and "true" values. In our case, we have two problems. The first one is that **we don't have any dataset**, we need to collect experiences from the environment. The second one is that **we don't have access to the "true" values**. An ideal objective would be to calculate the loss with respect to the "true" (optimal) action-value function q^* calculated over the experiences that we have collected:

$$L(\theta) = E_{s,a}[(q_*(s,a) - Q(s,a;\theta))^2]$$

Obviously, we are not talking about having access to q^* (otherwise, there's no need for learning), but we can use the same principles learned in generalized policy iteration, in which we **alternate between policy-evaluation and policy-improvement processes** to find good policies. However, because we're using non-linear function approximation, **convergence guarantees no longer exist**. In the NFQ implementation we do just that: we start with a randomly initialized action-value function (and implicit policy), then we evaluate the policy by sampling actions from it, we improve it with an exploration strategy (such as epsilon-greedy) and finally, we keep iterating until we reach the desired performance. So, **we use as "true values" the targets**, which depend on a prediction also coming from the network, the value of the next state, in a circular dependency:



As you may be thinking, **this circular dependency is bad**. It's not well behaved because **it doesn't respect several of the assumptions made in supervised learning problems**. However, we can go on and see what we can do with it. As you remember, there are different targets we can use for estimating the action-value function of a policy: Monte Carlo (MC) target, Temporal-Difference (TD) target, n-Step target, and Lambda target:

MC uses all rewards found in a trajectory to the terminal state

MC



TD uses the value of the next state as an estimate of all rewards to go

TD



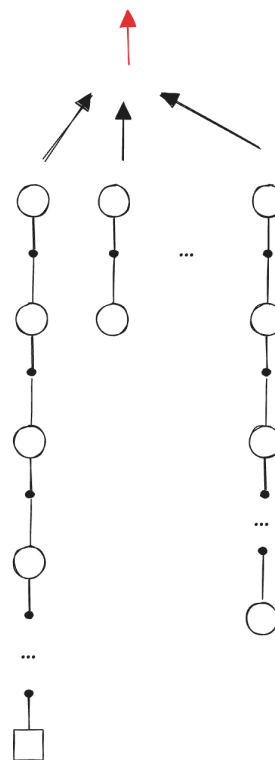
n-step is like TD, but instead of bootstrapping after one step, it uses n steps

n-step TD



Lambda mixes in an exponentially decaying fashion all n-step targets into one

Lambda



The target can be either on-policy (like in SARSA) or off-policy (like in Q-learning), depending on the way we bootstrap. The two main ways are to either use the action-value function of the action the agent will take at the landing state, or alternatively, to use the value of the action with the highest estimate at the next state:

$$Q^{\text{SARSA}}(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

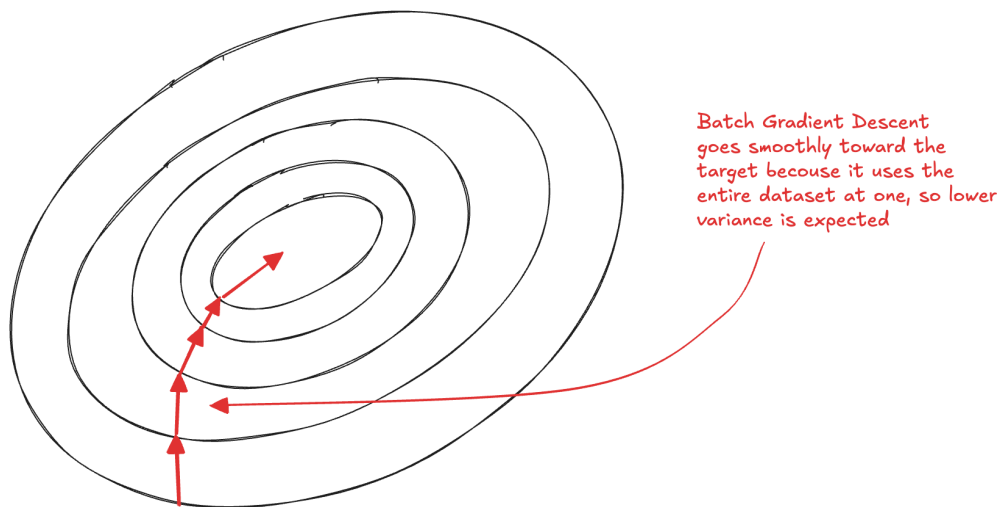
$$Q^{\text{Q-learning}}(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Using those rules we can define the loss function as the difference between the current Q-value and the new value that should be assigned. For example, a squared-error measure like the following can be used:

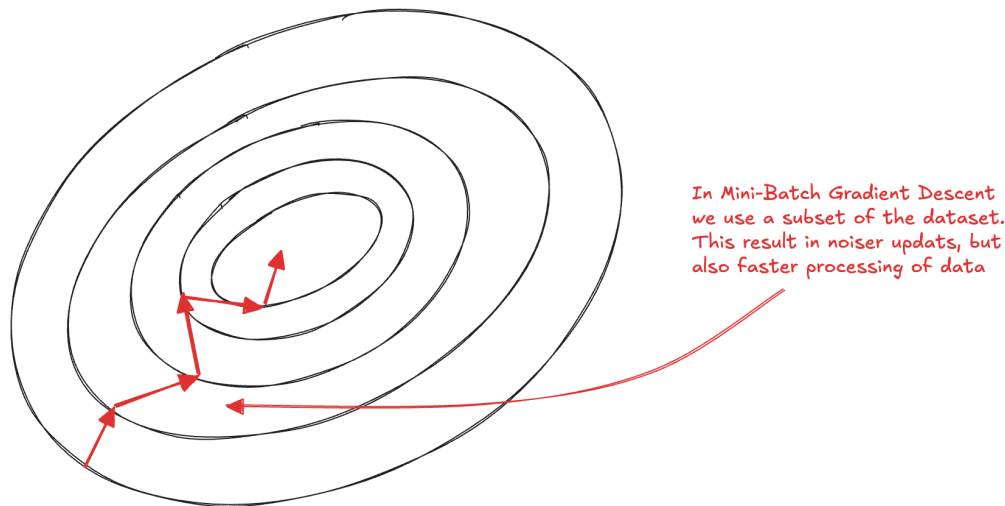
$$L(\theta) = E_{s,a} \left[(r + \gamma \max_a Q(s', a; \theta) - Q(s, a; \theta))^2 \right]$$

In order to minimize the loss function we can use a standard optimization methods in supervised learning, like **Batch Gradient Descent (BGD)**, it takes the entire dataset at once, calculates the gradient of the dataset and steps toward this gradient a little bit at a time. Then, it repeats

this cycle until convergence:

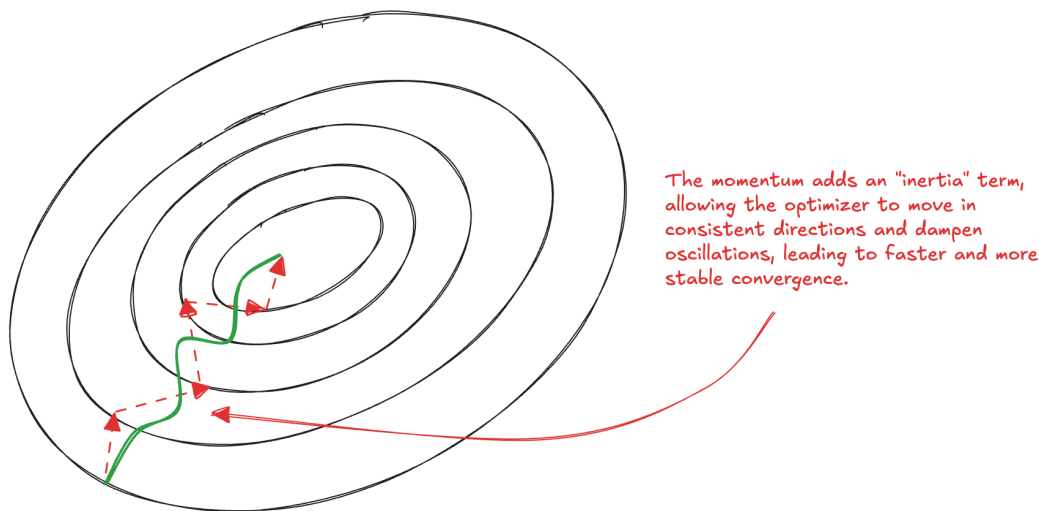


When we have a considerable dataset with millions of samples, BGD is too slow to be practical. Moreover, in reinforcement learning, **we don't even have a dataset in advance, so batch gradient descent isn't a practical method for our purpose**. An optimization method capable of handling smaller batches of data is called **Mini-Batch Gradient Descent (MGD)**, where we use only a fraction of the data at a time:



We can control the size of the mini-batches, which allows the processing of large datasets. At one extreme, we can set the size of mini-batch to the size of the dataset (BGD) at the other extreme, we can set the size to a single sample (**Stochastic Gradient Descent (SGD)**). The larger the batch, the lower the variance the steps of the optimization method have, but learning slows down considerably. An improved algorithm is called **Gradient Descent with Momentum**. This is a MBGD that updates the weights of the network in the direction of the moving average

of the gradients, instead of the gradient itself:



However, these algorithms require a couple of assumptions: data must be **independent and identically distributed (IID)** and targets must be **stationary**. In reinforcement learning we cannot ensure any of these assumptions. So choosing a robust optimization method to minimize the loss function can often make the difference between convergence and divergence. In practice, **Root Mean Square propagation (RMSprop)** (that reduces oscillations of Momentum by scaling the gradient in proportion to the average magnitude of recent gradients) and **Adaptive Moment Estimation (Adam)** (that steps in the direction of the velocity of the gradients as momentum, but it scales updates in proportion to the moving average of the magnitude of the gradients as RMSprop) are sensible choices for reinforcement learning:

```
learning_rate = 0.007;
optimizer = torch.optim.RMSprop(q.parameters(), lr=learning_rate);
```

All optimization methods are based on gradient calculation. It's important to note that when differentiating the loss function, the gradient is computed solely with respect to the predicted values, not the target values (this is a common source of error). In supervised learning, "predicted values" come from the model, while "true values" are constants provided in advance. In reinforcement learning, however, the "true values" depend on the predicted values themselves, as they are generated by the model. **Although this value is not a true constant, it should be treated as one during the optimization:**

$$\nabla_{\theta_i} L(\theta) = E_{s,a} \left[(r + \gamma \max_a Q(s', a; \theta) - Q(s, a; \theta)) \nabla_{\theta} Q(s, a; \theta) \right]$$

The problem with this on-line update rule is that if weights are adjusted for one certain state action pair, then unpredictable changes also occur at other places in the state-action space. Although in principle this could also have the positive effect of generalization, in many cases this

seems to be the main reason for unreliable and slow learning. So the basic idea of NFQ is to **update the neural value off-line, considering an entire set of transition experiences**. So we need to store a collection of experiences:

```
# define the batch size
batch_size = 1024;

# define the structured dtype for an experience tuple
experience_type = np.dtype([
    ('state',      np.float32, state_size),    # current state
    ('action',     np.int8),                   # action taken
    ('reward',     np.float32),                # reward received
    ('next_state', np.float32, state_size),    # next state
    ('failure',    np.int8)                    # terminal flag (1 if done)
])
```

To train the agent, we must collect batches of experiences by interacting with the environment. During this phase, we use an **exploration strategy** to balance the trade-off between exploring new actions and exploiting what the agent has already learned. The common choice is the epsilon-greedy policy:

```
# Exploration vs exploitation parameter
epsilon = 0.5

def epsilon_greedy(state):

    # Exploration
    if random.random() < epsilon:
        action = random_pi(state)

    # Exploitation
    else:
        action = nfq_pi(state)

    return action
```

We can now train the neural network by using the experiences collected from the environment to update its parameters. This is done by computing the loss function (which measures the discrepancy between predicted and target Q-values) and applying the backpropagation algorithm to adjust the network weights accordingly:

```
# define the discount factor
gamma = 0.99;

# define the number of optimization epochs
epochs = 40;

def optimize(batch):

    # convert experience fields to tensors
    states      = torch.from_numpy(batch['state'].copy()).float()
    actions     = torch.from_numpy(batch['action'].copy()).long()
    rewards     = torch.from_numpy(batch['reward'].copy()).float()
    next_states = torch.from_numpy(batch['next_state'].copy()).float()
    failures    = torch.from_numpy(batch['failure'].copy()).float()

    # perform multiple epochs of optimization
    for epoch in range(epochs):

        # compute target:  $r + \gamma \max_a Q(\text{next\_state}, a) * (1 - \text{done})$ 
        # remember to detach, we need to treat these values as constants!
        with torch.no_grad():
            q_next = q(next_states);
            max_q_next = q_next.max(dim=1).values;
            target = rewards + gamma * max_q_next * (1.0 - failures);

        # compute current Q(s, a)
        q_current_all = q(states);
        q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1);

        # compute the Mean Squared TD Error loss
        td_error = target - q_current;

        # compute the loss
        loss = (td_error ** 2).mean();

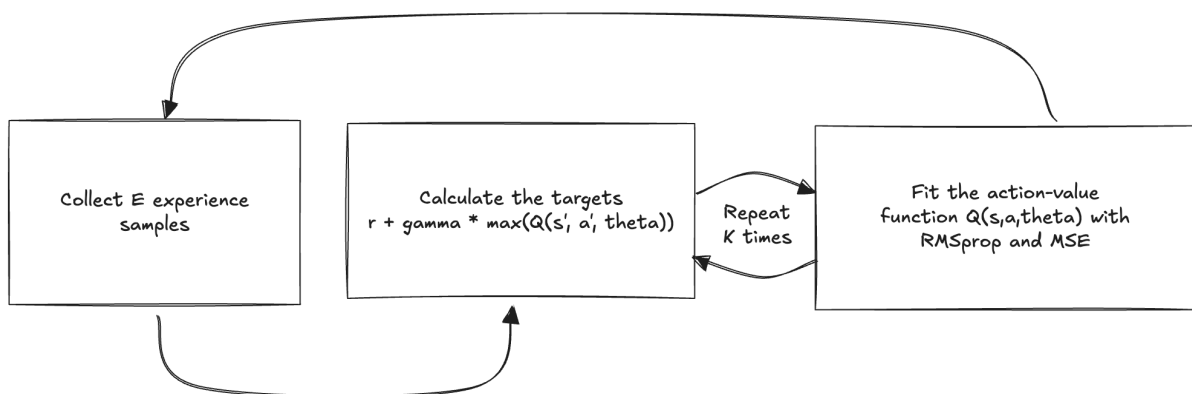
        # optimize network parameters
        optimizer.zero_grad();
        loss.backward();
        optimizer.step();

    return loss;
```

8.5 NFQ Algorithm

Now we can put things together and consider the full algorithm:

1. use a fully connected neural network with states as input and actions as output;
2. use an epsilon-greedy strategy to explore the environment and to collect some experiences (e.g. 1024 samples);
3. calculate the off-policy targets; 4 fit the action-value function using MSE for the loss function and RMSprop as optimizer;
4. repeats steps 3 and 4 a number of times (epochs) before going back to step 2;



We can write this process and apply it to the Cart-pole environment:

```
def nfq(max_episodes):

    # create a batch index
    index = 0;

    # create a score tracker for statistic purposes
    scores = [];

    # create an empty experience batch
    experiences_batch = np.empty(batch_size, dtype=experience_type);

    # train until the maximum number of episodes
    for episode in range(max_episodes):

        # reset the environment at the beginning of the episode
        state, _ = env.reset();
        done = False;
```

```
# interact with the environment until the episode is done
while not done:

    # select the action using the exploration policy
    action = epsilon_greedy(state);

    # perform the selected action
    next_state, reward, terminal, truncated, _ = env.step(action);
    done = terminal or truncated;
    failure = terminal and not truncated;

    # store the experience
    experiences_batch[index] = (state, action, reward, next_state, failure);
    index += 1;

    # if the experience collected so far is enough...
    if index == len(experiences_batch):
        # optimize the model
        optimize(experiences_batch);
        # and clear the buffer
        index = 0;
        experiences_batch = np.empty(shape=(len(experiences_batch),),
    dtype=experience_type);

    # update current state to next state
    state = next_state;

    # After each episode, evaluate the policy
    score = evaluate(nfq_pi, episodes=10);

    # store the score in the tracker
    scores.append(score);

    # print some informative logging
    message = 'Episode {:03}, score {:.05.1f}';
    message = message.format(episode+1, score);
    print(message, end='\r', flush=True);

return scores;
```

We can apply the algorithm to the Cart-pole environment:

```
nfq(max_episodes = 1100);
```

Episode 1100, score 127.8

We can evaluate the performance of the learned agent versus the random one:

```
agent_score = evaluate(nfq_pi, episodes=10);  
  
print("Random score", random_score);  
print("Agent score", agent_score);
```

Random score 21.921

Agent score 124.7

And we can see the agent in action:

```
show_policy(nfq_pi, n_episodes=1, max_steps=500);
```

Episode 1, Step 115



The agent is good enough to balance the pole for a considerable amount of time. However, we don't have any information about the performance of the algorithm, we can try to understand how the agent is learning by plotting some quantities over time setting up an experiment.

8.6 Experiment

To better understand how the agent learns over time, we can write a function to run a series of trainings and track the average score achieved by the algorithm after a fixed number of episodes. By repeating the experiment multiple times with different random seeds, we can assess the stability and robustness of the learning process. This helps determine whether the algorithm converges reliably or if its performance depends heavily on random initialization.

```
import joblib

def experiment(max_episodes):

    global q, optimizer;

    # List of random seeds to test algorithm stability
    seeds = (12, 34, 56, 78, 90);

    # Container to collect all experiment results
    results = [];

    # Run an independent training experiment per seed
    for seed in seeds:

        print("Experiment seed: ", seed);

        # Set all relevant random seeds for reproducibility
        torch.manual_seed(seed);
        np.random.seed(seed);
        random.seed(seed);

        # reset the environment
        env.reset(seed=seed);
        env.action_space.seed(seed)
        env.observation_space.seed(seed)

        # create a new model
        q = create_network();
        optimizer = torch.optim.RMSprop(q.parameters(), lr=learning_rate);

        # train the network
        scores = nfq(max_episodes);
```

```
        # smooth the result using a sliding window
        sliding_windows = 25;
        scores = np.convolve(scores, np.ones(sliding_windows)/sliding_windows,
        ↪ mode='valid');

        # collect the results
        results.append(scores);

        print("");

        # calculate max, min and average scores among experiments
        max_score = np.max(results, axis=0).T;
        min_score = np.min(results, axis=0).T;
        mean_score = np.mean(results, axis=0).T;

        # prepare the results
        experiment_results = {
            'max_score': max_score,
            'min_score': min_score,
            'mean_score': mean_score
        }

        # Save results
        joblib.dump(experiment_results, './nfq_results.joblib');

    return experiment_results;
```

We now define a specific set of hyperparameters controlling both the training process and the neural network architecture, and we execute a single training experiment to observe how the agent's performance evolves over time under these conditions:

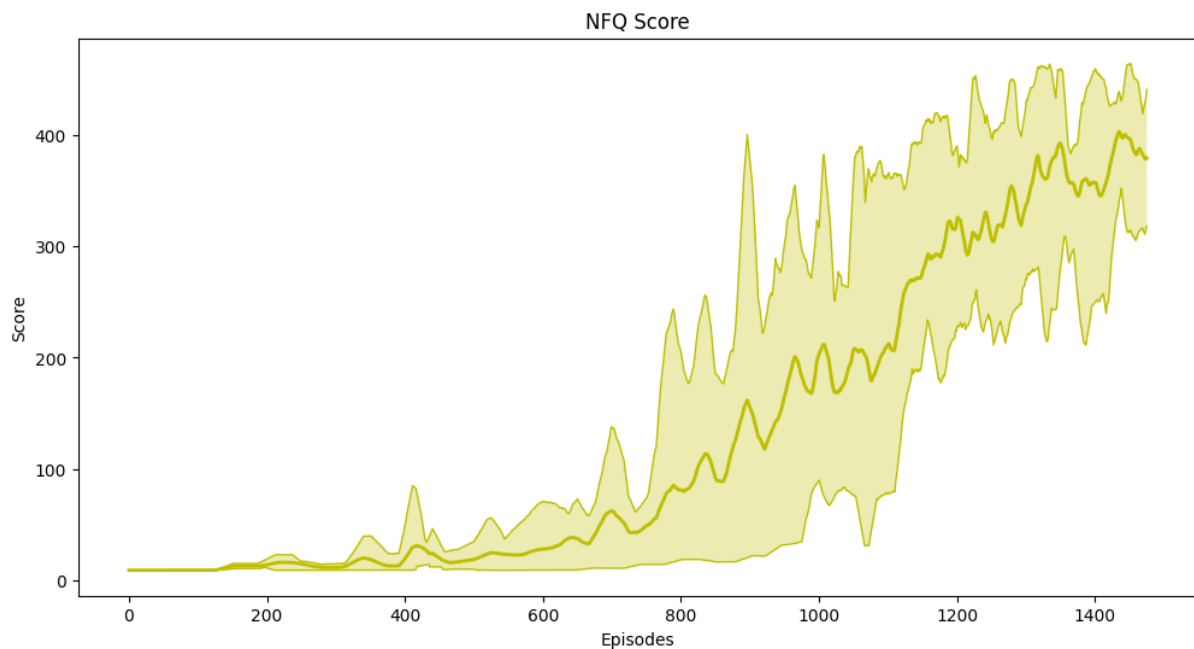
```
# Define the experiment setup and hyperparameters
gamma = 0.99;           # discount factor
learning_rate = 0.0005;  # step size for the optimizer
batch_size = 1024;      # number of experiences per batch
epochs = 16;            # optimization steps per batch
epsilon = 0.5           # exploration vs exploitation parameter
first_hidden_layer = 256; # size of the first hidden layer
second_hidden_layer = 128; # size of the second hidden layer

# Run the experiment
results = experiment(max_episodes=1500);
```

```
Experiment seed: 12
Episode 1500, score 321.3
Experiment seed: 34
Episode 1500, score 485.7
Experiment seed: 56
Episode 1500, score 408.6
Experiment seed: 78
Episode 1500, score 473.6
Experiment seed: 90
Episode 1500, score 441.8
```

While NFQ cannot be considered a state-of-the-art solution, it demonstrates decent performance in relatively simple environments like CartPole. As observed during the simulations, the algorithm converges differently when initialized with various random seeds. This variability indicates that the algorithm lacks stability, resulting in significant fluctuations in performance. We perform basic statistical analysis on the experimental data to calculate the maximum, minimum, and mean scores for each step. This helps us assess both the stability of the performance (how close the scores are to the mean) and its quality (how close the scores are to the maximum). we can plot the average scores obtained by the agent over different training episodes:

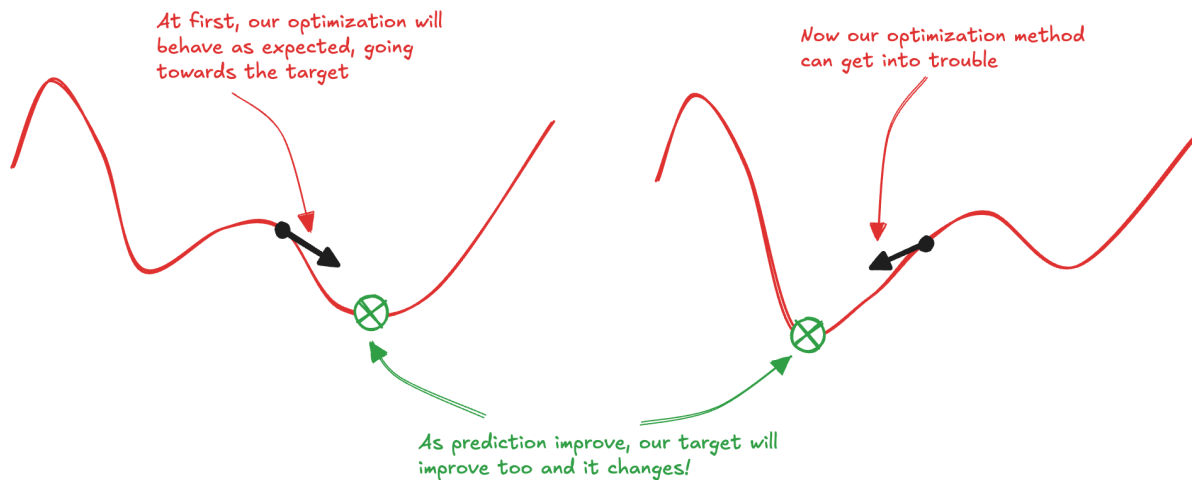
```
plt.figure(figsize=(12,6));
plt.title('NFQ Score');
plt.ylabel('Score');
plt.xlabel('Episodes');
episodes = range(len(results['max_score']));
plt.plot(results['max_score'], 'y', linewidth=1)
plt.plot(results['min_score'], 'y', linewidth=1)
plt.plot(results['mean_score'], 'y', linewidth=2)
plt.fill_between(episodes, results['max_score'], results['min_score'], facecolor='y',
    ↪ alpha=0.3);
plt.show();
```

The NFQ algorithm shows learning progress as the score increases, but there is **some instability** as indicated by the **high variance** in the scores and the growing gap between the max and min scores. The sliding average helps to highlight the trend in the data, showing that the agent is learning to balance the pole more effectively over time. Overall, in this simple environment, the algorithm is able to learn a good policy in a relatively short amount of time.

8.7 Challenges

There are two main issues with the NFQ algorithm. First, although we use a powerful function approximator capable of generalizing across many state-action pairs (which is a strength) this same property becomes a **source of instability**. The neural network updates the values of all similar states at once, and since our target values depend on the estimated values of the next state, the **learning targets themselves are non-stationary**. Each time the network's weights are updated, the targets shift as well, immediately invalidating the most recent update. This feedback loop causes **the training process to quickly become unstable**, as illustrated below:



NFQ mitigates this instability through the use of **batch learning**. Instead of updating the network after each individual experience, NFQ collects a batch of experiences and optimizes the network on this larger, more diverse set. Larger batches help by smoothing out noise and providing broader coverage of the state–action space. However, this approach introduces a second problem: **correlated samples**. The collected experiences come from consecutive time steps within the same trajectory and policy, meaning they are **not independent**. Consequently, the network learns from batches that violate the **IID (independent and identically distributed) assumption fundamental to most optimization methods**. In reinforcement learning, **samples are not independent** (each state depends on the previous one) and **not identically distributed** (the policy generating the data changes over time). While gradient-based optimizers can tolerate limited departures from IID conditions, reinforcement learning breaks this assumption almost entirely, requiring additional techniques to stabilize learning.

To better understand how hyperparameter choices affect NFQ’s performance and stability, we can conduct a systematic **grid search** by running multiple experiments with different configurations. The following code iterates over selected hyperparameter values, runs a complete experiment for each setup, and saves the corresponding learning curves as images. By comparing these saved plots, we can analyze how various settings influence learning speed, convergence, and robustness. We rank the results based on the **average score over the entire learning process**, which provides higher values to runs whose learning curves stay consistently high throughout training, rather than just peaking at the end. This metric helps **identify hyperparameter combinations that yield stable and effective learning** across the entire training duration:

```
import itertools

def run_grid_experiments(param_grid, max_episodes):
```

```
global gamma, batch_size, epochs, epsilon, learning_rate, first_hidden_layer,
    second_hidden_layer

# Extract the parameter names and their values,
# itertools.product to get all combinations
keys = list(param_grid.keys())
combos = list(itertools.product(*(param_grid[k] for k in keys)))
total = len(combos)

print(f"Running {total} experiments ... ")

# Store a summary of all experiments
summary = []

# Run each experiment
for idx, combo in enumerate(combos, start=1):

    # Get the parameters for this experiment
    params = dict(zip(keys, combo))

    # Print the experiment details
    print(f"\nExperiment {idx}/{total}")
    for k, v in params.items():
        print(f"    {k:>20s} = {v}")
    print("")

    # Run the experiment with the current parameters
    gamma=params["gamma"]
    batch_size=params["batch_size"]
    epochs=params["epochs"]
    epsilon=params["epsilon"]
    learning_rate=params["learning_rate"]
    first_hidden_layer=params["first_hidden_layer"]
    second_hidden_layer=params["second_hidden_layer"]

    exp_res = experiment(max_episodes=max_episodes)

    # Extract learning curve
    mean_score = exp_res["mean_score"]
    min_score = exp_res["min_score"]
    max_score = exp_res["max_score"]

    # Compute metrics
```

```

avg_mean = float(np.mean(mean_score))
final_mean = float(mean_score[-1])

# Build a descriptive title string
title_str = (
    f"NFQ Learning Performance (Exp {idx}/{total})\n"
    f"γ={params['gamma']}, lr={params['learning_rate']}, "
    f"bs={params['batch_size']}, ep={params['epochs']}, "
    f"ε={params['epsilon']}, h1={params['first_hidden_layer']}, "
    f"h2={params['second_hidden_layer']}"
)

# Generate the plot
plt.figure(figsize=(12, 6))
plt.title(title_str, fontsize=10)
plt.ylabel('Score')
plt.xlabel('Episodes')
episodes = range(len(mean_score))
plt.plot(mean_score, color='orange', linewidth=2, label='Mean score')
plt.fill_between(episodes, min_score, max_score,
                 color='orange', alpha=0.3,
                 label='Range (min-max)')

plt.legend()
plt.tight_layout()

# Create a filename based on parameters and save
filename = (f"nfq_gamma{params['gamma']}_ "
            f"lr{params['learning_rate']}_ "
            f"bs{params['batch_size']}_ "
            f"ep{params['epochs']}_ "
            f"eps{params['epsilon']}_ "
            f"h1_{params['first_hidden_layer']}_ "
            f"h2_{params['second_hidden_layer']}.png").replace('.', '_')
plt.savefig(filename, dpi=300, bbox_inches="tight")
plt.close()

# Append the summary
summary.append({
    "index": idx,
    **params,
    "avg_mean": avg_mean,
    "final_mean": final_mean
})

```

```
# Sort by average score
summary.sort(key=lambda r: (r["avg_mean"], r["final_mean"]), reverse=True)

# Print the best configuration
print("\nAll experiments completed.")
print(f"Best configuration (by average mean score): Experiment
      ↳ {summary[0]['index']}")
print(summary[0])

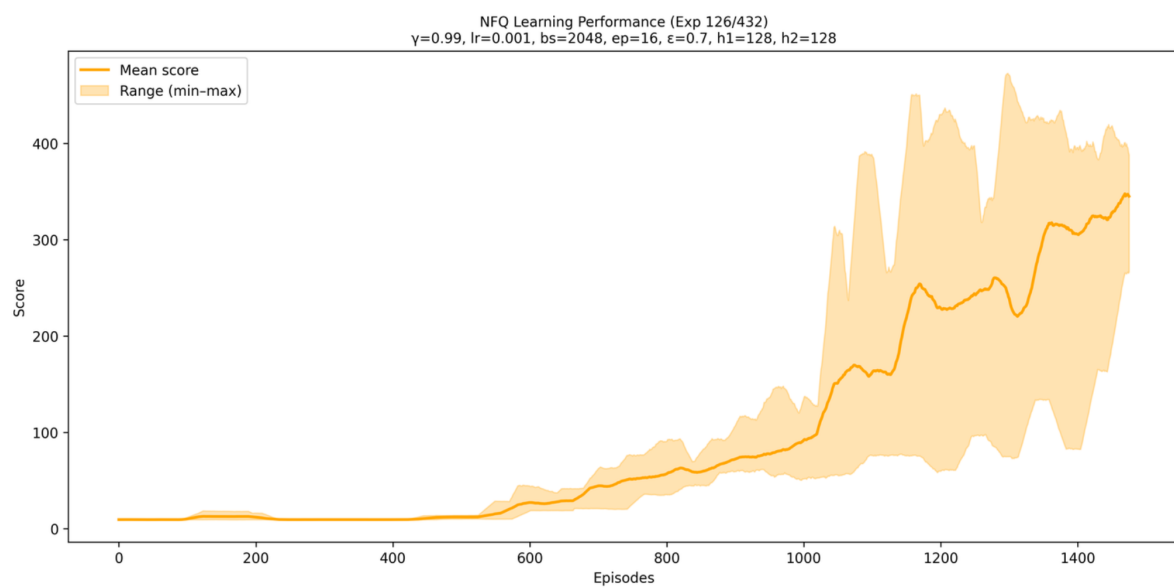
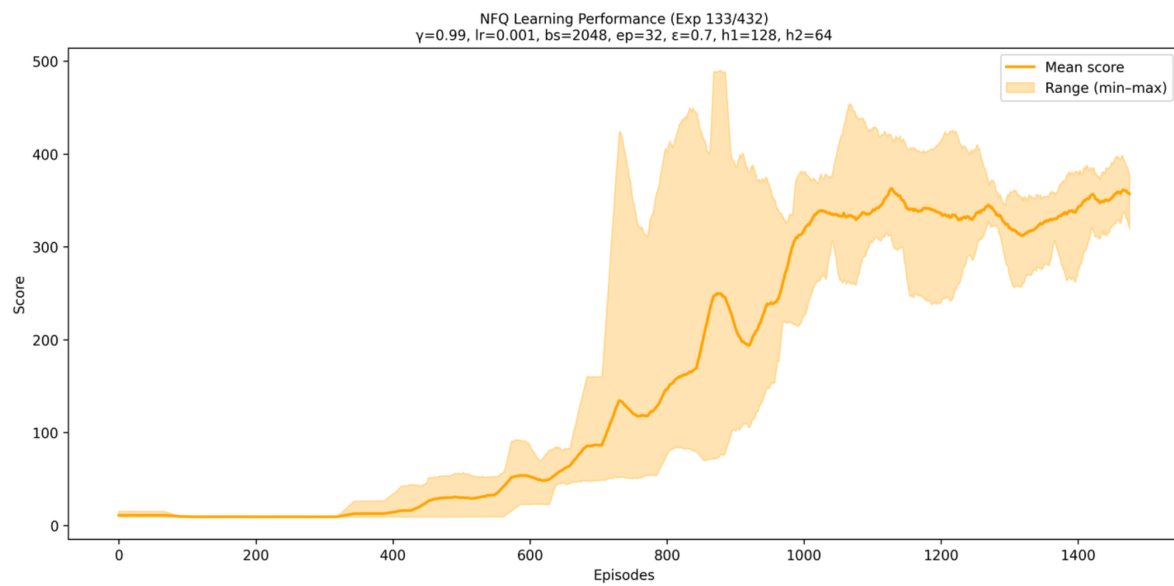
# Return the summary of all experiments
return summary
```

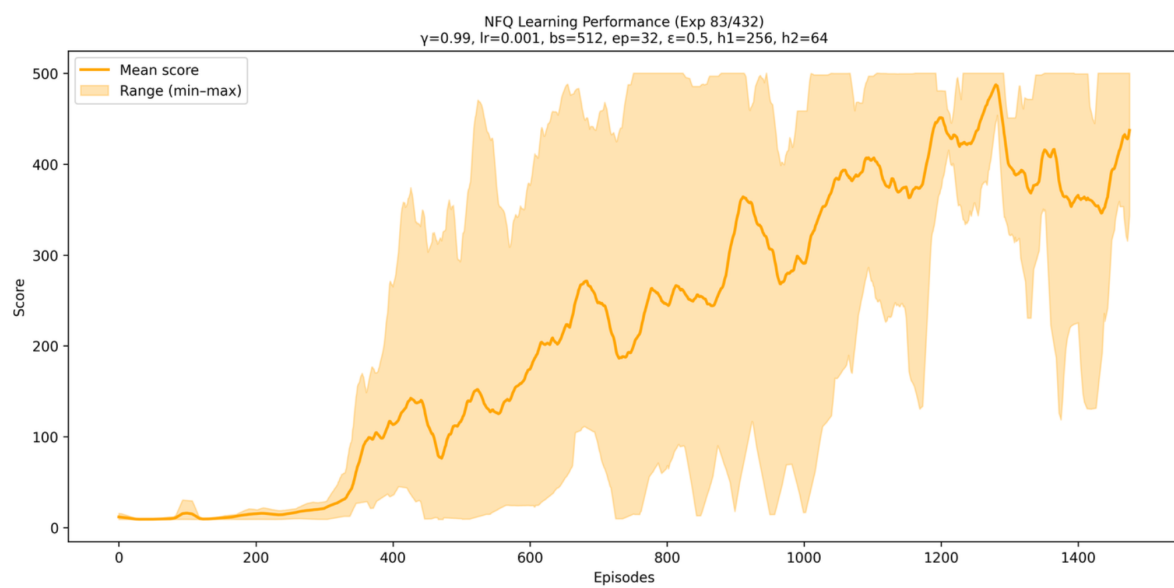
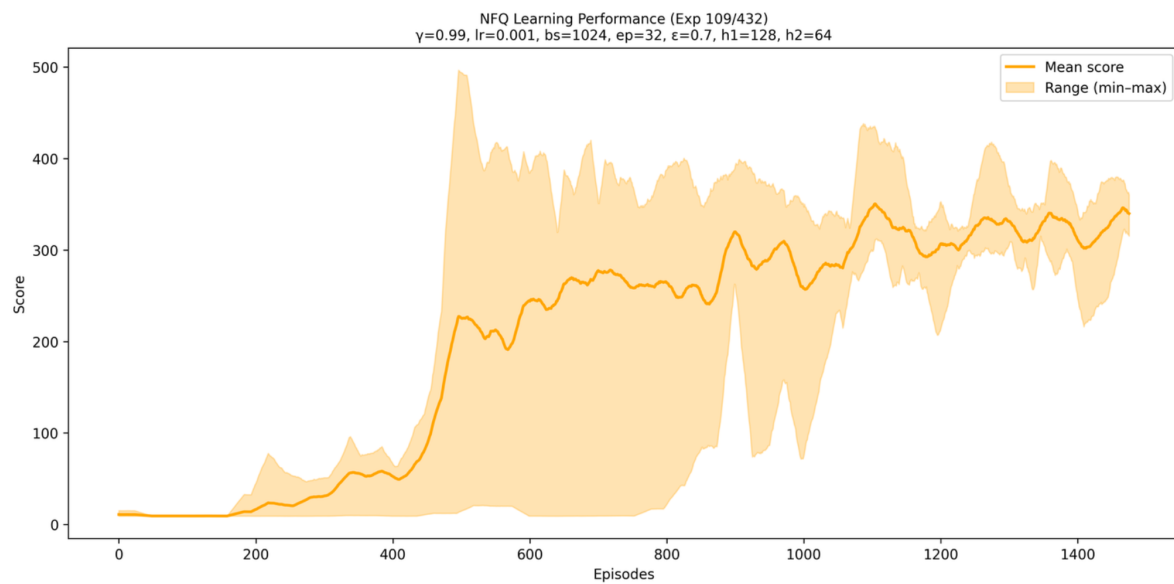
Do not run this code directly in the notebook, as it is computationally intensive.

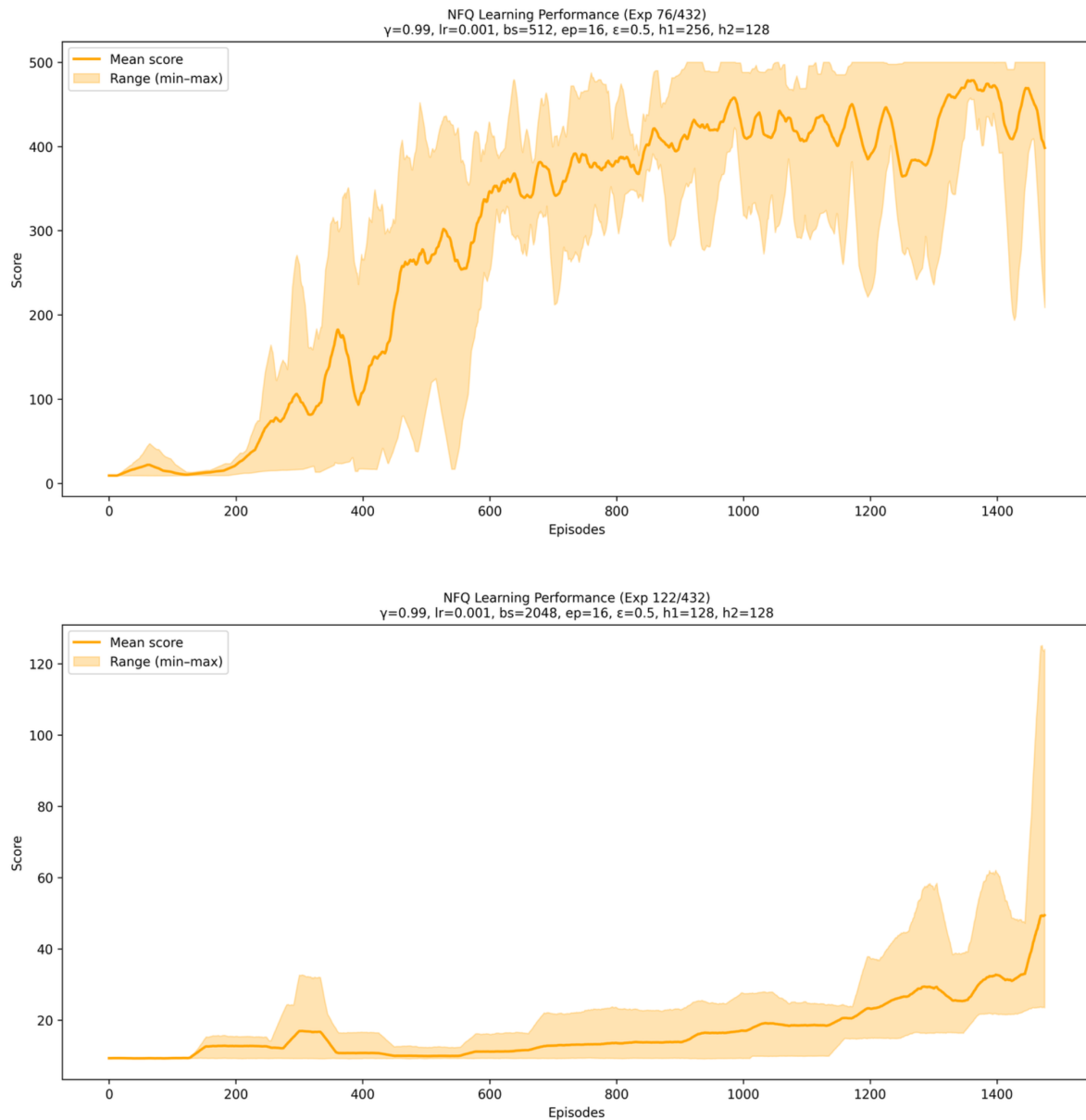
```
param_grid = {
    "gamma": [0.99, 1.00],
    "learning_rate": [0.0005, 0.001, 0.001, 0.005],
    "batch_size": [256, 512, 1024, 2048],
    "epochs": [8, 16, 32, 64, 128],
    "epsilon": [0.4, 0.5, 0.7],
    "first_hidden_layer": [128, 256, 512],
    "second_hidden_layer": [64, 128, 256]
}

#summary = run_grid_experiments(param_grid, max_episodes=1000)
```

In the following images, you can see some of the learning curves obtained with different hyperparameter combinations:







Across these NQ experiments, several patterns emerge that reveal how the main hyperparameters influence learning stability and performance. The experiments with a **higher exploration rate consistently show faster and more reliable progress**. This suggests that a broader exploration phase allows the agent to discover more rewarding state–action pairs, avoiding premature convergence to suboptimal behaviors. When the exploration rate is reduced, learning becomes sluggish or erratic: the mean score either plateaus at low values or increases only very late in training.

The batch size also plays an important role in shaping learning stability. **Larger batches pro-**

duce smoother curves and more consistent improvement, although the **first phase of learning tends to be slower**. As the number of samples decreases, the learning curve reacts faster but becomes increasingly noisy. The variability of the orange shaded region widens, indicating higher sensitivity to stochastic effects in the Q-target updates. In practice, this means that using very small batches accelerates feedback but destabilizes convergence, while large batches act as a regularizer, filtering out fluctuations and leading to steadier progress once the agent begins to improve.

The network architecture also influences the dynamics. The most balanced configurations, with **two hidden layers of 128 neurons each, achieve solid and consistent growth**. Reducing the second layer to 64 neurons slightly speeds up early learning without harming the final result. Larger architectures show wide oscillations in the score range. These networks may be **too expressive** for the amount of experience available, amplifying estimation noise and leading to overfitting of the Q-function to transient data patterns.

Overall, while some of these configurations eventually reach respectable scores, the collective picture suggests that **NFQ remains a rather brittle and inefficient learning method in this context**. Even in the best runs, progress is slow and unstable: the agent often spends hundreds of episodes with almost no improvement, and when performance finally increases, it tends to oscillate widely or collapse before stabilizing. The **strong dependence on hyperparameter tuning** (where small changes in exploration rate, batch size, or network width completely alter the learning dynamics) reveals that **NFQ lacks robustness**.

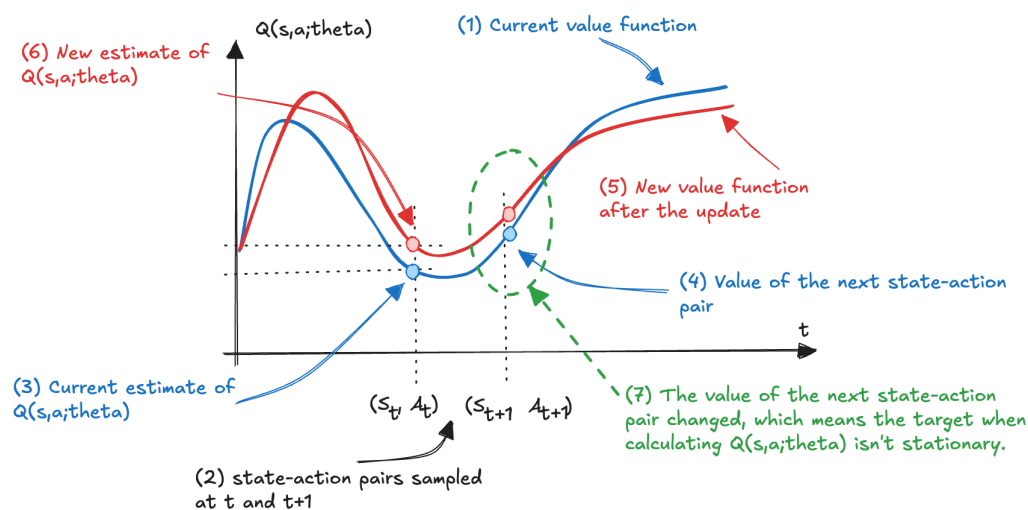
Moreover, the very large variance across seeds and experiments indicates that the algorithm **struggles to generalize** from replayed experience. It is highly sensitive to inaccuracies in the Q-targets. Errors can propagate through successive iterations and lead to instability or divergence. The fact that wider networks and smaller batches tend to explode in variance further confirms this weakness: NFQ does not regularize its function approximation effectively.

In short, despite occasional good results, the learning curves show that NFQ is not a particularly reliable or sample-efficient approach. These results highlight the algorithm's historical and conceptual limitations: NFQ was an important step toward value-function approximation, but in practice it remains fragile and outdated compared with more recent reinforcement learning methods.

9 Deep Q-Network (DQN)

The two problems that consistently show up in the naive application of function approximation to RL (like NFQ) are the **violations of the IID assumption** and the **non stationarity of targets**. In supervised learning, we obtain a full dataset in advance. We pre-process it, shuffle it,

and then split it into sets for training. One crucial step in this process is the **shuffling of the data set**. By doing so, we allow our optimization method to avoid developing overfitting biases, reduce the variance of the training process, speed up convergence, and overall learn a more general representation of the underlying data-generating process. In reinforcement learning data is **gathered online** (they come from a sequence, a time series, a trajectory) and the experience sample generated at time step $t+1$ correlates with the experience sample generated at time step t . Moreover, samples are not identically distributed, because they depend on the policy that generates the actions and the policy is changing through time (it improves). Also, in supervised learning, the targets used for training are fixed values on our dataset (fixed throughout the training process). In reinforcement learning **targets move with every training step** of the network. At every training update step, we optimize the approximate value function and therefore change the shape of the function, possibly the entire value function. Changing the value function means that the target values change as well, which, in turn, means the targets used are no longer valid. Because the targets come from the network, even before we use them, we can assume targets are invalid or biased at a minimum:



In order to address these issues, we can use a **replay buffer** to store experience samples, a **target network** to stabilize the learning process and **more advanced exploration strategies**. Again we use the cart-pole environment to illustrate the algorithm.

```
import gymnasium

env = gymnasium.make('CartPole-v1', render_mode="rgb_array");

state_size = env.observation_space.shape;
action_size = env.action_space.n;
```

```
print('State Space: ', state_size);
print('Action Space: ', action_size);
```

```
State Space:  (4,)
Action Space:  2
```

Again, we fix the seed of the random number generator in order to reproduce the results:

```
import torch;
import numpy as np;
import random;

# set the seeds for reproducibility of results
seed = 34;

torch.manual_seed(seed);
np.random.seed(seed);
random.seed(seed);

env.reset(seed=seed)
env.action_space.seed(seed)
env.observation_space.seed(seed)
```

```
34
```

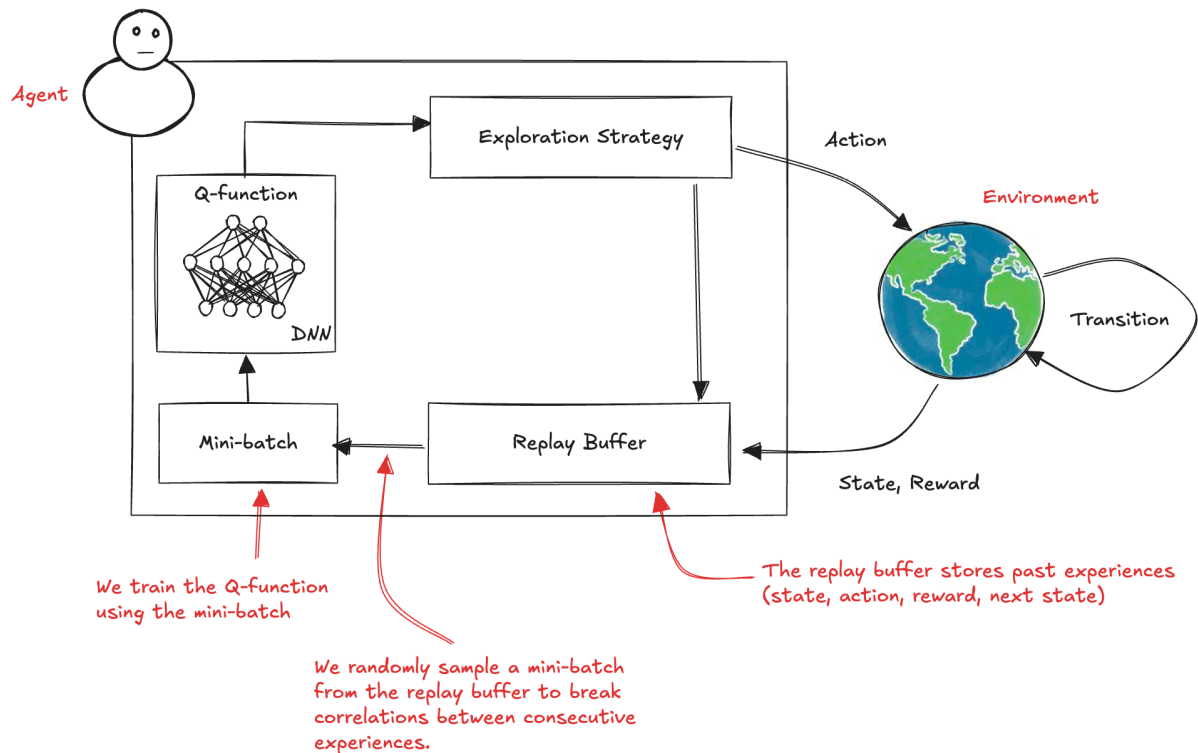
As usual, we define the baseline random policy:

```
def random_pi(state):
    # selects an action uniformly at random
    # from the environment's action space.
    return env.action_space.sample()
```

9.1 Experience Replay

In NFQ, we use a mini-batch of N samples, and train the network with this batch for K iterations, alternating between calculating new targets and optimizing the network. These N samples are temporally correlated, because most of them belong to the same trajectory. **Experience replay** consists of a data structure (**replay memory**, called D) that **holds $M \gg N$ experience samples for**

several steps, allowing the **sampling of mini-batches from a broad set of past experiences**. In that way, the training process can use a more diverse mini-batch for performing updates, and the agent no longer has to fit the model to the same small mini-batch for multiple iterations. The replay buffer **gives the impression that data are IID**, so that the optimization method is stable. Samples appear independent and identically distributed because of the sampling from multiple trajectories and even policies at once.



The only difference with NFQ is that **we're now obtaining the experiences we use for training by sampling uniformly at random the replay buffer D** , instead of using the online experiences as before:

$$L(\theta) = E_{(s,a) \sim U(D)} \left[(r + \gamma \max_a Q(s', a; \theta) - Q(s, a; \theta))^2 \right]$$

Unfortunately, the **implementation becomes a challenge when working with high-dimensional observations**, because poorly implemented replay buffers hit a **hardware memory limit** quickly. In image-based environments, for instance, where each state representation is a stack of the latest image frames, we probably don't have enough memory to naively store the experience samples. We can write a simple implementation just for the sake of understanding the concept, but in practice, we need to use a more efficient data structure. We can use a **circular buffer** to store the experiences, which is a fixed-size buffer that overwrites the oldest experiences with the newest ones when the buffer is full:

```
# define the structured dtype for an experience tuple
experience_type = np.dtype([
    ('state',      np.float32, state_size),  # current state
    ('action',     np.int8),                 # action taken
    ('reward',     np.float32),              # reward received
    ('next_state', np.float32, state_size),  # next state
    ('failure',    np.int8)                  # terminal flag (1 if done)
])
```

```
# Set the replay memory size
memory_size = 100000;

# Create the replay memory
replay_memory = {
    'size': memory_size,
    'buffer': np.empty(shape=(memory_size,), dtype=experience_type),
    'index': 0,
    'entries': 0
}
```

We write a method to store a new experience in the replay buffer:

```
def store_experience(experience):
    # store the experience in the buffer
    replay_memory['buffer'][replay_memory['index']] = experience;

    # update the number of experiences in the buffer
    replay_memory['entries'] = min(replay_memory['entries'] + 1,
    ↪ replay_memory['size']);

    # update index, if the memory is full, start from the begging
    replay_memory['index'] += 1;
    replay_memory['index'] = replay_memory['index'] % replay_memory['size'];
```

And a method to sample a mini-batch of experiences from the replay buffer:

```
# Set the batch size for sampling experiences
batch_size = 32;

# function to sample a batch of experiences from the replay memory
```

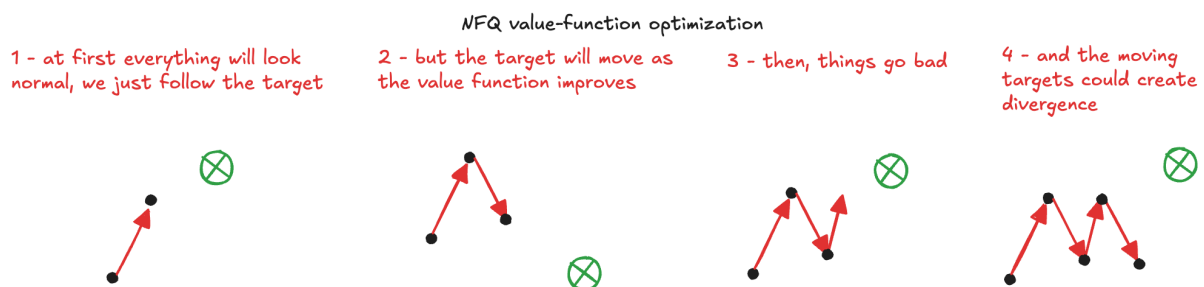
```
def sample_experiences():
    # select uniformly at random a batch of experiences from the memory
    idxs = np.random.choice(range(replay_memory['entries']), batch_size,
    ↪ replace=False);

    # return the batch of experiences
    experiences = replay_memory['buffer'][idxs];

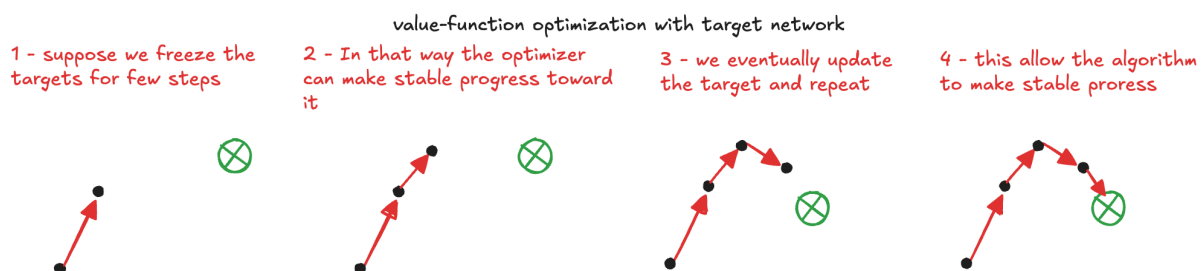
    return experiences;
```

9.2 Target networks

As we mentioned before, the targets used for training are not stationary:



One way to make the target values more stable is by using a separate network that remains fixed for several steps and is dedicated to calculating more consistent targets. This network is known as the **target network**:



This mitigates the issue of "chasing your own tail" by artificially creating **several small supervised learning problems** presented sequentially to the agent. Targets are fixed for as many steps as we fix our target network. It substantially reduces the chance of divergence. The only difference with NFQ is **the age of the neural network weights**:

$$L(\theta) = E_{(s,a) \sim U(D)} \left[(r + \gamma \max_a Q(s', a; \theta^-) - Q(s, a; \theta))^2 \right]$$

Notice that in practice we don't have two different networks. The target network is a **previous instance of the same neural network** that we freeze for a number of steps. The gradient update now has time to catch up to the target, which is much more stable when frozen. This adds stability to the updates. We use the same model architecture and "sometimes" update the weights of the target network to match the weights of the **online network**, which is the network we optimize on every step. In order to implement this idea, we can use a network architecture identical to the one used in NFQ:

```
import torch

first_hidden_layer = 512;
second_hidden_layer = 128;

def create_network():

    # Define a deep neural network using Sequential:
    # Each layer feeds directly into the next one.
    dnn = torch.nn.Sequential(
        # First fully connected layer maps state inputs to 512 hidden units
        torch.nn.Linear(state_size[0], first_hidden_layer),

        # ReLU activation introduces nonlinearity
        torch.nn.ReLU(),

        # Second fully connected layer (hidden layer with 128 units)
        torch.nn.Linear(first_hidden_layer, second_hidden_layer),

        # Another ReLU activation
        torch.nn.ReLU(),

        # Output layer: one unit per possible action
        # Produces Q-values
        torch.nn.Linear(second_hidden_layer, action_size)
    )

    # Return the constructed model
    return dnn;
```

```
online_q = create_network();
target_q = create_network();
```

The online network is trained, so we need an optimizer:

```
learning_rate = 0.007;
optimizer = torch.optim.RMSprop(online_q.parameters(), lr=learning_rate);
```

The target network is not trained, but sometimes we need to **update its weights to match the online network weights**:

```
def update_target():
    # copy the parameters from the online model to the target model
    for target, online in zip(target_q.parameters(), online_q.parameters()):
        target.data.copy_(online.data)
```

At the end of training, the online network will be used to implement the policy:

```
def dqn_pi(state):
    # convert the state into a tensor
    state = torch.as_tensor(state, dtype=torch.float32);

    # compute Q-values from the network
    q_values = online_q(state).detach().numpy().squeeze()

    # select greedy action
    action = int(np.argmax(q_values))

    # return the action
    return action;
```

That we can evaluate usgis the average reward over a number of episodes:

```
def evaluate(pi, episodes=1):

    # collect total rewards per episode
    rewards = [];

    # loop over episodes
    for episode in range(episodes):

        # reset the environment
        state, _ = env.reset();
        done = False;
        total_reward = 0.0;
```



```
# run an episode
while not done:
    action = pi(state);
    state, reward, terminal, truncated, _ = env.step(action)
    total_reward += reward;
    done = terminal or truncated;

# store the total reward
rewards.append(total_reward);

# return the average reward over the episodes
return np.mean(rewards);
```

By using the target network, we allow the weights to move consistently toward the targets before an update changes the optimization problem, but we also **slow down learning** because you're no longer training on up-to-date values. It's essential to **balance stability and speed** by tuning this hyperparameter. We can modify the optimizing method in order to use the target network to calculate the value of the next state, but the online network to calculate the values of the current state:

```
# define the discount factor
gamma = 0.99;

def optimize():

    # sample a batch of experiences
    batch = sample_experiences();

    # prepare the experience as tensors
    states      = torch.from_numpy(batch['state'].copy()).float();
    actions     = torch.from_numpy(batch['action'].copy()).long();
    rewards     = torch.from_numpy(batch['reward'].copy()).float();
    next_states = torch.from_numpy(batch['next_state'].copy()).float();
    failures    = torch.from_numpy(batch['failure'].copy()).float();

    # get the values of the Q-function at next state from the "target" network
    # remember to detach, we need to treat these values as constants
    q_target_next = target_q(next_states).detach();

    # get the max value
```

```
max_q_target_next = q_target_next.max(1)[0];

# one important step, often overlooked, is to ensure
# that failure states are grounded to zero
max_q_target_next *= (1 - failures.float())

# calculate the target
target = rewards + gamma * max_q_target_next;

# finally, we get the current estimate of Q(s,a)
# here we query the current "online" network
q_online_current = torch.gather(online_q(states), 1,
↪ actions.unsqueeze(1)).squeeze(1);

# create the errors
td_error = target - q_online_current;

# calculate the loss
loss = td_error.pow(2).mean();

# backward pass: compute the gradients
optimizer.zero_grad();
loss.backward();

# update model parameters
optimizer.step();
```

9.3 Exploration strategies

Exploration is a crucial aspect of reinforcement learning. In NFQ, we implemented an epsilon-greedy exploration strategy, where the agent acts randomly with a probability of epsilon. However, this is not the only approach (see the notebook on “Exploration vs Exploitation”). Now, we use a **decaying epsilon-greedy strategy**, where we start with a high epsilon value and gradually reduce it over time. This allows the agent to explore the environment more in the early stages of training and increasingly exploit its knowledge as training progresses. While simple and effective, other, more advanced strategies may also be worth exploring. It’s important to note that even small changes in hyperparameters can have a significant impact on performance.

```
# define decay parameters (max, min, steps)
epsilon_max = 1.0;
```

```
epsilon_min = 0.01;
epsilon_decay_steps = 10000;

# generate epsilons
epsilons = np.logspace(start=0, stop=-2, num=epsilon_decay_steps, base=10)

# normalize epsilons
epsilons = (epsilons - epsilon_min) / (epsilon_max - epsilon_min);

# scale epsilons to the desired range
epsilons = (epsilon_max - epsilon_min) * epsilons + epsilon_min;
```

```
import random

def epsilon_greedy(state, step):
    # get the epsilon value
    epsilon = epsilons[step] if step < epsilon_decay_steps else epsilon_min;

    # Exploration
    if random.random() < epsilon:
        action = random_pi(state)

    # Exploitation
    else:
        action = dqn_pi(state)

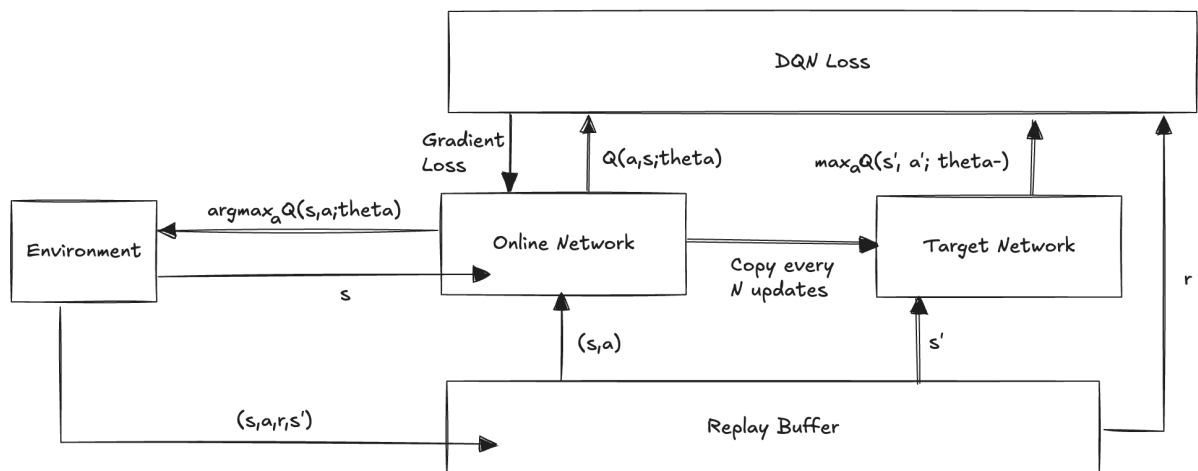
    return action;
```

9.4 DQN Algorithm

Deep Q-network (DQN) is one of the most popular deep RL algorithms and it started a series of research innovations that mark the history of RL. DQN claimed for the first time **superhuman level performance** on the **Atari benchmark** in which agents learned from raw pixel data from mere images (V. Mnih, et al. *"Playing Atari with Deep Reinforcement Learning"*, NIPS 2013 and V. Mnih, et al. *"Human-level control through deep reinforcement learning"*, Nature 2015). We now implement the full DQN algorithm, the main difference with NFQ are that it uses a replay buffer, a target network and a decaying epsilon-greedy strategy. The algorithm is as follows:

1. collect experience and insert it into the replay buffer;

2. randomly sample a mini-batch from the buffer and provide it in input to both the online and the target network;
3. use the target network to calculate the Q-values of the next state, but use the online network to calculate the Q-values of the current state;
4. compute the loss function and update the online network;
5. sometimes update the target network with the online network weights.



Let's implement the DQN algorithm in Python:

```
# define the size of the replay memory before starting the training
memory_start_size = 1000;

# define the number of steps between target network updates
target_update_steps = 10;

def dqn(max_episodes):

    # create a score tracker for statistic purposes
    scores = [];

    # counter for the number of steps
    step = 0;

    # update the target model with the online one
    update_target();

    # train until the maximum number of episodes
    for episode in range(max_episodes):
```

```
# reset the environment before starting the episode
state, _ = env.reset();
done = False;

# interact with the environment until the episode is done
while not done:

    # select the action using the exploration policy
    action = epsilon_greedy(state, step);

    # perform the selected action
    next_state, reward, terminal, truncated, _ = env.step(action);
    done = terminal or truncated;
    failure = terminal and not truncated;

    # store the experience into the replay buffer
    experience = (state, action, reward, next_state, failure);
    store_experience(experience);

    # optimize the online model after the replay buffer is large enough
    if replay_memory['entries'] > memory_start_size:
        optimize();

        # sometimes, synchronize the target model with the online model
        if step % target_update_steps == 0:
            update_target();

    # update current state to next state
    state = next_state;

    # update the step counter
    step += 1;

# After each episode, evaluate the policy
score = evaluate(dqn_pi, episodes=10);

# store the score in the tracker
scores.append(score);

# print some informative logging
message = 'Episode {:03}, score {:.05.1f}';
message = message.format(episode+1, score);
print(message, end='\n', flush=True);
```

```
return scores;
```

We can apply the algorithm to the CartPole environment, where tuning the hyperparameters is critical to optimizing the agent's performance.

```
# run the DQN algorithm
dqn(max_episodes=100);
```

Episode 100, score 239.2

From this run, it seems that DQN is much more efficient than NFQ in solving the Cart-pole environment. However, it is difficult to conclude something. To compare the performance of the two algorithms, we conduct an experiment similar to the one used for NFQ. This allows us to evaluate and compare the performance of both agents effectively.

```
import joblib

def experiment(max_episodes, filename):

    global online_q, target_q, optimizer, replay_memory, epsilons;

    # List of random seeds to test algorithm stability
    seeds = (12, 34, 56, 78, 90);

    # Container to collect all experiment results
    results = [];

    # Run an independent training experiment per seed
    for seed in seeds:

        print("Experiment seed: ", seed);

        # Set all relevant random seeds for reproducibility
        torch.manual_seed(seed);
        np.random.seed(seed);
        random.seed(seed);

        # reset the environment
        env.reset(seed=seed);
```

```
env.action_space.seed(seed)
env.observation_space.seed(seed)

# create online and target models
online_q = create_network();
target_q = create_network();
optimizer = torch.optim.RMSprop(online_q.parameters(), lr=learning_rate);

# create the replay memory
replay_memory = {
    'size': memory_size,
    'buffer': np.empty(shape=(memory_size,), dtype=experience_type),
    'index': 0,
    'entries': 0
}

# create the epsilon values
epsilons = np.logspace(start=0, stop=-2, num=epsilon_decay_steps, base=10)
epsilons = (epsilons - epsilon_min) / (epsilon_max - epsilon_min);
epsilons = (epsilon_max - epsilon_min) * epsilons + epsilon_min;

# train the network
scores = dqn(memory_start_size, target_update_steps, max_episodes);

# smooth the result using a sliding window
sliding_windows = 25;
scores = np.convolve(scores, np.ones(sliding_windows)/sliding_windows,
    ↪ mode='valid');

# collect the results
results.append(scores);

print("");

# calculate max, min and average scores among experiments
max_score = np.max(results, axis=0).T;
min_score = np.min(results, axis=0).T;
mean_score = np.mean(results, axis=0).T;

# prepare the results
experiment_results = {
    'max_score': max_score,
    'min_score': min_score,
```

```
        'mean_score': mean_score
    }

    # save permanently
    joblib.dump(experiment_results, filename);

    return experiment_results;
```

Then we run the experiment for the DQN agent on the cart-pole environment:

```
# Define the experiment setup and hyperparameters
gamma = 0.99;                # discount factor
memory_size = 100000;        # replay memory size
memory_start_size = 1000;    # minimum experiences before training
batch_size = 32;             # number of experiences per batch
target_update_steps = 100;    # steps between target network updates
first_hidden_layer = 512;     # size of the first hidden layer
second_hidden_layer = 128;    # size of the second hidden layer
learning_rate = 0.007;        # learning rate for the optimizer
epsilon_max = 1.0;           # maximum exploration rate
epsilon_min = 0.01;          # minimum exploration rate
epsilon_decay_steps = 10000;  # number of steps to decay epsilon

# Run the experiment
dqn_results = experiment(max_episodes=250, filename='./dqn_results.joblib');
```

```
Experiment seed: 12
Episode 250, score 214.5
Experiment seed: 34
Episode 250, score 113.5
Experiment seed: 56
Episode 250, score 141.0
Experiment seed: 78
Episode 250, score 046.5
Experiment seed: 90
Episode 250, score 101.8
```

While NFQ's performance fluctuates significantly, DQN demonstrates more consistent results and faster convergence, making it more suitable for this type of environment. It reach high score

in a hundred of episodes, while NFQ requires much more episodes to achieve similar results. DQN is more efficient algorithm, and it is the foundation for many other deep RL algorithms. Let's load the results of NFQ in order to compare the performances of the two algorithms:

```
nfq_results = joblib.load(' ../08 - Neural Fitted Q (NFQ)/nfq_results.joblib');

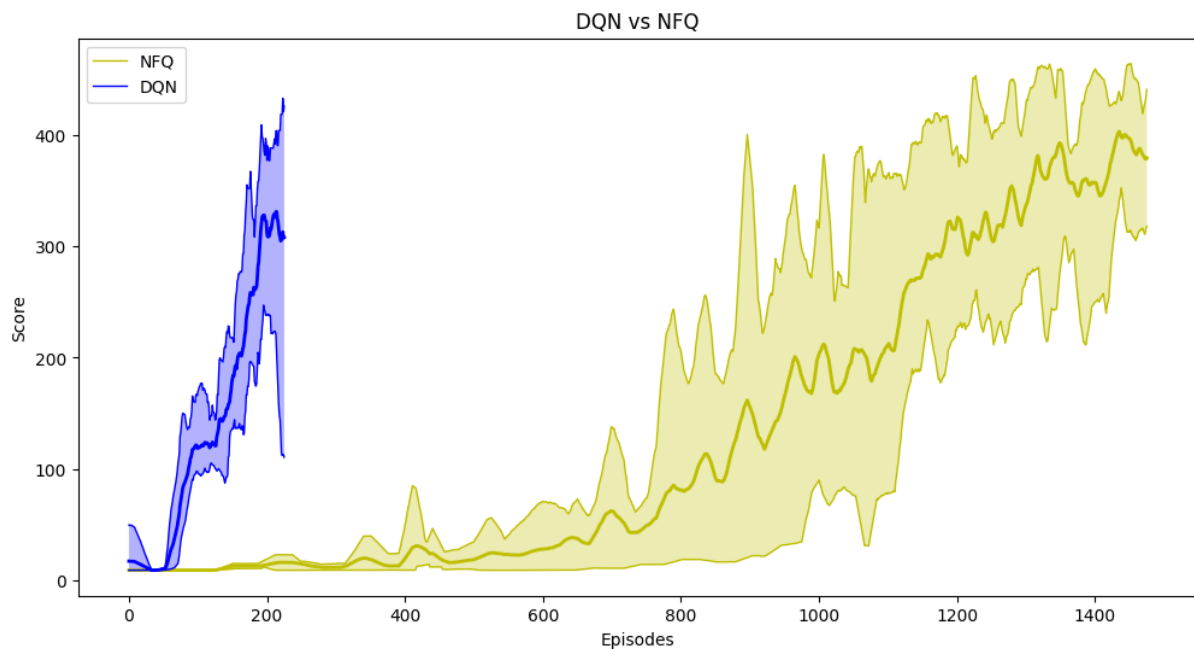
plt.figure(figsize=(12,6));
plt.title('DQN vs NFQ');
plt.ylabel('Score');
plt.xlabel('Episodes');

nfq_episodes = range(len(nfq_results['max_score']));
dqn_episodes = range(len(dqn_results['max_score']));

plt.plot(nfq_results['max_score'], 'y', linewidth=1, label="NFQ");
plt.plot(nfq_results['min_score'], 'y', linewidth=1);
plt.plot(nfq_results['mean_score'], 'y', linewidth=2);
plt.fill_between(nfq_episodes, nfq_results['min_score'], nfq_results['max_score'],
    ↪ facecolor='y', alpha=0.3);

plt.plot(dqn_results['max_score'], 'b', linewidth=1, label="DQN");
plt.plot(dqn_results['min_score'], 'b', linewidth=1);
plt.plot(dqn_results['mean_score'], 'b', linewidth=2);
plt.fill_between(dqn_episodes, dqn_results['min_score'], dqn_results['max_score'],
    ↪ facecolor='b', alpha=0.3);

plt.legend()
plt.show();
```



DQN is extrimelly **more powerful and less noisier** than NFQ. Experiment results show that the target network and the replay buffer are important for the stability of the algorithm.

9.5 Double Deep Q-Learning (DDQN)

In general, Q-learning **tends to overestimate action-value functions**, and the DQN agent is no different. We use the same off-policy TD target with the max operator, and taking the max of estimats (which are are often off-center) creates a **positive bias** and performance suffers. One way to better understand this positive bias and how we can address it is by unwrapping the max operator in the target calculations. Notice that the max of a Q-function is the same as the Q-function of the argmax action:

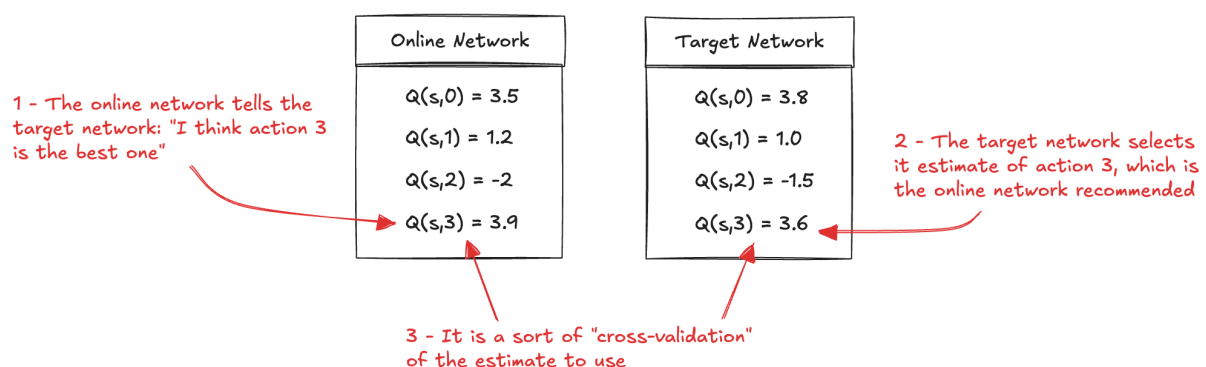
$$\max_a Q(s', a; \theta^-) = Q(s', \operatorname{argmax}_a Q(s', a; \theta^-); \theta^-)$$

so we can modify the loss expression as

$$L(\theta) = E_{(s,a) \sim U(D)} \left[(r + \gamma Q(s', \operatorname{argmax}_a Q(s', a; \theta^-); \theta^-) - Q(s, a; \theta))^2 \right]$$

There's no real difference between the two equations, since both are using the same Q-values for the target. All we're saying here is that taking the max is like asking the network "What's the value of the highest-valued action in state s?" But, we are really asking two questions with a single one. First, we do an argmax, which is equivalent to asking, "Which action is the highest-valued action in state s?" and then, we use that action to get its value, equivalent to asking, "What's the value of this action in state s?" One of the problems is that **we are asking both**

questions to the same Q-function, which shows bias in the same direction in both answers. In other words, the function approximator will answer, "I think this one is the highest-valued action in state s , and this is its value". An idea to reduce the chance of positive bias is to have **two instances of the action-value function** (van Hasselt et al. **"Deep Reinforcement Learning with Double Q-learning"** (2015) AAAI Conference, so we can ask one of the questions to one and the other question to the other. It's somewhat like getting a second doctor's opinion on health matters. In double learning, **one estimator selects the index of what it believes to be the highest-valued action, and the other estimator gives the value of this action**. Implementing this procedure creates **unnecessary overhead** (we end up with four networks, two for training and two target networks) and it creates a **slowdown in the training process** (since we'd be training only one of these networks at a time). Fortunately, there's a simple modification to the original learning procedure that adapts it to DQN and gives us substantial improvements without the extra overhead: **we can perform double learning with the other network we already have, which is the target network**. We use the online network (the network we're training) for answering the first question (to find the index of the best action), then we use the target network to ask the second question (evaluate the selected action):



So, the only difference in DDQN is that now we use the online weights to select the action, but still use the frozen weights to get the estimate:

$$L(\theta) = E_{(s,a) \sim U(D)} \left[(r + \gamma Q(s', \arg\max_a Q(s', a; \theta); \theta^-) - Q(s, a; \theta))^2 \right]$$

We can rewrite the optimize method in order to implement this idea:

```
def optimize():
    # sample a batch of experiences
    batch = sample_experiences();

    # prepare the experience as tensors
    states = torch.from_numpy(batch['state'].copy()).float();
```

```
actions = torch.from_numpy(batch['action'].copy()).long();
next_states = torch.from_numpy(batch['next_state'].copy()).float();
rewards = torch.from_numpy(batch['reward'].copy()).float();
failures = torch.from_numpy(batch['failure'].copy()).long();

# get the online model and target model predictions for the next state
# remember to detach because this part is fixed and should not be part
# of the gradient calculation
q_online_next = online_q(next_states).detach();
q_target_next = target_q(next_states).detach();

# get the best actions according to the online model
best_actions = q_online_next.max(1)[1];

# get the value estimations of the best actions from the target model
max_q_target_next = torch.gather(q_target_next, 1,
↪ best_actions.unsqueeze(1)).squeeze(1);
max_q_target_next = max_q_target_next * (1 - failures.float());

# calculate the target
target = rewards + gamma * max_q_target_next;

# finally, we get the current estimate of Q(s,a)
# here we query the current "online" network
q_online_current = torch.gather(online_q(states), 1,
↪ actions.unsqueeze(1)).squeeze(1);

# create the errors
td_error = target - q_online_current;

# calculate the loss
loss = td_error.pow(2).mean();

# backward pass: compute the gradients
optimizer.zero_grad();
loss.backward();

# update model parameters
optimizer.step();
```

We run the experiment in order to evaluate the impact of this idea:

```
ddqn_results = experiment(max_episodes=250, filename = './ddqn_results.joblib');
```

```
Experiment seed: 12
Episode 250, score 500.0
Experiment seed: 34
Episode 250, score 124.5
Experiment seed: 56
Episode 250, score 112.6
Experiment seed: 78
Episode 250, score 204.3
Experiment seed: 90
Episode 250, score 086.7
```

After the experiment, we save results for later use and we make similar statistical operations to create a graph of performance:

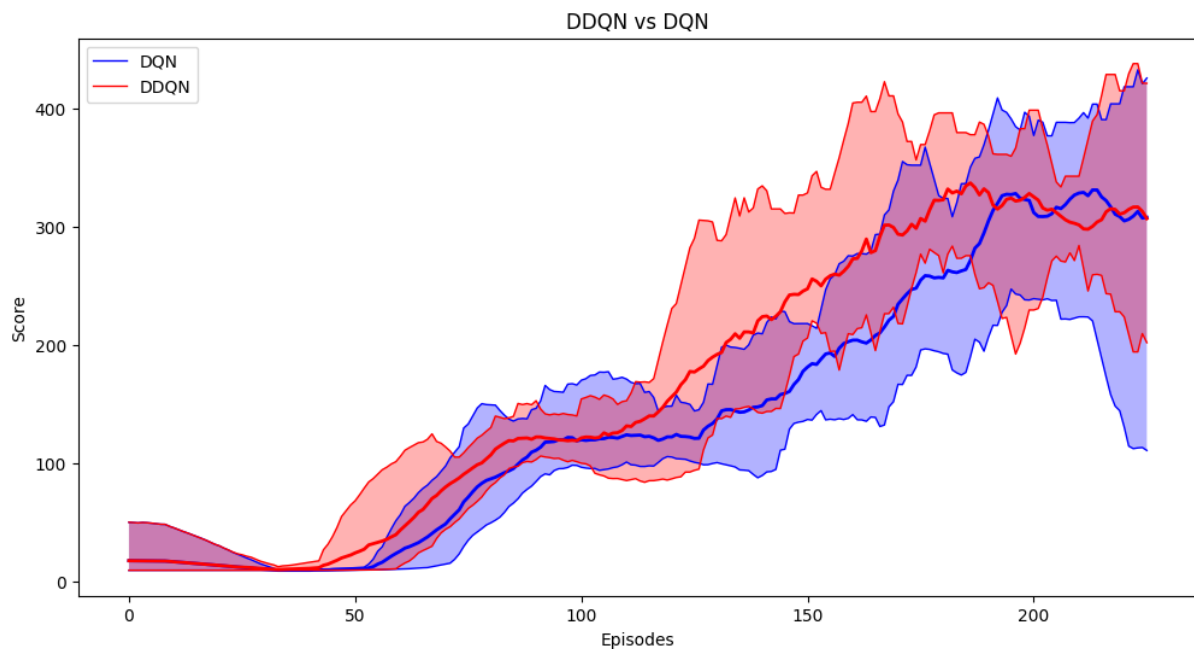
```
plt.figure(figsize=(12,6));
plt.title('DDQN vs DQN');
plt.ylabel('Score');
plt.xlabel('Episodes');

dqn_episodes = range(len(dqn_results['max_score']));
ddqn_episodes = range(len(ddqn_results['max_score']));

plt.plot(dqn_results['max_score'], 'b', linewidth=1, label="DQN");
plt.plot(dqn_results['min_score'], 'b', linewidth=1);
plt.plot(dqn_results['mean_score'], 'b', linewidth=2);
plt.fill_between(dqn_episodes, dqn_results['min_score'], dqn_results['max_score'],
    ↪ facecolor='b', alpha=0.3);

plt.plot(ddqn_results['max_score'], 'r', linewidth=1, label="DDQN");
plt.plot(ddqn_results['min_score'], 'r', linewidth=1);
plt.plot(ddqn_results['mean_score'], 'r', linewidth=2);
plt.fill_between(ddqn_episodes, ddqn_results['min_score'], ddqn_results['max_score'],
    ↪ facecolor='r', alpha=0.3);

plt.legend();
plt.show();
```



DQN and DDQN have similar performance in the cart-pole environment. DDQN seems **slightly more data efficient** since the number of samples it needs to pass the environment is lower than that of the DQN. However, this is a simple environment with a smooth reward function. In reality, DDQN should always give better performance than DQN in a more evident way.

9.6 Prioritized Experience Replay (PER)

The agent samples experience tuples from replay memory uniformly at random. Intuitively, this seems like an **inefficient way to replay experiences**. It feels wasteful for the agent to spend time "learning" from experiences that offer little relevance to its current state. While this issue is clear, it's more challenging to define a better sampling strategy. One might think that, since the agent aims to maximize cumulative discounted rewards, it should focus on replaying experiences with high rewards. However, this approach proved ineffective. The agent also needs other types of experiences! What we're really looking for are **experiences with unexpected value (surprising experiences)**. The agent forms expectations about the world, and when outcomes deviate significantly from these expectations, it presents a key learning opportunity. In reinforcement learning, this **measure of surprise** is captured by the TD error, which quantifies the difference between the agent's current estimate and the target value. The greater this difference, the more unexpected the experience is, making it a valuable opportunity for the agent to learn and adjust its understanding (Tom Schaul et al. **Prioritized Experience Replay** ICLR 2016):

$$|\delta| = |r + \gamma Q(s', \arg\max_a Q(s', a; \theta); \theta^-) - Q(s, a; \theta)|$$

We can **insert experience into the replay memory as a tuple containing also to the TD error**, so we can pull out the top experiences from the buffer **sorting** by this new value. However there are two issues with this approach. First, we are calculating the TD errors twice (before inserting it into the memory, and then again when we train the network), second we're ignoring the fact that TD errors change every time the network changes, because they're calculated using the network (bootstrapping). The solution can't be updating all of the TD errors every time step, it's simply not cost effective. A workaround for both these problems is to **update the TD errors only for experiences that are used to update the network (the replayed experiences)** and insert new experiences with the highest magnitude TD error in the memory. However, other issues arise: a TD error of zero in the first update means that experience will likely never be replayed again, moreover when using function approximator, errors shrink slowly, and this means that updates concentrate heavily in a small subset of the replay memory. For these reasons, we need a strategy for sampling experiences based on the TD errors, but **stochastically, not greedily**: in that way we can simultaneously ensure all experiences have a chance of being replayed, and that the probabilities of sampling experiences are monotonic in the absolute TD error. So, first we calculate priorities for each sample in the buffer based on TD errors adding a small constant (epsilon) to make sure zero TD error samples still have a chance of being replayed:

$$p_i = |\delta_i| + \epsilon$$

Then, we scale the priority by exponentiating it to a hyperparameter α that allows us to interpolate between uniform and prioritized sampling:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

When α is zero, all priorities are treated equally, resulting in uniform sampling. When α is one, the priorities directly correspond to the absolute TD error values. Intermediate values of α blend these two strategies, balancing between uniform sampling and prioritization. The scaled priorities are converted to actual probabilities by dividing by the sum of the values.

One issue with this approach is its sensitivity to outliers. Experiences with unusually high TD errors (whether due to noise or actual significance) are sampled more frequently than those with lower errors. This can lead to over-sampling such outliers, which may be undesirable. So prioritization introduces **sampling bias toward high-priority experiences**, meaning those with high TD errors are more likely to be selected, deviating from uniform sampling. Over-sampling high-priority experiences without adjustment can cause the model to overfit to these specific cases. To address this, we need to compensate for the bias during training. While **we want important experiences to be sampled more often, we must also reduce their impact by giving them lower weight during training**. This is done using **weighted importance sampling**, where

the TD errors are scaled by weights based on the sampling probabilities:

$$w_i = \left(\frac{1}{N P_i} \right)^\beta$$

We introduce the beta hyperparameter to control the degree of bias correction. When beta is zero, no correction is applied, while a value of one fully corrects the sampling bias. Early in training, when learning is highly unstable, the corrections are less critical. However, as training progresses, the importance of these corrections increases. To accommodate this, beta typically starts with a smaller value (commonly between 0.4 and 0.6) and gradually anneals toward one as training nears completion. Additionally, we normalize the importance-sampling weights by dividing them by the maximum weight, ensuring that the largest weight is one and all others scale proportionally. This normalization prevents the TD errors from growing too large, helping to maintain stability throughout training:

$$w_i = \frac{w_i}{\max(w_j)}$$

We modify the definition of replay memory in order to add the priority value to all sample and the parameters for the sampling algorithm:

```
# add the priority and weight fields to the experience type
experience_type = np.dtype([
    ('state', np.float32, (state_size)),
    ('action', np.int8),
    ('reward', np.float32),
    ('next_state', np.float32, (state_size)),
    ('failure', np.int8),
    ('priority', np.float32),
    ('weight', np.float32)
])
```

We modify the store method in order to initialize the priority of samples:

```
def store_experience(experience):

    # Calculate the priority for the sample using
    # the highest magnitude TD error in the memory
    priority = 1.0
    if replay_memory['entries'] > 0:
        priority =
    ↪ replay_memory['buffer'][:replay_memory['entries']]['priority'].max()
```



```

# initialize the weight to 1
weight = 1.0;

# add the priority and weight fields to the experience
experience = experience + (priority,);
experience = experience + (weight,);

# store the experience in the buffer
replay_memory['buffer'][replay_memory['index']] = experience;

# update the number of experiences in the buffer
replay_memory['entries'] = min(replay_memory['entries'] + 1,
↪ replay_memory['size']);

# update index, if the memory is full, start from the begging
replay_memory['index'] += 1;
replay_memory['index'] = replay_memory['index'] % replay_memory['size'];

```

We need a method to update the priority of samples that are used to update the network (the replayed experiences):

```

def update_experience(idxs, td_errors):
    # takes the array of experiences ids, and new TD error values
    # insert the absolute TD errors into the right place.
    replay_memory['buffer'][idxs]['priority'] = np.abs(td_errors);

```

Finally, we change the sample method in order to implement the prioritized sampling:

```

# hyperparameters for the prioritized experience replay
alpha = 0.5;
beta = 0.4;

def sample_experiences():
    # go from priorities to probabilities
    priorities = replay_memory['buffer'][:replay_memory['entries']]['priority'] +
↪ 1e-6;
    scaled_priorities = priorities ** alpha;

    probs = scaled_priorities / np.sum(scaled_priorities);

    # calculate the importance-sampling weights using the probabilities

```

```

weights = (replay_memory['entries'] * probs) ** beta;
weights = weights/weights.max();

# store the weights in the buffer
replay_memory['buffer'][:replay_memory['entries']]['weight'] = weights;

# sample experiences in the memory using the probabilities
idxs = np.random.choice(replay_memory['entries'], batch_size, replace=False,
↪ p=probs);

# return the batch of experiences
experiences = replay_memory['buffer'][idxs];

return experiences, idxs;

```

Now we can write the new expression for the loss function. First of all we are sampling from the replay buffer using the probabilities calculated by the priorities (P) and not uniformly at random (U). Moreover we use the importance-sampling weights to scale the TD errors:

$$L(\theta) = E_{(s,a) \sim P(D)} \left[w(r + \gamma Q(s', \arg\max_a Q(s', a; \theta); \theta^-) - Q(s, a; \theta))^2 \right]$$

We can implement this loss function in the optimize method:

```

def optimize():

    # sample a batch of experiences
    batch, idxs = sample_experiences();

    # prepare the experience as tensors
    states = torch.from_numpy(batch['state'].copy()).float();
    actions = torch.from_numpy(batch['action'].copy()).long();
    next_states = torch.from_numpy(batch['next_state'].copy()).float();
    rewards = torch.from_numpy(batch['reward'].copy()).float();
    failures = torch.from_numpy(batch['failure'].copy()).long();
    priorities = torch.from_numpy(batch['priority'].copy()).float();
    weights = torch.from_numpy(batch['weight'].copy()).float();

    # get the online model and target model predictions for the next state
    # remember to detach because this part is fixed and should not be part
    # of the gradient calculation
    q_online_next = online_q(next_states).detach();
    q_target_next = target_q(next_states).detach();

```

```
# get the best actions according to the online model
best_actions = q_online_next.max(1)[1];

# get the value estimations of the best actions from the target model
max_q_target_next = torch.gather(q_target_next, 1,
↪ best_actions.unsqueeze(1)).squeeze(1);
max_q_target_next = max_q_target_next * (1 - failures.float());

# calculate the target
target = rewards + gamma * max_q_target_next;

# finally, we get the current estimate of Q(s,a)
# here we query the current "online" network
q_online_current = torch.gather(online_q(states), 1,
↪ actions.unsqueeze(1)).squeeze(1);

# create the errors
td_errors = target - q_online_current;

# calculate the loss using the TD error downscaled by the weights
loss = (weights * td_errors).pow(2).mean();

# backward pass: compute the gradients
optimizer.zero_grad();
loss.backward();

# update model parameters
optimizer.step();

# update the priorities of the replayed batch
# using the absolute TD errors
priorities = np.abs(td_errors.detach().cpu().numpy());
update_experience(idxs, priorities.squeeze());
```

We run the experiment in order to evaluate the impact of this idea:

```
per_results = experiment(max_episodes=250, filename='./per_results.joblib');
```

Experiment seed: 12

Episode 250, score 128.4

```
Experiment seed: 34
Episode 250, score 500.0
Experiment seed: 56
Episode 250, score 500.0
Experiment seed: 78
Episode 250, score 500.0
Experiment seed: 90
Episode 250, score 217.0
```

After the experiment, we save results for later use and we make similar statistical operations to create a graph of performance:

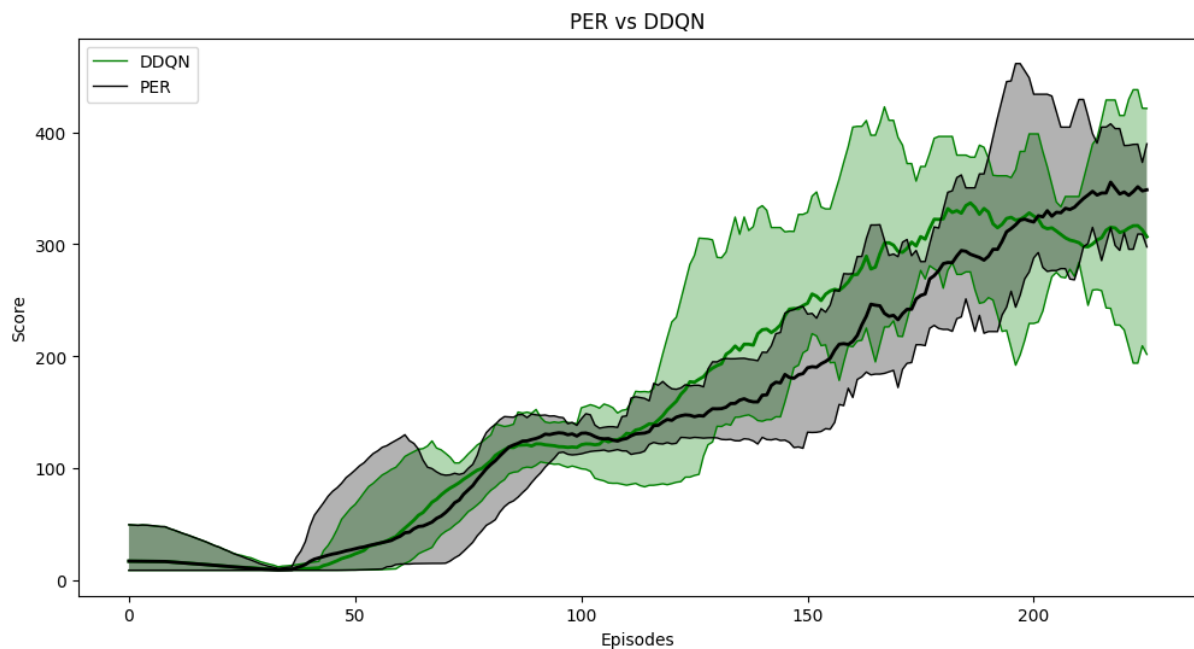
```
plt.figure(figsize=(12,6));
plt.title('PER vs DDQN');
plt.ylabel('Score');
plt.xlabel('Episodes');

ddqn_episodes = range(len(ddqn_results['max_score']));
per_episodes = range(len(per_results['max_score']));

plt.plot(ddqn_results['max_score'], 'g', linewidth=1, label="DDQN");
plt.plot(ddqn_results['min_score'], 'g', linewidth=1);
plt.plot(ddqn_results['mean_score'], 'g', linewidth=2);
plt.fill_between(ddqn_episodes, ddqn_results['min_score'], ddqn_results['max_score'],
    ↪ facecolor='g', alpha=0.3);

plt.plot(per_results['max_score'], 'k', linewidth=1, label="PER");
plt.plot(per_results['min_score'], 'k', linewidth=1);
plt.plot(per_results['mean_score'], 'k', linewidth=2);
plt.fill_between(per_episodes, per_results['min_score'], per_results['max_score'],
    ↪ facecolor='k', alpha=0.3);

plt.legend();
plt.show();
```



The prioritized replay buffer uses fewer samples than any of the previous methods. And as you can see it in the graphs below, it even makes things look more stable. PER is much slower than other methods, but this is an implementation-specific issue, if we get a high quality implementation of the replay buffer, this shouldn't happen.

9.7 Dueling Architecture

The dueling network is an improvement that applies to the network architecture and not the algorithm. Until now, we concentrated our efforts on making reinforcement learning **looks more like a supervised learning problem**: using the replay memory we made data more like an independent and identically distributed dataset, using two networks we made targets look more static. This helps stabilize training, but ignoring the fact that reinforcement learning problems are problems of their own isn't the smartest approach to solving these problems. Now we try to exploit the subtleties of value-based deep reinforcement learning agents, in particular the way the value functions relate to one another. More specifically, we can use the fact that the state-value function and the action-value function are related to each other through the **advantage-value function**:

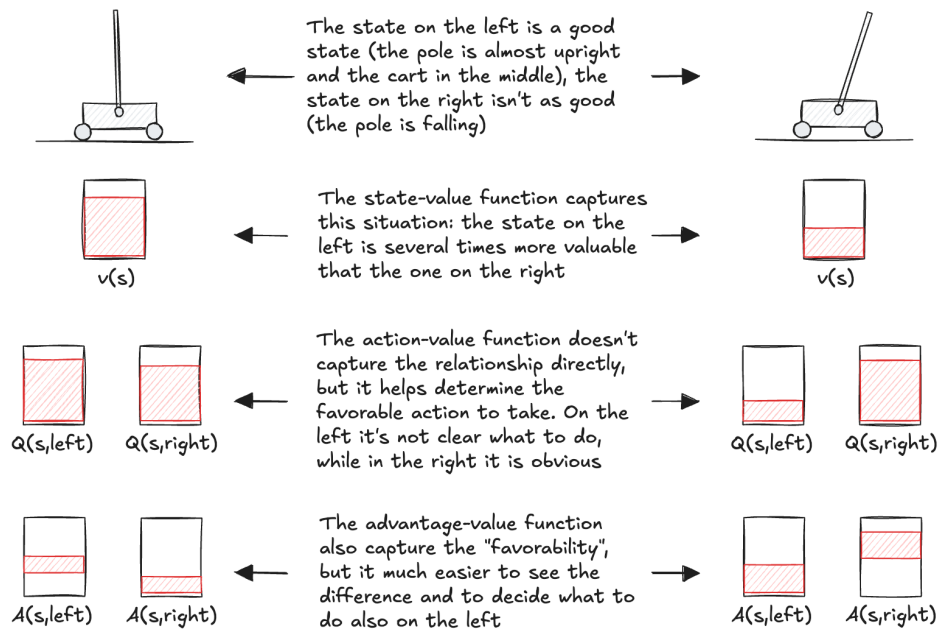
$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

we can define the **advantage-value function** as the **difference between the value of taking an action in a state and the value of choosing the policy action**:

$$a_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$$

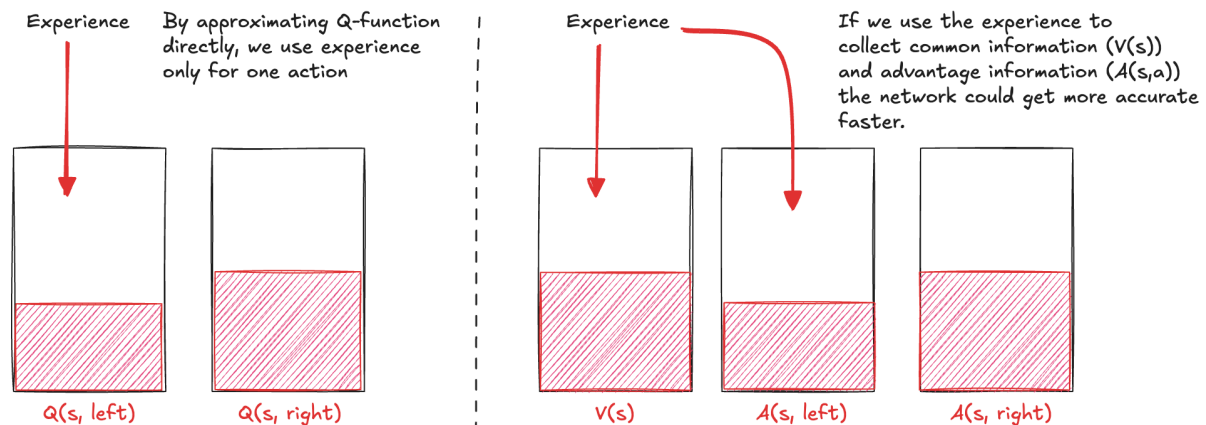
Considering the following example from cart-pole to better understand the idea of the advantage-value function:



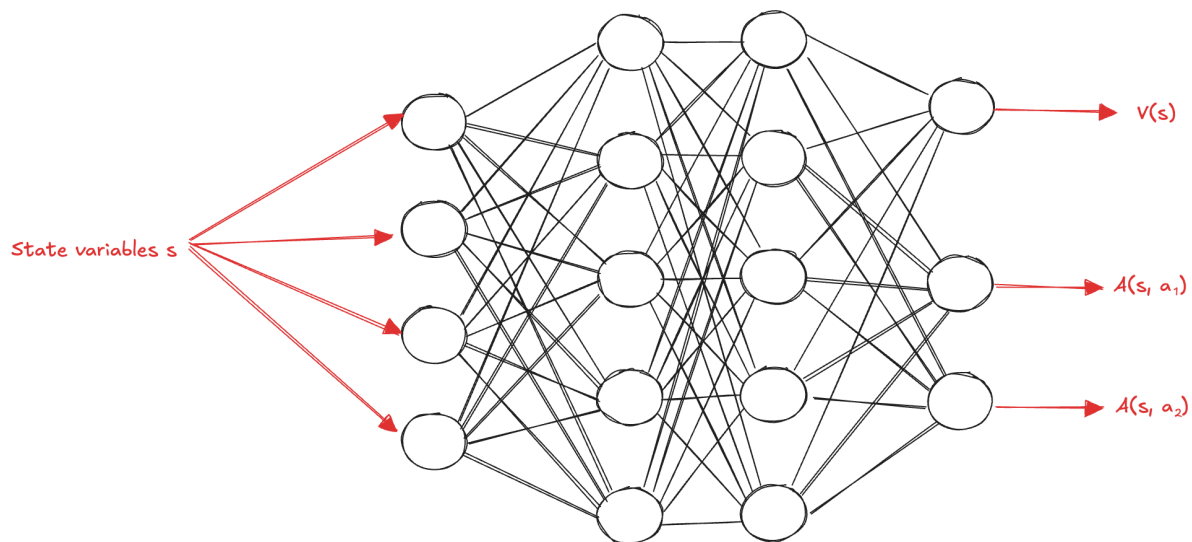
We can rewrite the equation and observe that the action-value function can be defined as the sum of the state-value function and the advantage-value function. This means that we can decompose the action-value function into two components: **one shared across all actions (state-value)**, and **another unique to each action (advantage-value)**:

$$q_{\pi}(s, a) = v_{\pi}(s) + a_{\pi}(s, a)$$

This means that we're learning the action-value function inefficiently, because **some information is shared between the nodes of a network**. However, when learning about the action-value of an action, we're ignoring the fact that we could use the same information to learn something about all other actions:



The bottom line is that **the values of actions depend on the values of states**, and it would be nice to leverage this fact. The **Dueling Network Architecture** (Ziyu et al. **Dueling Network Architectures for Deep Reinforcement Learning**, ICML 2016) consists of creating two separate estimators: one of the state-value function, and the other of the action-advantage function. We can use a single network sharing most of the internal nodes and layers, the layer before the output splits into two heads: an head for the state-value function and another head for the action-advantage function. The state-value function output layer always ends in a single node because the value of a state is always a single number, the output layer for the advantage-value function, however, outputs a vector of the same size as the number of actions:



Now, we need to find a way to **aggregate the two outputs from the network** and reconstruct the action-value function. How do we join the outputs? Unfortunately, we cannot simply compute the sum directly. Consider the Q -function, it is parameterized by weights of the shared layers (θ), weights of the advantage-value function head (α), and weights of the state-value

function head (beta):

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha)$$

Notice that we cannot uniquely recover the Q from V and A. Imagine a situation where all actions in a given state should have the same Q-value. The value of V can remain constant, and the algorithm can arbitrarily increase or decrease the advantage for different actions. This introduces ambiguity because it falsely suggests one action is better or worse, even though they should have equal value. This confuses the learning process by decoupling the real relation between the state value and the advantage value. To avoid ambiguity, the mean of the advantage should be subtracted, ensuring that V represents only the state's importance, while A models the **relative differences** between actions.. This normalization prevents the network from skewing the advantage values incorrectly:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

We implement the dueling network architecture in Python, however we cannot use the Sequential class of PyTorch, since we need to split the network into two heads, one to evaluate A and one to evaluate V. So need to implement a new class in order to describe the two separate branches (value and advantage) that must be combined with custom operations, which cannot be expressed using a simple Sequential model:

```
class DualHeadNetwork(torch.nn.Module):
    def __init__(self):
        super(DualHeadNetwork, self).__init__()

        # shared layers
        self.shared = torch.nn.Sequential(
            torch.nn.Linear(state_size[0], first_hidden_layer),
            torch.nn.ReLU(),
            torch.nn.Linear(first_hidden_layer, second_hidden_layer),
            torch.nn.ReLU());

        # value head
        self.value_head = torch.nn.Linear(second_hidden_layer, 1);

        # advantage head
        self.advantage_head = torch.nn.Linear(second_hidden_layer, action_size);

    def forward(self, state):
        x = self.shared(state)
        value = self.value_head(x)
```



```

    advantage = self.advantage_head(x)
    advantage = advantage - advantage.mean(dim=-1, keepdim=True)
    q_values = value + advantage
    return q_values

```

Now we can create the online and target networks using the dueling architecture:

```

def create_network():
    dnn = DualHeadNetwork();
    return dnn;

```

The rest of the architecture of the Dueling DDQN is almost identical to DDQN: the network outputs the action-value approximation, an off-policy TD target is used to evaluate policies, a replay memory is used to learning from past experiences, the optimizer is the RMSprop, an exponentially decaying epsilon-greedy strategy is used to improve policies and a greedy action selection strategy for the evaluation step. However we can observe that the DDQN agent use a target network that can be **outdated for several steps** before it gets a weight update. There are a couple of issues with this approach: on the one hand, we're freezing the weights for several steps and calculating estimates with progressively increasing stale data, on the other hand, a huge update is made to the network, that changes the whole landscape of the loss function all at once. **This update style seems to be both too conservative and too aggressive at the same time.** Of course, we got into this issue because we wanted our network not to move too quickly and therefore create instabilities. We can have similar results, but in a smooth manner: we can **slow down the target network instead of freezing it**. This technique is called **Polyak Averaging**, and it consists of mixing online network weights into the target network on every step. We create a new target network composed of a large percentage of the target network weights and a small percentage of the online network weights. In that way, the target network always lags, but by a much smaller gap. So, instead of making the target network equal to the online network every N time steps, we mix the target network with a tiny bit of the online network more frequently (even every time step):

$$\theta_i^- = \tau \theta_i + (1 - \tau) \theta_i^-$$

$$\alpha^- = \tau \alpha_i + (1 - \tau) \alpha_i^-$$

$$\beta_i^- = \tau \beta_i + (1 - \tau) \beta_i^-$$

where τ is the **mixing factor**. We can modify the update function in order to implement this idea:

```
# parameter for soft update
tau = 0.1

def update_target():
    for target, online in zip(target_q.parameters(), online_q.parameters()):

        # calculate the ratios we're taking from the target and online weights
        target_ratio = tau * target.data;
        online_ratio = (1.0 - tau) * online.data;

        # mix the weights and copy the new values into the target network
        mixed_weights = target_ratio + online_ratio;
        target.data.copy_(mixed_weights);
```

We can perform the experiment using the dueling improvements, setting the tau parameter at 0.01 and the target_update_steps to 1, the target network will be fully updated every 100 steps, as in our previous experiments, but this time more gradually and smoothly:

```
dueling_results = experiment(max_episodes=250, filename='./dueling_results.joblib');
```

```
Experiment seed: 12
Episode 250, score 500.0
Experiment seed: 34
Episode 250, score 500.0
Experiment seed: 56
Episode 250, score 101.2
Experiment seed: 78
Episode 250, score 088.1
Experiment seed: 90
Episode 250, score 159.3
```

```
plt.figure(figsize=(12,6));
plt.title('Dueling vs PER');
plt.ylabel('Score');
plt.xlabel('Episodes');

per_episodes = range(len(per_results['max_score']));
dueling_episodes = range(len(dueling_results['max_score']));
```

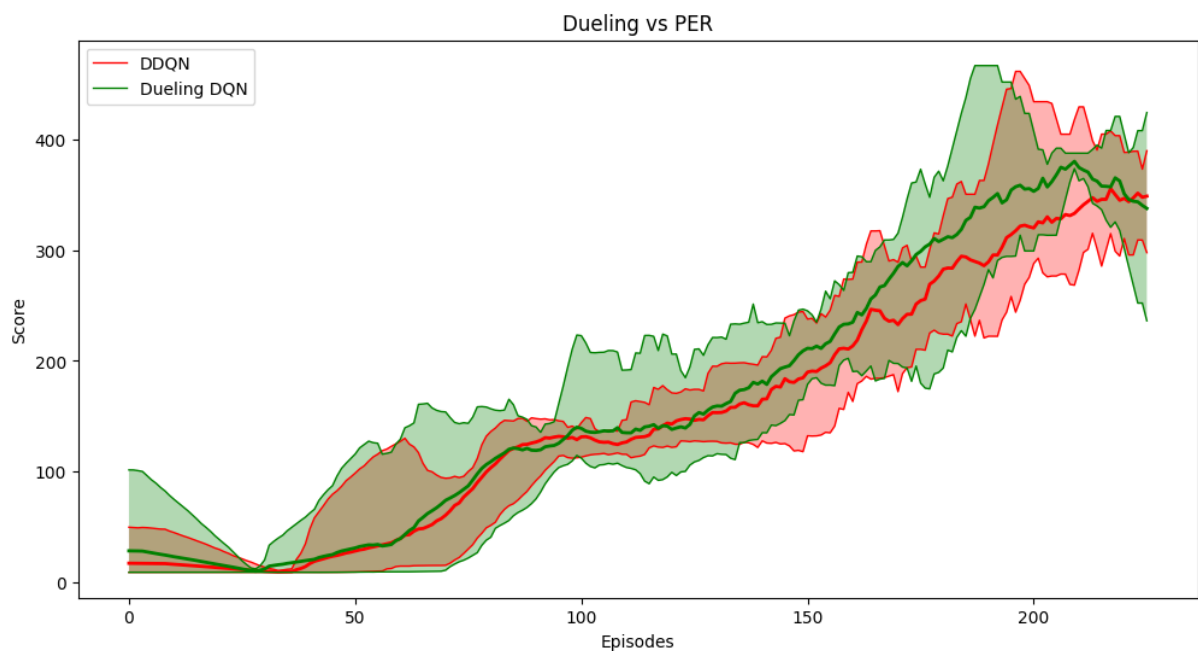
```

plt.plot(per_results['max_score'], 'r', linewidth=1, label="DDQN");
plt.plot(per_results['min_score'], 'r', linewidth=1);
plt.plot(per_results['mean_score'], 'r', linewidth=2);
plt.fill_between(per_episodes, per_results['min_score'], per_results['max_score'],
    ↳ facecolor='r', alpha=0.3);

plt.plot(dueling_results['max_score'], 'g', linewidth=1, label="Dueling DQN");
plt.plot(dueling_results['min_score'], 'g', linewidth=1);
plt.plot(dueling_results['mean_score'], 'g', linewidth=2);
plt.fill_between(dueling_episodes, dueling_results['min_score'],
    ↳ dueling_results['max_score'], facecolor='g', alpha=0.3);

plt.legend();
plt.show();

```



Dueling DDQN has similar performance in the cart-pole environment. It seems **more stable** (the max and min values are closer to the mean value), but the difference is not so evident in this simple environment.

9.8 The deadly triad

All of those improvements can be combined together in several different ways in order to achieve better performance and stability. For example you can consider the **Rainbow algorithm** (Hes-

sel et al. “**Rainbow: Combining Improvements in Deep Reinforcement Learning**”, AAAI, 2017). In any way, all those value-based methods are really **sensitive to hyperparameters**. Try it for yourself: change any hyperparameter and you will find more values that doesn’t work than values that do. Second, value-based methods **assume they interact with a Markovian environment** (state contains all information required by the agent) and this assumption dissipates as we move away from bootstrapping and value-based methods in general. Finally, **the combination of bootstrapping, off-policy learning, and function approximator** is known as **the deadly triad** (van Hasselt et al. “**Deep Reinforcement Learning and the Deadly Triad**”, 2018). Avoiding its pitfalls often requires careful use of target networks, replay buffers, double learning, appropriately small learning rates—and a bit of patience.

10 Policy Based Methods

Up to now, the methods we have discussed are **value-based methods**. These algorithms learn the values of actions (or state–action pairs) and then derive the policy by selecting the actions with the highest estimated value. The policy depends on the value function estimates, without those the policy would not exist.

A different family of methods, called **policy-based methods**, take a more direct approach: they learn a parameterized policy that maps states to actions without consulting a value function (Sutton et al., “**Policy Gradient Methods for Reinforcement Learning with Function Approximation**”, NIPS, 2000). The policy is written as:

$$\pi_{\theta}(a|s)$$

The goal is to **maximize a performance measure** $J(\theta)$ with respect to these parameters:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t)$$

The only essential requirement for a policy parameterization is that **it must be differentiable** with respect to θ , allowing gradient-based optimization.

In practice, to ensure sufficient exploration, it is important that the policy **does not collapse into a purely deterministic mapping**. A common choice is to parameterize it as a **softmax distribution** over action preferences, which assigns a probability to each action while still favoring those with higher estimated preference:

$$\pi_{\theta}(a|s) = \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

where $h(s, a, \theta)$ is the **preference function** for actions in states. It might be computed by a neural network, where the parameter is the vector of all the connection weights of the network, or it could simply be a linear function.

10.1 Objective function

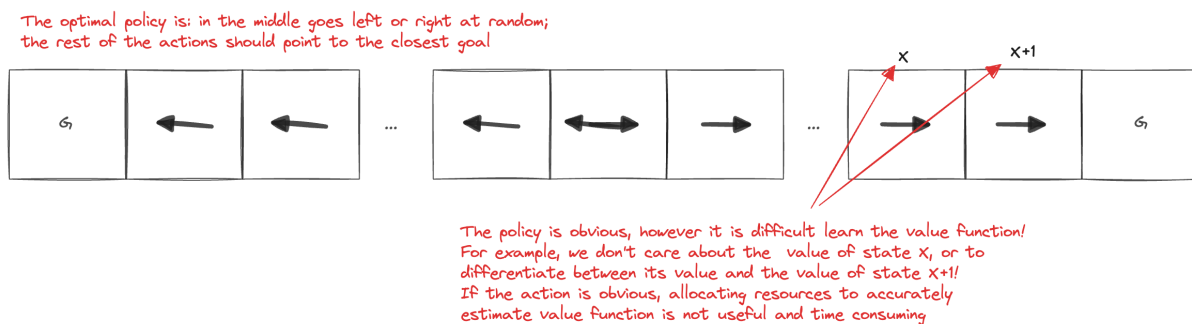
It is natural to define the performance measure as the **expected discounted return from the initial state**:

$$J(\pi_\theta) = E_{s_0 \sim p_0} [G_{\pi_\theta}(s_0)]$$

where p_0 is the distribution of the initial state, and G_{π_θ} is the total discounted return obtained under policy π_θ .

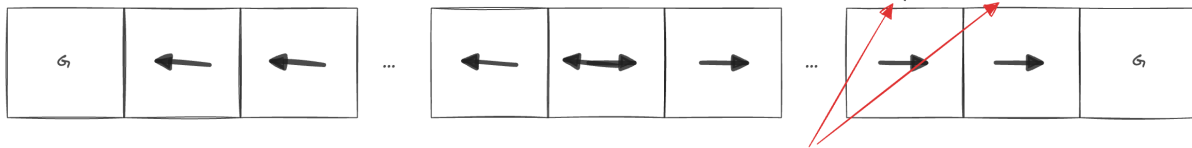
Learning **stochastic policies directly** has several advantages. They often achieve **better performance in partially observable environments**, since they can hedge decisions across states that appear identical under limited observations. Moreover, exploration is embedded in the learned function itself. In addition, learning a policy can be simpler than learning a value function, since estimating exact state or action values may be unnecessarily complex.

We can now **learn stochastic policies directly**, which often achieve better performance in partially observable environments. The key intuition is that by learning arbitrary action probabilities, the agent is not forced to rely strictly on the Markov assumption. In other words, when the agent cannot fully distinguish between different underlying states based on its observations, a deterministic policy may fail. In such cases, the optimal behavior is often to act randomly but with carefully chosen probabilities, allowing the agent to hedge its decisions across indistinguishable situations. Consider the foggy lake environment:



Interestingly, with stochastic policies, **exploration can be embedded in the learned function**. Moreover, learning stochastic policies could be **more straightforward for function approximation**. Sometimes value functions are **too much informative** for what's truly needed. It could be that calculating the exact value of a state or state-action pair is complicated or unnecessary. For example, consider a near-infinite corridor deterministic environment:

The optimal policy is: in the middle goes left or right at random;
the rest of the actions should point to the closest goal



The policy is obvious, however it is difficult learn the value function!
For example, we don't care about the value of state x , or to
differentiate between its value and the value of state $x+1$!
If the action is obvious, allocating resources to accurately
estimate value function is not useful and time consuming

Finally, value-based methods are **prone to oscillations and even divergence**, because small changes in the value function can translate into large, discontinuous shifts in the chosen actions. Such abrupt changes may lead the agent to follow entirely different trajectories, amplifying instability. By contrast, policy-based methods optimize the parameters of a stochastic policy directly through gradient ascent. This results in **actions being adjusted gradually and smoothly**, which often gives policy-based methods **more stable and reliable convergence** properties.

10.1.1 Policy Gradient Theorem

To improve a policy directly, we need a way to adjust its parameters so that performance improves. This requires computing the **gradient of the performance measure** with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} E_{s_0 \sim p_0} [G_{\pi(\theta)}]$$

Let's introduce the variable τ to denote a trajectory from a starting state following the policy:

$$\tau = (S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T)$$

and define its return as:

$$G(\tau) = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$$

we can express the performance measure as the expected return over all possible trajectories:

$$J(\theta) = E_{\tau \sim \pi_{\theta}} [G(\tau)]$$

The **probability of a trajectory** under the policy is given by:

$$p(\tau | \pi_{\theta}) = p_0(S_0) \pi_{\theta}(A_0 | S_0) P(S_1 | S_0, A_0) \pi_{\theta}(A_1 | S_1) P(S_2 | S_1, A_1) \dots P(S_T | S_{T-1}, A_{T-1}) =$$

$$= p_0(S_0) \prod_{t=0}^{T-1} \{P(S_{t+1} | S_t, A_t) \pi_{\theta}(A_t | S_t)\}$$

Using this notation, we can rewrite the objective as:

$$J(\theta) = E_{\tau \sim \pi_\theta} [G(\tau)] = \sum_{\tau} p(\tau | \pi_\theta) G(\tau)$$

Taking the gradient of this expression, we obtain:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} p(\tau | \pi_{\theta}) G(\tau)$$

We can't compute it exactly, because we don't have complete knowledge of the problem. It is a mix of factors that are under our control via the parameters (policy) and factors that are not known (transition probabilities and initial distribution). We can provide an estimation of the gradient, making some mathematical manipulations to remove the unknown factors. first, we can rewrite the gradient of the sum as the sum of the gradient:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} p(\tau | \pi_{\theta}) G(\tau)$$

Multiplying and dividing by $p(\tau | \pi_{\theta})$, we obtain the **likelihood ratio** form:

$$= \sum_{\tau} p(\tau | \pi_{\theta}) \frac{\nabla_{\theta} p(\tau | \pi_{\theta})}{p(\tau | \pi_{\theta})} G(\tau)$$

a mathematical result states that the gradient of a function divided by the function itself is equal to the gradient of the log of the function:

$$= \sum_{\tau} p(\tau | \pi_{\theta}) \nabla_{\theta} \log p(\tau | \pi_{\theta}) G(\tau) = E_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \log p(\tau | \pi_{\theta}) G(\tau) \right]$$

Substituting the definition of the transition probability:

$$= E_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \log \left(p_0(S_0) \prod_{t=0}^{T-1} P(S_{t+1} | S_t, A_t) \pi_{\theta}(A_t | S_t) \right) G(\tau) \right]$$

Substituting the log of the product with the sum of the logs:

$$= E_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \left(\log p_0(S_0) + \sum_{t=0}^{T-1} \log P(S_{t+1} | S_t, A_t) + \sum_{t=0}^{T-1} \log \pi_{\theta}(A_t | S_t) \right) G(\tau) \right]$$

Substituting the gradient of the sum with the sum of gradients:

$$= E_{\tau \sim \pi_{\theta}} \left[\left(\nabla_{\theta} \log p_0(S_0) + \nabla_{\theta} \sum_{t=0}^{T-1} \log P(S_{t+1} | S_t, A_t) + \nabla_{\theta} \sum_{t=0}^{T-1} \log \pi_{\theta}(A_t | S_t) \right) G(\tau) \right]$$

Neither initial state distribution or state transition dynamics are dependent of the policy parameters, so the derivate of both terms can be removed:

$$E_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \sum_{t=0}^{T-1} \log \pi_{\theta}(A_t | S_t) G(\tau) \right] =$$

Substituting the gradient of the sum with the sum of gradients:

$$E_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G(\tau) \right]$$

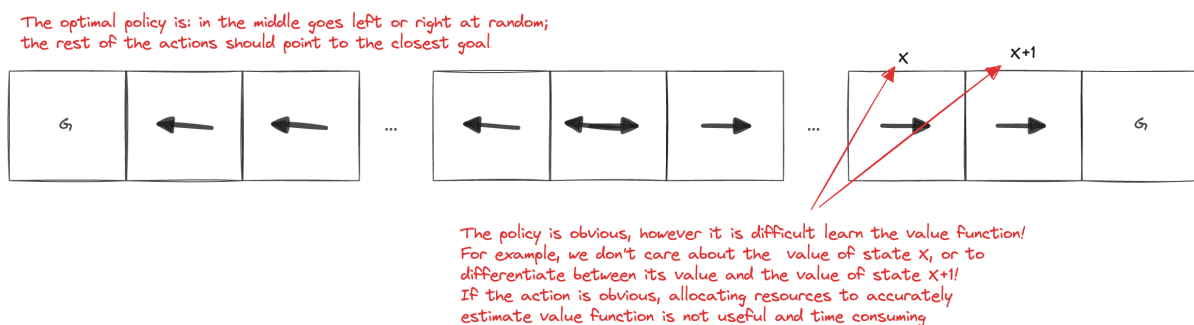
Finally, estimating the expectation with the mean over several trajectories, we obtain the **policy gradient theorem** that allow us to estimate the gradient using trajectory samples:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left\{ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) G(\tau^{(i)}) \right\}$$

The resulting formulation provides a practical means to estimate the policy gradient without access to the transition model. In practice, we collect several complete trajectories by following the current policy, compute for each the total discounted return, and then use this return to weight the log-probabilities of all the actions taken along that trajectory when updating the policy parameters.

10.1.2 Causality

Considering the policy gradient theorem formulation, we use the total return of the trajectory to weight the log-probabilities of all the actions taken along that trajectory. However, it is better to use only the **causal reward** that come after each action, because we know we can't go back in time and current actions cannot be responsible for past reward:



For each step in the trajectory, we calculate the return just from that step and use this value as the score to weight the log probability of the action taken at that time step:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left\{ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) G_t(\tau^{(i)}) \right\}$$

10.2 REINFORCE algorithm

The REINFORCE algorithm (R.J. Williams, “**Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning**”, Machine Learning (1992)) was one of the first attempts to implement the idea of estimating the gradient using the full return and using a neural network to represent the policy. As in the previous chapter, we use the CartPole environment as an example:

```
import gymnasium

env = gymnasium.make('CartPole-v1', render_mode="rgb_array");

state_size = env.observation_space.shape;
action_size = env.action_space.n;

print('State Space: ', state_size);
print('Action Space: ', action_size);
```

State Space: (4,)

Action Space: 2

First of all, we implement the parametric policy as a neural network, that takes the state as input and outputs a probability distribution over the actions:

```
import torch

def create_network(first_hidden_layer, second_hidden_layer):

    # Define a deep neural network using Sequential:
    # Each layer feeds directly into the next one.
    dnn = torch.nn.Sequential(
        # First fully connected layer maps state inputs to 512 hidden units
        torch.nn.Linear(state_size[0], first_hidden_layer),

        # ReLU activation introduces nonlinearity
        torch.nn.ReLU(),

        # Second fully connected layer (hidden layer with 128 units)
        torch.nn.Linear(first_hidden_layer, second_hidden_layer),

        # Another ReLU activation
```

```
        torch.nn.ReLU(),

        # Output layer: one unit per possible action
        # Produces raw scores (logits) for each action
        torch.nn.Linear(second_hidden_layer, action_size)
    )

    # Return the constructed model
    return dnn;
```

```
first_hidden_layer= 512;
second_hidden_layer= 128;
pi = create_network(first_hidden_layer, second_hidden_layer);
```

The network will be trained, so we need an optimizer:

```
learning_rate = 0.001;
optimizer = torch.optim.RMSprop(pi.parameters(), lr=learning_rate);
```

To optimize the policy, we define the loss function corresponding to the negative of the performance measure, since most optimization algorithms are designed to perform gradient descent. The optimizer then updates the policy parameters by taking a gradient descent step according to this objective.

```
def optimize(rewards, logpas, discounts):

    # convert rewards and discounts to tensors
    rewards = torch.as_tensor(rewards, dtype=torch.float32)
    discounts = torch.as_tensor(discounts, dtype=torch.float32)

    # number of time steps
    steps = rewards.shape[0]

    # compute causal returns:
    causal_return = torch.empty(steps, dtype=torch.float32)
    for step in range(steps):
        causal_return[step] = torch.sum(discounts[:steps - step] * rewards[step:])

    # stack log-probabilities
    logpas = torch.stack(logpas).squeeze(-1)
```

```
# compute loss
loss = -(causal_return * logpas).mean()

# optimize the model
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

In policy-based reinforcement learning, there is **no need to implement explicit exploration strategies**. The policy neural network itself produces a probability distribution over actions, which inherently defines both exploitation and exploration. We can write a function to compute the action preferences produced by the policy network and, depending on a greedy flag, either samples an action from the resulting probability distribution or selects the one with the highest likelihood. In both cases, we also return the log-probability of the chosen action, which is required for the update. This design unifies exploration and exploitation within a single function: sampling enables the agent to explore during training, while greedy selection allows for deterministic evaluation of the learned policy:

```
def select_action(state, greedy=False):

    # convert state to tensor
    state = torch.as_tensor(state, dtype=torch.float32)

    # compute action preferences
    prefs = pi(state)

    # create a categorical distribution over actions
    dist = torch.distributions.Categorical(logits=prefs)

    # select the most probable action (greedy)
    if greedy:
        action = torch.argmax(prefs, dim=-1)
    # or sample an action (stochastic policy)
    else:
        action = dist.sample()

    # log-probability (used in REINFORCE loss)
    logpa = dist.log_prob(action)

    # return values
```

```
return int(action.item()), logpa
```

We need a function to evaluate the performance of the agent, in this case we run several episodes selecting actions greedily and compute the average return

```
import numpy as np

def evaluate(epochs=5):

    # collect total rewards per episode
    rewards = []

    # loop over episodes
    for episode in range(epochs):

        # reset the environment
        state, _ = env.reset();
        done = False;
        total_reward = 0.0;

        # run an episode
        while not done:
            action, _ = select_action(state, greedy=True);
            state, reward, terminated, truncated, _ = env.step(action)
            total_reward += reward;
            done = terminated or truncated;

        # store the total reward
        rewards.append(total_reward);

    # return the average reward over the episodes
    return float(np.mean(rewards))
```

Finally, we can write the complete algorithm, where we collect a full trajectory and then perform an optimization step:

```
def reinforce(gamma, max_episodes):

    # calculate the discounts to be used in the episodes
```

```
max_steps = 1000;
discounts = gamma ** np.arange(max_steps)

# create a score tracker for statistic purposes
scores = [];

# train until for the maximum number of episodes
for episode in range(max_episodes):

    # reset the environment at the beginning of the episode
    state, _ = env.reset()
    done = False;

    # log the rewards and the log of action probabilities
    logpas = []
    rewards = []

    # interact with the environment until the episode is done
    while not done:

        # select the action using the policy
        action, logpa = select_action(state);

        # collect an experience tuple as usual
        next_state, reward, terminal, truncated, _ = env.step(action);
        done = terminal or truncated

        # save the log probability and the reward
        logpas.append(logpa)
        rewards.append(reward)

        # update current state to next state
        state = next_state;

    # At the end of the episode, run one optimization step
    optimize(rewards, logpas, discounts);

    # Evaluate the policy
    score = evaluate(episodes=10);

    # store the score in the tracker
    scores.append(score);
```

```
# print some informative logging
message = 'Episode {:03}, score {:.05.1f}';
message = message.format(episode+1, score);
print(message, end='\r', flush=True);

return scores
```

We can write an experiment to evaluate the REINFORCE algorithm in the cart-pole environment so that we can easily compare it and the value-based methods:

```
import joblib
import random

def experiment(gamma,
              learning_rate,
              first_hidden_layer,
              second_hidden_layer,
              max_episodes):

    global pi, optimizer;

    # List of random seeds to test algorithm stability
    seeds = (12, 34, 56, 78, 90);

    # Container to collect all experiment results
    results = [];

    # Run an independent training experiment per seed
    for seed in seeds:

        print("Experiment seed: ", seed);

        # Set all relevant random seeds for reproducibility
        torch.manual_seed(seed);
        np.random.seed(seed);
        random.seed(seed);

        # reset the environment
        env.reset(seed=seed);
        env.action_space.seed(seed)
        env.observation_space.seed(seed)
```

```
# create a new model
pi = create_network(first_hidden_layer, second_hidden_layer);
optimizer = torch.optim.RMSprop(pi.parameters(), lr=learning_rate);

# train the network
scores = reinforce(gamma, max_episodes);

# smooth the result using a sliding window
sliding_windows = 25;
scores = np.convolve(scores, np.ones(sliding_windows)/sliding_windows,
↪ mode='valid');

# collect the results
results.append(scores);

print("");

max_score = np.max(results, axis=0).T;
min_score = np.min(results, axis=0).T;
mean_score = np.mean(results, axis=0).T;

experiment_results = {
    'max_score': max_score,
    'min_score': min_score,
    'mean_score': mean_score
}

# Save results
joblib.dump(experiment_results, '../models/reinforce_results.joblib');

return experiment_results;
```

We now define a specific set of hyperparameters controlling both the training process and the neural network architecture, and we execute a single training experiment to observe how the agent's performance evolves over time under these conditions:

```
# Define the experiment setup and hyperparameters
gamma = 0.99;           # discount factor
learning_rate = 0.001;  # step size for the optimizer
first_hidden_layer = 64; # size of the first hidden layer
second_hidden_layer = 32; # size of the second hidden layer
```

```
# Number of training episodes per experiment
max_episodes = 1500;

# run the experiment
reinforce_results = experiment(gamma, learning_rate, first_hidden_layer,
    ↪ second_hidden_layer, max_episodes);
```

```
Experiment seed: 12
Episode 1500, score 125.3
Experiment seed: 34
Episode 1500, score 156.3
Experiment seed: 56
Episode 1500, score 136.7
Experiment seed: 78
Episode 1500, score 439.9
Experiment seed: 90
Episode 1500, score 500.0
```

We can load the results of NFQ in order to compare the performances of the two algorithms:

```
nfq_results = joblib.load('../models/nfq_results.joblib');
```

As for value-based algorithms, we can plot the performance of the algorithm over episodes:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6));
plt.title('REINFORCE vs NFQ');
plt.ylabel('Score');
plt.xlabel('Episodes');

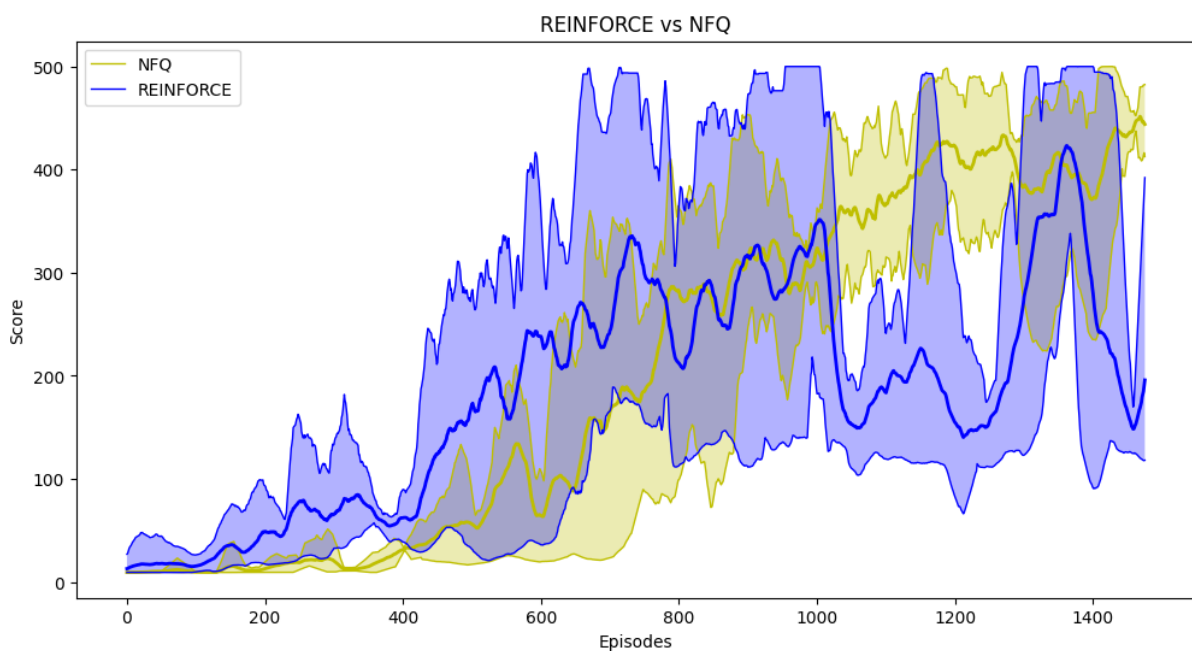
nfq_episodes = range(len(nfq_results['max_score']));
reinforce_episodes = range(len(reinforce_results['max_score']));

plt.plot(nfq_results['max_score'], 'y', linewidth=1, label="NFQ");
plt.plot(nfq_results['min_score'], 'y', linewidth=1);
plt.plot(nfq_results['mean_score'], 'y', linewidth=2);
plt.fill_between(nfq_episodes, nfq_results['min_score'], nfq_results['max_score'],
    ↪ facecolor='y', alpha=0.3);
```



```
plt.plot(reinforce_results['max_score'], 'b', linewidth=1, label="REINFORCE");
plt.plot(reinforce_results['min_score'], 'b', linewidth=1);
plt.plot(reinforce_results['mean_score'], 'b', linewidth=2);
plt.fill_between(reinforce_episodes, reinforce_results['min_score'],
    ↪ reinforce_results['max_score'], facecolor='b', alpha=0.3);

plt.legend()
plt.show();
```



The REINFORCE algorithm performs **well on simple problems** and, in theory, it converges to a local optimum under suitable conditions. However, because it relies entirely on Monte Carlo estimates of the return to compute its gradient, the resulting updates **suffer from high variance**. Every random factor (the initial state, the stochastic transitions of the environment, and the randomness of the policy itself) accumulates along the trajectory. As a result, the return becomes a very noisy training signal, making learning unstable and inefficient. In practice, this means that REINFORCE, while elegant and conceptually important, is too sample-inefficient to be used on complex tasks. Nevertheless, it remains a valuable reference algorithm: it is simple, lightweight, and easy to implement.

10.3 Vanilla Policy Gradient (VPG)

The core idea behind policy gradient methods is simple: **increase the probability of actions that lead to higher returns** and **decrease the probability of those that lead to lower returns**, gradually improving the policy toward the optimal one. However, since we estimate returns using noisy Monte Carlo samples, the resulting gradient estimates often have **high variance**, which can slow down learning. For example, in an environment like Cart-Pole, where all rewards are positive, it's difficult to tell which actions are merely good and which are truly the best, requiring a large amount of data to make that distinction reliably.

It would be helpful if we had a way to **distinguish between actions taken in the same state**. To achieve this, we can use the **advantage function**, which measures how much better (or worse) an action is compared to the average action in that state.

$$A_{\pi}(s,a) = q_{\pi}(s,a) - v_{\pi}(s)$$

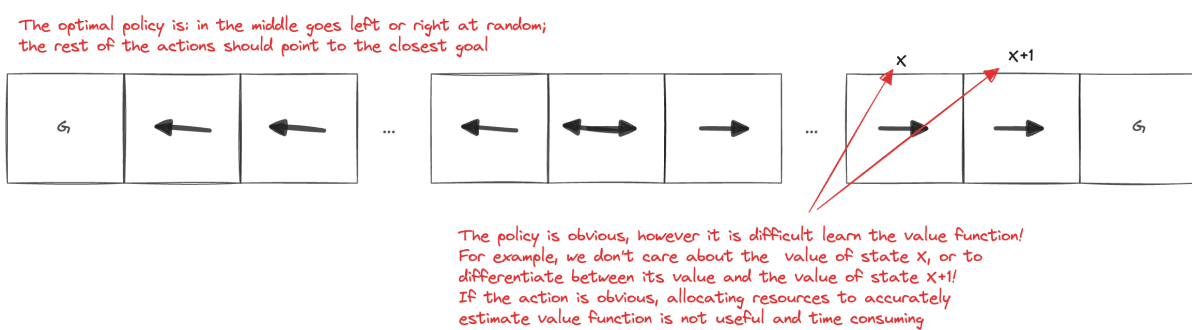
In practice, we can express the advantage at time step t approximately as

$$A(S_t, A_t) = R_t + \gamma R_{t+1} + \dots + \gamma^{T-t} R_T - v_{\pi}(S_t)$$

An estimate of it is the return minus the estimated expected return from that state:

$$A(S_t, A_t) = G_t - V(S_t)$$

Using the advantage somewhat **centers scores around zero**: better-than-average actions have a positive score (increasing probabilities), worse-than-average have a negative score (decreasing probabilities). However, we need **two neural networks**, one for learning the policy and one for learning the state-value function in order to estimate the advantage:



The performance measure for the value function is simple, the mean squared Monte Carlo error:

$$J_v(\phi) = -\frac{1}{m} \sum_{i=0}^m \left[\left(G_t(\tau^{(i)}) - V(s_t^{(i)}) \right)^2 \right]$$

For the policy network, we can update the performance measure and its gradient using the advantage instead of the return:

$$J_{\pi}(\theta) = E_{\tau \sim \pi_{\theta}} [G(\tau)] = \sum_{\tau} p(\tau | \pi_{\theta}) (G_t(\tau) - V(s_t))$$

$$\nabla_{\theta} J_{\pi}(\theta) = \frac{1}{m} \sum_{i=1}^m \left\{ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_t(\tau^{(i)}) - V(s_t^{(i)})) \right\}$$

It is essential to add another improvement. We want our policy to exploit what it has already learned, but also to explore other possibilities (to avoid premature convergence to suboptimal actions). However, during training, the policy's probability distribution often collapses too early onto a single action (especially when early rewards are noisy or misleading). This makes the agent **overconfident** and prevents it from exploring alternative, possibly better, behaviors. We can use a measures of uncertainty or randomness of the distribution, like the entropy:

is to add an **entropy term** to to the loss function. We can interpret entropy in many different ways, from the amount of information we can gain by sampling from a distribution, to the number of ways we can order a set. In this context, we can think of entropy in a straightforward way. A uniform distribution, which has evenly distributed samples, has high entropy. If we have two samples, both with a 50% chance, then the entropy is the highest it can be for a two-sample set, conversely, if we have two samples, one with 100% chance, then the entropy is the lowest. The way to use entropy in policy-gradient methods is to add the weighted entropy to the performance function **to encourage having evenly distributed actions**. That way, a policy (with evenly distributed actions) yields the highest entropy and contributes to maximize the performance. On the other hand, converging on a single action (which means entropy is zero) doesn't increase the performance measure.

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{T-1} \nabla_{\theta} [\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_t(\tau^{(i)}) - V(s_t^{(i)}; \phi)) + \beta H(s_t^{(i)}; \theta)]$$

This algorithm is called **Vanilla policy gradient**. To implement it, we need a secondo functon to create the state-value new neural network that has a single output:

```
import torch

def create_value_network(first_hidden_layer, second_hidden_layer):

    # Define a deep neural network using Sequential:
    # Each layer feeds directly into the next one.
    dnn = torch.nn.Sequential(
        # First fully connected layer maps state inputs to 512 hidden units
        torch.nn.Linear(state_size[0], first_hidden_layer),
```

```
        # ReLU activation introduces nonlinearity
        torch.nn.ReLU(),

        # Second fully connected layer (hidden layer with 128 units)
        torch.nn.Linear(first_hidden_layer, second_hidden_layer),

        # Another ReLU activation
        torch.nn.ReLU(),

        # Output layer: xxx
        torch.nn.Linear(second_hidden_layer, 1)
    )

    # Return the constructed model
    return dnn;
```

```
first_hidden_layer= 512;
second_hidden_layer= 128;
v = create_value_network(first_hidden_layer, second_hidden_layer);
```

We need also to the select_action method of the first network (the policy network) in order to calculate also the entropy term:

```
def select_action(state, greedy=False):

    # convert state to tensor
    state = torch.as_tensor(state, dtype=torch.float32)

    # compute action preferences
    prefs = pi(state)

    # create a categorical distribution over actions
    dist = torch.distributions.Categorical(logits=prefs)

    # select the most probable action (greedy)
    if greedy:
        action = torch.argmax(prefs, dim=-1)
    # or sample an action (stochastic policy)
    else:
        action = dist.sample()
```

```
# log-probability (used in REINFORCE loss)
logpa = dist.log_prob(action)

# calculate the entropy
entropy = dist.entropy()

# return values
return int(action.item()), logpa, entropy
```

The optimize method is different from the previous one, because we have two neural networks to optimize:

```
def optimize(rewards, logpas, entropies, discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for all steps
    returns = np.array([np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
    returns = torch.FloatTensor(returns)

    # prepare the log probabilities, the entropies and the values
    logpas = torch.cat(logpas)
    entropies = torch.cat(entropies)
    values = torch.cat(values)

    # estimate the advantages
    advantage = returns - values

    # calculate the policy loss
    beta = 0.001
    policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

    # calculate the value loss
    value_loss = advantage.pow(2).sum()

    # optimize the policy network
    policy_optimizer.zero_grad()
    policy_loss.backward()
```

```
policy_optimizer.step()

# optimize the value network
value_optimizer.zero_grad()
value_loss.backward()
value_optimizer.step()
```

Finally, we can write the complete algorithm, similar to REINFORCE, but with the addition of the value network and the entropy term:

```
def vpg(env_name, seed, parameters):

    # unpack the parameters
    gamma = parameters['gamma']
    learning_rate = parameters['learning_rate']
    max_time = parameters['max_time']
    max_episodes = parameters['max_episodes']
    max_score = parameters['max_score']
    episodes_window = parameters['episodes_window']

    # create the environment
    env = gymnasium.make(env_name)

    # access the state and action sizes of the environment
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    # calculate the discounts to be used in the episodes
    max_steps = 1000
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma)

    # set the seeds for reproducibility of results
    torch.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)
    env.np_random = np.random.Generator(np.random.PCG64(seed))

    # create the policy model and its optimizer
    policy_model = PolicyApproximator(state_size, action_size)
    policy_optimizer = torch.optim.RMSprop(policy_model.parameters(),
    ↪ lr=learning_rate)
```

```
# create the value model and its optimizer
value_model = ValueApproximator(state_size)
value_optimizer = torch.optim.RMSprop(value_model.parameters(), lr=learning_rate)

# episode stats trackers
episode_steps = []
episode_return = []
episode_seconds = []
episode_scores = []

# total stats trackers
total_step = 0
training_time = 0
results = np.empty((max_episodes+1, 4))
results[:] = np.nan

# train until we reach a terminal condition
for episode in count():

    # get the start time of the episode
    episode_start = time.time()

    # set return to 0
    discounted_return = 0

    # reset the environment before starting the episode
    state = env.reset()[0]

    # log the rewards, the log of action probabilities, entropies and values
    logpas = []
    rewards = []
    entropies = []
    values = []

    # interact with the environment until the episode is done
    for step in count():

        # select the action using the policy
        action, logpa, entropy = policy_model.select_action(state)

        # collect an experience tuple as usual
        next_state, reward, is_terminal, is_truncated, info = env.step(action)
        is_done = is_terminal or is_truncated
```

```
is_failure = is_terminal and not is_truncated

# calculate the value using the value network
value = value_model(state)

# save the log probability, entropy, reward and value
logpas.append(logpa)
entropies.append(entropy)
rewards.append(reward)
values.append(value)

# update current state to next state
state = next_state

# update the return
discounted_return += reward

if is_done:
    gc.collect()
    break

# then, run one optimization step
optimize(policy_model, value_model, rewards, logpas, values, entropies,
↪ policy_optimizer, value_optimizer, discounts)

# evaluate the learned policy after the episode
score = evaluate(policy_model, env)

# timing stuff
episode_elapsed = time.time() - episode_start

# update episode stats
episode_steps.append(step)
episode_return.append(discounted_return)
episode_seconds.append(episode_elapsed)
episode_scores.append(score)

# update total stats
total_step += step
training_time += episode_elapsed

# average stats over some recent episodes
mean_return = np.mean(episode_return[-episodes_window:])
```



```

mean_score = np.mean(episode_scores[-episodes_window:])

# report results
results[episode] = total_step, mean_return, mean_score, training_time

# informative print logging
message = 'episode {:04}, steps {:06}, average score {:05.1f}'
message = message.format(episode, total_step, mean_score)
print(message, end='\r', flush=True)

# check max episodes achievement criteria
if (episode ≥ max_episodes):
    print(message, ' --> reached_max_episodes ')
    break

# check max time achievement criteria
if (training_time ≥ max_time):
    print(message, ' --> reached_max_time ')
    break

# check max score achievement criteria
if (mean_score ≥ max_score):
    print(message, ' --> reached_max_score ')
    break

return policy_model, score, results

```

We can try the algorithm in the cartpole environment:

```

vpg_results, vpg_agent, vpg_score = run_experiment(name = 'vpg_results', algorithm =
↳ vpg)

```

TypeError

Traceback (most recent call last)

Cell In[25], line 1

----> 1 vpg_results, vpg_agent, vpg_score = run_experiment(name = 'vpg_results', algorithm = v

Cell In[13], line 14, in run_experiment(name, env_name, algorithm, seeds, parameters)

10 best_eval_score = float('-inf')

12 for seed in seeds:

```

13     # create the agent
--> 14     agent, score, result = algorithm(env_name, seed, parameters)
16     # save the results the best agent
17     results.append(result)

```

Cell In[24], line 4, in vpg(env_name, seed, parameters)

```

1 def vpg(env_name, seed, parameters):
2
3     # unpack the parameters
----> 4     gamma = parameters['gamma']
5     learning_rate = parameters['learning_rate']
6     max_time = parameters['max_time']

```

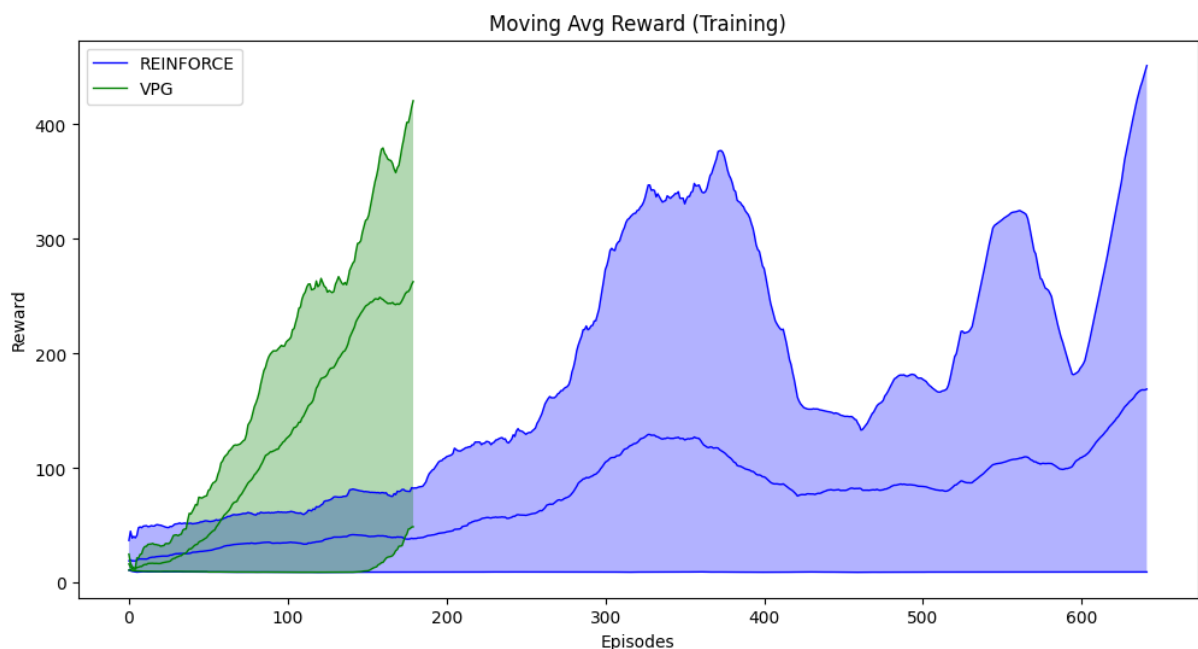
TypeError: 'NoneType' object is not subscriptable

We can compare VPG with REINFORCE, Notice how VPG is more sample efficient, instead REINFORCE is too inefficient to be a practical algorithm.

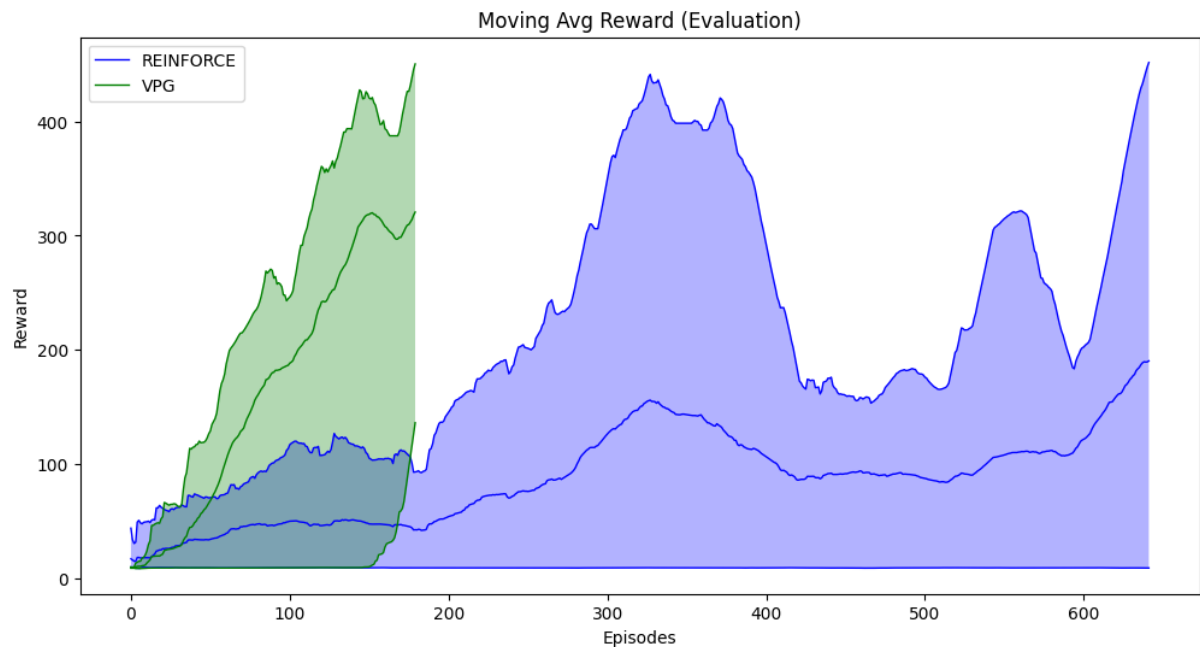
```

show_result('avg_training_reward', 'REINFORCE', reinforce_results, 'VPG', vpg_results)

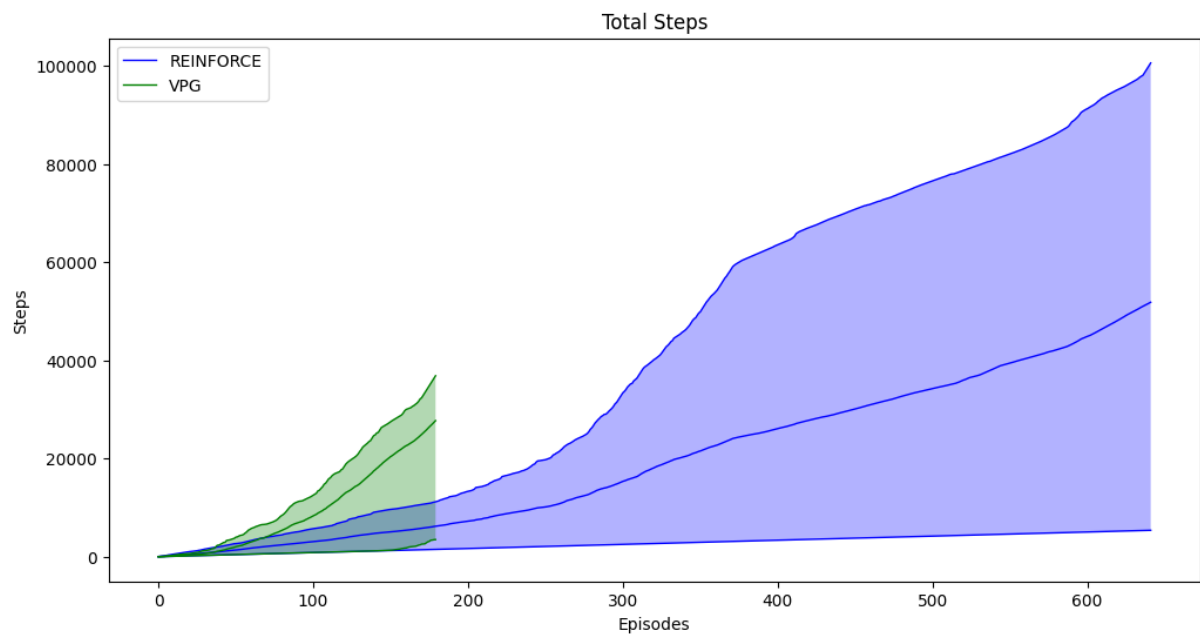
```



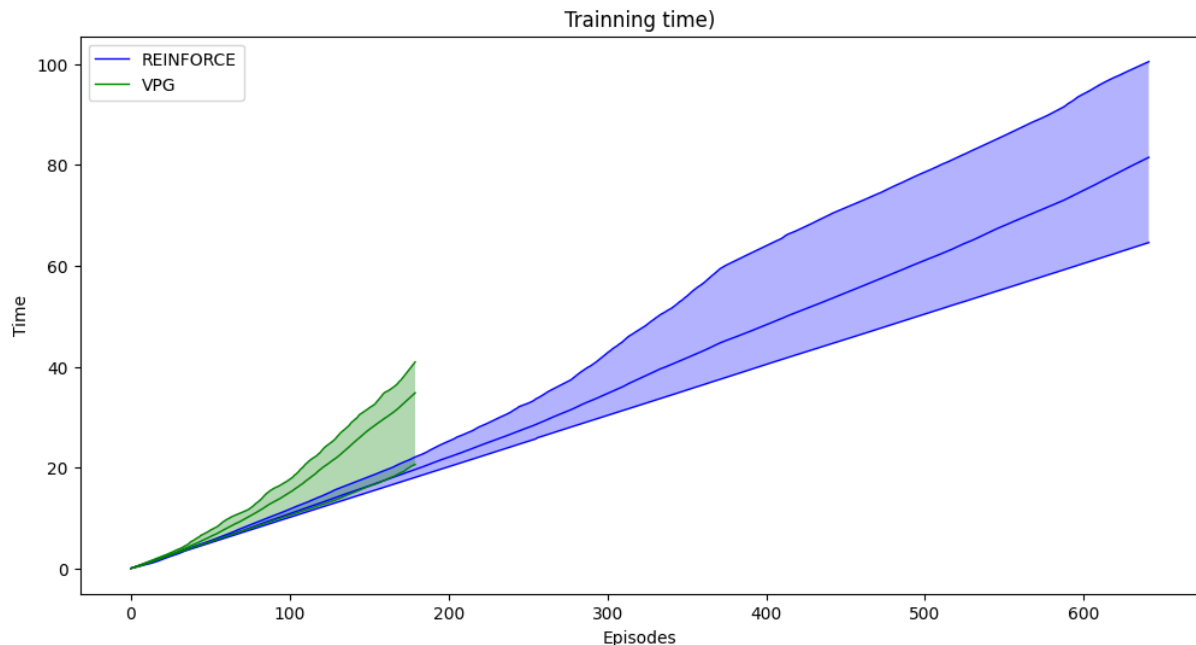
```
show_result('avg_evaluation_reward','REINFORCE', reinforce_results, 'VPG',  
           ↪ vpg_results)
```



```
show_result('total_steps','REINFORCE', reinforce_results, 'VPG', vpg_results)
```



```
show_result('training_time', 'REINFORCE', reinforce_results, 'VPG', vpg_results)
```



VPG is a pretty robust method for simple problems; it is, for the most part, unbiased because it uses an unbiased target for learning both the policy and value function. That is, it uses Monte Carlo returns, which are complete actual returns experienced directly in the environment, without any bootstrapping. The only bias in the entire algorithm is because we use function approximation, which is inherently biased.

10.4 Asynchronous Advantage Actor-Critic (A3C)

One of the main sources of variance in DRL algorithms is how **correlated and non-stationary** online samples are. In value-based methods, we use a replay buffer to uniformly sample mini-batches of (for the most part) independent and identically distributed data. Unfortunately, this is limited to off-policy methods, because on-policy agents cannot reuse data generated by previous policies. In on-policy methods (like the policy-based methods), instead of using the replay buffer, we can have **multiple workers generating experience on multiple instances of the environment** in parallel and asynchronously updating the policy and value function. This can help to de-correlate the data used for training and to reduce the variance of the algorithm.

The **Asynchronous Advantage Actor-Critic (A3C)** algorithm (Vlad Mnih et al. “**Asynchronous Methods for Deep Reinforcement Learning**”, ICML 2016) uses concurrent actors

to generate a broad set of experience samples in parallel. Moreover, it also uses **n-step returns with bootstrapping** to learn the policy and value function.

This behavior can be implemented using **multiple threads**, one for each worker, however Python threads cannot take advantage of CPU with multiple cores, due to internal implementation details. The workaround is to use **multiple processes** using the multiprocessing module. Unfortunately, the multiprocessing library has problems in running in an iterative environment like Jupyter, so we will use the threading library. We need to create a work function that will be executed by each thread. In the following example, the work function just waits for a random time before printing its rank:

```
import time

def work(rank, sleep_time):
    time.sleep(sleep_time)
    print(rank)
```

Then we create a pool of threads, execute them in parallel and finally wait for the threads to finish:

```
import random

from threading import Thread

def main():
    # number of workers
    n_workers = 4

    print('start')
    if __name__ == '__main__':

        workers = [Thread(target=work, args=(rank, random.uniform(1, 3),)) for rank in
↵ range(n_workers)]

        # start workers
        [w.start() for w in workers]

        # wait for all workers to finish
        [w.join() for w in workers]

    print('end')
```

```
main()
```

```
start
1
0
3
2
end
```

As you can see, workers are running in parallel: since they wait for a different amount of time, the order of the output is not deterministic. However, we can see that the rank of the thread is always the same, because the pool of threads is created with a fixed number of workers. Moreover, the "end" string is always printed at the end, because the main thread waits for the completion of all the workers.

Moreover, when writing concurrent programs we may need to share data or resources between threads. Since threads live in the same memory space, they have access to the same (public) data. Thus, we can for example simply use a global variable to which all threads have read and write access. Moreover, typically we need to protect with a mutual exclusion lock some shared variables in order to prevent race conditions. Python provides a mutual exclusion lock for use with processes via the Lock class. In this example, we create a shared counter and we use a lock to synchronize access to it:

```
def work(rank, iterations, lock):

    # all threads can access this shared variable
    global counter

    for _ in range(iterations):

        time.sleep(random.uniform(1, 10)/100)

        # lock the shared variable
        lock.acquire()

        counter += 1
        print(counter, " - int's me ", rank)

    # unlock the shared variable
```

```
lock.release()
```

```
from threading import Lock

# number of workers
n_workers = 4

print('start')

# number of iterations per worker
iterations = 5

# shared variable
counter = 0

# create a lock
lock = Lock()

# create workers
workers = [Thread(target=work, args=(rank, iterations, lock)) for rank in
    ↪ range(n_workers)]

# start workers
[w.start() for w in workers]

# wait for all workers to finish
[w.join() for w in workers]

print ('end')
```

start

```
1 - int's me 1
2 - int's me 2
3 - int's me 0
4 - int's me 3
5 - int's me 2
6 - int's me 0
7 - int's me 3
8 - int's me 0
```

```
9  - int's me  1
10 - int's me  2
11 - int's me  3
12 - int's me  0
13 - int's me  2
14 - int's me  1
15 - int's me  0
16 - int's me  1
17 - int's me  3
18 - int's me  3
19 - int's me  2
20 - int's me  1
end
```

We can write the work function used by each agent in the A3C algorithm. The agent interacts with the environment and update the global networks parameters:

```
# create a global object to share information between the workers
shared_info = {}
```

```
def agent_work(rank, lock):

    # access the global shared variables
    global shared_info

    # create a different seed for each worker, since we need different
    # experiences from each one of them
    local_seed = shared_info['seed'] + rank

    # create the local environment
    env = gymnasium.make(shared_info['env_name'])

    # create a uniquely seeded environment for each worker
    env.np_random = np.random.Generator(np.random.PCG64(local_seed))

    # access the state and action sizes of the environment
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    # create the local policy model
```



```
# initialize weights with the weights of the shared policy model
local_policy_model = PolicyApproximator(state_size, action_size)
local_value_model = ValueApproximator(state_size)
local_policy_model.load_state_dict(shared_info['shared_policy_model'].state_dict())

# do the same thing with the value model
local_value_model.load_state_dict(shared_info['shared_value_model'].state_dict())

# increment the global episodes counter
with lock:
    shared_info['episode'] = shared_info['episode'] + 1
    local_episode = shared_info['episode']

# start the training loop, until the worker is signaled to get out of it
# we reach a terminal condition
while not shared_info['out_signal']:

    # get the start time of the episode
    episode_start = time.time()

    # set return to 0
    discounted_return = 0

    # reset the environment before starting the episode
    state = env.reset()[0]

    # log the rewards, the log of action probabilities, entropies and values
    logpas = []
    rewards = []
    entropies = []
    values = []

    n_steps_start = 0

    # interact with the environment
    for step in count():

        # select the action using the local policy
        action, logpa, entropy = local_policy_model.select_action(state)

        # collect an experience tuple as usual
        next_state, reward, is_terminal, is_truncated, info = env.step(action)
```

```
is_done = is_terminal or is_truncated
is_failure = is_terminal and not is_truncated

# calculate the value using the value network
value = local_value_model(state)

# save the log probability, entropy, reward and value
logpas.append(logpa)
entropies.append(entropy)
rewards.append(reward)
values.append(value)

# update current state to next state
state = next_state

# update the return
discounted_return += reward

# if we collected enough steps or if the episode is done
# calculate the n-step return and optimize the model
if is_done or step - n_steps_start == shared_info['max_n_steps']:

    # if it's a failure, then the value of the next state is 0; otherwise,
    ↪ we bootstrap
    next_value = 0 if is_failure else value.detach().item()
    rewards.append(next_value)

    # optimize the model
    optimize(logpas, entropies, rewards, values,
            local_policy_model, local_value_model,
            shared_info['shared_policy_model'],
            shared_info['shared_value_model'],
            shared_info['shared_policy_optimizer'],
            shared_info['shared_value_optimizer'],
            shared_info['discounts'])

    # load the shared updated models parameters into the local models
    lo-
    ↪ cal_policy_model.load_state_dict(shared_info['shared_policy_model'].state_dict())
    ↪ local_value_model.load_state_dict(shared_info['shared_value_model'].state_dict())

    logpas = []
```

```

        entropies = []
        rewards = []
        values = []

        n_steps_start = step

    if is_done:
        break

    # evaluate the local policy after the episode
    score = evaluate(local_policy_model, env)

    # update total stats
    episode_elapsed = time.time() - episode_start
    with lock:
        shared_info['total_steps'] += step
        shared_info['training_time'] += episode_elapsed / shared_info['n_workers']

    # update global episode stats
    with lock:
        shared_info['episode_steps'].append(step)
        shared_info['episode_return'].append(discounted_return)
        shared_info['episode_seconds'].append(episode_elapsed)
        shared_info['episode_scores'].append(score)

    # average stats over some recent episodes
    episodes_window = shared_info['episodes_window']
    mean_return = np.mean(shared_info['episode_return'][-episodes_window:])
    mean_score = np.mean(shared_info['episode_scores'][-episodes_window:])

    # report results
    shared_info['results'][local_episode] = step, mean_return, mean_score,
    ↪ episode_elapsed / shared_info['n_workers']

    # informative print logging
    message = 'worker {:02} - episode {:04}, steps {:06}, average score {:05.1f}'
    message = message.format(rank, local_episode, shared_info['total_steps'],
    ↪ mean_score)
    with lock:
        print(message, end='\r', flush=True)

    # check if we should stop
    if shared_info['out_signal'] == 1:

```

```

        break

    # check max episodes achievement criteria
    if (local_episode > shared_info['max_episodes']):
        print(message, ' --> reached_max_episodes ')
        shared_info['out_signal'] = 1
        break

    # check max time achievement criteria
    if (shared_info['training_time'] > shared_info['max_time']):
        print(message, ' --> reached_max_time ')
        shared_info['out_signal'] = 1
        break

    # check max score achievement criteria
    if (mean_score > shared_info['max_score']):
        print(message, ' --> reached_max_score ')
        shared_info['out_signal'] = 1
        break

    # else go work on another episode by
    # getting a new episode id from the
    # global protected counter
    with lock:
        shared_info['episode'] = shared_info['episode'] + 1
        local_episode = shared_info['episode']

env.close()
del env

```

Workers append the value of the next state, whether terminal or not, to the reward sequence before calling the optimize function (in VPG we were using the full returns for the advantage estimate). In that way, the reward variable contains all rewards from the partial trajectory and the state-value estimate of that last state. We can see this as having the partial return (the sequence of rewards) and the **predicted remaining return** in the same place. You can realize that this is an **n-step return**: the agent go out for n-steps collecting rewards, and then **bootstrap** after that n-th state (or before if he lands on a terminal state that comes first):

$$A(S_t, A_t) = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}) - V(S_t) = G_{t:t+n} - V(S_{t+n+1})$$

Notice the bootstrapping, this is what makes this algorithm an **actor-critic method**. Now, We

can use that n-step return to improve the value function estimate:

$$\displaystyle L_v(\phi) = \frac{1}{m} \sum \lim_{i \rightarrow \infty} \left[\left(G_{t:t+n}^{(i)} - V(s_{t+n+1}^{(i)}; \phi) \right)^2 \right]$$

We also use this n-step advantage estimate for updating the policy networks:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^n \nabla_{\theta} [\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_{t:t+n}^{(i)} - V(s_{t+n+1}^{(i)}; \phi) + \beta H(s_{t+n+1}^{(i)}; \theta))]$$

We can write the optimize model function, which similar to the one of VPG however this time we are optimizing the shared models:

```
def optimize(logpas, entropies, rewards, values,
            local_policy_model, local_value_model,
            shared_policy_model, shared_value_model,
            shared_policy_optimizer, shared_value_optimizer,
            discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for the n-step steps
    # remember: rewards includes the bootstrapping value
    returns = np.array([np.sum(discounts[T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
    returns = torch.FloatTensor(returns[:-1])

    # prepare the log probabilities, the entropies and the values
    logpas = torch.cat(logpas)
    entropies = torch.cat(entropies)
    values = torch.cat(values)

    # estimate the advantages
    advantage = returns - values

    # calculate the policy loss
    beta = 0.001
    policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

    # calculate the value loss
    value_loss = advantage.pow(2).sum()

    # optimize the policy network
```

```
# notice, we now zero the shared policy optimizer
shared_policy_optimizer.zero_grad()
policy_loss.backward()

# iterating over all local and shared policy network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_policy_model.parameters(),
    ↪ shared_policy_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_policy_optimizer.step()

# optimize the value network
shared_value_optimizer.zero_grad()
value_loss.backward()

# iterating over all local and shared value network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_value_model.parameters(),
    ↪ shared_value_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_value_optimizer.step()
```

One of the most critical aspects of A3C is that **its network updates are asynchronous and lock-free**. As we already discussed, having a shared model creates a need for a blocking mechanism to prevent workers from overwriting other updates, however this would be inefficient. Instead, A3C uses an update style called a **Hogwild!**, in which workers access to shared model with the possibility of overwriting each other's work, which is shown to achieve a near-optimal rate of convergence and outperforming alternative schemes that use locking by an order of magnitude: F. Niu et al. "**HOGWILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent**", NIPS 2011.

We can write the A3C algorithm, which is similar to VPG, but with the addition of the n-step returns and the multiple workers:

```
def a3c(env_name, seed, parameters):

    # unpack the parameters
    gamma = parameters['gamma']
    learning_rate = parameters['learning_rate']
    max_time = parameters['max_time']
    max_episodes = parameters['max_episodes']
    max_score = parameters['max_score']
    episodes_window = parameters['episodes_window']
    n_workers = parameters['n_workers']
    max_n_steps = parameters['max_n_steps']

    # access the global object
    global shared_info

    # define the seed for workers
    shared_info['seed'] = seed

    # set the seeds for reproducibility of results
    torch.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)

    # define the number of workers
    shared_info['n_workers'] = n_workers

    # define the number of steps to collect before optimizing (n-step)
    shared_info['max_n_steps'] = max_n_steps

    # provide gamma and learning rate to workers
    shared_info['gamma'] = gamma
    shared_info['learning_rate'] = learning_rate

    # provide environment information to workers
    shared_info['env_name'] = 'CartPole-v1'

    # provide stopping thresholds to workers
    shared_info['max_time'] = max_time
    shared_info['max_episodes'] = max_episodes
    shared_info['max_score'] = max_score
    shared_info['episodes_window'] = episodes_window

    # create an environment in order to get information
```

```
# to create the shared models and to perform final evaluation
env = gymnasium.make(env_name)

# access the state and action sizes of the environment
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# episode stats trackers
shared_info['episode_steps'] = []
shared_info['episode_return'] = []
shared_info['episode_seconds'] = []
shared_info['episode_scores'] = []

# total stats trackers
shared_info['total_steps'] = 0
shared_info['results'] = np.empty((max_episodes+1+shared_info['n_workers'], 4))

# calculate the discounts to be used in the episodes
max_steps = 1000
shared_info['discounts'] = np.logspace(start=0, stop=max_steps, num=max_steps,
↪ base=gamma)

# create the policy model and its optimizer
shared_info['shared_policy_model'] = PolicyApproximator(state_size, action_size)
shared_info['shared_policy_optimizer'] =
↪ torch.optim.RMSprop(shared_info['shared_policy_model'].parameters(),
↪ lr=learning_rate)

# create the value model and its optimizer
shared_info['shared_value_model'] = ValueApproximator(state_size)
shared_info['shared_value_optimizer'] =
↪ torch.optim.RMSprop(shared_info['shared_value_model'].parameters(),
↪ lr=learning_rate)

# initialize shared episode counter
shared_info['episode'] = 0

# initialize the training time
shared_info['training_time'] = 0

# create a lock in order to protect the concurrent access to variables
lock = Lock()
```



```

# create a shared quit signal to inform the workers
# when a stop condition is reached (max episodes, max time or max score)
shared_info['out_signal'] = 0

# create workers
workers = [Thread(target=agent_work, args=(rank, lock,)) for rank in range(
↪ shared_info['n_workers'])]

# start workers
[w.start() for w in workers]

# wait for all workers to finish
[w.join() for w in workers]

# final evaluation
final_eval_score = evaluate(shared_info['shared_policy_model'], env, episodes=10)
env.close()
del env

return shared_info['shared_policy_model'], final_eval_score,
↪ shared_info['results']

```

We can try the algorithm in the cart-pole environment:

```

parameters['n_workers'] = 8
parameters['max_n_steps'] = 50

a3c_results, a3c_agent, a3c_score = run_experiment(name = 'a3c_results', algorithm =
↪ a3c)

```

worker 04 - episode 0019, steps 000206, average score 008.9

```

worker 01 - episode 3001, steps 025120, average score 009.2 --> reached_max_episodes
worker 04 - episode 3000, steps 031575, average score 009.4 --> reached_max_episodes
worker 01 - episode 3001, steps 026713, average score 009.3 --> reached_max_episodes
worker 07 - episode 3000, steps 025187, average score 009.2 --> reached_max_episodes
worker 01 - episode 1193, steps 010065, average score 009.3

```

```
show_result('avg_training_reward','VPG', vpg_results, 'A3C', a3c_results)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[197], line 1
----> 1 show_result('avg_training_reward','VPG', vpg_results, 'A3C', a3c_results)

NameError: name 'vpg_results' is not defined
```

```
show_result('avg_evaluation_reward','VPG', vpg_results, 'A3C', a3c_results)
```

One of the main things to notice is how VPG is more sample efficient than the more complex A3C methods. This is mostly because A3C uses multiple workers, which initially cost lots of data to get only a bit of progress.

```
show_result('total_steps','VPG', vpg_results, 'A3C', a3c_results)
```

```
show_result('training_time','VPG', vpg_results, 'A3C', a3c_results)
```

In terms of time, the parallel method should be fast, however this cannot be seen in the graph because the algorithm is not implemented in a real parallel way on multiple cores, due to the limitation of the threading library.

10.5 Generalized advantage estimation (GAE)

We can combine multiple n-step bootstrapping targets into a single target, creating a more robust targets than a single n-step: the lambda-target. The **Generalized advantage estimation (GAE)** John Schulman et al. “**High-dimensional Continuous Control Using Generalized Advantage Estimation**”, ICLR 2016 uses an exponentially weighted combination of n-step action-advantage function targets in order to reduce the variance of policy-gradient estimates at the cost of some bias:

$$A^1(S_t, A_t) = R_t + \gamma V(S_{t+1}) - V(S_t)$$

$$A^2(S_t, A_t) = R_t + \gamma R_{t+1} + \gamma^2 V(S_{t+2}) - V(S_t)$$

$$A^3(S_t, A_t) = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 V(S_{t+3}) - V(S_t)$$

...

$$A^n(S_t, A_t) = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}) - V(S_t)$$

which we can mix to make an estimate, analogous to TD-lambda, but for advantages:

$$A^{\{\text{GAE}(\gamma, \lambda)\}}(S_t, A_t) = \sum_{l=0}^{\infty} \{(\gamma \lambda)^l \delta_{t+l}\}$$

Values of $\lambda=0$ returns the one-step advantage estimate and $\lambda=1$ returns the infinite-step advantage estimate.

We can implement the GAE optimize logic model in Python:

```
def optimize(logpas, entropies, rewards, values,
            local_policy_model, local_value_model,
            shared_policy_model, shared_value_model,
            shared_policy_optimizer, shared_value_optimizer,
            discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for the n-step steps
    # remember: rewards includes the bootstrapping value
    returns = np.array([np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
    returns = torch.FloatTensor(returns[:-1])

    # prepare the log probabilities, the entropies and the values
    logpas = torch.cat(logpas)
    entropies = torch.cat(entropies)
    values = torch.cat(values)

    # estimate the advantages
    advantage = returns - values

    # calculate the policy loss
    beta = 0.001
    policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

    # calculate the value loss
    value_loss = advantage.pow(2).sum()
```

```
# optimize the policy network
# notice, we now zero the shared policy optimizer
shared_policy_optimizer.zero_grad()
policy_loss.backward()

# iterating over all local and shared policy network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_policy_model.parameters(),
    ↪ shared_policy_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_policy_optimizer.step()

# optimize the value network
shared_value_optimizer.zero_grad()
value_loss.backward()

# iterating over all local and shared value network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_value_model.parameters(),
    ↪ shared_value_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_value_optimizer.step()
```

We can try the algorithm in the cart-pole environment:

```
parameters['n_workers'] = 8
parameters['max_n_steps'] = 50

gae_results, gae_agent, gae_score = run_experiment(name = 'gae_results', algorithm =
    ↪ a3c)
```

```
show_result('avg_training_reward','VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('avg_evaluation_reward','VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('total_steps','VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('training_time','VPG', vpg_results, 'GAE', gae_results)
```

10.6 Advantage Actor-Critic (A2C)

Advantage actor-critic (A2C) is the **synchronous version of A3C**. Updating the neural network in a Hogwild!-style can be chaotic, yet introducing a lock mechanism lowers A3C performance considerably. In A2C, we move the workers from the agent down to the environment. Instead of having multiple actor-learners, we have multiple actors with a single learner.

We can also use a single neural network for both the policy and the value function. Sharing a model can be particularly beneficial when learning from images, because feature extraction can be compute-intensive. However, model sharing can be challenging due to the potentially different scales of the policy and value function updates.

```
import torch

class PolicyApproximator(torch.nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyApproximator, self).__init__()

        # activation function (ReLU)
        self.activation = torch.nn.functional.relu

        # input layer
```

```
self.input_layer = torch.nn.Linear(state_size, 512)

# hidden layer
self.hidden_layer = torch.nn.Linear(512, 128)

# output layer
self.output_layer = torch.nn.Linear(128, action_size)

def forward(self, state):

    # convert the state into a tensor
    if not isinstance(state, torch.Tensor):
        state = torch.Tensor(state)

    # pass it through the input layer
    x = self.activation(self.input_layer(state))

    # then pass for the hidden layer
    x = self.activation(self.hidden_layer(x))

    # finally, pass for the output layer
    preferences = self.output_layer(x)

    return preferences

def select_action(self, state):

    # get the preferences over actions
    preferences = self.forward(state)

    # sample the action from the probability distribution
    dist = torch.distributions.Categorical(logits=preferences)
    action = dist.sample()

    # calculate the log probability of the action
    # needed for the loss
    logpa = dist.log_prob(action).unsqueeze(-1)

    return action.item(), logpa

def select_greedy_action(self, state):

    # select the action with the highest preference
```

```

preferences = self.forward(state)
action = np.argmax(preferences.detach())

return action.item()

```

11 Policy Based Methods

Until now, almost all the methods considered are **value-based methods**, they learn the values of actions and then select actions based on the estimated values. Their policies would not even exist without the value function estimates. However, we can consider a different approach. **Policy-based methods** learn directly a parameterized policy (Sutton et al., “**Policy Gradient Methods for Reinforcement Learning with Function Approximation**”, NIPS, 2000) to select actions **without consulting a value function**:

$$\pi_{\theta}(a|s, \theta)$$

In that case, the objective is to maximize the performance of the parameterized policy based on the gradient of some **performance measure J** with respect to the policy parameter:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

The policy can be parameterized in many different ways, with the only essential requirement being that it is **differentiable with respect to its parameters**, so that gradient-based optimization can be applied. In practice, to **encourage sufficient exploration**, it is important that the policy does **not collapse into a fully deterministic choice** of actions. A common solution is to parameterize the policy as a **softmax distribution** over action preferences, which assigns a probability to each action while still favoring those with higher estimated preference

$$\pi_{\theta}(a|s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$$

where **h** is the **preference function** for actions in states, which can be parameterized arbitrarily. For example, it might be computed by a neural network, where the parameter is the vector of all the connection weights of the network, or it could simply be a linear function.

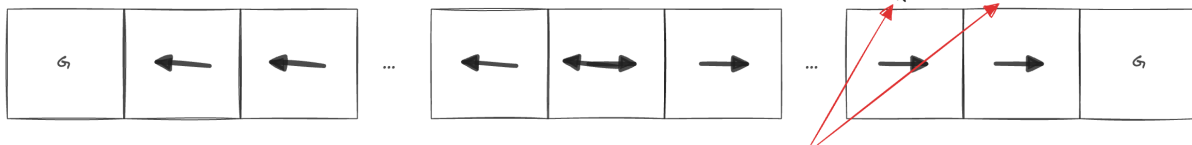
It is rather natural to define the performance measure as the **expected discounted return from the initial state**:

$$J(\theta) = E_{s_0 \sim p_0} [G_{\pi_{\theta}}]$$

where p_0 is the distribution of the initial state and $G_{\pi_{\theta}}$ is the discounted return from the initial state under policy π_{θ} .

We can now **learn stochastic policies directly**, which often achieve better performance in partially observable environments. The key intuition is that by learning arbitrary action probabilities, the agent is not forced to rely strictly on the Markov assumption. In other words, when the agent cannot fully distinguish between different underlying states based on its observations, a deterministic policy may fail. In such cases, the optimal behavior is often to act randomly but with carefully chosen probabilities, allowing the agent to hedge its decisions across indistinguishable situations. Consider the foggy lake environment:

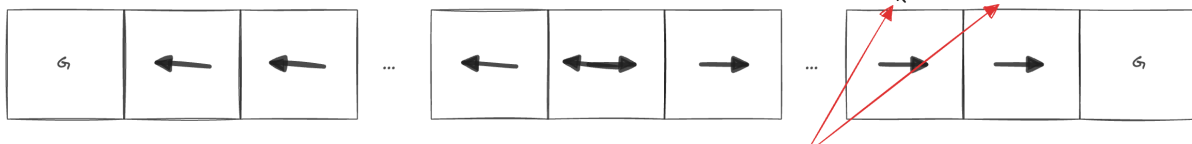
The optimal policy is: in the middle goes left or right at random;
the rest of the actions should point to the closest goal



The policy is obvious, however it is difficult learn the value function!
For example, we don't care about the value of state x , or to differentiate between its value and the value of state $x+1$!
If the action is obvious, allocating resources to accurately estimate value function is not useful and time consuming

Interestingly, with stochastic policies, **exploration can be embedded in the learned function**. Moreover, learning stochastic policies could be **more straightforward for function approximation**. Sometimes value functions are **too much informative** for what's truly needed. It could be that calculating the exact value of a state or state-action pair is complicated or unnecessary. For example, consider a near-infinite corridor deterministic environment:

The optimal policy is: in the middle goes left or right at random;
the rest of the actions should point to the closest goal



The policy is obvious, however it is difficult learn the value function!
For example, we don't care about the value of state x , or to differentiate between its value and the value of state $x+1$!
If the action is obvious, allocating resources to accurately estimate value function is not useful and time consuming

Finally, value-based methods are **prone to oscillations and even divergence**, because small changes in the value function can translate into large, discontinuous shifts in the chosen actions. Such abrupt changes may lead the agent to follow entirely different trajectories, amplifying instability. By contrast, policy-based methods optimize the parameters of a stochastic policy directly through gradient ascent. This results in **actions being adjusted gradually and smoothly**, which often gives policy-based methods **more stable and reliable convergence** properties.

11.1 Policy Gradient Theorem

In order to learn the policy directly, we need to find the gradient of performance metric with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} E_{s_0 \sim p_0} [G_{\pi(\theta)}]$$

To simplify notation, let's use a variable τ to represent a trajectory:

$$\tau = S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$$

we can abuse notation and use G function to obtain the return of the trajectory:

$$G(\tau) = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$$

We can also express the **probability of a trajectory** as the probability of the initial states, then the action decided by the policy, then the probability of a transition, and so on until we have the product of all the probabilities that make the trajectory likely:

$$p(\tau | \pi_{\theta}) = p_0(S_0) \pi_{\theta}(A_0 | S_0) P(S_1 | S_0, A_0) \pi_{\theta}(A_1 | S_1) P(S_2 | S_1, A_1) \dots P(S_T | S_{T-1}, A_{T-1}) = p_0 \prod_{t=0}^{T-1} \{P(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)\}$$

Of course, we don't know the distribution of the start state or the probability of the transitions, because we don't have the model of the environment.

Using this notation, we can rewrite the objective as:

$$J(\theta) = E_{\tau \sim \pi_{\theta}} [G(\tau)] = \sum_{\tau} p(\tau | \pi_{\theta}) G(\tau)$$

The problem is that we can't just evaluate the gradient of this score function because we don't have complete knowledge of the problem, it is a mix of factors that are under our control via the parameters (policy) and factors that are not known (environment model).

In order to provide an estimation of the gradient of the objective function (without knowing the transition function), we have to make some mathematical manipulations in order to remove the unknown factors from the equation. We can rewrite the gradient of the sum as the sum of the gradient:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} p(\tau | \pi_{\theta}) G(\tau) = \sum_{\tau} \nabla_{\theta} p(\tau | \pi_{\theta}) G(\tau)$$

we then multiply every term in the sum by and rewrite in order to obtain the likelihood ratio:

$$= \sum_{\tau} \left\{ \frac{p(\tau | \pi_{\theta})}{p(\tau | \pi_{\theta})} \right\} \nabla_{\theta} p(\tau | \pi_{\theta}) G(\tau) = \sum_{\tau} p(\tau | \pi_{\theta}) \frac{\nabla_{\theta} p(\tau | \pi_{\theta})}{p(\tau | \pi_{\theta})} G(\tau) =$$

now we can use a mathematical result called **likelihood ratio**, which states that the gradient of the function divided by the function itself is equal to the gradient of the log of a function:

$$\frac{\nabla_{\theta} p(\tau|\pi_{\theta})}{p(\tau|\pi_{\theta})} = \nabla_{\theta} \log p(\tau|\pi_{\theta})$$

Thanks for this formula, we can estimate the gradient using m trajectory samples:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log p(\tau^{(i)}|\pi_{\theta}) G(\tau^{(i)})$$

where $\tau^{(i)}$ is a trajectory sampled.

We still have some mathematics work in order to simplify the formula, we can substitute the definition of the transition probability:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log \left[p_0 \prod_{t=0}^{T-1} P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)}) \pi_{\theta}(a_t^{(i)}|s_t^{(i)}) \right] G(\tau^{(i)}) =$$

and we know that the log of a product is equal to the sum of the logs:

$$= \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \left[\log p_0 + \sum_{t=0}^{T-1} \log P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)}) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t^{(i)}|s_t^{(i)}) \right] G(\tau^{(i)})$$

we also know that the gradient of the sum is equal to the sum of gradient:

$$= \frac{1}{m} \sum_{i=1}^m \left[\nabla_{\theta} \log p_0 + \nabla_{\theta} \sum_{t=0}^{T-1} \log P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)}) + \nabla_{\theta} \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t^{(i)}|s_t^{(i)}) \right] G(\tau^{(i)}) =$$

since neither initial state distribution or state transition dynamics are dependent of θ , the derivative of both terms can be removed:

$$= \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t^{(i)}|s_t^{(i)}) G(\tau^{(i)})$$

finally, we can rewrite the gradient of the sum as the sum of gradients and the formula for estimating the gradient of the objective function is:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)}|s_t^{(i)}) G(\tau^{(i)})$$

We have a way to compute the policy gradient without knowing anything about the environment's transition function: we first collect a full trajectory τ and calculate the full discounted

return $G(\tau)$ and then we use that to weight the log probabilities of every action taken in that trajectory: $A_t, A_{t+1}, \dots, A_{T-1}$.

However, it is better to use only the **causal reward** that come after each action, because we know we can't go back in time and current actions cannot be responsible for past reward.

For each step in the trajectory, we calculate the return just from that step and use that value as the score to weight the log probability of the action taken at that time step:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left\{ \sum_{t=0}^{T-1} \left(\nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \right) G_t(\tau^{(i)}) \right\}$$

11.2 REINFORCE

The REINFORCE algorithm (R.J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning", Machine Learning (1992)) was one of the first attempts to implements the idea, estimating the gradient using the full Monte Carlo returns and using a neural network to represent the policy. First of all we implement the parametric policy as a neural network, that takes the state as input and outputs a probability distribution over the actions:

```
import torch

class PolicyApproximator(torch.nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyApproximator, self).__init__()

        # activation function (ReLU)
        self.activation = torch.nn.functional.relu

        # input layer
        self.input_layer = torch.nn.Linear(state_size, 512)

        # hidden layer
        self.hidden_layer = torch.nn.Linear(512, 128)

        # output layer
        self.output_layer = torch.nn.Linear(128, action_size)

    def forward(self, state):

        # convert the state into a tensor
```

```
    if not isinstance(state, torch.Tensor):
        state = torch.Tensor(state)

    # pass it through the input layer
    x = self.activation(self.input_layer(state))

    # then pass for the hidden layer
    x = self.activation(self.hidden_layer(x))

    # finally, pass for the output layer
    preferences = self.output_layer(x)

    return preferences

def select_action(self, state):

    # get the preferences over actions
    preferences = self.forward(state)

    # sample the action from the probability distribution
    dist = torch.distributions.Categorical(logits=preferences)
    action = dist.sample()

    # calculate the log probability of the action
    # needed for the loss
    logpa = dist.log_prob(action).unsqueeze(-1)

    return action.item(), logpa

def select_greedy_action(self, state):

    # select the action with the highest preference
    preferences = self.forward(state)
    action = np.argmax(preferences.detach())

    return action.item()
```

We write the optimization method in order to calculate the performance value. Notice that, in order to have a loss function (to be used in the gradient descent algorithm), we can use the negative of the performance measure as the objective function. Then we use the optimizer to make a step of gradient descent.

```
def optimize(model, rewards, logpas, discounts, optimizer):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for all steps
    returns = np.array([np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
    returns = torch.FloatTensor(returns)

    # prepare the log probabilities of the actions taken in the episode
    logpas = torch.cat(logpas)

    # calculate the policy loss
    loss = -(returns * logpas).sum()

    # optimize the model
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

This time we don't need to create methods to implement epsilon-greedy and greedy strategies, because we can use directly the output of the neural network as a probability distribution over the actions. In the evaluate method, instead of selecting only in a greedy way, **we can sample from the learned stochastic policy**. The correct thing to do depends on the environment, but sampling is the safe bet.

```
def evaluate(approximator, env, episodes=1):
    rewards = []
    for _ in range(episodes):
        state = env.reset()[0]
        rewards.append(0)
        while True:
            action = approximator.select_greedy_action(state)
            state, reward, is_terminal, is_truncated, info = env.step(action)
            rewards[-1] += reward
            is_done = is_terminal or is_truncated
            if is_done:
                break
    return np.mean(rewards)
```

Finally, we can write the complete algorithm, where we collect a full trajectory and then perform an optimization step:

```
import time
import numpy as np
import random
from itertools import count
import gymnasium
import gc

def reinforce(env_name, seed, parameters):

    # unpack the parameters
    gamma = parameters['gamma']
    learning_rate = parameters['learning_rate']
    max_time = parameters['max_time']
    max_episodes = parameters['max_episodes']
    max_score = parameters['max_score']
    episodes_window = parameters['episodes_window']

    # create the environment
    env = gymnasium.make(env_name)

    # access the state and action sizes of the environment
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    # calculate the discounts to be used in the episodes
    max_steps = 1000
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma)

    # set the seeds for reproducibility of results
    torch.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)
    env.np_random = np.random.Generator(np.random.PCG64(seed))

    # create the policy model
    policy_model = PolicyApproximator(state_size, action_size)

    # create the optimizer
    optimizer = torch.optim.RMSprop(policy_model.parameters(),
                                     lr=learning_rate)
```

```
# episode stats trackers
episode_steps = []
episode_return = []
episode_seconds = []
episode_scores = []

# total stats trackers
total_step = 0
training_time = 0
results = np.empty((max_episodes+1, 4))
results[:] = np.nan

# train until we reach a terminal condition
for episode in count():

    # get the start time of the episode
    episode_start = time.time()

    # set return to 0
    discounted_return = 0

    # reset the environment before starting the episode
    state = env.reset()[0]

    # log the rewards and the log of action probabilities
    logpas = []
    rewards = []

    # interact with the environment until the episode is done
    for step in count():

        # select the action using the policy
        action, logpa = policy_model.select_action(state)

        # collect an experience tuple as usual
        next_state, reward, is_terminal, is_truncated, info = env.step(action)
        is_done = is_terminal or is_truncated
        is_failure = is_terminal and not is_truncated

        # save the log probability and the reward
        logpas.append(logpa)
        rewards.append(reward)
```

```
# update current state to next state
state = next_state

# update the return
discounted_return += reward

if is_done:
    gc.collect()
    break

# then, run one optimization step
optimize(policy_model, rewards, logpas, discounts, optimizer)

# evaluate the learned policy after the episode
score = evaluate(policy_model, env)

# timing stuff
episode_elapsed = time.time() - episode_start

# update episode stats
episode_steps.append(step)
episode_return.append(discounted_return)
episode_seconds.append(episode_elapsed)
episode_scores.append(score)

# update total stats
total_step += step
training_time += episode_elapsed

# average stats over some recent episodes
mean_return = np.mean(episode_return[-episodes_window:])
mean_score = np.mean(episode_scores[-episodes_window:])

# report results
results[episode] = total_step, mean_return, mean_score, training_time

# informative print logging
message = 'episode {:04}, steps {:06}, average score {:05.1f}'
message = message.format(episode, total_step, mean_score)
print(message, end='\r', flush=True)

# check max episodes achievement criteria
```



```
    if (episode ≥ max_episodes):
        print(message, ' --> reached_max_episodes ')
        break

    # check max time achievement criteria
    if (training_time ≥ max_time):
        print(message, ' --> reached_max_time ')
        break

    # check max score achievement criteria
    if (mean_score ≥ max_score):
        print(message, ' --> reached_max_score ')
        break

    return policy_model, score, results
```

We can run REINFORCE in the cart-pole environment so that we can easily compare it and the DQN value-based method.

```
def run_experiment( name = 'reinforce',
                    env_name = 'CartPole-v1',
                    algorithm = reinforce,
                    seeds = (12, 34, 56, 78, 90),
                    parameters = None):

    # variables to collect results
    results = []
    best_agent = None
    best_eval_score = float('-inf')

    for seed in seeds:
        # create the agent
        agent, score, result = algorithm(env_name, seed, parameters)

        # save the results the best agent
        results.append(result)
        if score > best_eval_score:
            best_eval_score = score
            best_agent = agent

    np.save(".".join(['../models/', name]), results)
```

```
return results, best_agent, best_eval_score
```

```
parameters = {
    'gamma': 0.99,
    'learning_rate': 0.0007,
    'max_episodes': 3000,
    'max_time': 4*60,
    'max_score': 450,
    'episodes_window': 50
}

reinforce_results, reinforce_agent, reinforce_score = run_experiment(name =
    ↪ 'reinforce_results', algorithm = reinforce, parameters = parameters)
```

episode 0012, steps 000624, average score 048.9

episode 0641, steps 100552, average score 451.5 --> reached_max_score
 episode 0995, steps 120646, average score 133.3 --> reached_max_time
 episode 0173, steps 001448, average score 009.5 --> reached_max_time
 episode 0338, steps 045626, average score 413.7 --> reached_max_time
 episode 0704, steps 005949, average score 009.2 --> reached_max_time

We can load the results of DQN in order to compare the performances of the two algorithms:

```
dqn_results = np.load('../models/dqn_results.npy')
```

As for value-based algorithms, we can plot the performance of the algorithm over episodes:

```
import matplotlib.pyplot as plt

def show_result(what, alg_a_name, alg_a_results, alg_b_name, alg_b_results):

    plt.figure(figsize=(12,6))

    # steps, train_reward, eval_score, time
    alg_a_max = np.max(alg_a_results, axis=0).T
    alg_a_min = np.min(alg_a_results, axis=0).T
    alg_a_mean = np.mean(alg_a_results, axis=0).T
    alg_a_episodes = np.arange(len(alg_a_mean[0]))
```

```
alg_b_max = np.max(alg_b_results, axis=0).T
alg_b_min = np.min(alg_b_results, axis=0).T
alg_b_mean = np.mean(alg_b_results, axis=0).T
alg_b_episodes = np.arange(len(alg_b_mean[0]))

if (what == 'total_steps'):
    index = 0;
    title = 'Total Steps'
    label = 'Steps'
elif(what == 'avg_training_reward'):
    index = 1;
    title = 'Moving Avg Reward (Training)'
    label = 'Reward'
elif(what == 'avg_evaluation_reward'):
    index = 2;
    title = 'Moving Avg Reward (Evaluation)'
    label = 'Reward'
elif(what == 'training_time'):
    index = 3;
    title = 'Trainning time)'
    label = 'Time'
else:
    raise ValueError('Not sure what you want to plot')

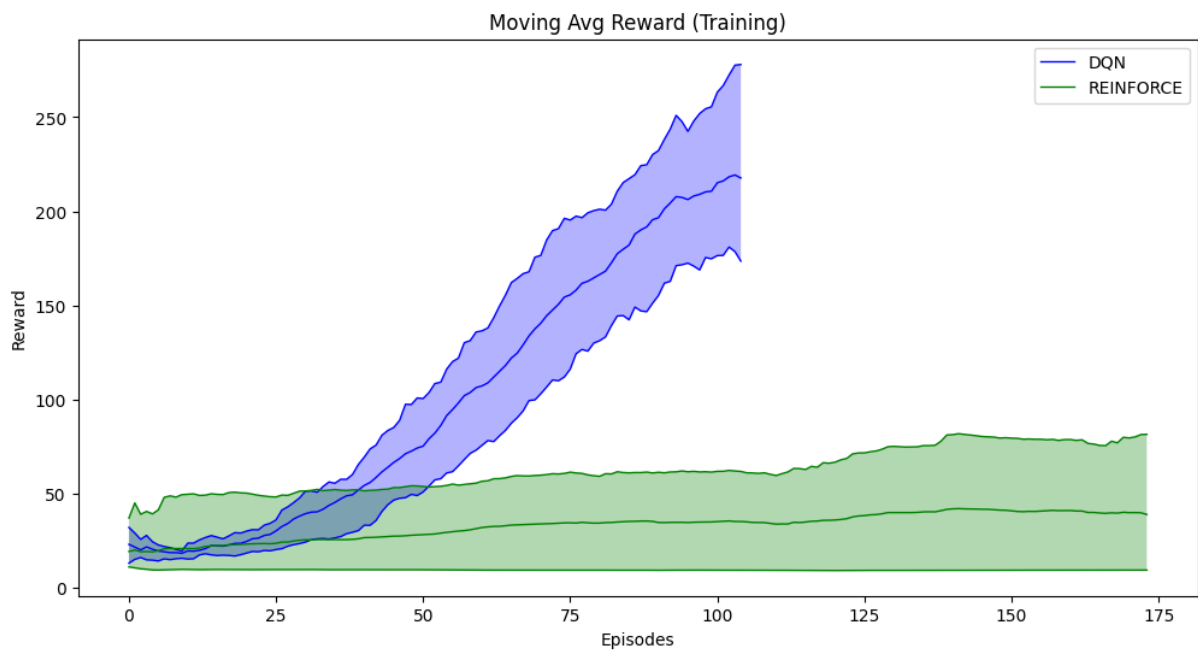
plt.plot(alg_a_max[index], 'b', linewidth=1, label=alg_a_name)
plt.plot(alg_a_min[index], 'b', linewidth=1)
plt.plot(alg_a_mean[index], 'b', linewidth=1)
plt.fill_between(alg_a_episodes, alg_a_min[index], alg_a_max[index],
↪ facecolor='b', alpha=0.3)

plt.plot(alg_b_max[index], 'g', linewidth=1, label=alg_b_name)
plt.plot(alg_b_min[index], 'g', linewidth=1)
plt.plot(alg_b_mean[index], 'g', linewidth=1)
plt.fill_between(alg_b_episodes, alg_b_min[index], alg_b_max[index],
↪ facecolor='g', alpha=0.3)

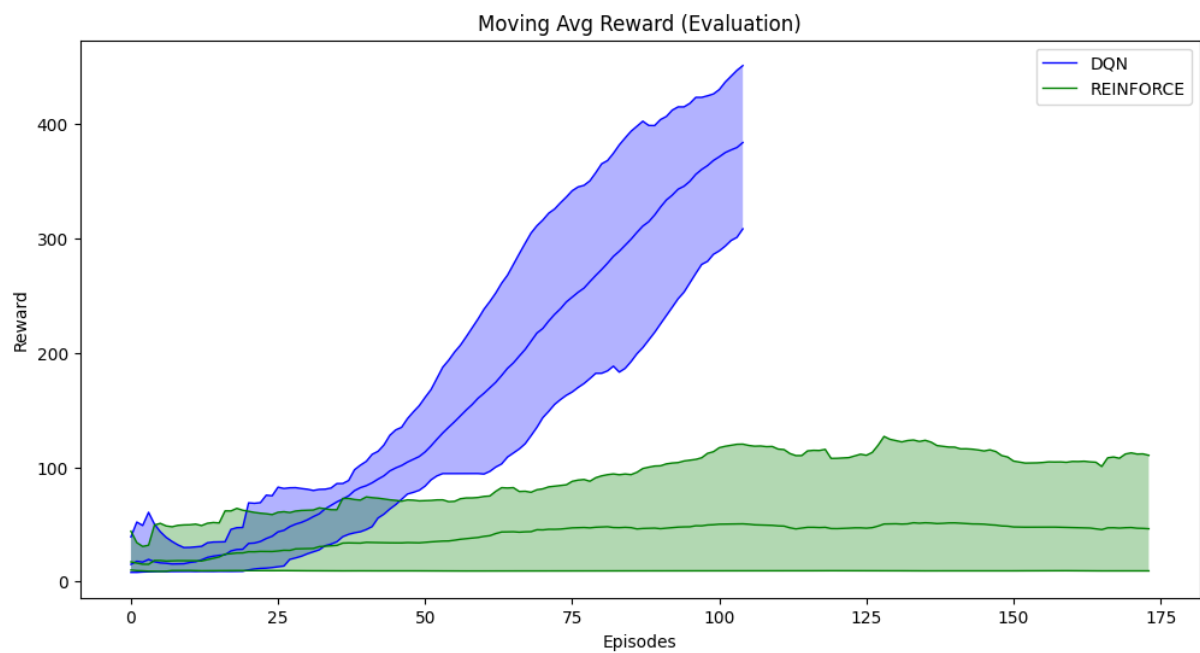
plt.title(title)
plt.ylabel(label)
plt.xlabel('Episodes')
plt.legend()

plt.show()
```

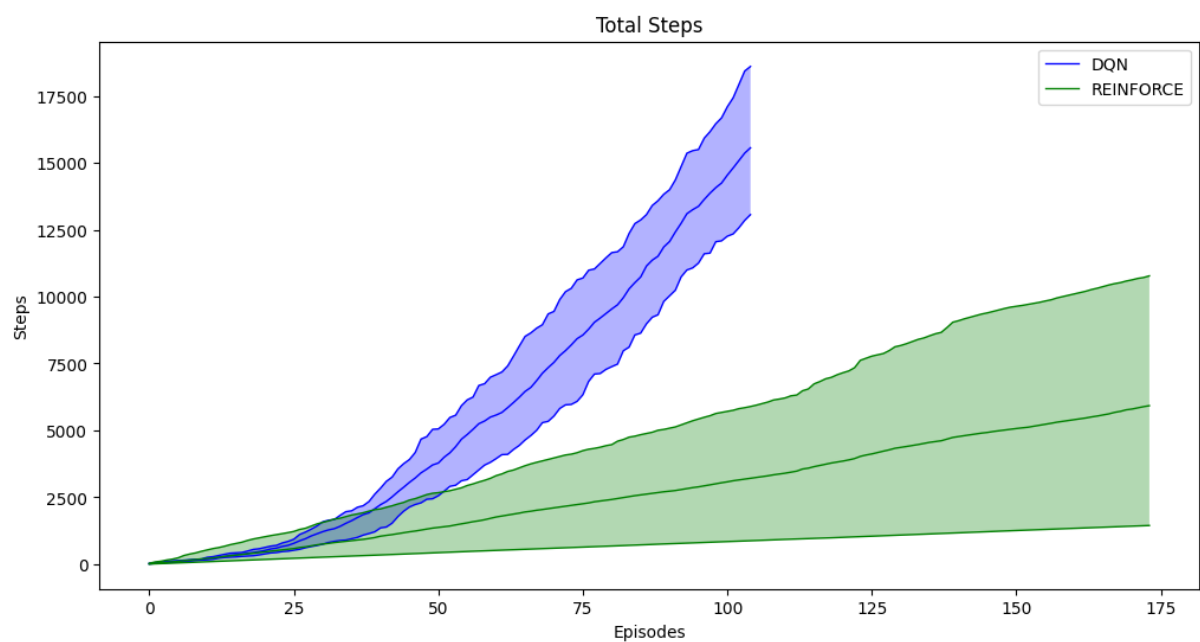
```
show_result('avg_training_reward','DQN', dqn_results, 'REINFORCE', reinforce_results)
```



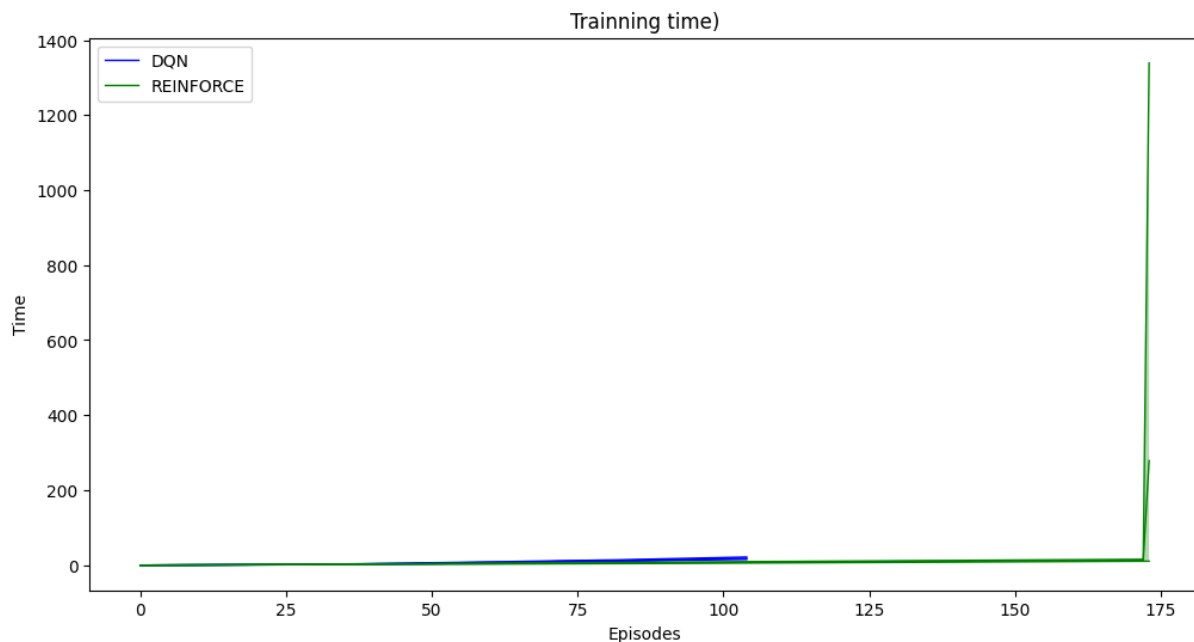
```
show_result('avg_evaluation_reward','DQN', dqn_results, 'REINFORCE',  
↪ reinforce_results)
```



```
show_result('total_steps','DQN', dqn_results, 'REINFORCE', reinforce_results)
```



```
show_result('training_time','DQN', dqn_results, 'REINFORCE', reinforce_results)
```



The REINFORCE algorithm **works well in simple problems**, and it has **convergence guarantees**. However, because it uses the full Monte Carlo returns for calculating the gradient, **its variance is a problem**. The accumulation of random events along a trajectory (including the initial state) and the randomness associated with the stochastic policy is compounded inside the return, making it a high-variance signal that's challenging to interpret. So, REINFORCE alone is **too inefficient** to be a practical algorithm. However, in terms of training time, we can see how it uses few resources.

11.3 Vanilla Policy Gradient (VPG)

The key idea underlying policy gradient is to push up the probabilities of actions that lead to higher return, and push down the probabilities of actions that lead to lower return, until we arrive at the optimal policy. However, using a (noisy) Monte Carlo estimate of the return introduces high variance in the policy gradient estimate, which can make learning slow. Imagine an environment (such as the cart-pole) in which all rewards are positive, in order to accurately separate good actions from the best actions, we need a lot of data.

It would be handy if we could use something that allows us to **differentiate the values of actions in the same state**, to do this we can exploit the **action-advantage function**:

$$a_{\pi}(s,a) = q_{\pi}(s,a) - v_{\pi}(s)$$

we can say that the advantage function is approximately the following:

$$A(S_t, A_t) = R_t + \gamma R_{t+1} + \dots + \gamma^{T-1} R_T - v_\pi(S_t)$$

An estimate of it is the return minus the estimated expected return from that state:

$$A(S_t, A_t) = G_t - V(S_t)$$

Using the advantage somewhat **centers scores around zero**: better-than-average actions have a positive score (increasing probabilities), worse-than-average have a negative score (decreasing probabilities). However, we need two neural networks, one for learning the policy as before and one for learning the state-value function in order to estimate the advantage:

The loss for the value function is simple, the mean squared Monte Carlo error:

$$L_v(\phi) = \frac{1}{m} \sum_{i=0}^m \left[\left(G_t(\tau^{(i)}) - V(s_t^{(i)}; \phi) \right)^2 \right]$$

For the policy network, we can update the gradient of the performance measure using the estimated advantage :

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left\{ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_t(\tau^{(i)}) - V(s_t^{(i)}; \phi)) \right\}$$

However, another essential improvement is to add an **entropy term** to the loss function. We can interpret entropy in many different ways, from the amount of information we can gain by sampling from a distribution, to the number of ways we can order a set. In this context, we can think of entropy in a straightforward way. A uniform distribution, which has evenly distributed samples, has high entropy. If we have two samples, both with a 50% chance, then the entropy is the highest it can be for a two-sample set, conversely, if we have two samples, one with 100% chance, then the entropy is the lowest. The way to use entropy in policy-gradient methods is to add the weighted entropy to the performance function **to encourage having evenly distributed actions**. That way, a policy (with evenly distributed actions) yields the highest entropy and contributes to maximize the performance. On the other hand, converging on a single action (which means entropy is zero) doesn't increase the performance measure.

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{T-1} \nabla_{\theta} [\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_t(\tau^{(i)}) - V(s_t^{(i)}; \phi)) + \beta H(s_t^{(i)}; \theta)]$$

This algorithm is called **Vanilla policy gradient**. We may be wondering, is this a first example of actor-critic? It's learning a policy and a value-function, so it seems it should be. However, according to the definition accepted in the field, only methods that learn the value function using bootstrapping should be called actor-critic, because it's bootstrapping that adds bias to the value function, and thus makes it a "critic". Therefore, REINFORCE and VPG aren't considered actor-critic methods. But beware of the lingo, it's not consistent.

We need a new neural network to approximate the state-value function:

```
class ValueApproximator(torch.nn.Module):
    def __init__(self, state_size):
        super(ValueApproximator, self).__init__()

        # define the activation function (ReLU)
        self.activation = torch.nn.functional.relu

        # defining the input layer
        self.input_layer = torch.nn.Linear(state_size, 512)

        # creating the hidden layers
        self.hidden_layers = torch.nn.ModuleList()
        self.hidden_layers.append(torch.nn.Linear(512, 128))

        # connecting the output layer
        self.output_layer = torch.nn.Linear(128, 1)

    def forward(self, state):
        # take in the raw state and convert it into a tensor
        # also unsqueezing it in order to change it's shape
        if not isinstance(state, torch.Tensor):
            state = torch.Tensor(state)

        # pass it through the input layer
        x = self.activation(self.input_layer(state))

        # then pass for all hidden layers
        for hidden_layer in self.hidden_layers:
            x = self.activation(hidden_layer(x))

        # finally, pass for the output layer
        value = self.output_layer(x)

        return value
```

We need also to the select_action method of the first network (the policy network) in order to calculate also the entropy term:

```
def select_action(self, state):
```



```
# get the preferences over actions
preferences = self.forward(state)

# sample the action from the probability distribution
dist = torch.distributions.Categorical(logits=preferences)
action = dist.sample()

# calculate the log probability of the actions, it is needed for the loss
logpa = dist.log_prob(action).unsqueeze(-1)

# calculate the entropy
entropy = dist.entropy().unsqueeze(-1)

return action.item(), logpa, entropy

PolicyApproximator.select_action = select_action
```

The optimize method is different from the previous one, because we have two neural networks to optimize:

```
def optimize(policy_model, value_model,
             rewards, logpas, values, entropies,
             policy_optimizer, value_optimizer,
             discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for all steps
    returns = np.array([np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
    returns = torch.FloatTensor(returns)

    # prepare the log probabilities, the entropies and the values
    logpas = torch.cat(logpas)
    entropies = torch.cat(entropies)
    values = torch.cat(values)

    # estimate the advantages
    advantage = returns - values
```

```
# calculate the policy loss
beta = 0.001
policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

# calculate the value loss
value_loss = advantage.pow(2).sum()

# optimize the policy network
policy_optimizer.zero_grad()
policy_loss.backward()
policy_optimizer.step()

# optimize the value network
value_optimizer.zero_grad()
value_loss.backward()
value_optimizer.step()
```

Finally, we can write the complete algorithm, similar to REINFORCE, but with the addition of the value network and the entropy term:

```
def vpg(env_name, seed, parameters):

    # unpack the parameters
    gamma = parameters['gamma']
    learning_rate = parameters['learning_rate']
    max_time = parameters['max_time']
    max_episodes = parameters['max_episodes']
    max_score = parameters['max_score']
    episodes_window = parameters['episodes_window']

    # create the environment
    env = gymnasium.make(env_name)

    # access the state and action sizes of the environment
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    # calculate the discounts to be used in the episodes
    max_steps = 1000
    discounts = np.logspace(start=0, stop=max_steps, num=max_steps, base=gamma)

    # set the seeds for reproducibility of results
```

```
torch.manual_seed(seed)
np.random.seed(seed)
random.seed(seed)
env.np_random = np.random.Generator(np.random.PCG64(seed))

# create the policy model and its optimizer
policy_model = PolicyApproximator(state_size, action_size)
policy_optimizer = torch.optim.RMSprop(policy_model.parameters(),
↪ lr=learning_rate)

# create the value model and its optimizer
value_model = ValueApproximator(state_size)
value_optimizer = torch.optim.RMSprop(value_model.parameters(), lr=learning_rate)

# episode stats trackers
episode_steps = []
episode_return = []
episode_seconds = []
episode_scores = []

# total stats trackers
total_step = 0
training_time = 0
results = np.empty((max_episodes+1, 4))
results[:] = np.nan

# train until we reach a terminal condition
for episode in count():

    # get the start time of the episode
    episode_start = time.time()

    # set return to 0
    discounted_return = 0

    # reset the environment before starting the episode
    state = env.reset()[0]

    # log the rewards, the log of action probabilities, entropies and values
    logpas = []
    rewards = []
    entropies = []
    values = []
```

```
# interact with the environment until the episode is done
for step in count():

    # select the action using the policy
    action, logpa, entropy = policy_model.select_action(state)

    # collect an experience tuple as usual
    next_state, reward, is_terminal, is_truncated, info = env.step(action)
    is_done = is_terminal or is_truncated
    is_failure = is_terminal and not is_truncated

    # calculate the value using the value network
    value = value_model(state)

    # save the log probability, entropy, reward and value
    logpas.append(logpa)
    entropies.append(entropy)
    rewards.append(reward)
    values.append(value)

    # update current state to next state
    state = next_state

    # update the return
    discounted_return += reward

    if is_done:
        gc.collect()
        break

    # then, run one optimization step
    optimize(policy_model, value_model, rewards, logpas, values, entropies,
    ↪ policy_optimizer, value_optimizer, discounts)

    # evaluate the learned policy after the episode
    score = evaluate(policy_model, env)

    # timing stuff
    episode_elapsed = time.time() - episode_start

    # update episode stats
    episode_steps.append(step)
```

```
episode_return.append(discounted_return)
episode_seconds.append(episode_elapsed)
episode_scores.append(score)

# update total stats
total_step += step
training_time += episode_elapsed

# average stats over some recent episodes
mean_return = np.mean(episode_return[-episodes_window:])
mean_score = np.mean(episode_scores[-episodes_window:])

# report results
results[episode] = total_step, mean_return, mean_score, training_time

# informative print logging
message = 'episode {:04}, steps {:06}, average score {:05.1f}'
message = message.format(episode, total_step, mean_score)
print(message, end='\r', flush=True)

# check max episodes achievement criteria
if (episode ≥ max_episodes):
    print(message, ' --> reached_max_episodes ')
    break

# check max time achievement criteria
if (training_time ≥ max_time):
    print(message, ' --> reached_max_time ')
    break

# check max score achievement criteria
if (mean_score ≥ max_score):
    print(message, ' --> reached_max_score ')
    break

return policy_model, score, results
```

We can try the algorithm in the cartpole environment:

```
vpg_results, vpg_agent, vpg_score = run_experiment(name = 'vpg_results', algorithm =
↪ vpg)
```

TypeError Traceback (most recent call last)

Cell In[25], line 1

----> 1 vpg_results, vpg_agent, vpg_score = run_experiment(name = 'vpg_results', algorithm = v

Cell In[13], line 14, in run_experiment(name, env_name, algorithm, seeds, parameters)

10 best_eval_score = float('-inf')

12 for seed in seeds:

13 # create the agent

---> 14 agent, score, result = algorithm(env_name, seed, parameters)

16 # save the results the best agent

17 results.append(result)

Cell In[24], line 4, in vpg(env_name, seed, parameters)

1 def vpg(env_name, seed, parameters):

2

3 # unpack the parameters

----> 4 gamma = parameters['gamma']

5 learning_rate = parameters['learning_rate']

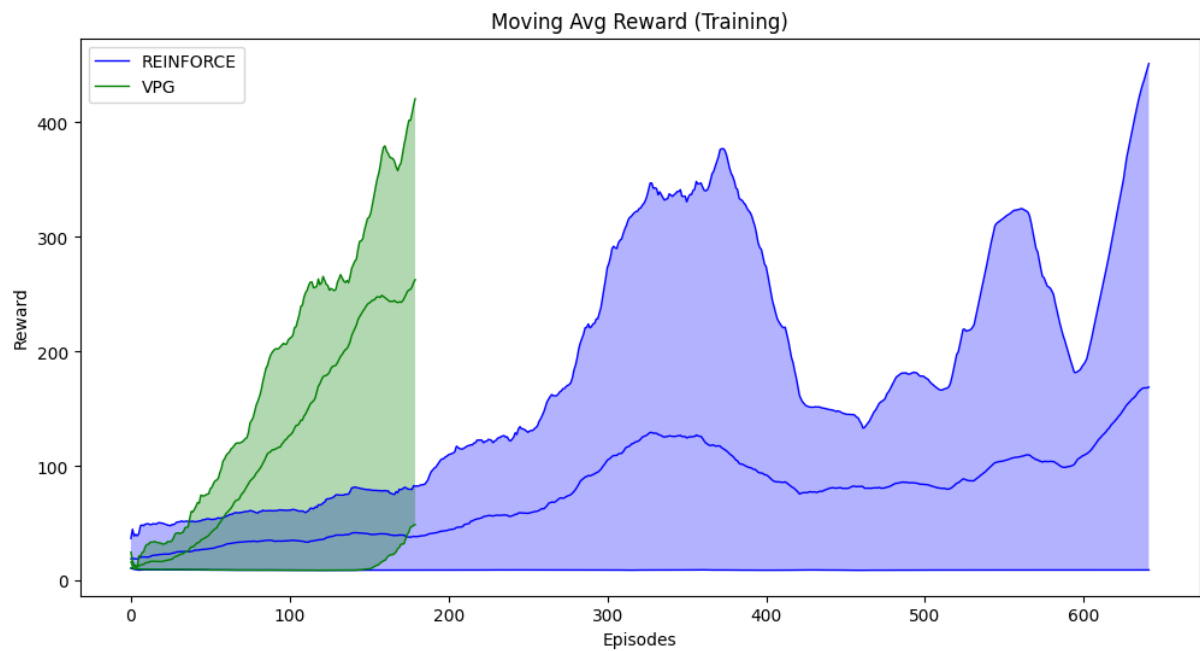
6 max_time = parameters['max_time']

TypeError: 'NoneType' object is not subscriptable

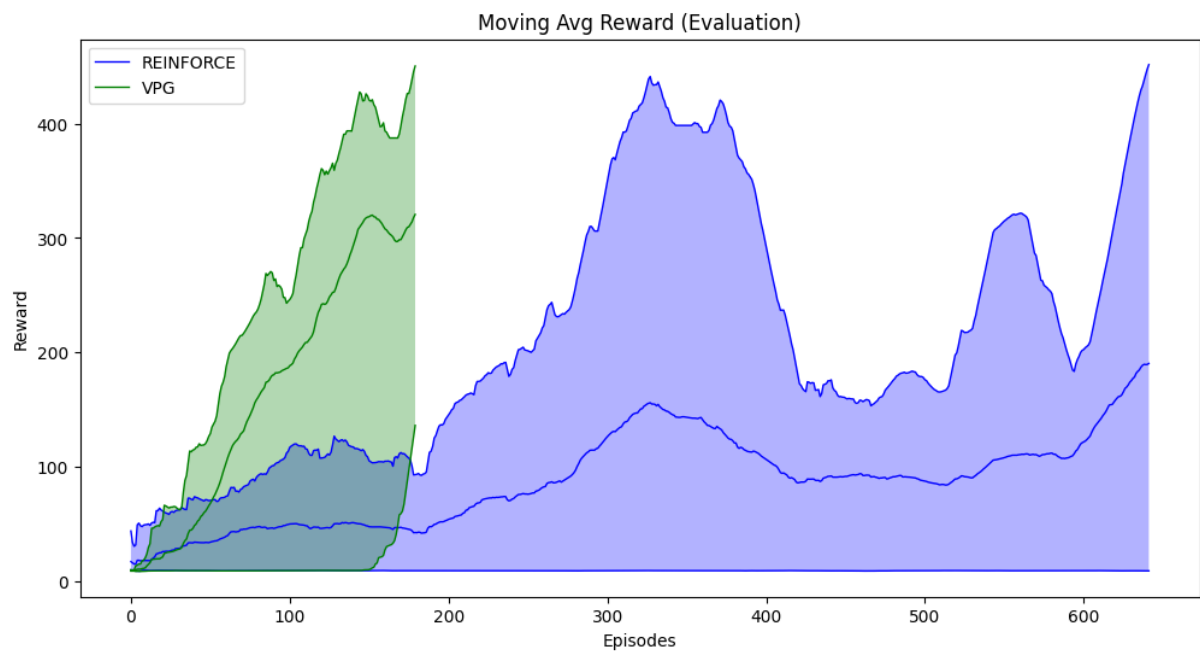
We can compare VPG with REINFORCE, Notice how VPG is more sample efficient, instead REINFORCE is too inefficient to be a practical algorithm.

```
show_result('avg_training_reward', 'REINFORCE', reinforce_results, 'VPG', vpg_results)
```

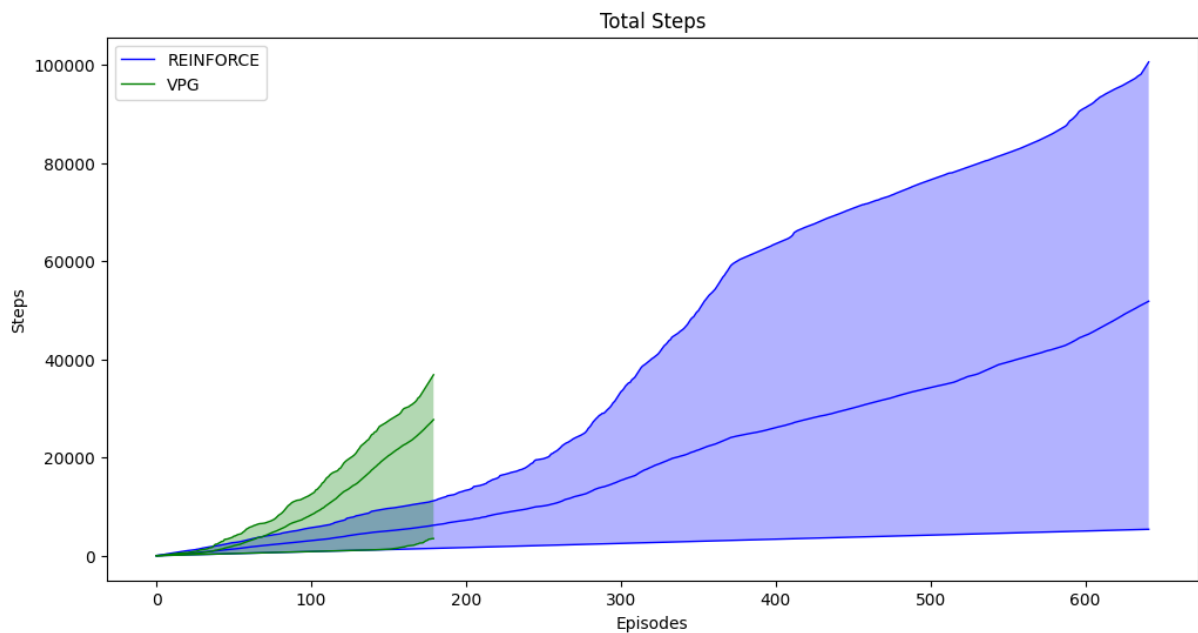
↪



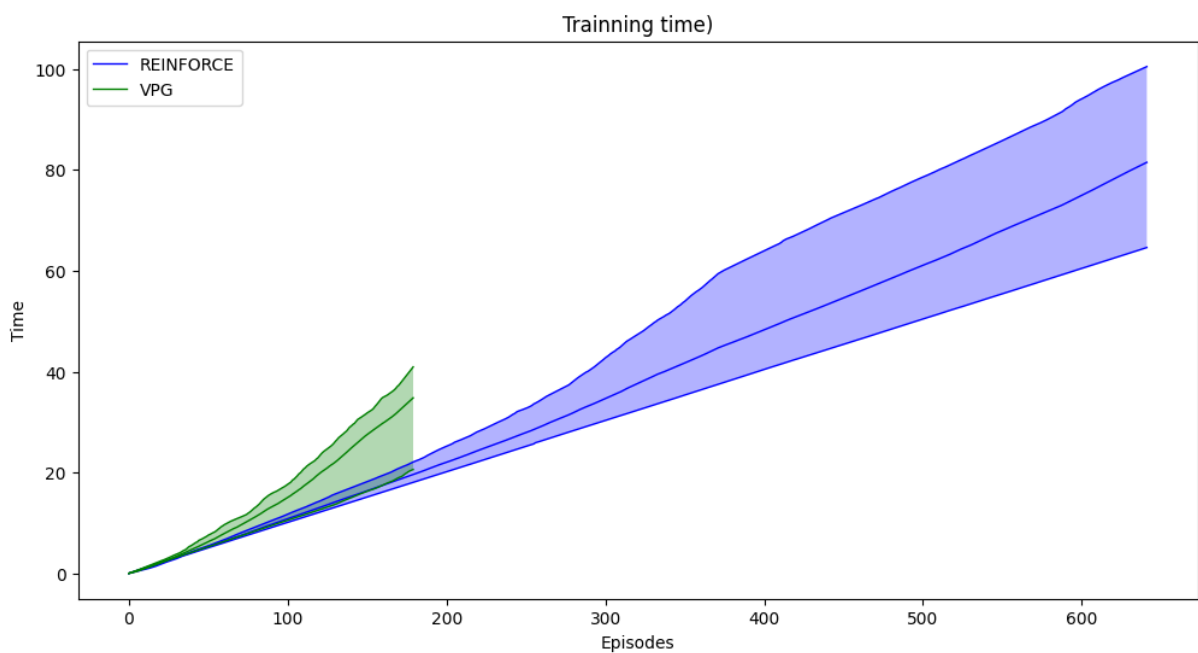
```
show_result('avg_evaluation_reward', 'REINFORCE', reinforce_results, 'VPG',  
           ↪ vpg_results)
```



```
show_result('total_steps', 'REINFORCE', reinforce_results, 'VPG', vpg_results)
```



```
show_result('training_time', 'REINFORCE', reinforce_results, 'VPG', vpg_results)
```



VPG is a pretty robust method for simple problems; it is, for the most part, unbiased because it uses an unbiased target for learning both the policy and value function. That is, it uses Monte Carlo returns, which are complete actual returns experienced directly in the environment, without any bootstrapping. The only bias in the entire algorithm is because we use function approx-

imation, which is inherently biased.

11.4 Asynchronous Advantage Actor-Critic (A3C)

One of the main sources of variance in DRL algorithms is how **correlated and non-stationary** online samples are. In value-based methods, we use a replay buffer to uniformly sample mini-batches of (for the most part) independent and identically distributed data. Unfortunately, this is limited to off-policy methods, because on-policy agents cannot reuse data generated by previous policies. In on-policy methods (likes the policy-based methods), instead of using the replay buffer, we can have **multiple workers generating experience on multiple instances of the environment** in parallel and asynchronously updating the policy and value function. This can help to de-correlate the data used for training and to reduce the variance of the algorithm.

The **Asynchronous Advantage Actor-Critic (A3C)** algorithm (Vlad Mnih et al. *"Asynchronous Methods for Deep Reinforcement Learning"*, ICML 2016) uses concurrent actors to generate a broad set of experience samples in parallel. Moreover, it also uses **n-step returns with bootstrapping** to learn the policy and value function.

This behavior can be implemented using **multiple threads**, one for each worker, however Python threads cannot take advantage of CPU with multiple cores, due to internal implementation details. The workaround is to use **multiple processes** using the multiprocessing module. Unfortunately, the multiprocessing library has problems in running in an iterative environment like Jupyter, so we will use the threading library. We need to create a work function that will be executed by each thread. In the following example, the work function just waits for a random time before printing its rank:

```
import time

def work(rank, sleep_time):
    time.sleep(sleep_time)
    print(rank)
```

Then we create a pool of threads, execute them in parallel and finally wait for the threads to finish:

```
import random

from threading import Thread

def main():
```

```
# number of workers
n_workers = 4

print('start')
if __name__ == '__main__':

    workers = [Thread(target=work, args=(rank, random.uniform(1, 3),)) for rank in
↪ range(n_workers)]

    # start workers
    [w.start() for w in workers]

    # wait for all workers to finish
    [w.join() for w in workers]

    print('end')

main()
```

```
start
1
0
3
2
end
```

As you can see, workers are running in parallel: since they wait for a different amount of time, the order of the output is not deterministic. However, we can see that the rank of the thread is always the same, because the pool of threads is created with a fixed number of workers. Moreover, the "end" string is always printed at the end, because the main thread waits for the completion of all the workers.

Moreover, when writing concurrent programs we may need to share data or resources between threads. Since threads live in the same memory space, they have access to the same (public) data. Thus, we can for example simply use a global variable to which all threads have read and write access. Moreover, typically we need to protect with a mutual exclusion lock some shared variables in order to prevent race conditions. Python provides a mutual exclusion lock for use with processes via the Lock class. In this example, we create a shared counter and we use a lock to synchronize access to it:

```
def work(rank, iterations, lock):

    # all threads can access this shared variable
    global counter

    for _ in range(iterations):

        time.sleep(random.uniform(1, 10)/100)

        # lock the shared variable
        lock.acquire()

        counter += 1
        print(counter, " - int's me ", rank)

        # unlock the shared variable
        lock.release()
```

```
from threading import Lock

# number of workers
n_workers = 4

print('start')

# number of iterations per worker
iterations = 5

# shared variable
counter = 0

# create a lock
lock = Lock()

# create workers
workers = [Thread(target=work, args=(rank, iterations, lock)) for rank in
    ↪ range(n_workers)]

# start workers
[w.start() for w in workers]

# wait for all workers to finish
```

```
[w.join() for w in workers]

print ('end')
```

start

```
1 - int's me 1
2 - int's me 2
3 - int's me 0
4 - int's me 3
5 - int's me 2
6 - int's me 0
7 - int's me 3
8 - int's me 0
9 - int's me 1
10 - int's me 2
11 - int's me 3
12 - int's me 0
13 - int's me 2
14 - int's me 1
15 - int's me 0
16 - int's me 1
17 - int's me 3
18 - int's me 3
19 - int's me 2
20 - int's me 1
end
```

We can write the work function used by each agent in the A3C algorithm. The agent interacts with the environment and update the global networks parameters:

```
# create a global object to share information between the workers
shared_info = {}
```

```
def agent_work(rank, lock):

    # access the global shared variables
```

```
global shared_info

# create a different seed for each worker, since we need different
# experiences from each one of them
local_seed = shared_info['seed'] + rank

# create the local environment
env = gymnasium.make(shared_info['env_name'])

# create a uniquely seeded environment for each worker
env.np_random = np.random.Generator(np.random.PCG64(local_seed))

# access the state and action sizes of the environment
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# create the local policy model
# initialize weights with the weights of the shared policy model
local_policy_model = PolicyApproximator(state_size, action_size)
local_value_model = ValueApproximator(state_size)
local_policy_model.load_state_dict(shared_info['shared_policy_model'].state_dict())
local_value_model.load_state_dict(shared_info['shared_value_model'].state_dict())

# increment the global episodes counter
with lock:
    shared_info['episode'] = shared_info['episode'] + 1
    local_episode = shared_info['episode']

# start the training loop, until the worker is signaled to get out of it
# we reach a terminal condition
while not shared_info['out_signal']:

    # get the start time of the episode
    episode_start = time.time()

    # set return to 0
    discounted_return = 0

    # reset the environment before starting the episode
    state = env.reset()[0]
```

```
# log the rewards, the log of action probabilities, entropies and values
logpas = []
rewards = []
entropies = []
values = []

n_steps_start = 0

# interact with the environment
for step in count():

    # select the action using the local policy
    action, logpa, entropy = local_policy_model.select_action(state)

    # collect an experience tuple as usual
    next_state, reward, is_terminal, is_truncated, info = env.step(action)
    is_done = is_terminal or is_truncated
    is_failure = is_terminal and not is_truncated

    # calculate the value using the value network
    value = local_value_model(state)

    # save the log probability, entropy, reward and value
    logpas.append(logpa)
    entropies.append(entropy)
    rewards.append(reward)
    values.append(value)

    # update current state to next state
    state = next_state

    # update the return
    discounted_return += reward

    # if we collected enough steps or if the episode is done
    # calculate the n-step return and optimize the model
    if is_done or step - n_steps_start == shared_info['max_n_steps']:

        # if it's a failure, then the value of the next state is 0; otherwise,
        # we bootstrap
        next_value = 0 if is_failure else value.detach().item()
        rewards.append(next_value)
```

```

        # optimize the model
        optimize(logpas, entropies, rewards, values,
                  local_policy_model, local_value_model,
                  shared_info['shared_policy_model'],
                  shared_info['shared_value_model'],
                  shared_info['shared_policy_optimizer'],
                  shared_info['shared_value_optimizer'],
                  shared_info['discounts'])

        # load the shared updated models parameters into the local models
        lo-
↪ local_policy_model.load_state_dict(shared_info['shared_policy_model'].state_dict())
↪ local_value_model.load_state_dict(shared_info['shared_value_model'].state_dict())

        logpas = []
        entropies = []
        rewards = []
        values = []

        n_steps_start = step

    if is_done:
        break

    # evaluate the local policy after the episode
    score = evaluate(local_policy_model, env)

    # update total stats
    episode_elapsed = time.time() - episode_start
    with lock:
        shared_info['total_steps'] += step
        shared_info['training_time'] += episode_elapsed / shared_info['n_workers']

    # update global episode stats
    with lock:
        shared_info['episode_steps'].append(step)
        shared_info['episode_return'].append(discounted_return)
        shared_info['episode_seconds'].append(episode_elapsed)
        shared_info['episode_scores'].append(score)

    # average stats over some recent episodes

```

```
    episodes_window = shared_info['episodes_window']
    mean_return = np.mean(shared_info['episode_return'][-episodes_window:])
    mean_score = np.mean(shared_info['episode_scores'][-episodes_window:])

    # report results
    shared_info['results'][local_episode] = step, mean_return, mean_score,
    ↪ episode_elapsed / shared_info['n_workers']

    # informative print logging
    message = 'worker {:02} - episode {:04}, steps {:06}, average score {:05.1f}'
    message = message.format(rank, local_episode, shared_info['total_steps'],
    ↪ mean_score)
    with lock:
        print(message, end='\n', flush=True)

    # check if we should stop
    if shared_info['out_signal'] == 1:
        break

    # check max episodes achievement criteria
    if (local_episode >= shared_info['max_episodes']):
        print(message, ' --> reached_max_episodes ')
        shared_info['out_signal'] = 1
        break

    # check max time achievement criteria
    if (shared_info['training_time'] >= shared_info['max_time']):
        print(message, ' --> reached_max_time ')
        shared_info['out_signal'] = 1
        break

    # check max score achievement criteria
    if (mean_score >= shared_info['max_score']):
        print(message, ' --> reached_max_score ')
        shared_info['out_signal'] = 1
        break

    # else go work on another episode by
    # getting a new episode id from the
    # global protected counter
    with lock:
        shared_info['episode'] = shared_info['episode'] + 1
        local_episode = shared_info['episode']
```



```
env.close()
del env
```

Workers append the value of the next state, whether terminal or not, to the reward sequence before calling the optimize function (in VPG we were using the full returns for the advantage estimate). In that way, the reward variable contains all rewards from the partial trajectory and the state-value estimate of that last state. We can see this as having the partial return (the sequence of rewards) and the **predicted remaining return** in the same place. You can realize that this is an **n-step return**: the agent goes out for n-steps collecting rewards, and then **bootstrap** after that n-th state (or before if he lands on a terminal state that comes first):

$$\begin{aligned} A(S_t, A_t) &= R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}) - V(S_t) \\ G_{t:t+n} - V(S_{t+n+1}) & \end{aligned}$$

Notice the bootstrapping, this is what makes this algorithm an **actor-critic method**. Now, we can use that n-step return to improve the value function estimate:

$$L_v(\phi) = \frac{1}{m} \sum_{i=0}^m \left[\left(G_{t:t+n}^{(i)} - V(s_{t+n+1}^{(i)}; \phi) \right)^2 \right]$$

We also use this n-step advantage estimate for updating the policy networks:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^n \nabla_{\theta} [\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_{t:t+n}^{(i)} - V(s_{t+n+1}^{(i)}; \phi) + \beta H(s_{t+n+1}^{(i)}; \theta))]$$

We can write the optimize model function, which is similar to the one of VPG however this time we are optimizing the shared models:

```
def optimize(logpas, entropies, rewards, values,
            local_policy_model, local_value_model,
            shared_policy_model, shared_value_model,
            shared_policy_optimizer, shared_value_optimizer,
            discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for the n-step steps
    # remember: rewards includes the bootstrapping value
    returns = np.array([np.sum(discounts[T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
```

```
returns = torch.FloatTensor(returns[:-1])

# prepare the log probabilities, the entropies and the values
logpas = torch.cat(logpas)
entropies = torch.cat(entropies)
values = torch.cat(values)

# estimate the advantages
advantage = returns - values

# calculate the policy loss
beta = 0.001
policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

# calculate the value loss
value_loss = advantage.pow(2).sum()

# optimize the policy network
# notice, we now zero the shared policy optimizer
shared_policy_optimizer.zero_grad()
policy_loss.backward()

# iterating over all local and shared policy network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_policy_model.parameters(),
    ↪ shared_policy_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_policy_optimizer.step()

# optimize the value network
shared_value_optimizer.zero_grad()
value_loss.backward()

# iterating over all local and shared value network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_value_model.parameters(),
    ↪ shared_value_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad
```

```
# once the gradients are copied into the shared optimizer, we run an optimization
↳ step
shared_value_optimizer.step()
```

One of the most critical aspects of A3C is that **its network updates are asynchronous and lock-free**. As we already discussed, having a shared model creates a need for a blocking mechanism to prevent workers from overwriting other updates, however this would be inefficient. Instead, A3C uses an update style called a **Hogwild!**, in which workers access to shared model with the possibility of overwriting each other's work, which is shown to achieve a near-optimal rate of convergence and outperforming alternative schemes that use locking by an order of magnitude: F. Niu et al. **"HOGWILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent"**, NIPS 2011.

We can write the A3C algorithm, which is similar to VPG, but with the addition of the n-step returns and the multiple workers:

```
def a3c(env_name, seed, parameters):

    # unpack the parameters
    gamma = parameters['gamma']
    learning_rate = parameters['learning_rate']
    max_time = parameters['max_time']
    max_episodes = parameters['max_episodes']
    max_score = parameters['max_score']
    episodes_window = parameters['episodes_window']
    n_workers = parameters['n_workers']
    max_n_steps = parameters['max_n_steps']

    # access the global object
    global shared_info

    # define the seed for workers
    shared_info['seed'] = seed

    # set the seeds for reproducibility of results
    torch.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)

    # define the number of workers
```

```
shared_info['n_workers'] = n_workers

# define the number of steps to collect before optimizing (n-step)
shared_info['max_n_steps'] = max_n_steps

# provide gamma and learning rate to workers
shared_info['gamma'] = gamma
shared_info['learning_rate'] = learning_rate

# provide environment information to workers
shared_info['env_name'] = 'CartPole-v1'

# provide stopping thresholds to workers
shared_info['max_time'] = max_time
shared_info['max_episodes'] = max_episodes
shared_info['max_score'] = max_score
shared_info['episodes_window'] = episodes_window

# create an environment in order to get information
# to create the shared models and to perform final evaluation
env = gymnasium.make(env_name)

# access the state and action sizes of the environment
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# episode stats trackers
shared_info['episode_steps'] = []
shared_info['episode_return'] = []
shared_info['episode_seconds'] = []
shared_info['episode_scores'] = []

# total stats trackers
shared_info['total_steps'] = 0
shared_info['results'] = np.empty((max_episodes+1+shared_info['n_workers'], 4))

# calculate the discounts to be used in the episodes
max_steps = 1000
shared_info['discounts'] = np.logspace(start=0, stop=max_steps, num=max_steps,
↪ base=gamma)

# create the policy model and its optimizer
shared_info['shared_policy_model'] = PolicyApproximator(state_size, action_size)
```

```
shared_info['shared_policy_optimizer'] =
↳ torch.optim.RMSprop(shared_info['shared_policy_model'].parameters(),
↳ lr=learning_rate)

# create the value model and its optimizer
shared_info['shared_value_model'] = ValueApproximator(state_size)
shared_info['shared_value_optimizer'] =
↳ torch.optim.RMSprop(shared_info['shared_value_model'].parameters(),
↳ lr=learning_rate)

# initialize shared episode counter
shared_info['episode'] = 0

# initialize the training time
shared_info['training_time'] = 0

# create a lock in order to protect the concurrent access to variables
lock = Lock()

# create a shared quit signal to inform the workers
# when a stop condition is reached (max episodes, max time or max score)
shared_info['out_signal'] = 0

# create workers
workers = [Thread(target=agent_work, args=(rank, lock,)) for rank in range(
↳ shared_info['n_workers'])]

# start workers
[w.start() for w in workers]

# wait for all workers to finish
[w.join() for w in workers]

# final evaluation
final_eval_score = evaluate(shared_info['shared_policy_model'], env, episodes=10)
env.close()
del env

return shared_info['shared_policy_model'], final_eval_score,
↳ shared_info['results']
```

We can try the algorithm in the cart-pole environment:

```
parameters['n_workers'] = 8
parameters['max_n_steps'] = 50

a3c_results, a3c_agent, a3c_score = run_experiment(name = 'a3c_results', algorithm =
↳ a3c)
```

worker 04 - episode 0019, steps 000206, average score 008.9

worker 01 - episode 3001, steps 025120, average score 009.2 --> reached_max_episodes
 worker 04 - episode 3000, steps 031575, average score 009.4 --> reached_max_episodes
 worker 01 - episode 3001, steps 026713, average score 009.3 --> reached_max_episodes
 worker 07 - episode 3000, steps 025187, average score 009.2 --> reached_max_episodes
 worker 01 - episode 1193, steps 010065, average score 009.3

```
show_result('avg_training_reward','VPG', vpg_results, 'A3C', a3c_results)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[197], line 1
----> 1 show_result('avg_training_reward','VPG', vpg_results, 'A3C', a3c_results)

NameError: name 'vpg_results' is not defined
```

```
show_result('avg_evaluation_reward','VPG', vpg_results, 'A3C', a3c_results)
```

One of the main things to notice is how VPG is more sample efficient than the more complex A3C methods. This is mostly because A3C uses multiple workers, which initially cost lots of data to get only a bit of progress.

```
show_result('total_steps','VPG', vpg_results, 'A3C', a3c_results)
```

```
show_result('training_time','VPG', vpg_results, 'A3C', a3c_results)
```

In terms of time, the parallel method should be fast, however this cannot be seen in the graph because the algorithm is not implemented in a real parallel way on multiple cores, due to the limitation of the threading library.

11.5 Generalized advantage estimation (GAE)

We can combine multiple n-step bootstrapping targets into a single target, creating a more robust targets than a single n-step: the lambda-target. The **Generalized advantage estimation (GAE)** John Schulman et al. "**High-dimensional Continuous Control Using Generalized Advantage Estimation**", ICLR 2016 uses an exponentially weighted combination of n-step action-advantage function targets in order to reduce the variance of policy-gradient estimates at the cost of some bias:

$$A^1(S_t, A_t) = R_t + \gamma V(S_{t+1}) - V(S_t)$$

$$A^2(S_t, A_t) = R_t + \gamma R_{t+1} + \gamma^2 V(S_{t+2}) - V(S_t)$$

$$A^3(S_t, A_t) = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 V(S_{t+3}) - V(S_t)$$

...

$$A^n(S_t, A_t) = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}) - V(S_t)$$

which we can mix to make an estimate, analogous to TD-lambda, but for advantages:

$$A^{\{\text{GAE}(\gamma, \lambda)\}}(S_t, A_t) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

Values of $\lambda=0$ returns the one-step advantage estimate and $\lambda=1$ returns the infinite-step advantage estimate.

We can implement the GAE optimize logic model in Python:

```
def optimize(logpas, entropies, rewards, values,
            local_policy_model, local_value_model,
            shared_policy_model, shared_value_model,
            shared_policy_optimizer, shared_value_optimizer,
            discounts):

    # get the length of the episode
    T = len(rewards)

    # calculate the sum of discounted returns for the n-step steps
    # remember: rewards includes the bootstrapping value
    returns = np.array([np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

    # format the variables as a tensor
```

```
returns = torch.FloatTensor(returns[:-1])

# prepare the log probabilities, the entropies and the values
logpas = torch.cat(logpas)
entropies = torch.cat(entropies)
values = torch.cat(values)

# estimate the advantages
advantage = returns - values

# calculate the policy loss
beta = 0.001
policy_loss = -(advantage.detach() * logpas + beta * entropies).sum()

# calculate the value loss
value_loss = advantage.pow(2).sum()

# optimize the policy network
# notice, we now zero the shared policy optimizer
shared_policy_optimizer.zero_grad()
policy_loss.backward()

# iterating over all local and shared policy network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_policy_model.parameters(),
    ↪ shared_policy_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad

# once the gradients are copied into the shared optimizer, we run an optimization
    ↪ step
shared_policy_optimizer.step()

# optimize the value network
shared_value_optimizer.zero_grad()
value_loss.backward()

# iterating over all local and shared value network parameters
# and copy every gradient from the local to the shared model
for local_param, shared_param in zip(local_value_model.parameters(),
    ↪ shared_value_model.parameters()):
    if shared_param.grad is None:
        shared_param._grad = local_param.grad
```



```
# once the gradients are copied into the shared optimizer, we run an optimization
↳ step
shared_value_optimizer.step()
```

We can try the algorithm in the cart-pole environment:

```
parameters['n_workers'] = 8
parameters['max_n_steps'] = 50

gae_results, gae_agent, gae_score = run_experiment(name = 'gae_results', algorithm =
↳ a3c)
```

```
show_result('avg_training_reward', 'VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('avg_evaluation_reward', 'VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('total_steps', 'VPG', vpg_results, 'GAE', gae_results)
```

```
show_result('training_time', 'VPG', vpg_results, 'GAE', gae_results)
```

11.6 Advantage Actor-Critic (A2C)

Advantage actor-critic (A2C) is the **synchronous version of A3C**. Updating the neural network in a Hogwild!-style can be chaotic, yet introducing a lock mechanism lowers A3C performance considerably. In A2C, we move the workers from the agent down to the environment. Instead of having multiple actor-learners, we have multiple actors with a single learner.

We can also use a single neural network for both the policy and the value function. Sharing a model can be particularly beneficial when learning from images, because feature extraction can be compute-intensive. However, model sharing can be challenging due to the potentially different scales of the policy and value function updates.



12 title

13 title

14 title